

MÉTODOS DE OPTIMIZACIÓN METAHEURÍSTICA PARA INGENIERÍA

ECOE
EDICIONES



Óscar Danilo Montoya Giraldo
Luis Fernando Grisales Noreña
Rubén Iván Bolaños

MÉTODOS DE OPTIMIZACIÓN METAHEURÍSTICA PARA INGENIERÍA

Con aplicaciones en el software Julia

**ÓSCAR DANILO MONTOYA GIRALDO
LUIS FERNANDO GRISALES NOREÑA
RUBÉN IVÁN BOLAÑOS**

Montoya Giraldo, Óscar Danilo, autor

Métodos de optimización metaheurística para ingeniería / Óscar Danilo Montoya Giraldo, autor, Luis Fernando Grisales Noreña, autor, Rubén Iván Bolaños, autor. -- Primera edición. -- Bogotá: Ecoe Ediciones, 2025.

253 páginas, 17x24 cm (Computación y tecnología de la información. Programación y desarrollo de software)

Incluye datos curriculares de los autores -- Incluye referencias bibliográficas.

ISBN 978-958-508-378-3 (impreso) 978-958-508-379-0 (pdf)

1. Optimización en ingeniería 2. Métodos heurísticos 3. Metaheurística 4. Optimización matemática 5. Algoritmos 6. Lenguajes de programación -- Julia I. Montoya Giraldo, Óscar Danilo, autor, II. Grisales Noreña, Luis Fernando, autor, III. Bolaños, Rubén Iván, autor.

CDD: 515.19 Ed. 23

nla-



Área: Computación y tecnología de la información

Subárea: Programación y desarrollo de software



© Óscar Danilo Montoya
© Luis Fernando Grisales Noreña
© Rubén Iván Bolaños

© Ecoe Ediciones S.A.S.
info@ecoeediciones.com
www.ecoeediciones.com
Carrera 19 # 63 C 32
Teléfono: (+57) 321 226 46 09
Bogotá, Colombia

Primera edición: Bogotá, 2025

ISBN: 978-958-508-378-3

ISBN (PDF): 978-958-508-379-0

Coordinadora editorial: Ana María Rueda G.
Coordinadora de producción editorial:
Alejandra Rondón Forero
Asistente administrativo editorial:
Edwin Corzo S.
Representante comercial de servicios
editoriales: Liliana María Lache
Carátula: Laura Yillian Garnica
Impresión: Carvajal Soluciones de
Comunicación S.A.S.
Carrera 69 #15-24

La reproducción total de esta obra, ya sea en formato físico o digital, está estrictamente prohibida sin la autorización expresa del titular de los derechos. Asimismo, cualquier reproducción parcial de este libro, con o sin fines comerciales, en formato físico o digital, requiere la autorización previa

Agradecimientos

El presente libro fue producido en el marco del proyecto de investigación: *Desarrollo de una metodología de optimización para la gestión óptima de recursos energéticos distribuidos en redes de distribución de energía eléctrica*, registrado ante el Centro de Investigaciones y Desarrollo Científico de la Universidad Distrital Francisco José de Caldas. Código interno 1643-012-2020.

Adicionalmente, este libro se publicó en el marco del proyecto ANID/FONDECYT de iniciación en investigación 2024: *Smart energy management methods for improving the economic, technical, and environmental indexes of the alternating current microgrids including variable generation and demand profiles*, identificado con el No. 11240006. Empleando los espacios del centro Tecnológico de Conversión de Energía de la Universidad de Talca.

Este libro también es un resultado del programa *Sistema Inteligente de Cubicaje como aporte a la industria 4.0* ejecutado por la Empresa Integra S.A., con código 6438-852-72034. El cual incluye los siguientes proyectos de investigación: i) *Desarrollo de un sistema guía de visión por computadora para un manipulador industrial embebido dentro de un Sistema Inteligente de Cubicaje* cód 72052, ii) *Desarrollo de un sistema de trayectorias libres de colisiones en un manipulador industrial embebido dentro un Sistema Inteligente de Cubicaje* cód 72060, iii) *Desarrollo e implementación de un sistema de empaquetamiento de mercancías regular e irregular con llegada de mercancía en línea embebido dentro de Sistema Inteligente de Cubicaje* cód 72062.

Los autores quieren agradecer a la Red para la Integración a Gran Escala de Energías Renovables en Sistemas Eléctricos (RIBIERSE-CYTED) de la cual hace parte el grupo de investigación en compatibilidad e interferencia electromagnética (GCEM), así como el semillero de investigación en circuitos y sistemas (SICSUD) por el soporte brindado a este libro resultado de investigación. Para más información

sobre RIBIERSE consulte la siguiente página web: RIBIERSE-CYTED.

Por otra parte, los autores agradecen el soporte brindado por la Empresa Integra S.A. mediante la disposición de su Unidad de I+D+i y su grupo de Investigación TransFórmate, reconocidos por Minciecnias. Así mismo, agradecemos al Fondo Nacional de Financiamiento para la Ciencia, la Tecnología y la Innovación, Fondo Francisco José de Caldas por la financiación recibida en el marco del programa: *Sistema Inteligente de Cubicaje como aporte a la industria 4.0* Código 6438-85272034. El cual incluye los proyectos: i) 72052, ii) 72060 y iii) 72062.

Finalmente, queremos expresar nuestro agradecimiento a todos los estudiantes que hacen parte del Semillero de Investigación en Circuitos y Sistemas de la Universidad Distrital Francisco José de Caldas (SICSUD), así como al Laboratorio de Investigación en Sistemas Eléctricos de Potencia (LOPSE) de la Universidad de Talca por sus valiosos aportes durante el proceso de construcción y validación de los resultados presentados en este libro.

Sobre los autores

Ó. D. MONTOYA GIRALDO, (Obando, Valle del Cauca, Colombia, 1989) Doctor en Ingeniería de la Universidad Tecnológica de Pereira, Magíster en Ingeniería Eléctrica de la Universidad Tecnológica de Pereira e Ingeniero Electricista Universidad Tecnológica de Pereira. Actualmente es profesor asociado adscrito al Proyecto Curricular de Ingeniería Eléctrica de la Facultad de Ingeniería de la Universidad Distrital Francisco José de Caldas. Ha publicado más de trescientos artículos en revistas y conferencias registradas en las bases de datos Scopus y WoS. Investigador Asociado en Minciencias, Miembro Senior el Instituto de Ingenieros Electricistas y Electrónicos de Estados Unidos (IEEE, USA) y miembro del grupo de Investigación en Compatibilidad e Interferencia Electromagnética (GCEM) adscrito a la Facultad de Ingeniería de la Universidad Distrital Francisco José de Caldas con clasificación A1 en Minciencias. odmontoyag@udistrital.edu.co

L. F. GRISALES-NOREÑA, (Cartago, Valle del Cauca, Colombia, 1990) Doctor en Energías Renovables de la Universidad de Jaén de España. Doctor en Ingeniería Automática de la Universidad Nacional de Colombia, Magíster en Ingeniería Eléctrica de la Universidad Tecnológica de Pereira e Ingeniero Electricista Universidad Tecnológica de Pereira. Profesor conferenciante de Departamento de Ingeniería Eléctrica de la Facultad de Ingeniería de la Universidad de Talca-Chile. Ha publicado más de ciento sesenta artículos en revistas y conferencias registradas en las bases de datos Scopus y WoS. Investigador Senior en Minciencias y Director alterno del Magíster en Ciencias de la Ingeniería con mención en Conversión de Energía de la Universidad de Talca . luis.grisales@utalca.cl

R. I. BOLAÑOS, (Garzón, Huila, Colombia, 1988) Doctor en Ingeniería, Magíster en Ingeniería Eléctrica e Ingeniero Electricista de la Universidad Tecnológica de Pereira. Actualmente es docente ocasional adscrito al Programa de Mecatrónica y Electromecánica de la Facultad de Ingenierías del Instituto Tecnológico Metropolitano de Medellín. Ha publicado diferentes artículos en revistas y conferencias registradas en las bases

de datos Scopus y WoS. Investigador junior en Minciencias, Miembro del Instituto de Ingenieros Electricistas y Electrónicos de Estados Unidos (IEEE, USA), miembro de los grupos de Investigación TransFórmate, avalado por la empresa Integra S.A. con reconocimiento de Minciencias en la categoría A1 (rbolanos@integra.com.co), y Materiales Avanzados y Energía (MATYER) en la línea de investigación eficiencia energética adscrita a la Facultad de Ingenierías del Instituto Tecnológico Metropolitano con clasificación A1 en Minciencias (rubenbolanos@itm.edu.co).

Resumen ejecutivo

La optimización en ingeniería y ciencia es una área de las matemáticas en constante desarrollo, que ha tomado auge en el mundo moderno debido al desarrollo de la computación y la necesidad humana de la búsqueda de mejores soluciones para problemas con recursos limitados. En este sentido, este libro presenta diferentes herramientas de optimización pertenecientes a la familia de los algoritmos de optimización metaheurística para abordar problemas de optimización en tres dominios diferentes. El primer dominio se concentra en problemas de optimización de naturaleza discreta, donde las variables de decisión pertenecen al conjunto de los números enteros, más precisamente al conjunto de los números binarios. Para ello se estudia uno de los problemas más clásicos en investigación de operaciones, como lo es el caso del problema de la mochila. El segundo dominio analizado se concentra en problemas de optimización de naturaleza continua, donde las variables de decisión pertenecen al conjunto de los números reales y cuya principal dificultad radica en que el espacio de soluciones es de dimensión infinita. Para ilustrar este tipo de problemas, se estudia un problema clásico en ingeniería eléctrica, conocido como despacho económico de centrales térmicas multiperíodo. El tercer dominio estudiado, corresponde a una combinación de los dos casos anteriores, es decir, un problema de optimización que combina variables definidas en el conjunto de los números reales y variables en el conjunto de los números binarios. Para este caso, se diseña un problema de programación no lineal entero mixto que ilustra la complejidad de este tipo de problemas, que son muy comunes en ingeniería y ciencia.

Para cada uno de los problemas de optimización estudiado, este libro contribuye con la explicación y aplicación detallada de tres técnicas de optimización, con lo cual, se tiene un total de nueve algoritmos de optimización metaheurística que han sido cuidadosamente seleccionados dentro del gran universo de posibilidades que ofrecen las técnicas metaheurísticas, para abordar problemas de optimización complejos en ingeniería y ciencia. Dentro de las técnicas seleccionadas, se destacan algoritmos bio-inspirados y técnicas inspiradas en matemática y probabilidad. Además, estas técnicas se agrupan en métodos evolutivos de naturaleza poblacional y de trayectoria,

lo que garantiza un grupo de posibilidades al lector de seleccionar dentro del abanico de posibilidades, la técnica de optimización metaheurística que mejor se adapte a su área de interés. Dentro de los algoritmos bio-inspirados, se tienen: (i) el optimizador por enjambre de partículas, (ii) el algoritmo JAYA, (iii) el algoritmo de búsqueda de cuervos, (iv) el algoritmo genético poblacional y de Chu & Beasley, (v) el algoritmo de enjambre de salpas; además, por el lado de las técnicas inspiradas en matemáticas y probabilidad, se tienen: (vi) el algoritmo de senos y cosenos, (vii) el algoritmo de optimización de aprendizaje incremental, (viii) el algoritmo de búsqueda por vórtices, y (ix) el algoritmo de optimización de distribución normal generalizada.

Como principal aporte de este libro de investigación, se presentan dos elementos relevantes. El primero de ellos corresponde a la formulación de los problemas de optimización en diferentes dominios y su solución mediante técnicas exactas disponibles en el software Julia, y en segundo corresponde a la descripción detallada de cada una de las técnicas metaheurísticas y la programación de estas en el software Julia a manera de tutorial, lo que le permitirá al lector comprender la lógica de cada algoritmo, así como su algoritmia y programación.

Nótese que todos los códigos desarrollados en este libro están disponibles en la página web: <https://github.com/odmontoya/METODOS-DE-OPTIMIZACION-METAHEURSTICA-PARA-INGENIERIA>.

Contenido

1	Programación matemática	1
1.1	Modelo general de un problema de optimización	1
1.1.1	Optimización sin restricciones	2
1.1.2	Optimización con restricciones	2
1.1.3	Naturaleza de los modelos de optimización	3
1.2	Metodologías de solución	4
1.2.1	Métodos exactos	5
1.2.2	Técnicas heurísticas y metaheurísticas	6
1.3	Algunas aplicaciones en ingeniería	6
2	Modelos de optimización en el software Julia	9
2.1	Formulación general del problema de la mochila	9
2.1.1	Implementación en el software Julia	10
2.2	Despacho económico de centrales de generación térmica	13
2.2.1	Modelo matemático clásico	14
2.2.2	Modelo considerando pérdidas de potencia	15
2.2.3	Implementación en el software Julia	16
2.3	Un problema de programación no lineal entera mixta	21
2.3.1	Complejidad del problema de optimización	22
2.3.2	Implementación en el software Julia	23
2.4	Algunos datos sobre Julia	26
3	Optimización metaheurística	29
3.1	Posible clasificación	30
3.1.1	Técnicas de optimización bio-inspiradas	31
3.1.2	Técnicas de optimización inspiradas en matemáticas	32
3.1.3	Técnicas de optimización inspiradas en física	33
3.1.4	Algoritmos inspirados en comportamientos sociales	33
3.2	Estrategias basadas en población y en trayectorias	34

3.2.1	Optimización basada en poblaciones	34
3.2.2	Optimización basada en trayectorias	35
3.3	Algoritmia y programación	35
3.4	Función adaptación y codificación	37
3.4.1	Construcción de la función de adaptación	38
3.4.2	Estructura general de una codificación	39
3.5	Función de adaptación y codificación en el problema de la mochila . .	40
3.6	Función de adaptación y codificación en el problema de despacho óptimo de centrales térmicas	41
3.7	Función de adaptación y codificación en el problema entero mixto . .	42
3.8	Selección de algoritmos para el desarrollo de este libro	43

I Técnicas metaheurísticas en dominio discreto 47

4	Algoritmo de senos y cosenos	49
4.1	Filosofía e inspiración	49
4.2	Algoritmo de senos y cosenos	51
4.2.1	Población inicial	51
4.2.2	Reglas de avance	52
4.2.3	Sustitución en la población	54
4.2.4	Criterios de parada	55
4.2.5	Implementación computacional	56
4.3	Implementación en el software Julia	57
5	Algoritmo de aprendizaje incremental basado en poblaciones	63
5.1	Filosofía e inspiración	64
5.2	Algoritmo de aprendizaje incremental basado en poblaciones	66
5.2.1	Inicialización de parámetros	67
5.2.2	Población inicial	67
5.2.3	Reglas de avance	69
5.2.4	Criterio de parada	70
5.2.5	Implementación computacional	71
5.3	Implementación en el software Julia	72
6	Algoritmo genético	81
6.1	Filosofía e inspiración	82
6.2	Algoritmo Genético	83
6.2.1	Algoritmo Genético poblacional	83
6.2.1.1	Población inicial	84

6.2.1.2	Función de adaptación	85
6.2.1.3	Reglas de avance	85
6.2.1.4	Sustitución en la población	88
6.2.1.5	Criterio de parada	90
6.2.1.6	Implementación computacional	90
6.2.1.7	Implementación en el software Julia	91
6.2.2	Algoritmo Genético de Chu-Beasley	96
6.2.2.1	Sustitución en la población	96
6.2.2.2	Criterio de parada	97
6.2.2.3	Implementación computacional	97
6.2.2.4	Implementación en el software Julia	98

II Técnicas metaheurísticas en dominio continuo 105

7	Algoritmo de optimización de búsqueda de cuervos	107
7.1	Filosofía e inspiración	108
7.2	Algoritmo de búsqueda de cuervos	109
7.2.1	Inicialización de parámetros	109
7.2.2	Población y memoria inicial	110
7.2.3	Reglas de avance	112
7.2.4	Criterio de parada	114
7.2.5	Implementación computacional	115
7.3	Implementación en el software Julia	116
8	Algoritmo JAYA	127
8.1	Filosofía e inspiración	127
8.2	Algoritmo JAYA	129
8.2.1	Inicialización de parámetros	130
8.2.2	Población inicial	130
8.2.3	Evaluación de la función adaptación y selección del mejor y peor individuo	132
8.2.4	Reglas de avance	133
8.2.5	Sustitución en la población	134
8.2.6	Criterios de parada	135
8.2.7	Implementación computacional	136
8.3	Implementación en el software Julia	137
9	Algoritmo de optimización de búsqueda por vórtices	147
9.1	Filosofía e inspiración	148

9.2	Algoritmo de búsqueda de vórtices	149
9.2.1	Centro de la hiper-elipse y vecindario	149
9.2.2	Reglas de avance	151
9.2.3	Reducción del radio	152
9.2.4	Criterios de parada	152
9.2.5	Implementación computacional	153
9.3	Implementación en el software Julia	154

III Técnicas metaheurísticas en dominio discreto-continuo 163

10	Algoritmo de optimización de distribución normal generalizada	165
10.1	Filosofía e inspiración	166
10.2	Algoritmo de optimización de distribución normal generalizada	167
10.2.1	Población inicial	167
10.2.2	Reglas de avance	168
10.2.2.1	Búsqueda local	169
10.2.2.2	Búsqueda global	170
10.2.3	Sustitución en la población	172
10.2.4	Criterios de parada	173
10.2.5	Implementación computacional	173
10.3	Implementación en el software Julia	175
11	Algoritmo de optimización por cúmulo de partículas	183
11.1	Filosofía e inspiración	184
11.2	Algoritmo PSO	186
11.2.1	Inicialización de parámetros	186
11.2.2	Población inicial	187
11.2.3	Evaluación de la función adaptación y selección de mejor solución local y global	190
11.2.4	Reglas de avance	190
11.2.5	Sustitución de la mejor solución local y global	192
11.2.6	Criterios de parada	192
11.2.7	Implementación computacional	193
11.3	Implementación en el software Julia	195
12	Algoritmo de optimización por enjambre de Salpas	203
12.1	Filosofía e inspiración	204
12.2	Algoritmo por enjambre de salpas	206

12.2.1	Inicialización de parámetros	206
12.2.2	Población inicial	206
12.2.3	Función de adaptación	207
12.2.4	Reglas de avance	208
12.2.5	Sustitución de la mejor solución	209
12.2.6	Criterios de parada	210
12.2.7	Implementación computacional	210
12.3	Implementación en el software Julia	210
13	Conclusiones, recomendaciones y trabajos futuros	219
13.1	Conclusiones	219
13.2	Recomendaciones	224
13.3	Trabajos futuros	226

Lista de figuras

2.1	Representación de un sistema eléctrico de potencia	13
2.2	Proyección del problema de optimización al espacio tridimensional de la función $z = 4 \sin(8x_1) + 0.66x_1^2 + 3 \sin(6x_2) + 0.50x_2^2$	23

Lista de tablas

2.1	Coeficientes de la función objetivo y límites de generación para las plantas térmicas	16
2.2	Perfil diario de demanda (donde P^D es la demanda total acumulada) .	17
2.3	Información paramétrica para el modelo de optimización no lineal entero mixto	24

1

Programación matemática

Este capítulo presenta una introducción básica a los problemas de optimización en ingeniería y ciencia, partiendo de su formulación general y la clasificación de los modelos matemáticos. Además, se presentan algunos comentarios acerca de las metodologías para la resolución de estos problemas a través de metodologías exactas y técnicas de optimización metaheurística.

1.1 Modelo general de un problema de optimización

En ingeniería y ciencia, el concepto de optimización es ampliamente conocido y aplicado, para dar un contexto a esta palabra, en este libro se entenderá por optimización, lo siguiente:

Optimización corresponde a una palabra con origen en el latín *Optimum*, por lo cual, la Real Academia de la Lengua Española define el verbo **optimizar** como **Buscar la mejor manera de realizar una actividad**. En este contexto, la optimización se entenderá en este libro como el proceso de hallar la mejor solución posible para un

modelo matemático.

Un par de ejemplos que ilustran el concepto de optimización se presentan a continuación.

- i. Un jugador de fútbol que actúa en la posición de delantero tiene como objetivo anotar la mayor cantidad de goles en la valla rival, por cuanto, este es un problema de optimización en la que se busca maximizar un objetivo, que es el número de goles a marcar.
- ii. Ahora, otro jugador de fútbol que actúa en la posición de arquero tiene como objetivo mantener su valla invicta, con lo cual, este es un problema de optimización en la que se busca el mínimo global del problema, el cual es recibir cero goles durante la mayor cantidad de partidos disputados.

A partir de los ejemplos anteriores, se concluye que un problema de optimización podrá tener diferentes sentidos de análisis, es decir, maximizar o minimizar un objetivo, con lo cual desde el punto de vista matemático podrá ser formulado empleando modelos restringidos o irrestringidos (Yang, 2010). A continuación, se ilustran más detalles al respecto.

1.1.1 Optimización sin restricciones

Este tipo de problemas de optimización, también conocidos como, problemas de optimización irrestringida, son aquellos que únicamente tienen una función objetivo y el dominio de las variables de decisión pertenece a un conjunto determinado. La formulación de este tipo de problemas se presenta en (1.1).

$$\begin{aligned} \min / \max \quad & f(\mathbf{x}) \\ & \mathbf{x} \in \mathbb{R}^n, \end{aligned} \tag{1.1}$$

donde $\mathbf{x}$ corresponde al vector de variables de decisión definido para este caso en el dominio de los números reales con dimensión n , siendo $f(\mathbf{x})$ la función objetivo de interés.

1.1.2 Optimización con restricciones

La optimización con restricciones corresponde al caso práctico de mayor interés en ingeniería y ciencia, puesto que para optimizar un proceso o actividad, siempre se

MÉTODOS DE OPTIMIZACIÓN METAHEURÍSTICA PARA INGENIERÍA: Con aplicaciones en el software Julia

dispondrá de un nivel de recursos limitados (Rao, 2009). La estructura general de un problema de optimización con restricciones se presenta a continuación.

Función objetivo

$$\min / \max f(\mathbf{x})$$

Restricciones

$$g_j(\mathbf{x}) = 0, \{j = 1, 2, \dots, G\} \quad (1.2)$$

$$h_k(\mathbf{x}) \leq 0, \{k = 1, 2, \dots, H\}$$

$$\mathbf{x}^{\min} \leq \mathbf{x} \leq \mathbf{x}^{\max},$$

donde $g_j(\mathbf{x})$ corresponde a la j -ésima restricción de igualdad, $h_k(\mathbf{x})$ es la k -ésima restricción de desigualdad, $\mathbf{x}^{\min}$ y $\mathbf{x}^{\max}$ representan los límites máximos y mínimos aplicables al vector de variables de decisión $\mathbf{x}$.

El conjunto de restricciones del modelo matemático (1.2) define el espacio de soluciones admisible para las variables de decisión $\mathbf{x}$, lo cual implica que será el espacio donde estas podrán tomar valores, tal que, la función objetivo sea maximizada o minimizada, según sea el caso.

1.1.3 Naturaleza de los modelos de optimización

Una de las principales características de la optimización en ingeniería y ciencia corresponde a la naturaleza del modelo de optimización, es decir, el dominio de las variables de decisión, así como la estructura de la función objetivo y las restricciones (Garces, 2021). En este sentido, se tiene lo siguiente:

- i. Las variables de decisión, i.e., el vector $\mathbf{x}$ podrá definirse en el conjunto de los números reales, lo cual implicará que el problema es dominio continuo. También podrá ser definido en el conjunto de los números naturales, es decir, variables discontinuas, lo que llevará al modelo a una estructura discreta. Finalmente, las variables podrán ser combinación de los dos conjuntos anteriores, lo cual definirá un problema entero mixto.
- ii. Las funciones $f(\mathbf{x})$ y las restricciones $g_j(\mathbf{x})$ y $h_k(\mathbf{x})$ podrán tener estructuras lineales o no lineales, lo cual definirá la naturaleza del modelo de optimización.

A continuación se presentan las clasificaciones más comunes en cuanto a la estructura de los modelos matemáticos (Conejo et al., 2006).

MÉTODOS DE OPTIMIZACIÓN METAHEURÍSTICA PARA INGENIERÍA: Con aplicaciones en el software Julia

1. **Programación lineal:** Un modelo de programación lineal tendrá lugar cuando en el modelo matemático dado en (1.2), la función objetivo y las restricciones estén compuestas de funciones lineales, en general, hiperplanos, y las variables de decisión pertenezcan al conjunto de los números reales.
2. **Programación lineal entera:** Un modelo de programación lineal entera tendrá lugar cuando en el modelo matemático dado en (1.2), la función objetivo y las restricciones estén compuestas de funciones lineales, en general, hiperplanos, y las variables de decisión pertenezcan al conjunto de los números enteros.
3. **Programación lineal entera mixta:** Un modelo de programación lineal entera tendrá lugar cuando en el modelo matemático dado en (1.2), la función objetivo y las restricciones estén compuestas de funciones lineales, en general, hiperplanos, y las variables de decisión combinen el conjunto de números enteros con el conjunto de los números reales.
4. **Programación no lineal:** Un modelo de programación no lineal tendrá lugar cuando en el modelo matemático dado en (1.2), la función objetivo sea no lineal o en su defecto alguna de las restricciones tenga forma no lineal y las variables de decisión pertenezcan al conjunto de los números reales.
5. **Programación no lineal entera:** Un modelo de programación no lineal entera tendrá lugar cuando en el modelo matemático dado en (1.2), la función objetivo sea no lineal o en su defecto alguna de las restricciones tenga forma no lineal y las variables de decisión pertenezcan al conjunto de los números enteros.
6. **Programación no lineal entera mixta:** Un modelo de programación no lineal entera mixta tendrá lugar cuando en el modelo matemático dado en (1.2), la función objetivo sea no lineal o en su defecto alguna de las restricciones tenga forma no lineal y las variables de decisión combinen el conjunto de números enteros con el conjunto de los números reales.

1.2 Metodologías de solución

La metodología de solución aplicable a un problema de optimización con la estructura (1.2) dependerá de la naturaleza del modelo de optimización estudiado, así por ejemplo, un problema de programación lineal podrá ser resuelto a través del método *simplex* o sus variantes o por medio de algoritmos de optimización metaheurística, así pues, la selección del método de optimización dependerá esencialmente de la estructura del modelo y del tamaño del espacio de soluciones, i.e., de la dimensión del problema.

1.2.1 Métodos exactos

Los métodos exactos para problemas de optimización con la estructura definida en (1.2) corresponden esencialmente a técnicas de optimización basadas en procedimientos analíticos (Yang, 2010). Estos procedimientos se definen en función de la naturaleza matemática del problema, es decir, el tipo de modelo estudiado. Por ejemplo, en el caso, de problemas de programación no lineal, se reescribe el modelo de optimización mediante multiplicadores de *Lagrange* por medio de las condiciones de optimalidad de *Karush-Kuhn-Tucker* (Garces, 2021). Una vez construido el modelo ampliado, se aplican técnicas clásicas de optimización para problemas irrestrictos, como el caso de gradiente descendente, Netwon-Raphson o el método de penalización de la función objetivo (Rao, 2019).

En el caso de problemas de optimización con variables de dominio discreto o combinadas con variables de dominio continuo, se emplean metodologías que desacoplan el problema discreto del problema continuo usando estrategias de ramificación y sondeo (i.e., *branch & bound*) o ramificación y corte (i.e., *branch & cut*), las cual definen el valor de las variables discretas por medio de un árbol de decisión y una vez definidas se resuelve el problema lineal o no lineal resultante (Borchers, 2008).

Otra área de investigación en las metodologías analíticas, son aquellas que se dedican a explorar y explotar las propiedades geométricas del espacio de soluciones, como es el caso de la optimización convexa (Garcés Ruiz, 2020). Esta es una disciplina que estudia las propiedades del modelo matemático, con el fin de garantizar analíticamente la existencia de la solución óptima global y su hallazgo por medio de algoritmos como el de puntos interiores, estudiando las propiedades geométricas del modelo matemático. Por ejemplo, los problemas de programación lineal, problemas de cono de segundo orden o de programación semidefinida pertenecen a la familia de modelos convexos (Cambini and Martein, 2009), y por tanto, será posible hallar la solución óptima global de este, ya que geométricamente se conoce que esta existe, si el espacio de soluciones es no nulo.

La principal dificultad de los métodos de optimización exacta radica en la dimensión del problema de optimización, es decir, número de restricciones y número de variables, por lo cual, aunque se conozca de la existencia del óptimo global, una solución mediante métodos exactos podrá tomar tiempos de solución exponencial, con lo cual no será resoluble de manera práctica. Por esta razón se requerirá de soluciones de buena calidad cercanas al óptimo global, que sólo podrán ser alcanzadas con métodos aproximados (i.e., relajaciones) o técnicas metaheurísticas.

1.2.2 Técnicas heurísticas y metaheurísticas

En ingeniería y ciencia, una de las vertientes de mayor auge en las últimas dos décadas, corresponde a las metodologías de optimización basadas en algoritmos de origen heurístico o metaheurístico.

1. **Técnicas heurísticas:** Una estrategia de optimización heurística se caracteriza por ser un procedimiento para solucionar un problema complejo de optimización por medio de una aproximación intuitiva, en la que la estructura matemática del problema se utiliza de forma inteligente para obtener una buena solución de manera eficiente y con tiempos de cómputo reducidos (Díaz-López et al., 2016).
2. **Técnicas metaheurísticas:** Una estrategia de optimización metaheurística es un método aproximado de solución que son diseñados para problemas optimización complejos, en los que las estrategias heurísticas clásicas no son efectivos. Las técnicas metaheurísticas proporcionan un marco general para crear nuevos algoritmos híbridos, combinando diferentes conceptos derivados de la inteligencia artificial, la evolución biológica y la teoría de estadística y probabilidad, entre otros (César-Vélez and Montoya, 2007).

La principal ventaja de las metodologías de optimización metaheurística es que no trabajan sobre la geometría del problema de optimización, sino que emplean formulaciones basadas en funciones de penalización para explorar el espacio de soluciones dando la posibilidad de salirse de la región factible, con el objetivo de explorar nuevas regiones del espacio de soluciones en las que puedan existir soluciones de buena calidad (Sörensen and Glover, 2013).

La diferencia fundamental entre las estrategias de optimización heurística y metaheurística radica en la capacidad de los últimos para escapar de soluciones óptimas locales en búsqueda de la solución óptima global, o una solución cercana a esta, mientras que las estrategias heurísticas tienden a tener convergencia a óptimos locales sin la posibilidad de escapar de estos (Blocho, 2020).

1.3 Algunas aplicaciones en ingeniería

La programación matemática es un campo en constante auge y evolución, ya que el avance tecnológico, permite resolver en sistemas de cómputo avanzados, problemas

MÉTODOS DE OPTIMIZACIÓN METAHEURÍSTICA PARA INGENIERÍA: Con aplicaciones en el software Julia

impensados décadas atrás. En este contexto, la optimización es una área transversal a todas las ingenierías y la ciencia en general, ya que como se mencionó anteriormente, el avance de la humanidad está soportado en la posibilidad de hacer más y mejores cosas a partir de recursos limitados, lo cual puede tomarse como una definición informal de lo que es optimización.

Innumerable cantidad de aplicaciones de la optimización en ingeniería y ciencia existen en la actualidad, desde el diseño de medicamentos para enfermedades graves hasta la exploración espacial, sin embargo, a continuación se listan algunas de las aplicaciones en ingeniería eléctrica, electrónica, mecánica e industrial, entre otras ingenierías.

1. En ingeniería eléctrica, la programación matemática se concentran en proveer soluciones para problemas de optimización complejos, los cuales se relacionan con la gestión y la operación eléctrica del sistema eléctrico en su conjunto. En este sentido, algunas de las aplicaciones pueden ser: diseño de rutas para líneas de transmisión de alta y extra alta tensión (Jabr, 2013), despacho económico de centrales de generación de energía (térmicas, hidráulicas, renovables, entre otras) (El-Hawary, 1979), localización y dimensionamiento de estaciones de recarga de vehículos eléctricos (Vandet and Rich, 2023), estimación de parámetros en máquinas y dispositivos eléctricos (Rodríguez-Abreo et al., 2022) y diseño de sistemas de suministro eléctrico para zonas rurales sin acceso a electricidad (Montoya et al., 2021b), entre otros problemas de optimización.
2. En ingeniería electrónica, la programación matemática corresponde a una herramienta fundamental para el diseño y empaquetamiento de circuitos electrónicos integrados (Suwa and Hadim, 2010), diseño de estrategias de control para sistemas robóticos (Rojas-Quintero et al., 2021), seguimiento de trayectorias para vehículos no tripulados y tecnología de drones (Yang et al., 2023), así como para el diseño óptimo de sistemas de telecomunicaciones (Guttula et al., 2021), entre otras aplicaciones relevantes.
3. En ingeniería mecánica, la programación matemática es una pieza fundamental en el desarrollo de nuevos materiales (Huang and Li, 2021), diseño de maquinaria y herramienta (Ma et al., 2023), diseño de álabes para turbinas de generación de energía eléctrica (Song et al., 2020) o sistemas de tracción para sistemas de carga pesada (Mastinu and Gobbi, 2001), entre otras aplicaciones.
4. En ingeniería industrial, las aplicaciones de la optimización y programación matemática son muy variadas, e incluso, es posible afirmar que esta rama de la ingeniería es la responsables de los grandes avances en optimización gracias a la investigación de operaciones (Carter et al., 2018). Desde mediados

MÉTODOS DE OPTIMIZACIÓN METAHEURÍSTICA PARA INGENIERÍA: Con aplicaciones en el software Julia

del siglo XX, la optimización y sus aplicaciones en investigación operativa marcaron el destino político y económico a nivel mundial (Ansio et al., 1980). En este sentido, la programación matemática juega un papel fundamental en el desarrollo de sistemas logísticos para distribución de mercancías (Pečený et al., 2020), operación de sistemas portuarios (Roh et al., 2021), diseño de sistemas de flujo de trabajo en industrias (Dauzère-Pérès et al., 2024) y planeación de esquemas de producción agrícola (Zhai et al., 2023), entre otros.

Teniendo en cuenta las aplicaciones de la optimización en ingeniería y ciencia, así como su potencial para la contribución al desarrollo económico y social de la humanidad, este libro pretende ser una herramienta para el aprendizaje de diversos métodos de optimización de naturaleza metaheurística para aplicaciones en ingeniería con énfasis en estudiantes de pregrado de últimos semestres, así como cursos de posgrado en programas de maestría y doctorado en ingeniería.

2

Modelos de optimización en el software Julia

Este capítulo presenta la formulación matemática de tres problemas de optimización clásicos en ingeniería y ciencia. El primer problema corresponde al denominado problema de la mochila, el cual es de naturaleza binaria. El segundo problema de optimización que se presenta, corresponde al despacho económico de centrales de generación de energía eléctrica de origen térmico, el cual es de dominio continuo; mientras que el tercer problema de optimización, corresponde a un problema entero-mixto de naturaleza no lineal que combina variables binarias con funciones parabólicas y sinusoidales. Para cada uno de estos problemas se presenta su formulación matemática, así como su implementación computacional en el software Julia.

2.1 Formulación general del problema de la mochila

El problema de la mochila es uno de los problemas de programación lineal entera mas conocidos de la literatura especializada (Salkin and De Kluyver, 1975), el cual tiene como objetivo obtener el mayor beneficio económico en la adquisición de N productos,

teniendo en cuenta dos restricciones típicas asociadas a las características físicas de la mochila o elemento contenedor, siendo estas el volumen máximo admisible ($V_{\max}$) y el peso máximo ($W_{\max}$) que esta soporta. Matemáticamente el problema de la mochila puede ser formulado considerando variables de decisión de tipo binario o de naturaleza discreta (i.e., perteneciente a los números naturales). A continuación, se presenta el modelo matemático que representa el problema de la mochila.

$$\text{Fun. Obj.} \quad \max B_n = \sum_{k \in \mathbf{N}} C_k x_k, \quad (2.1)$$

$$\text{s.a.:} \quad \sum_{k \in \mathbf{N}} V_k x_k \leq V_{\max}, \quad \{\forall k \in \mathbf{N}\}, \quad (2.2)$$

$$\sum_{k \in \mathbf{N}} W_k x_k \leq W_{\max}, \quad \{\forall k \in \mathbf{N}\}, \quad (2.3)$$

$$x_k \in \{0, 1\}, \quad \{\forall k \in \mathbf{N}\}, \quad (2.4)$$

donde B_n representa el valor de la función objetivo relacionada con el beneficio neto esperado, C_k corresponde al factor de costo aplicable al producto x_k (variable de decisión), siendo W_k y V_k el peso y volumen asociados al k -ésimo producto. En este modelo matemático, $\mathbf{N}$ es el conjunto que contiene el número de productos disponibles.

Observación 2.1. Nótese que el modelo matemático (2.1)–(2.4) corresponde a la versión binaria del problema de la mochila; sin embargo, para el caso discreto, la variable x_k será definida entre $x_k^{\min}$ y $x_k^{\max}$, siendo el conjunto de decisión $\mathbf{N} \subset \mathbb{R}$ (Cacchiani et al., 2022).

2.1.1 Implementación en el software Julia

Con el fin de ilustrar la implementación del problema de la mochila definido por las ecuaciones (2.1) y (2.4), se consideran los siguientes vectores (nótese que se han escrito en forma matricial por efectos de formato en este documento):

$$C = \begin{bmatrix} 236 & 484 & 500 & 136 & 380 & 299 & 481 & 419 & 419 & 301 \\ 226 & 467 & 224 & 202 & 474 & 241 & 287 & 209 & 319 & 231 \\ 111 & 182 & 233 & 339 & 412 & 251 & 460 & 235 & 341 & 439 \end{bmatrix}$$

$$W = \begin{bmatrix} 67 & 34 & 83 & 49 & 33 & 75 & 17 & 34 & 92 & 15 \\ 62 & 19 & 64 & 45 & 91 & 34 & 85 & 68 & 23 & 45 \\ 86 & 60 & 84 & 60 & 92 & 43 & 19 & 51 & 78 & 13 \end{bmatrix}$$

$$V = \begin{bmatrix} 40 & 45 & 50 & 25 & 28 & 25 & 50 & 35 & 21 & 30 \\ 31 & 41 & 29 & 39 & 21 & 30 & 39 & 21 & 43 & 50 \\ 47 & 37 & 45 & 39 & 31 & 45 & 29 & 21 & 43 & 47 \end{bmatrix}$$

además, los valores de volumen máximo y peso máximo admisibles para este problema son $V_{\max} = 861$ y $W_{\max} = 1215$, respectivamente.

```
using JuMP, HiGHS, DataFrames
ks = DataFrames.DataFrame([
(236, 40, 67), (484, 45, 34), (500, 50, 83), (136, 25, 49),
(380, 28, 33), (299, 25, 75), (481, 50, 17), (419, 35, 34),
(419, 21, 92), (301, 30, 15), (226, 31, 62), (467, 41, 19),
(224, 29, 64), (202, 39, 45), (474, 21, 91), (241, 30, 34),
(287, 39, 85), (209, 21, 68), (319, 43, 23), (231, 50, 45),
(111, 47, 86), (182, 37, 60), (233, 45, 84), (339, 39, 60),
(412, 31, 92), (251, 45, 43), (460, 29, 19), (235, 21, 51),
(341, 43, 78), (439, 47, 13),])
DataFrames.rename!(ks, [:C, :V, :W])
C = ks.C'; V = ks.V'; W = ks.W';
Vmax = 861; Wmax = 1215;
n = size(C,2); N = 1:n;
mochila = Model(HiGHS.Optimizer)
@variable(mochila, x[k in N], Bin)
@objective(mochila, Max, sum(C[k]*x[k] for k in N))
@constraint(mochila, sum(W[k]*x[k] for k in N) <= Wmax)
@constraint(mochila, sum(V[k]*x[k] for k in N) <= Vmax)
JuMP.optimize!(mochila)
println(Int.(round.(value.(x))))
```

En la implementación anterior, el comando `JuMP` es empleado para definir el entorno de optimización en el software Julia, sobre el cual se puede emplear el solucionador de problemas de programación lineal o lineal entera denominado `HiGHS`. Además, el comando `DataFrames` permite organizar información de forma estructurada de manera matricial. Una vez sean seleccionados los parámetros relacionados con los vectores de costo, volumen y peso, se define el número de variables del problema empleando el comando `n = size(C,2)`. Posteriormente, se define el modelo matemático a través de un objeto con el comando `mochila = Model(HiGHS.Optimizer)`. Nótese que dentro de este objeto matemático, se construye el modelo matemático, como sigue:

MÉTODOS DE OPTIMIZACIÓN METAHEURÍSTICA PARA INGENIERÍA: Con aplicaciones en el software Julia

- i. La palabra reservada `variable(mochila, x[k in N], Bin)` asigna al objeto `mochila`, una variable denominada `x` cuya dimensión es `N` (i.e., un vector).
- ii. El comando `objective(mochila, Max, sum(C[k]*x[k] for k in N))` le asigna al objeto matemático `mochila`, la dirección de optimización, que para el caso de la mochila es maximización, i.e., `Max`, además, se construye la función objetivo a partir de una estructura de suma.
- iii. El comando `constraint(mochila, sum(W[k]*x[k] for k in N) <= Wmax)` permite asignar al objeto `mochila`, las restricciones, en este caso se agrega la restricción de peso máximo a manera de ejemplo.
- iv. Una vez construido el modelo matemático, se emplea el comando `JuMP.optimize!(mochila)` para resolver el modelo matemático implementado. Nótese que el comando `println(round.(value.(x))')` imprime en pantalla el vector solución.

A continuación se ilustra la solución obtenida para este problema.

```
Solving report
Status          Optimal
Primal bound    8416
Dual bound      8416
Gap             0% (tolerance: 0.01%)
Solution status feasible
                8416 (objective)
[0 1 1 0 1 1 1 1 1 1 1 1 0 1 1
 1 1 1 1 1 0 0 0 1 1 1 1 1 1 1]
```

donde se observa que la solución óptima para este problema corresponde a 8416 unidades.

2.2 Despacho económico de centrales de generación térmica

El problema de despacho económico de centrales de generación térmica es uno de los problemas más clásicos de optimización en ingeniería eléctrica (Soroudi, 2017). Este problema consiste en determinar el nivel de generación de energía de cada central de generación térmica para satisfacer la demanda de energía eléctrica para un período determinado. En la Figura 2.1 se presenta el esquema general de un sistema eléctrico de potencia, compuesto por seis nodos, i.e., $b_1, b_2, \dots, b_6$, tres generadores localizados en los nodos b_1, b_2 y b_3 , i.e., p_1^g, p_2^g y p_3^g , cuatro consumos de potencia constante denotados como P_3^d, P_4^d, P_5^d y P_6^d ; además de siete líneas de transmisión que denotan la interconexión eléctrica entre nodos.

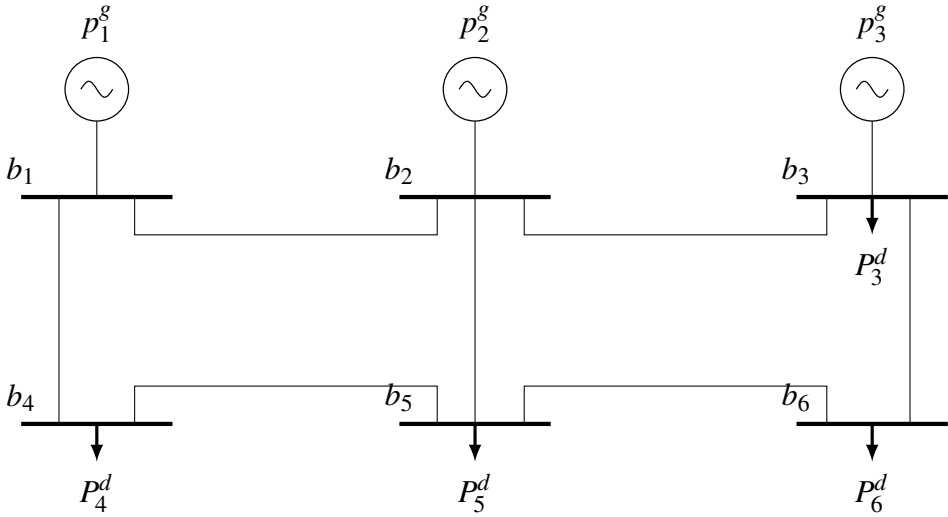


Figura 2.1: Representación de un sistema eléctrico de potencia

En la literatura especializada, el problema de despacho económico de plantas térmicas en sistemas eléctricos puede ser abordado empleando la formulación ideal (i.e., sin pérdidas) (Soroudi, 2017) y la formulación

teniendo en cuenta un modelo aproximado de pérdidas de energía en el sistema de transmisión, también conocido como modelo de nodo único con pérdidas (Montoya, 2017; Abdelaziz et al., 2016). A continuación se presentan los principales detalles de cada modelo de optimización.

2.2.1 Modelo matemático clásico

La formulación del modelo clásico asume que dado un sistema eléctrico de potencia predominantemente térmico, el costo de operación para un período determinado es una función no lineal de los costos de producción de cada central térmica (i.e., plantas de carbón, gas natural, nucleares o diésel). Además, se asume que la red de transmisión es ideal (pérdidas eléctricas nulas), y que por tanto, la energía entregada a las cargas puede fluir entre las centrales térmicas y las cargas sin ninguna limitación. Aunque este es un modelo ideal, es empleado como referencia para saber el costo de referencia mínimo de operación de un sistema eléctrico, y cuantificar la operación real, el efecto de la red de transmisión durante la operación real del sistema (Abdelaziz et al., 2016).

A continuación se presenta matemáticamente el problema de despacho económico ideal para sistemas eléctricos con generación predominantemente térmica. La función objetivo, corresponde a la minimización de los costos totales de producción de energía durante el período de análisis, cuya estructura cuadrática se presenta en la Ecuación (2.5).

$$\min E_c = \sum_{k \in \mathbf{G}} \sum_{h \in \mathbf{H}} \left(a_k \left(p_{k,h}^g \right)^2 + b_k p_{k,h}^g + c_k \right) \Delta_h, \quad (2.5)$$

donde E_c representa el costo total de operación del sistema, $p_{k,h}^g$ es el nivel de generación de potencia en la k -ésima planta térmica para el período h , a_k y b_k representan los coeficientes de costo variable de la k -ésima planta de generación térmica y c_k modela sus costos fijos de operación; además, Δ_h corresponde a la longitud del período de análisis,

típicamente con valores de una hora o fracciones de ella, $\mathbf{G}$ corresponde al conjunto de generadores analizados y $\mathbf{H}$ al conjunto de períodos.

El objetivo fundamental del problema de despacho económico de plantas térmicas es que se garantice el equilibrio de potencia entre generación y consumo, teniendo en cuenta los límites operativos de cada planta. Lo cual se formula en (2.6) y (2.7).

$$\sum_{k \in \mathbf{G}} p_{k,h}^g \geq \sum_{j \in \mathbf{D}} P_{j,h}^d, \quad \{\forall h \in \mathbf{H}\}, \quad (2.6)$$

$$P_k^{g,\min} \leq p_{k,h}^g \leq P_k^{g,\max} \quad \{\forall k \in \mathbf{G}, \forall h \in \mathbf{H}\}, \quad (2.7)$$

donde $P_{m,h}^d$ representa la potencia consumida por la j -ésima demanda, $P_k^{g,\min}$ y $P_k^{g,\max}$ representan las potencias mínimas y máximas asociadas a la k -ésima planta térmica, y $\mathbf{D}$ corresponde al conjunto de demandas.

Es importante mencionar que para un problema de despacho económico de centrales de generación térmica como el formulado de (2.5) a (2.7), si el conjunto $\mathbf{H}$ tiene cardinalidad H y el conjunto $\mathbf{G}$ tiene cardinalidad G , entonces el número de variables del problema será $N_v = H \cdot G$. Por ejemplo, para un sistema como el de la Figura 2.1 con tres plantas térmicas, si se considera un día de operación, el problema de despacho económico tendrá un total de 72 variables de decisión.

2.2.2 Modelo considerando pérdidas de potencia

Cuando son consideradas las pérdidas en un sistema de potencia para el caso de despacho económico de plantas térmicas, se suele emplear un modelo aproximado que relaciona las pérdidas de potencia (i.e., $P_{L,h}$) para cada período como una función cuadrática de las potencia generadas, esto puede ser modelado fácilmente como se define en (2.8) (Urazel and Keskin, 2023).

$$P_{L,h} = \sum_{k \in \mathbf{G}} \sum_{m \in \mathbf{G}} B_{km} p_{k,h}^g p_{m,h}^g, \quad \{h \in \mathbf{H}\} \quad (2.8)$$

donde B_{km} es el coeficiente de pérdidas cuadrático que relaciona las plantas térmicas k y m (Abdelaziz et al., 2016).

Teniendo en cuenta el modelo de las pérdidas, el problema de despacho económico de plantas de generación térmica multi-período se puede formular según se define de (2.9) a (2.11).

$$\text{Fun. Obj.} \quad \min E_c = \sum_{k \in \mathbf{G}} \sum_{h \in \mathbf{H}} \left(a_k \left(p_{k,h}^g \right)^2 + b_k p_{k,h}^g + c_k \right) \Delta_h, \quad (2.9)$$

$$\text{s.a.:} \quad \sum_{k \in \mathbf{G}} p_{k,h}^g \geq \sum_{j \in \mathbf{D}} P_{j,h}^d + \sum_{k \in \mathbf{G}} \sum_{m \in \mathbf{G}} B_{km} p_{k,h}^g p_{m,h}^g, \quad \{\forall h \in \mathbf{H}\}, \quad (2.10)$$

$$P_k^{g,\min} \leq p_{k,h}^g \leq P_k^{g,\max} \quad \{\forall k \in \mathbf{G}, \forall h \in \mathbf{H}\}. \quad (2.11)$$

2.2.3 Implementación en el software Julia

Para la implementación en el software Julia, se considera un sistema compuesto por tres centrales de generación térmica (Soroudi, 2017) y período de análisis de veinticuatro horas (Montoya, 2017). Los datos asociados a los coeficientes de cada una de las plantas térmicas, así como la variación de demanda se presentan en las Tablas 2.1 y 2.2.

Tabla 2.1: Coeficientes de la función objetivo y límites de generación para las plantas térmicas

Planta	a (USD/MWh ²)	b (USD/MWh)	c (USD)	$P_k^{g,\min}$ (MW)	$P_k^{g,\max}$ (MW)
G_1	0.006085	10.04025	136.9125	5	150
G_2	0.005915	9.760576	59.15500	15	100
G_3	0.005250	8.662500	328.1250	50	250

Tabla 2.2: Perfil diario de demanda (donde P^D es la demanda total acumulada)

Per. h (h)	P^D (MW)	Per. h (h)	P^D (MW)	Per. h (h)	P^D (MW)	Per. h (h)	P^D (MW)
1	210	7	150	13	310	19	450
2	230	8	100	14	320	20	460
3	200	9	80	15	350	21	470
4	180	10	130	16	380	22	420
5	240	11	190	17	400	23	350
6	180	12	280	18	420	24	220

Con el objetivo de incluir el efecto de las pérdidas en el modelo matemático, como se definió en (2.8), a continuación se presentan los coeficientes de la matriz B , i.e.,

$$B = \begin{bmatrix} 0.00013630 & 0.00006750 & 0.00007839 \\ 0.00006750 & 0.00015450 & 0.00009828 \\ 0.00007839 & 0.00009828 & 0.00016140 \end{bmatrix} \text{ (1/MW)}$$

A continuación se ilustra la implementación del modelo de despacho económico con pérdidas definido de (2.9) a (2.11) en el software Julia.

En primer lugar, se ilustra la implementación de los datos del problema empleando la herramienta DataFrames. Nótese que la matriz de coeficientes ha sido denominada B, los datos de las plantas de generación dados en la Tabla 2.1 se han denominado Generadores y los datos de demanda en la Tabla 2.2 se han definido como Periodos.

```
using JuMP, Ipopt, DataFrames
B = [0.00013630  0.00006750  0.00007839
      0.00006750  0.00015450  0.00009828
      0.00007839  0.00009828  0.00016140];
Generadores = DataFrames.DataFrame([
(1, 0.006085, 10.04025, 136.9125, 5, 150),
(2, 0.005915, 9.760576, 59.15500, 15, 100),
(3, 0.005250, 8.662500, 328.1250, 50, 250),])
DataFrames.rename!(Generadores, [:g,:a,:b,:c,:pmin,:pmax]);
a = Generadores.a; b = Generadores.b; c = Generadores.c;
pmin = Generadores.pmin; pmax = Generadores.pmax;
Periodos = DataFrames.DataFrame([
(1, 210), (7, 150), (13, 310), (19, 450),
```

```
(2, 230), (8, 100), (14, 320), (20, 460),  
(3, 200), (9, 80 ), (15, 350), (21, 470),  
(4, 180), (10, 130), (16, 380), (22, 420),  
(5, 240), (11, 190), (17, 400), (23, 350),  
(6, 180), (12, 280), (18, 420), (24, 220),])  
DataFrames.rename!(Periodos, [:h, :PD]);  
PD = Periodos.PD; t = size(PD,1);  
g = size(Generadores,1); H = 1:t; G = 1:g;
```

Nótese que las operaciones $H = 1:t$ y $G = 1:g$ definen las dimensiones de los conjuntos asociados a los períodos y plantas térmicas, respectivamente.

En segundo lugar, se realiza la implementación del modelo matemático, como se ilustra a continuación.

```
alpha = 1;  
DE = Model(Ipopt.Optimizer)  
@variable(DE, pmin[k] <= Pg[k] in G, h in H) <= pmax[k])  
@NLobjective(DE, Min, sum(sum(a[k]*Pg[k,h]^2 +  
    b[k]*Pg[k,h] + c[k] for k in G) for h in H))  
for h in H  
@NLconstraint(DE, sum(Pg[k,h] for k in G) == PD[h] +  
    alpha*sum(sum(Pg[k,h]*B[k,m]*Pg[m,h] for m in G)  
    for k in G));  
end  
JuMP.optimize!(DE)  
@show objective_value(DE)  
using Printf  
for k in G  
    @printf("Gen. %d \t", k)  
end  
@printf("\n")  
for h in H  
    for k in G  
        @printf("%0.5f \t", value.(Pg[k,h]))  
    end  
    @printf("\n")  
end
```

En esta implementación, es importante notar que:

- i. El parámetro `alpha = 1` es empleado para evaluar el problema de despacho económico considerando pérdidas de potencia, es decir, el modelo (2.9)–(2.11). Sin embargo, en el caso de querer el modelo de optimización sin pérdidas, solo bastará con hacer `alpha = 0`.
- ii. El modelo matemático es construido a través del objeto `DE = Model(Ipopt.Optimizer)`, en el cual se observa que se ha seleccionado el solucionador `Ipopt` disponible para el paquete de optimización `Jump`, el cual corresponde a una herramienta de optimización para problemas de programación no lineal.
- iii. El comando `@variable(DE, pmin[k] <= Pg[k in G, h in H] <= pmax[k])` permite definir una variable de tipo matricial, al tiempo que se pueden asignar sus límites inferiores y superiores, siempre que estos sean constantes.
- iv. La palabra reservada `@NLobjective(DE, Min, --)` define la naturaleza de la función objetivo, que para el caso del despacho económico de centrales térmicas, es de tipo no lineal. Además, la dirección de optimización en este caso corresponde a minimización.
- v. El comando `@NLconstraint(DE, --)` le asigna al objeto `DE` (es decir, al modelo matemático), el conjunto de restricciones asociadas con la generación, la demanda y las pérdidas para cada período. Estas se implementan por medio de un ciclo `for` sobre el conjunto `H`

Una vez implementado el modelo matemático del despacho económico, este es solucionado con el comando `Jump.optimize!(DE)`, se imprime el valor de función objetivo a través del comando `@show objective_value(DE)`. Finalmente, el resto del código emplea el

MÉTODOS DE OPTIMIZACIÓN METAHEURÍSTICA PARA INGENIERÍA: Con
aplicaciones en el software Julia

entorno `Printf` para imprimir los resultados de generación de cada planta térmica. Estos resultados se ilustran a continuación

```
EXIT: Optimal Solution Found.
objective_value(DE) = 82269.96375969377
Gen. 1      Gen. 2      Gen. 3
29.17829    45.04027    141.52118
8.07154     24.91132    120.22265
65.20667    79.20309    177.49918
130.71395   100.00000   244.37465
36.29713    51.81026    148.66810
5.00000     15.00000    81.42220
68.87067    82.66329    181.13095
136.25712   100.00000   250.00000
25.63468    41.66672    137.95668
5.00000     15.00000    60.87358
79.93195    93.09340    192.06420
147.26231   100.00000   250.00000
18.57869    34.94223    130.84546
5.00000     16.40787    111.08522
92.75667    100.00000   205.00750
114.33776   100.00000   227.42801
39.87253    55.20681    152.25059
22.10151    38.30072    134.39812
103.51150   100.00000   216.19304
79.93195    93.09340    192.06420
18.57869    34.94223    130.84546
54.28266    68.87117    166.64136
114.33776   100.00000   227.42801
32.73241    48.42144    145.09165
```

Nótese que el valor de función objetivo `objective_value(DE)` = 82269.96375969377 corresponde al costo diario en dólares asociado a la producción de energía cuando se incluyen las pérdidas. Se deja al lector la evaluación del modelo clásico de despacho económico sin pérdidas.

2.3 Un problema de programación no lineal entera mixta

En el campo de la optimización y sus aplicaciones en ingeniería, es común encontrar problemas de naturaleza no lineal que involucran variables de dominio continuo, con variables de naturaleza discreta, lo que da origen a los modelos matemáticos de programación no lineal entera mixta (Grisales-Noreña et al., 2022). Para ilustrar esta clase de problemas, considérese el problema de optimización que se define de (2.12) a (2.15).

$$\text{Fun. Obj.} \quad \min z = \sum_{k \in \mathbf{N}} (a_k \sin(b_k x_k) + c_k x_k^2) y_k, \quad (2.12)$$

$$\text{s.a.:} \quad \sum_{k \in \mathbf{N}} y_k \leq \frac{N}{2}, \quad (2.13)$$

$$x_k^{\min} \leq x_k \leq x_k^{\max} \quad \{\forall k \in \mathbf{N}\}, \quad (2.14)$$

$$y_k \in \{0, 1\} \quad \{\forall k \in \mathbf{N}\}. \quad (2.15)$$

donde y_k es una variable binaria, x_k es una variable de dominio continuo, z es la función objetivo, a_k , b_k y c_k son constantes positivas y N corresponde a la cardinalidad del conjunto $\mathbf{N}$.

Nótese que el modelo de optimización (2.12)–(2.15) pertenece a la familia de modelos de programación no lineal entera mixta debido a que:

- i. Existe la combinación entre variables binarias y continuas.
- ii. Existe producto de variables, y también hace presencia funciones cuadráticas y sinusoidales.

Además, para un conjunto de N elementos el número de variables de este modelo matemático será $2N$.

2.3.1 Complejidad del problema de optimización

Con el objetivo de ilustrar la complejidad de este problema de optimización, considérese $N = 4$, tal que los valores de los vectores a , b y c se presentan a continuación.

$$a = \begin{bmatrix} 4 \\ 3 \\ 10 \\ 1 \end{bmatrix}, b = \begin{bmatrix} 8 \\ 6 \\ 4 \\ 10 \end{bmatrix}, c = \begin{bmatrix} 0.66 \\ 0.50 \\ 0.87 \\ 0.74 \end{bmatrix}, \quad (2.16)$$

Además, se asume que los valores de los límites inferiores y superiores para las variable continua son: $x_k^{\min} = [-2.5, -1.5, -3, -5]$ y $x_k^{\max} = [2.5, 1.5, 5, 4]$ están definidos como ± 5 . Ahora, teniendo en cuenta que la solución óptima de este problema se presenta cuando la restricción (2.13) está activa, es decir, cuando se toma la igualdad, y dado que $N = 4$, entonces, la solución óptima se encuentra cuando dos de las cuatro variables binarias se encuentra activas, lo que da origen a seis posibles casos. A continuación se analiza el caso en que las variables y_1 y y_2 están activas, es decir, toman el valor de 1, mientras que las variables y_3 y y_4 permanecen inactivas, es decir, toman el valor de 0. Bajo este este escenario, y teniendo en cuenta los parámetros (2.16), el modelo matemático (2.12)–(2.15) se reduce como sigue:

$$\begin{aligned} \text{Fun. Obj.} \quad \min z &= 4 \sin(8x_1) + 0.66x_1^2 + 3 \sin(6x_2) + 0.50x_2^2 \\ \text{s.a.:} \quad &-2.5 \leq x_1 \leq 2.5 \\ &-1.5 \leq x_2 \leq 1.5 \end{aligned}$$

En la Figura 2.2, se ilustra el comportamiento no lineal de la función z en el dominio de las variables de decisión x_1 y x_2 .

La principal característica de esta función, es su característica no lineal, donde existen múltiples puntos mínimos, lo cual se plantea como un reto en el problema de optimización, ya que la algoritmo de optimización que se escoja para su solución debe estar en la capacidad de escapar a óptimos locales y llegar al óptimo global.

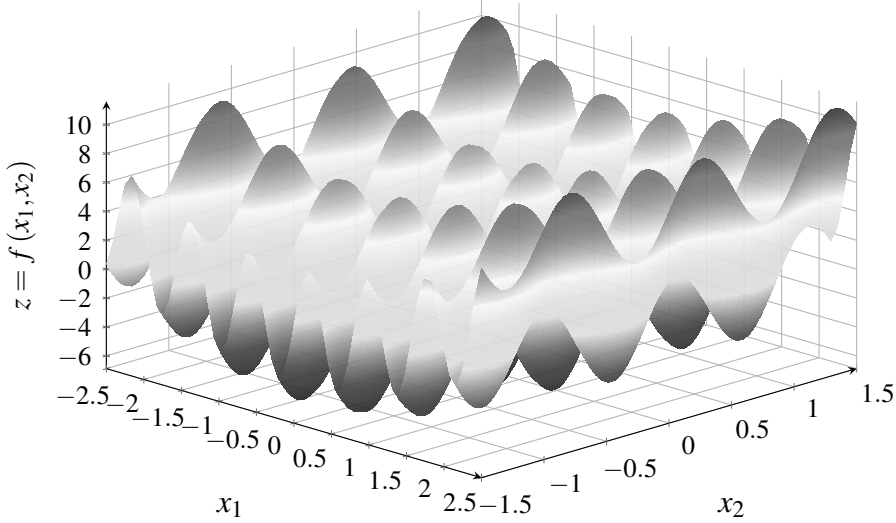


Figura 2.2: Proyección del problema de optimización al espacio tridimensional de la función $z = 4 \sin(8x_1) + 0.66x_1^2 + 3 \sin(6x_2) + 0.50x_2^2$.

Además, es importante tener en cuenta, que existen seis funciones como esta, una para cada combinación de variables (considerando que el óptimo se encuentra para cuando dos variables binarias se activan), con lo cual se debe explorar en el espacio donde estas funciones están definidas, con el fin de encontrar la mejor solución óptima posible para este problema.

2.3.2 Implementación en el software Julia

Con el fin de ilustrar la implementación computacional del problema del problema de de optimización no lineal entero mixto presentado definido de (2.12) a (2.15), considérese la información paramétrica para los vectores a , b y c , y los valores mínimos y máximos que se aplican a las variables de decisión, que se listan en la Tabla 2.3. Nótese que en este caso la dimensión del problema es $N = 10$.

Tabla 2.3: Información paramétrica para el modelo de optimización no lineal entero mixto

k	a_k	b_k	c_k	$x_k^{\min}$	$x_k^{\max}$
1	5	10	0.11	-6	5
2	1	9	0.50	-6	8
3	9	10	0.61	-13	10
4	1	4	0.17	-7	12
5	7	1	0.44	-15	5
6	2	7	0.56	-9	9
7	2	1	0.45	-11	11
8	8	4	0.60	-5	5
9	3	5	0.79	-14	14
10	1	9	0.51	-8	14

Para implementar en el software Julia, el modelo de programación no lineal entero mixto definido por (2.12)–(2.15), en primer lugar se definen los datos del sistema y el número de variables de decisión, tal como se muestra a continuación.

```
using JuMP, Ipopt, Juniper, DataFrames
Parametros = DataFrames.DataFrame([
(1, 5, 10, 0.11, -6, 5), (2, 1, 9, 0.50, -6, 8),
(3, 9, 10, 0.61, -13, 10), (4, 1, 4, 0.17, -7, 12),
(5, 7, 1, 0.44, -15, 5), (6, 2, 7, 0.56, -9, 9),
(7, 2, 1, 0.45, -11, 11), (8, 8, 4, 0.60, -5, 5),
(9, 3, 5, 0.79, -14, 14), (10, 1, 9, 0.51,
-8, 14),]);
DataFrames.rename!(Parametros, [:k, :a, :b, :c, :xmin, :xmax]);
a = Parametros.a; b = Parametros.b; c = Parametros.c;
xmin = Parametros.xmin; xmax = Parametros.xmax;
n = size(Parametros,1); N = 1:n;
```

En segundo lugar se construye el modelo matemático, tal como se ilustra a continuación.

```
optimizer = Juniper.Optimizer
nl_solver = optimizer_with_attributes(Ipopt.Optimizer)
MINLP = Model(optimizer_with_attributes(optimizer,
    "nl_solver"=>nl_solver))
@variable(MINLP, xmin[k]<=x[k in N]<= xmax[k])
@variable(MINLP, y[k in N], Bin)
@objective(MINLP, Min, sum(y[k]*(a[k]*sin(b[k]*x[k])
    + c[k]*x[k]^2) for k in N))
@constraint(MINLP, sum(y[k] for k in N) <= n/2)
JuMP.optimize!(MINLP)
using Printf
@printf("Binaria\tContinua\n")
for k in N
    @printf("%0.0f\t%0.5f \n",
        abs.(value.(y[k])), value.(x[k]))
end
```

En esta implementación, se observa que:

- i. Se ha empleado el solucionador Juniper en conjunto con el solucionador Ipopt para abordar el problema de optimización en el entorno JuMP. Esto se realiza, debido a que el modelo bajo estudio es de naturaleza no lineal entera mixta, con lo cual el modelo se descompone de manera interna en dos problemas, siendo el primero de naturaleza discreta y el segundo no lineal.
- ii. El comando Bin dentro de la palabra reservada @variable(MINLP, y[k in N], Bin) indica que a la variable y definida en el conjunto N se le asigna la condición de variable binaria.

Finalmente, el comando JuMP.optimize!(MINLP) permite resolver el modelo matemático, y a través del entorno Printf se imprimen los resultados en pantalla, como se observa a continuación.

```
EXIT: Optimal Solution Found.  
Status of relaxation: LOCALLY_SOLVED  
Relaxation Obj: -30.850090779384114  
Obj: -30.850090779384114  
Binaria Continua  
1      -0.15701  
0      0.17683  
1      -0.15687  
0      0.40208  
1      -1.39457  
0      -0.22186  
0      10.97443  
1      -0.38905  
1      -0.30768  
0      0.17693
```

de donde se observa que la función objetivo toma el valor de $\text{Obj}:-30.850090779384114$. Sin embargo, es importante mencionar que el solucionador define esta solución, como un óptimo local, esto debido a que por la naturaleza no lineal y no convexa de este problema de optimización, no es posible garantizar el hallazgo del óptimo global.

2.4 Algunos datos sobre Julia

El software Julia es una herramienta de programación de alto nivel con la capacidad para combinar la programación secuencial, con diferentes herramientas de optimización para problemas de ingeniería y ciencia, los cuales abarcan desde programación lineal hasta programación no lineal entera mixta (Montoya et al., 2024b). Este software fue desarrollado e impulsado desde el año 2012 y en la actualidad es considerado una herramienta de desarrollo muy potente y versátil para resolver problemas complejos en ingeniería y ciencia (Bezanson et al., 2017).

En este libro, la versión de Julia empleada corresponde a Julia 1.9.2 sobre el entorno de programación Visual Studio Code 1.85.1 (Microsoft, 2015). Nótese que en la instalación de paquetes, se requieren los siguien-

MÉTODOS DE OPTIMIZACIÓN METAHEURÍSTICA PARA INGENIERÍA: Con aplicaciones en el software Julia

tes comandos en la ventaja de comandos del software: `import Pkg`, luego se adicional el siguiente comando `Pkg.add("paquete")`, donde la palabra “paquete” será reemplazada por el entorno de interés. Así por ejemplo, la instalación de algunas librerías y paquetes de optimización se realiza, tal como se ilustra a continuación.

```
import Pkg
Pkg.add("DataFrames")      # Manejo matricial de datos.
Pkg.add("JuMP")            # Entorno para optimizar.
Pkg.add("Juniper")         # Sol. para modelos MINLP
Pkg.add("Ipopt")           # Sol. para modelos no lineales
Pkg.add("HiGHS")           # Sol. para problemas lineales
Pkg.add("Cbc")             # Sol. para problemas lineales
Pkg.add("Printf")          # Visualizacion en pantalla
Pkg.add("LinearAlgebra")   # Operaciones matriciales
Pkg.add("Random")          # Generacion de datos aleatorios
Pkg.add("Statistics")      # Analisis estadistico
Pkg.add("BenchmarkTools")  # Calculo del tiempo
```

Para mayores detalles sobre el entorno de programación Julia y todas sus herramientas, se recomienda al lector visitar las siguientes páginas web: <https://julialang.org/> y <https://jump.dev/JuMP.jl/stable/>.

3

Optimización metaheurística

Este capítulo presenta una breve revisión sobre los métodos y las técnicas de optimización metaheurística, en el cual se revisa una posible clasificación de las metodologías existentes en la literatura especializada, así como de la lógica algorítmica de estos métodos. Además, se presenta la formulación general de un problema de optimización a través del método de penalizaciones, en lo que se conoce como funciones de desempeño o adaptación, incluyendo las codificaciones propuestas para los problemas de la mochila, el despacho económico de centrales térmicas y el problema de optimización no lineal entero mixto estudiados a lo largo del libro.

3.1 Posible clasificación

El área de optimización en ingeniería y ciencia evoluciona debido a la necesidad humana de resolver problemas de la vida real de manera eficiente, lógica y estructurada (Abualigah et al., 2022). El modelo de optimización que representa dichos problemas de la vida real puede tener diferentes formas: desde programación lineal hasta modelos de programación no lineal de enteros mixtos (Sioshansi and Conejo, 2017), como se vio en el Capítulo 1. Además, los modelos matemáticos en optimización pueden tener una única función objetivo o múltiples objetivos en conflicto. Debido a la complejidad de algunos problemas de optimización en ingeniería y ciencia, los métodos de optimización exactos son ineficientes por varias razones (Montoya et al., 2022b):

- i. La dimensión del espacio de soluciones y el número de variables.
- ii. La estructura geométrica del problema, es decir, las no linealidades y no convexidades de la función objetivo y/o el conjunto de restricciones.
- iii. La imposibilidad de asegurar el hallazgo de la solución óptima global o una solución cercana a esta, debido a la convergencia hacia óptimos locales.

Además, para la mayoría de estos problemas de optimización, los tiempos de procesamiento aumentan con formas no polinómicas, razón por la cual la mayoría de los problemas de optimización pertenecen a la familia de problemas no polinomiales completos (*NP-hard*, por sus siglas en inglés). Estas complicaciones hacen necesario emplear métodos de solución alternativos para abordar múltiples modelos de programación, con el objetivo de encontrar soluciones adecuadas (óptimos locales) que requieran un bajo esfuerzo computacional y sean implementables en cualquier entorno de programación (Abdel-Basset et al., 2018).

Estos algoritmos de optimización se conocen como métodos de optimización metaheurísticos o combinatorios (se prefiere el término metaheurístico, ya que el término combinatorio hace referencia particularmente a una rama de la optimización aplicada a problemas con variables discretas). La principal característica de estos algoritmos es que inician la exploración del espacio de soluciones a partir de una solución inicial (una única solución o un conjunto de soluciones), la cual avanza durante el proceso de iteración utilizando diferentes reglas de evolución. Los algoritmos que utilizan una única solución se conocen como algoritmos de optimización basados en trayectorias (Venkateswarlu, 2021), mientras que los métodos que funcionan con un grupo de soluciones se conocen como optimizadores basados en poblaciones (Agushaka and Ezugwu, 2022).

Las metodologías de optimización metaheurística son ampliamente aceptadas en ciencia e ingeniería, ya que trabajan directamente con la formulación exacta del problema mediante el uso de factores de penalización para explorar y explotar el espacio de soluciones, con la principal ventaja de que se pueden utilizar soluciones no factibles durante el proceso de evolución para encontrar regiones del espacio de solución con excelentes valores de función objetivo (Dahal et al., 2008). Dependiendo de la filosofía que inspira a los optimizadores metaheurísticos, estos se pueden agrupar en muchas familias diferentes. Aquí presentamos una posible clasificación de los métodos de optimización combinatoria más comunes aplicados en ciencia e ingeniería.

3.1.1 Técnicas de optimización bio-inspiradas

Estos algoritmos están inspirados en los procesos biológicos y comportamientos de los seres vivos en la naturaleza. Los algoritmos genéticos, inspirados en la teoría de la evolución de Darwin, son los ejemplos más clásicos de su tipo. Trabajan con una población inicial de padres que evoluciona durante el proceso de iteración aplicando tres conceptos principales (Montoya et al., 2022b): (i) la selección, (ii) la recombinación, y

(iii) la mutación. Este proceso produce un conjunto de descendientes que serán parte de la población si mejoran los peores valores de la función objetivo de algunos padres y son diferentes de ellos. En la literatura actual, la versión de algoritmos genéticos más implementada corresponde al algoritmo de Chu & Beasley dada su eficiencia computacional en términos de tiempos de procesamiento, ya que solo reemplaza un individuo de la población en cada iteración (Katoch et al., 2020).

Los algoritmos bio-inspirados también pueden basarse en el comportamiento de grupos de seres vivos en busca de alimento. Algunas de las metodologías más reconocidas, por dar algunos ejemplos, son: (i) la optimización por enjambre de partículas, cuya inspiración puede atribuirse a manadas de aves o peces (Shami et al., 2022), (ii) el algoritmo de búsqueda por cuervos (Askarzadeh, 2016a), (iii) el algoritmo de enjambre de salpas (Mirjalili et al., 2017a), (iv) el algoritmo de optimización de las ballenas (Mirjalili and Lewis, 2016) y (v) los algoritmos basados en colonias, es decir, colonia de abejas y colonia de hormigas (BasheerJasser et al., 2014)

Una revisión completa de las técnicas de optimización bio-inspiradas podrá ser encontrada en la propuesta presentada por los autores de (Jakšić et al., 2023)

3.1.2 Técnicas de optimización inspiradas en matemáticas

Estos algoritmos de optimización corresponden a enfoques que se basan en funciones no lineales, métodos numéricos, comportamientos estadísticos o flujo de permutación distribuido (Montoya et al., 2022b). La principal característica de estos algoritmos, es que explotan las propiedades de algunas funciones y distribuciones estadísticas bien conocidas para modelar diferentes comportamientos presentes en la naturaleza. Las técnicas más comunes en esta línea de inspiración son: (i) el algoritmo seno-coseno (Mirjalili, 2016), (ii) el optimizador metaheurístico basado

en gradiente (Ahmadianfar et al., 2020), (iii) el algoritmo metaheurístico de Newton (Gholizadeh et al., 2020), (iv) el algoritmo de distribución normal generalizada (Zhang et al., 2020) y (v) la optimización de Monte Carlo (Dickman and Gilman, 1989), entre otros.

3.1.3 Técnicas de optimización inspiradas en física

Esta familia de algoritmos se basa en los comportamientos observados en la naturaleza que no están relacionados con procesos biológicos. En general, se basan en la observación y experimentación física (Biswas et al., 2013). Algunos de los algoritmos de optimización inspirados en la física más reconocidos y utilizados son (i) el algoritmo de búsqueda gravitacional (Mittal et al., 2020), (ii) el optimizador de agujeros negros (Deeb et al., 2022), (iii) el optimizador de supernovas (Hudaib and Fakhouri, 2017), (iv) el algoritmo de búsqueda de vórtices (Doğan and Ölmez, 2015a), (v) el algoritmo de búsqueda galáctica (Muthiah-Nakarajan and Noel, 2016), (vi) el algoritmo de optimización del multiverso (Mirjalili et al., 2015) y (vii) el algoritmo de recocido simulado (Tovey, 1988).

3.1.4 Algoritmos inspirados en comportamientos sociales

Los algoritmos de inspiración social son metodologías de optimización de búsqueda que emulan las interacciones humanas en diferentes entornos. Estos algoritmos explotan el comportamiento de los humanos en grupos para resolver problemas de la vida real y aprender nuevas habilidades (Montoya et al., 2022b). Algunos de los algoritmos de inspiración social más comunes son: (i) el optimizador de personas nómadas o migrantes (Lin et al., 2022), (ii) la optimización basada en la enseñanza-aprendizaje (Rao et al., 2011), (iii) el algoritmo de optimización del aprendizaje y la socio-evolución (Kumar et al., 2018), (iv) la optimización de la memoria humana (Zhu et al., 2024), (v) la búsqueda mental (Mousavirad and Ebrahimpour-Komleh, 2017), y (vi) el algoritmo de evolución cultural (Awad et al., 2013).

Observación 3.1. En general, el uso de algoritmos de optimización metaheurística en ciencia e ingeniería ofrece cientos de posibilidades respecto a algoritmos de diferentes familias (Montoya et al., 2022b). Además, su investigación también está en desarrollo continuo, i.e., varios artículos y mucha literatura cada año proporcionan múltiples optimizadores nuevos que intentan emular situaciones y procesos observados en la naturaleza para alcanzar soluciones óptimas para problemas de ingeniería y de la vida real.

3.2 Estrategias basadas en población y en trayectorias

Los algoritmos de optimización metaheurística pueden clasificarse en dos grandes familias, la primera de ellas, se conoce como los algoritmos de optimización basados en población (Agushaka and Ezugwu, 2022), mientras que la segunda gran familia corresponde a los métodos de búsqueda basados en trayectorias (Abu Khurma et al., 2022).

3.2.1 Optimización basada en poblaciones

Los algoritmos de optimización basados en poblaciones, son todas aquellas técnicas de búsqueda que evolucionan a partir de un conjunto inicial de soluciones, llamado población inicial o que también se conoce como el conjunto de individuos o padres (dependiendo de la técnica estudiada); sin embargo, su principal característica es que estos individuos son transformados en cada iteración por medio de un conjunto de reglas de avance que permiten su evolución, dejando en cada generación (o iteración) los individuos menos aptos y quedándose con aquellos que mejoran la función objetivo de interés. Los algoritmos poblacionales más comunes son el algoritmo genético (Katoch et al., 2020) y la optimización por enjambre de partículas (Shami et al., 2022).

3.2.2 Optimización basada en trayectorias

Las metodologías de optimización basadas en trayectoria exploran y explotan el espacio de soluciones partiendo de una única solución inicial, la cual es cuidadosamente seleccionada, con el fin de tener un buen punto de arranque para la metodología. A partir de esta solución se genera un conjunto de soluciones alrededor de esta en lo que se conoce como generación del vecindario. Este grupo de soluciones es generado tal que se tengan pequeñas variaciones respecto de la solución actual o que se garantice una distribución uniforme alrededor de esta. Los algoritmos más comunes que basados en búsqueda por trayectorias corresponden al algoritmo de búsqueda tabú (Pirim et al., 2008) y al algoritmo de búsqueda de vórtices (Doğan and Ölmez, 2015a).

3.3 Algoritmia y programación

Uno de los principales retos en la implementación de técnicas de optimización metaheurística en la solución de problemas de ingeniería y ciencia corresponde a la interpretación que le da el lector a cada una de las reglas del algoritmo estudiado; además, dado que la lógica de programación siempre es diferente de persona a persona, se tendrá una versión particular de cada algoritmo al final del proceso. Adicionalmente, uno de los detalles clave de la implementación de técnicas de optimización corresponde al ajuste y selección de los parámetros internos de este, los cuales no son genéricos y dependen de cada problema estudiado, razón por la cual, dos versiones de una misma técnica de optimización pueden diferir sustancialmente en la respuesta final y su desempeño estadístico.

Teniendo en cuenta lo anterior, los Algoritmos 1 y 2 describen los principales pasos en la implementación de técnicas de optimización metaheurística, donde t representa el contador de iteraciones y $t_{\max}$ el número de iteraciones máximas.

Data: Datos del problema de optimización bajo análisis

Definir los límites de las variables de decisión;

Construir la función de adaptación o equivalente;

Definir los parámetros del algoritmo y criterios de parada;

Construir la población inicial atendiendo a la factibilidad de las variables;

Evaluar la población inicial en la función adaptación o equivalente;

Seleccionar el mejor individuo de la población;

for $t = 1 : t_{\max}$ **do**

 Aplicar las reglas de avance para construir la población descendiente;

 Verificar la factibilidad de las soluciones generadas atendiendo a los
 límites de las variables;

 Evaluar la población descendiente en la función adaptación o
 equivalente;

 Aplicar las reglas de sustitución de la población;

if *¿Se cumple algún criterio de parada?* **then**

 Terminar el proceso de exploración y explotación del espacio de
 soluciones;

break;

Result: Reportar la mejor solución encontrada

Algoritmo 1: Algoritmos basados en técnicas de exploración poblacional

Observación 3.2. En los pseudocódigos presentados en los Algoritmos 1 y 2 existe una diferencia sutil, pero relevante en la implementación de metodologías de optimización basadas en metaheurísticas, y es que aunque ambas estrategias de exploración y explotación del espacio de soluciones emplean conjuntos de soluciones durante el avance, en el caso de los métodos poblacionales se sustituye regularmente la población inicial por su descendencia si cumple los criterios de reemplazo, mientras que los algoritmos basados en trayectoria, son guiados por una única solución, que se obtiene a partir de un número reducido de vecinos, que por lo general, son generados por estrategias heurísticas especializadas.

Data: Datos del problema de optimización bajo análisis

Definir los límites de las variables de decisión;

Construir la función de adaptación o equivalente;

Definir los parámetros del algoritmo y criterios de parada;

Generar el punto inicial de la búsqueda atendiendo a los límites de las variables;

Evaluar el punto generado en la función adaptación o equivalente;

for $t = 1 : t_{\max}$ **do**

 Generar el vecindario a partir de estrategias heurísticas;

 Verificar la factibilidad de las soluciones generadas atendiendo a los límites de las variables;

 Evaluar el vecindario en la función adaptación o equivalente;

 Seleccionar el nuevo punto de la trayectoria como la mejor solución encontrada en el vecindario, siempre que mejore el punto anterior;

if *¿Se cumple algún criterio de parada?* **then**

 Terminar el proceso de exploración y explotación del espacio de soluciones;

break;

Result: Reportar la mejor solución encontrada

Algoritmo 2: Algoritmos basados en técnicas de exploración por trayectorias

Desde el punto de vista computacional, la implementación de los Algoritmos 1 y 2 dependerá fundamentalmente del conocimiento que se tenga del problema a resolver, ya que en el caso de los métodos basados en trayectoria, el éxito de la metodología, dependerá en gran medida de la capacidad de generación de estrategias heurísticas para construir el vecindario, mientras que en los métodos basados en población, esto suele hacerse de manera estructurada con reglas de avance previamente establecidas.

3.4 Función adaptación y codificación

Uno de los principales retos en la asertiva aplicación de técnicas de optimización metaheurística corresponde a la transformación del modelo

matemático de optimización empleando el método de penalizaciones, lo cual permite convertir un problema de optimización restringido, en un problema irrestringido equivalente (Cortés-Caicedo et al., 2024). Adicionalmente, se requiere de una selección adecuada de la codificación (vector que representa la solución), tal que permita la aplicación de las etapas de exploración del espacio de soluciones de manera estructurada.

3.4.1 Construcción de la función de adaptación

Para ilustrar de forma genérica la construcción de la función de adaptación, considere el problema de optimización con restricciones de desigualdad que se presenta en (3.1)

Función objetivo

$$\min / \max f(\mathbf{x})$$

Restricciones

$$\begin{aligned} h_k(\mathbf{x}) &\leq 0, \{k = 1, 2, \dots, H\} \\ \mathbf{x}^{\min} &\leq \mathbf{x} \leq \mathbf{x}^{\max}, \end{aligned} \quad (3.1)$$

donde $f(\mathbf{x})$ representa la función objetivo que puede ser de minimización o maximización, la cual depende de las variables de decisión contenidas en el vector $\mathbf{x}$, $h_k(\mathbf{x})$ representa la k -ésima restricción de desigualdad y $\mathbf{x}^{\min}$ y $\mathbf{x}^{\max}$ los límites máximos y mínimos aplicables a las variables de decisión.

Observación 3.3. La estructura de una función de adaptación no es única y dependerá del sentido de optimización del problema, ya que la función de adaptación lleva las restricciones a la función objetivo mediante penalizaciones que pretenden deteriorar el valor de esta cuando no son satisfechas.

La estructura general de una función de adaptación $f_a(\mathbf{x})$ se presenta en (3.2).

$$f_a(\mathbf{x}) = f(\mathbf{x}) + \sum_{k=1}^H \alpha_k \max \{h_k(\mathbf{x}), 0\} \quad (3.2)$$

siendo α_k el factor de penalización correspondiente a la violación de cada restricción. Nótese que en el caso de problemas de minimización, los valores de α_k serán números reales positivos, mientras que en el caso de problemas de maximización, los valores de α_k estarán definidos para los números reales negativos. Además, la función $\max \{a, b\}$ retornará el valor de a si y sólo si $a \geq b$, en caso contrario reportará el valor de b .

Una estructura equivalente para escribir la función adaptación (3.2) es empleando funciones de tipo mínimo, como sigue:

$$f_a(\mathbf{x}) = f(\mathbf{x}) - \sum_{k=1}^H \alpha_k \min \{-h_k(\mathbf{x}), 0\} \quad (3.3)$$

donde el signo de $h_k(\mathbf{x})$ ha sido cambiado, ya que las funciones mínimo se suele emplean para restricciones de mayor igual que.

Es importante mencionar que en una función de adaptación es posible combinar funciones de máximo y mínimo en su estructura, ya que lo importante es penalizar las restricciones siempre que estas no sean satisfechas lo cual se logra mediante el deterioro de la función objetivo en su sentido de optimización, en otras palabras, si es de maximización las penalizaciones, si las hubiere, deben ser negativas; mientras que en el caso de minimización, estas deberán ser positivas.

3.4.2 Estructura general de una codificación

La codificación, es quizá, uno de los aspectos más relevantes en el diseño de estrategias de optimización metaheurística, ya que es la forma

en la que evolucionará la exploración y la explotación del espacio de soluciones. Sin j -ésimo individuo solución en la iteración t .

$$x_j^t = [x_{1,t}, x_{2,t}, \dots, x_{k,t}, \dots, x_{n,t}] \quad (3.4)$$

siendo n el número de variables, y los valores en $x_{k,t}$ son generados, tal que están contenidos entre $x_k^{\min}$ y $x_k^{\max}$.

3.5 Función de adaptación y codificación en el problema de la mochila

En el problema de optimización de la mochila visto en el Capítulo 2, cuya estructura matemática se presentó en las ecuaciones (2.1)–(2.4), y que por facilidad se escribe nuevamente en el modelo (3.5):

$$\begin{aligned} \text{Fun. Obj.} \quad \max B_n &= \sum_{k \in \mathbf{N}} C_k x_k, \\ \text{s.a.:} \quad \sum_{k \in \mathbf{N}} V_k x_k &\leq V_{\max}, \{ \forall k \in \mathbf{N} \}, \\ \sum_{k \in \mathbf{N}} W_k x_k &\leq W_{\max}, \{ \forall k \in \mathbf{N} \}, \\ x_k &\in \{0, 1\}, \{ \forall k \in \mathbf{N} \}, \end{aligned} \quad (3.5)$$

se propone la siguiente función de adaptación:

$$\begin{aligned} f_a(\mathbf{x}) &= \left(\sum_{k \in \mathbf{N}} C_k x_k \right) - \alpha_1 \max \left\{ \left(\sum_{k \in \mathbf{N}} W_k x_k \right) - W_{\max}, 0 \right\} \\ &\quad - \alpha_2 \max \left\{ \left(\sum_{k \in \mathbf{N}} V_k x_k - V_{\max} \right), 0 \right\} \end{aligned} \quad (3.6)$$

siendo α_1 y α_2 dos constantes reales positivas.

Ahora, teniendo en cuenta que para este problema, las variables de decisión son de naturaleza binaria (ver modelo (3.5)), la codificación

propuesta, se presenta en la Ecuación (3.7).

$$\mathbf{x} = [0, 1, 1, \dots, x_k, \dots, 0, 1, \dots, x_n] \quad (3.7)$$

donde los valores de x_k sólo podrán ser números binarios, es decir, un valor de cero o un valor de uno.

3.6 Función de adaptación y codificación en el problema de despacho óptimo de centrales térmicas

En el problema de optimización relacionado con el despacho óptimo de centrales térmicas presentado en el en el Capítulo 2, cuya estructura matemática se definió en las ecuaciones (2.9)–(2.11), y que por facilidad se escribe nuevamente en el modelo (3.8):

$$\begin{aligned} \text{Fun. Obj.} \quad \min E_c &= \sum_{k \in \mathbf{G}} \sum_{h \in \mathbf{H}} \left(a_k \left(p_{k,h}^g \right)^2 + b_k p_{k,h}^g + c_k \right) \Delta_h, \\ \text{s.a.:} \quad \sum_{k \in \mathbf{G}} p_{k,h}^g &\geq \sum_{j \in \mathbf{D}} P_{j,h}^d + \sum_{k \in \mathbf{G}} \sum_{m \in \mathbf{G}} B_{km} p_{k,h}^g p_{m,h}^g, \quad \{\forall h \in \mathbf{H}\}, \\ P_k^{g,\min} &\leq p_{k,h}^g \leq P_k^{g,\max} \quad \{\forall k \in \mathbf{G}, \forall h \in \mathbf{H}\}. \end{aligned} \quad (3.8)$$

se propone la siguiente función de adaptación, con α_1 una constante real positiva:

$$\begin{aligned} f_a(\mathbf{x}) &= \sum_{k \in \mathbf{G}} \sum_{h \in \mathbf{H}} \left(a_k \left(p_{k,h}^g \right)^2 + b_k p_{k,h}^g + c_k \right) \Delta_h \\ &+ \alpha_1 \max \left\{ \left(\sum_{h \in \mathbf{H}} \left(\sum_{j \in \mathbf{D}} P_{j,h}^d + \sum_{k \in \mathbf{G}} \left(\sum_{m \in \mathbf{G}} B_{km} p_{k,h}^g p_{m,h}^g \right) - p_{k,h}^g \right) \right), 0 \right\} \end{aligned} \quad (3.9)$$

Ahora, teniendo en cuenta que para este problema, las variables de decisión, donde el vector $\mathbf{x}$ estará compuesto de las potencias generadas en cada planta, (ver modelo (3.8)), la codificación propuesta, se presenta en la Ecuación (3.10).

$$\mathbf{x} = [P_{1,h_{\min}}^g, \dots, P_{1,h_{\max}}^g, |P_{2,h_{\min}}^g, \dots, P_{2,h_{\max}}^g, \dots | \dots P_{G,h_{\min}}^g, \dots, P_{G,h_{\max}}^g] \quad (3.10)$$

siendo G en número máximo de generadores, y $h_{\min}$ y $h_{\max}$ los extremos del conjunto de períodos, i.e., $\mathbf{H}$.

3.7 Función de adaptación y codificación en el problema entero mixto

En el problema de optimización de tipo no lineal entero mixto definido en el Capítulo 2, el cual matemáticamente tiene la estructura (2.12)–(2.15), y que por facilidad se escribe nuevamente en el modelo (3.11):

$$\begin{aligned} \text{Fun. Obj.} \quad \min z &= \sum_{k \in \mathbf{N}} (a_k \sin(b_k x_k) + c_k x_k^2) y_k, \\ \text{s.a.:} \quad \sum_{k \in \mathbf{N}} y_k &\leq \frac{N}{2}, \\ x_k^{\min} &\leq x_k \leq x_k^{\max} \quad \{\forall k \in \mathbf{N}\}, \\ y_k &\in \{0, 1\} \quad \{\forall k \in \mathbf{N}\}. \end{aligned} \quad (3.11)$$

se propone la siguiente función de adaptación, con α_1 una constante real positiva:

$$f_a(\mathbf{x}) = \sum_{k \in \mathbf{N}} (a_k \sin(b_k x_k) + c_k x_k^2) y_k + \alpha_1 \max \left\{ \left(\sum_{k \in \mathbf{N}} y_k \right) - \frac{N}{2}, 0 \right\} \quad (3.12)$$

Para este problema, existen varias codificaciones que pueden ser formuladas, sin embargo, en aras de simplicidad, se presenta la estructura discreto-continua dada en (3.13)

$$\mathbf{x} = [0, 1, \dots, y_k, \dots, 0, \dots, y_n \mid x_1, x_2, \dots, x_k, \dots, x_n] \quad (3.13)$$

donde los valores de x_k estarán contenidos en el rango $x_k^{\min}$ y $x_k^{\max}$.

3.8 Selección de algoritmos para el desarrollo de este libro

Este libro tiene como objetivo ser una herramienta de aprendizaje para estudiantes de pregrado y posgrado, presentando de manera didáctica, pero con un enfoque investigativo, un total de nueve técnicas metaheurísticas de optimización. Estas técnicas han sido cuidadosamente seleccionadas para abordar problemas de optimización no lineal de naturaleza continua, discreta y mixta, de manera que puedan ser replicables en diversos problemas de optimización en ingeniería y ciencia.

Los métodos seleccionados se se pueden clasificar en dos grupos, teniendo en cuenta la inspiración que dio origen a estos. En primer lugar, tenemos los algoritmos bio-inspirados (Pop et al., 2022), y en segundo lugar, los métodos basados en funciones matemáticas y distribuciones de probabilidad (R. et al., 2024). A continuación, se listan los métodos metaheurísticos de interés en este libro.

1. Algoritmo de optimización senos y cosenos (Mirjalili, 2016).
2. Algoritmo de aprendizaje incremental basado en poblaciones (Cerberio et al., 2012).
3. Algoritmo genético (Holland, 1975).
4. Algoritmo de optimización de búsqueda de cuervos (Askarzadeh, 2016b).
5. Algoritmo JAYA (Venkata Rao, 2019).
6. Algoritmo de optimización de búsqueda por vórtices (Doğan and Ölmez, 2015a).
7. Algoritmo de optimización de distribución normal generalizada (Zhang et al., 2020).

8. Algoritmo de optimización por cúmulo de partículas (Kennedy and Eberhart, 1995).
9. Algoritmo de optimización por enjambre de salpas (Mirjalili et al., 2017b).

Nótese que la selección de los algoritmos anteriores obedece a la experiencia de los autores en relación con la aplicación de estos métodos de optimización en la solución de problemas en ingeniería eléctrica y afines, los cuales han demostrado eficacia y han logrado publicaciones reconocidas en este campo de estudio.

A continuación se listan las publicaciones de los autores de este libro, que demuestran el conocimiento experto en las temáticas abordadas en este.

1. O. D. Montoya, W. Gil-González, R. I. Bolaños, D. F. Muñoz-Torres, J. C. Hernández and L. F. Grisales-Noreña, "Effective Power Coordination of BESUs in Distribution Grids via the Sine-Cosine Algorithm," 2024 IEEE Green Technologies Conference (GreenTech), Springdale, AR, USA, 2024, pp. 111-115, <https://doi.org/10.1109/GreenTech58819.2024.10520484>
2. Grisales-Noreña, Luis Fernando, Daniel Gonzalez Montoya, and Carlos Andres Ramos-Paja. 2018. "Optimal Sizing and Location of Distributed Generators Based on PBIL and PSO Techniques" *Energies* 11, no. 4: 1018. <https://doi.org/10.3390/en11041018>
3. L.F. Grisales-Noreña, Oscar Danilo Montoya, Carlos Andrés Ramos-Paja, "An energy management system for optimal operation of BSS in DC distributed generation environments based on a parallel PSO algorithm," *Journal of Energy Storage*, Volume 29, 2020, 101488, ISSN 2352-152X, <https://doi.org/10.1016/j.est.2020.101488>

4. Cortés-Caicedo, Brandon, Luis Fernando Grisales-Noreña, Oscar Danilo Montoya, Miguel-Angel Perea-Moreno, and Alberto-Jesus Perea-Moreno. 2022. "Optimal Location and Sizing of PV Generation Units in Electrical Networks to Reduce the Total Annual Operating Costs: An Application of the Crow Search Algorithm" *Mathematics* 10, no. 20: 3774. <https://doi.org/10.3390/math10203774>
5. O. D. Montoya, W. Gil-González and L. F. Grisales-Noreña, "Vortex Search Algorithm for Optimal Power Flow Analysis in DC Resistive Networks With CPLs," in *IEEE Transactions on Circuits and Systems II: Express Briefs*, vol. 67, no. 8, pp. 1439-1443, Aug. 2020, <https://doi.org/10.1109/TCSII.2019.2938530>
6. Oscar Danilo Montoya, Cristian H. De Angelo, Guillermo Bossio, Parametric estimation in three-phase induction motors using torque data via the generalized normal distribution optimizer, *Results in Engineering*, Volume 23, 2024, 102446, ISSN 2590-1230, <https://doi.org/10.1016/j.rineng.2024.102446>
7. Montoya, Oscar Danilo, Alexander Molina-Cabrera, Luis Fernando Grisales-Noreña, Ricardo Alberto Hincapié, and Mauricio Granada. 2021. "Improved Genetic Algorithm for Phase-Balancing in Three-Phase Distribution Networks: A Master-Slave Optimization Approach" *Computation* 9, no. 6: 67. <https://doi.org/10.3390/computation9060067>
8. Cortés-Caicedo, Brandon, Luis Fernando Grisales-Noreña, and Oscar Danilo Montoya. 2022. "Optimal Selection of Conductor Sizes in Three-Phase Asymmetric Distribution Networks Considering Optimal Phase-Balancing: An Application of the Salp Swarm Algorithm" *Mathematics* 10, no. 18: 3327. <https://doi.org/10.3390/math10183327>

Parte I

Técnicas metaheurísticas en dominio discreto

4

Algoritmo de senos y cosenos

En este capítulo se presenta la descripción general del algoritmo de senos y cosenos para problemas de dominio continuo, así como su adaptación mediante redondeo a problemas de optimización de naturaleza discreta. El algoritmo de senos y cosenos es una técnica de optimización metaheurística de tipo poblacional inspirada en la complementariedad de las funciones seno y coseno, las cuales son empleadas para explorar y explotar el espacio de soluciones. Este algoritmo es implementado en el software Julia para resolver el problema de la mochila.

4.1 Filosofía e inspiración

El algoritmo de senos y cosenos corresponde a una técnica de optimización metaheurística de naturaleza poblacional de la familia de las técnicas inspiradas en matemáticas. Este algoritmo fue inicialmente

propuesto por Mirjalili (2016) para resolver el problema de flujo de potencia óptimo en sistemas eléctricos de potencia predominantemente térmicos. Las principales características de esta técnica de optimización se resumen a continuación (Bansal et al., 2023).

- i. Evolucionan a partir de una población inicial creada de manera aleatoria dentro del rango de valores permitidos para las variables de decisión.
- ii. Emplea las funciones trigonométricas de seno y coseno para generar nuevas soluciones potenciales a partir de la solución actual y la mejor solución contenida en la población. En la selección de la función trigonométrica se aplica una regla probabilística, en la cual cada función (i.e., seno o coseno) tienen la misma probabilidad de ser seleccionadas.
- iii. Se aplica una regla lineal de decrecimiento para reducir el impacto de las funciones seno y coseno en cada solución de la población, con lo cual se pasa de la etapa de exploración del espacio de soluciones, a la etapa de explotación del mismo.
- iv. Sólo cuenta con tres parámetros de control para explorar y explotar el espacio de soluciones, siendo estos el tamaño de la población, la pendiente de la regla de decrecimiento y el número de iteraciones.

La técnica de optimización metaheurística de senos y cosenos ha sido ampliamente empleada en la literatura especializada para resolver problemas de optimización de naturaleza continua (Gabis et al., 2021), así como adaptaciones exitosas en problemas de variables discretas (Montoya et al., 2021a). Algunos de estas aplicaciones son: estimación de parámetros en máquinas eléctricas y células fotovoltaicas (Bocanegra et al., 2021; Montoya et al., 2020a), diagnósticos médicos (Ye et al., 2022), optimización de funciones complejas de naturaleza multimodal (Li et al., 2023), despacho óptimo de potencia en sistemas de transmisión

(Weng et al., 2023) y diseño óptimo de resortes helicoidales (Rodríguez-Cabal et al., 2021), entre otros.

4.2 Algoritmo de senos y cosenos

Considerando las exitosas aplicaciones presentadas en la literatura especializada de la técnica de optimización de senos y cosenos, el objetivo de esta sección es presentar de forma general este método de optimización, con énfasis particular en sus reglas de avance, corrección de las soluciones y criterios de parada, entre otros aspectos relevantes, tal como se verá en las siguientes subsecciones.

4.2.1 Población inicial

El algoritmo de senos y cosenos es una técnica de optimización inspirada en la complementariedad de las funciones seno y coseno para explorar y explotar el espacio de soluciones (Montoya et al., 2024a). Para esto, como todas las técnicas de optimización poblacional, este algoritmo evoluciona a partir de un conjunto de potenciales soluciones denominado población inicial (Montoya et al., 2021a). Para crear una población inicial en un problema de optimización general (discreto o continuo) se definen dos aspectos iniciales, siendo el primero el tamaño de la población n_s , y el segundo el número de variables n_v . Donde n_s es definido por el programador, mientras que n_v está predefinido por el problema de optimización en estudio. Por lo anterior, la población inicial X^t es un arreglo bidimensional (una matriz) de dimensiones $n_s \times n_v$ con $t = 0$, siendo t el contador de iteraciones. En la Ecuación

(4.1) se presenta la estructura general de la población inicial.

$$X^t = \begin{bmatrix} x_{1,1} & x_{1,2} & \cdots & x_{1,j} & \cdots & x_{1,n_v} \\ x_{2,1} & x_{2,2} & \cdots & x_{2,j} & \cdots & x_{2,n_v} \\ \vdots & \vdots & \ddots & \vdots & \ddots & \vdots \\ x_{k,1} & x_{k,2} & \cdots & x_{k,j} & \cdots & x_{k,n_v} \\ \vdots & \vdots & \ddots & \vdots & \ddots & \vdots \\ x_{n_s,1} & x_{n_s,2} & \cdots & x_{n_s,j} & \cdots & x_{n_s,n_v} \end{bmatrix}, \quad (4.1)$$

siendo $x_{k,j}$ el valor asignado a la j -ésima variable de decisión en la k -ésima fila de la población, donde para generar este valor, se aplica la regla definida en (4.2).

$$x_{k,j} = x_j^{\min} + \alpha_j \left(x_j^{\max} - x_j^{\min} \right), \quad \{j = 1, 2, \dots, n_v, k = 1, 2, \dots, n_s\} \quad (4.2)$$

donde α_j es un valor aleatorio en el rango de cero a uno con distribución normal, y $x_j^{\min}$ y $x_j^{\max}$ representan los límites inferiores y superiores aplicables a la j -ésima variable.

Observación 4.1. En el caso de problemas de optimización de naturaleza discreta, la generación de las variables de decisión a través de la Ecuación (4.2) son redondeadas al valor entero más cercano. Esto garantizará la factibilidad de cada una de las soluciones contenidas en $x_{k,j}$.

4.2.2 Reglas de avance

Una vez se ha creado la población inicial X^t en $t = 0$, el paso siguiente, es determinar la mejor solución contenida en la población, la cual se denominará x_{best}^t (Montoya et al., 2024a).

Observación 4.2. Para determinar la mejor solución en la iteración t se debe evaluar cada uno de los n_s potenciales soluciones contenidas en la población X^t en la función objetivo del problema o función de adaptación, ya que esta definirá la mejor solución, i.e., x_{best}^t , como:

$$x_{\text{best}}^t = \{x_k^t / x_k^t \rightarrow \min(f(x_k^t)), \forall k = 1, 2, \dots, n_s\}, \quad (4.3)$$

donde x_k^t es la k -ésima solución contenida en la población y $f(x_k^t)$ corresponde a la función objetivo (también conocida como función de adaptación o desempeño) evaluada en la solución x_k^t .

Es importante notar que la Ecuación (4.3) es aplicable a problemas de minimización. Sin embargo, en el caso de problemas de maximización, se cambia la función mínimo por la función máximo.

Una vez definida la mejor solución actual, i.e., x_{best}^t , se aplica la regla de avance definida en (4.4).

$$y_k^t = \begin{cases} x_k^t + r_1 \sin(r_2) \cdot |r_3 x_{\text{best}}^t - x_k^t| & r_4 \leq \frac{1}{2} \\ x_k^t + r_1 \cos(r_2) \cdot |r_3 x_{\text{best}}^t - x_k^t| & r_4 > \frac{1}{2} \end{cases}, \quad \{\forall k = 1, 2, \dots, n_s\}, \quad (4.4)$$

donde y_k^t corresponde a la potencial nueva solución, r_1 representa la función lineal de decrecimiento, r_2 es un vector de variables aleatorias con distribución uniforme entre $[0, 2\pi]$ con dimensión $1 \times n_v$, mientras que r_3 y r_4 son valor aleatorios en el rango de cero a uno generados distribución de probabilidad uniforme. Es importante mencionar que la notación $v \cdot w$ representa el producto elemento a elemento entre los vectores v y w ; además, el parámetro r_1 se calcula en cada iteración como:

$$r_1 = a \left(1 - \frac{t}{t_{\text{max}}} \right), \quad (4.5)$$

siendo t_{max} el número máximo de iteraciones y a un factor de escala definido por el programador. Según los autores de (Mirjalili, 2016) este escalar toma el valor de dos, i.e., $a = 2$.

Observación 4.3. Nótese que cada potencial solución y_k^t debe ser revisada con el fin de asegurar el cumplimiento de los límites aplicables a las variables de decisión. Por lo anterior, en la Ecuación (4.6) se presenta la regla de corrección es aplicable a cada variable.

$$y_{k,j}^t = \begin{cases} y_{k,j}, & x_j^{\min} \leq y_{k,j} \leq x_j^{\max} \\ x_j^{\min}, & y_{k,j} < x_j^{\min} \\ x_j^{\max}, & y_{k,j} > x_j^{\max} \end{cases}, \quad \{\forall j = 1, 2, \dots, n_v, \forall k = 1, 2, \dots, n_s\}. \quad (4.6)$$

Como se explicó anteriormente, las soluciones contenidas en y_k^t después del proceso de revisión de los límites máximos y mínimos dados por la regla (4.6) deben ser redondeadas a su parte entera, siempre que se trate de problemas de optimización de naturaleza discreta, con el objetivo de preservar la factibilidad en estos casos de optimización.

4.2.3 Sustitución en la población

Una vez se han generado las potenciales nuevas soluciones, i.e., y_k^t , se requiere conocer si estas harán parte de la población en la siguiente iteración, para lo anterior, es necesario conocer el valor de la función de desempeño $f(y_k^t)$. Teniendo en cuenta este valor, la regla de sustitución de la población se define en la Ecuación (4.7).

$$x_k^t = \begin{cases} y_k^t & f(y_k^t) < f(x_k^t) \\ x_k^t & f(y_k^t) \geq f(x_k^t) \end{cases}, \quad \{\forall k = 1, 2, \dots, n_s\}. \quad (4.7)$$

Es importante mencionar que la regla de substitución de la población definida en (4.7) es aplicable a problemas de minimización; sin embargo, en el caso de maximización se cambian los signos de menor que y mayor igual que, por símbolos de mayor que y menor igual que, respectivamente.

4.2.4 Criterios de parada

En la aplicación de técnicas de optimización metaheurística, debido a que la solución del problema es aproximada mediante programación secuencial, es típico encontrar dos criterios de parada comúnmente empleados en la literatura especializada, como siguen (Montoya et al., 2024a):

- i. El proceso de exploración y explotación del espacio de soluciones se da por terminado cuando se alcanza el número máximo de iteraciones preestablecido por el programador, o usuario del algoritmo.
- ii. Cuando durante $m_{\max}$ número de iteraciones la función de desempeño no presenta ninguna mejora en su valor.

Nótese que para implementar el segundo criterio de parada, se requiere de la implementación de un contador de no mejoras de la función objetivo. En caso de implementar este criterio de parada, el valor de $m_{\max}$ es predefinido por el usuario y se recomienda como un valor entre 10 y 30% del total de iteraciones. Este porcentaje depende del problema analizado y pasa a ser un parámetro de optimización para la estrategia.

4.2.5 Implementación computacional

En el Algoritmo 3 se describe la implementación general de la técnica de optimización de senos y cosenos para problemas de optimización.

Data: Datos del problema de optimización bajo análisis

Definir los límites inferiores de las variables, i.e., $x^{\min}$ y $x^{\max}$;
 Definir los valores de n_s , $t_{\max}$, $m_{\max}$ y determinar el valor de n_v ;
 Crear la población inicial dada en (4.1) empleando la regla dada en (4.2) con $t = 0$;
 Evaluar la función de adaptación $f(x_k^t) \forall k = 1, 2, \dots, n_s$;
 Determinar la mejor solución x_{best}^t aplicando la Ecuación (4.3);
 Hacer $m = 0$;
for $t = 1 : t_{\max}$ **do**
 Obtener el valor del parámetro r_1 empleando (4.5);
 for $k = 1 : n_s$ **do**
 Generar los números aleatorios para el vector r_2 , así como para r_3 y r_4 ;
 Generar el vector y_k^t empleando la regla de avance dada en (4.4);
 Verificar los límites de las variables de decisión empleando (4.6);
 Evaluar la función de adaptación para y_k^t , i.e., $f(y_k^t)$;
 Aplicar la regla de sustitución en la población dada por (4.7);
 Actualizar la mejor solución x_{best}^t ;
 if $f(x_{\text{best}}^t) == f(x_{\text{best}}^{t-1})$ **then**
 Hacer $m = m + 1$;
 else
 Hacer $m = 0$;
 if $m \geq m_{\max}$ **then**
 Reportar la mejor solución actual contenida en x_{best}^t ;
 break;
 end for
end for

Result: Reportar la mejor solución encontrada, i.e., x_{best}^t

Algoritmo 3: Algoritmo de optimización de senos y cosenos

4.3 Implementación en el software Julia

En esta sección se describe la aplicación del algoritmo de optimización de senos y cosenos al problema de la mochila y el cual está dado por el modelo matemático (2.1)–(2.4). Para ilustrar la implementación del algoritmo de senos y cosenos se presentan los siguientes pasos secuenciales.

Para implementar el problema de la mochila empleando el algoritmo de senos y cosenos, en primer lugar se realiza la carga de la información del sistema de prueba, tal como se ilustra a continuación.

```
using DataFrames
ks = DataFrames.DataFrame([
(236, 40, 67), (484, 45, 34), (500, 50, 83), (136, 25, 49),
(380, 28, 33), (299, 25, 75), (481, 50, 17), (419, 35, 34),
(419, 21, 92), (301, 30, 15), (226, 31, 62), (467, 41, 19),
(224, 29, 64), (202, 39, 45), (474, 21, 91), (241, 30, 34),
(287, 39, 85), (209, 21, 68), (319, 43, 23), (231, 50, 45),
(111, 47, 86), (182, 37, 60), (233, 45, 84), (339, 39, 60),
(412, 31, 92), (251, 45, 43), (460, 29, 19), (235, 21, 51),
(341, 43, 78), (439, 47, 13),])
DataFrames.rename!(ks, [:C, :V, :W])
C = ks.C'; V = ks.V'; W = ks.W';
Vmax = 861; Wmax = 1215; n = size(C,2); P = 1:n;
```

En segundo lugar, se construye la función adaptación para el problema de optimización la cual se denomina `fitnessf(C,V,W,Vmax,Wmax,x,P)`, como se muestra a continuación.

```
function fitnessf(C,V,W,Vmax,Wmax,x,P)
    Penalty = 100;
    Vol = sum(V[p]*x[p] for p in P);
    Wei = sum(W[p]*x[p] for p in P);
    Objval = sum(C[p]*x[p] for p in P) -
    Penalty*(max(Vol - Vmax,0) + max(Wei - Wmax,0));
    return Objval
end
```

En tercer, se ilustra el proceso iterativo del algoritmo de senos y cosenos, el cual se ha construido a partir de una función denominada `function ASC(X, lb, ub, tmax, a, Nv, Ns)`. Que recibe la parametrización del algoritmo, es decir, límites superiores e inferiores de las variables, población inicial y número de iteraciones, entre otros.

```
function ASC(X, lb, ub, tmax, a, Nv, Ns)
for t in 1:tmax
    global xbest = X[Ns, :];
    for k in 1:Ns
        global r1 = a*(1 - t/tmax)
        global r2 = 2*pi*rand(Float64, (Nv+1, 1));
        global r3 = rand(Float64);
        global r4 = rand(Float64);
        if isless(r4, 1/2)
            global y = X[k, :] +
                r1*sin.(r2).*abs.(r3.*xbest-X[k, :]);
        else
            global y = X[k, :] +
                r1*cos.(r2).*abs.(r3.*xbest-X[k, :]);
        end
        global y = (round.(y));
        for j = 1:Nv
            if y[j, 1] < lb[j, 1]
                global y[j, 1] = lb[j, 1];
            elseif y[j, 1] > ub[j, 1]
                global y[j, 1] = ub[j, 1];
            end
        end
        global y[Nv+1, 1] =
            fitnessf(C, V, W, Vmax, Wmax, y[1:Nv, 1], P);
        if X[k, Nv+1] < y[Nv+1, 1]
            global X[k, :] = y;
        end
    end
    global X = X[sortperm(X[:, Nv+1]), :];
end
return xbest
end
```

Un detalle importante de la implantación anterior corresponde al uso de la función `sortperm()`, la cual permite ordenar un vector o una matriz en forma ascendente. En el caso de matrices, estas se ordenan respecto de una columna específica, así pues, para el caso de la población inicial, esta es ordenada respecto de la última columna, que es donde se guardado el valor de la función de adaptación, para ello se emplea el comando `X = X[sortperm(X[:, Nv+1]), :]`.

Observación 4.4. En la implementación de la función `ASC(X, lb, ub, tmax, a, Nv, Ns)`, que determina la secuencia general del algoritmo de senos y cosenos, no se ha implementado el criterio de parada local por no mejoras consecutivas de la función objetivo, por lo cual se invita al lector a proponer las modificaciones necesarias para realizar dicha implementación.

Con el fin de desarrollar una evaluación estadística básica de la implementación de la metodología de optimización de senos y cosenos para resolver el problema de la mochila, se propone realizar múltiples veces la ejecución del algoritmo, que como se muestra a continuación, se realizó para 100 repeticiones, i.e., `rep = 100`.

```
using Printf, Statistics, BenchmarkTools
t1 = time(); rep = 100;
Ns = 1000; Nv = n; tmax = 100; a = 2;
lb = zeros(Nv,1); ub = ones(Nv,1);
global Guardar = zeros(rep,Nv+1);
    for r = 1:rep
        global X = rand(Float64,Ns,Nv+1);
        X = round.(X);
        for k in 1:Ns
            local x = X[k,1:Nv];
            global X[k,Nv+1] =
                fitnessf(C,V,W,Vmax,Wmax,x,P);
        end
    end
```

```
        global X = X[sortperm(X[:, Nv+1]), :];
        ASC(X, lb, ub, tmax, a, Nv, Ns)
        global Guardar[r, :] = xbest;
    end
global Guardar = Guardar[sortperm(Guardar[:, Nv+1]), :];
et = time() - t1;
@printf("Valor maximo: %2.1f\n", Guardar[rep, Nv+1])
@printf("Valor minimo: %2.1f\n", Guardar[1, Nv+1])
@printf("Valor promedio: %2.1f\n", mean(Guardar[:, Nv+1]))
@printf("Desviacion estandar: %2.1f\n", std(Guardar[:, Nv+1]))
@printf("Maximo - Media: %2.1f\n", Guardar[rep, Nv+1] - mean(Guardar[:, Nv+1]))
println("Tiempo promedio: ",
round(et/rep, digits = 4), " s");
println("xbest:", Int.(Guardar[rep, 1:Nv])');

```

Es esta implementación es importante mencionar que:

- i. Se emplea la librería `Printf` para realizar la impresión en pantalla de resultado final y el análisis estadístico, la librería `Statistics` para habilitar las funciones `mean()` y `std()`, que permiten el cálculo del promedio y la desviación estándar; además de la librería `BenchmarkTools` para evaluar el tiempo de ejecución del algoritmo a través del comando `@time()`.
- ii. La población inicial es creada, redondeada a los valores admisibles de las variables, evaluada y almacenada en la variable `X`, cada vez que se hace una ejecución, con el objetivo de realizar exploraciones y explotaciones del espacio de búsqueda desde diferentes puntos iniciales.
- iii. En esta implementación se han definido como parámetros del algoritmo, un tamaño de población de 1000 individuos y un nú-

mero máximo de iteraciones de 100, además el parámetro a del algoritmo de senos y cosenos se ha fijado en 2.

- iv. La matriz `Guardar[rep, 1:Nv]` contiene todas las soluciones del proceso de repetición, y como este es un problema de maximización, y la matriz ha sido ordenada en forma ascendente, siempre la mejor solución estará en la última fila, y la peor solución en la primera.

Nótese que una vez se realiza la ejecución del algoritmo de senos y cosenos con sus correspondientes repeticiones, se obtienen los resultados que se muestran a continuación.

```
Valor maximo: 8416.0
Valor minimo: 8248.0
Valor promedio: 8395.5
Desviacion estandar: 27.4
Maximo - Media: 20.5
Tiempo promedio: 1.1907 s
xbest:[0 1 1 0 1 1 1 1 1 1 1 1 0 1 1
        1 1 1 1 1 0 0 0 1 1 1 1 1 1 1]
```

Los resultados obtenidos muestran que: (i) la mejor solución encontrada en el proceso iterativo, i.e., 8416.0 corresponde al valor reportado por la metodología de solución exacta con el solver `HIGHS` para problemas de programación lineal entera estudiado en el Capítulo 2; (ii) los valores mínimo y promedio obtenidos en esta evaluación fueron 8248.0 y 8395.5, con una desviación estándar en valor real de 20.5, y (iii) en el tiempo promedio de ejecución del algoritmo fue de 1.1907 s.

Observación 4.5. Los resultados obtenidos en esta simulación son únicamente con fines ilustrativos, ya que estos dependen del computador en el que sea ejecutado el algoritmo.

5

Algoritmo de aprendizaje incremental basado en poblaciones

Este capítulo presenta las principales características y proceso de convergencia del algoritmo de aprendizaje incremental basado en poblaciones (PBIL), el cual es un algoritmo de optimización formulado para resolver problemas discretos. Este asigna una probabilidad a la opción discreta que puede tomar cada variable dentro del problema de optimización, la cual aumenta y disminuye acorde al impacto que tenga cada opción dentro de la solución del problema analizado. Siendo las posibles soluciones del problema contenidas en una matriz de individuos, que se construye a partir del porcentaje asignado a cada opción que componen las variables. Es importante resaltar que, a pesar de que este algoritmo emplea una población para explorar el espacio de solución, el control de

la exploración lo genera la matriz de probabilidades. La cual aumenta la probabilidad de las opciones contenidas en la mejor solución en cada iteración y reduce las probabilidades de las que no, estando la solución final del problema codificada dentro de la matriz de probabilidades. El proceso de convergencia controlado por la entropía de dicha matriz, el cual finaliza la exploración del algoritmo cuando la dispersión de los datos reduce a un nivel fijado por el optimizador entre un rango real entre [0-1]. Para interpretar la forma de avance y convergencia del PBIL, en este capítulo se abordó el problema de la mochila.

5.1 Filosofía e inspiración

El algoritmo PBIL pertenece a la familia de los algoritmos de estimación de distribución (Ceberio et al., 2012), los cuales emplean técnicas de aprendizaje automático para estimar la probabilidad de distribución asociada a la mejor solución hallada dentro de una población (Baluja, 1994). El PBIL, actualiza en cada iteración las probabilidades de selección de las diferentes opciones contenidas en las variables del problema, empleando una matriz de probabilidades. A partir de esta matriz es generada una población de individuos iteración a iteración, respetando las probabilidades de aparición de cada opción dentro de las diferentes variables que componen el problema. Evaluando la función adaptación de los individuos que componen la población en cada iteración, actualizando la matriz de probabilidades en función del individuo con el mejor desempeño.

El algoritmo PBIL emplea dos parámetros de control, el primero corresponde a la tasa de aprendizaje que controla la exploración del espacio de solución, la cual toma valores en un rango real de [1-0] en orden decente. El algoritmo PBIL contiene cuatro tipos principales de tasas de aprendizaje: lineal, exponencial, sigmoidea y en forma de campana (Bolanos et al., 2012). Sin embargo, en la literatura ha sido demostrado que la que mejor rendimiento presenta es la sigmoidea, empleada en este

documento (Grisales-Noreña et al., 2018). La convergencia del PBIL es controlada por la entropía de la matriz de probabilidades, segundo parámetro de control. Este mide la dispersión de la matriz de probabilidades, convergiendo cuando este parámetro alcanza un valor de cero, con un rango de operación real entre $[1-0]$. Siendo un valor de entropía de uno, una matriz totalmente dispersa. Las principales características del algoritmo PBIL se describen a continuación.

- i. Su evolución es controlada por una matriz de probabilidades (PM^t), actualizada en cada iteración t del algoritmo. Esta matriz inicializa con todas las opciones de cada variable con la misma probabilidad ($1/n$), donde n son las opciones que puede tomar cada variable dentro del problema. En el caso de la mochila existen dos opciones: (0) o (1), en el caso de no llevar o llevar un producto.
- ii. A partir de la matriz de probabilidades en cada iteración se genera una población de individuos X^t , la cual respeta el porcentaje de probabilidad de cada opción de las variables que conforman el problema. De esta manera, la población en cada iteración va convergiendo a la solución del problema, dando peso a las opciones que permiten encontrar el mejor valor de función adaptación. En el caso de la mochila, maximizar el costo.
- iii. A partir de la información contenida en PM^t , se calcula iteración a iteración la entropía de dicha matriz (E^t). Finalizando el proceso iterativo en el caso de que este variable alcance un valor de cero o una tolerancia asignada al algoritmo ($E^t \leq E_{tol}$). En el caso en que se cumpla el criterio de convergencia se decodifica la solución contenida en PM^t y se imprime como solución. En otro caso, se actualiza el valor de la tasa de aprendizaje en cada iteración (LR^t). Generando la actualización de la matriz de probabilidades con los valores de E^t , LR^t y las variables contenidas en la mejor solución hallada en cada solución.

- iv. Dentro de la población, se selecciona el individuo con la mejor solución para actualizar la matriz de probabilidades. Aumentando la probabilidad a las opciones contenidas dentro de este y reduciéndolas a las que no.
- v. A diferencia de otros algoritmos explicados en este documento, el PBIL no controla el número de iteraciones, ni es posible aplicarle el criterio de convergencia de iteraciones de no mejora. Dado que este permite empeorar la solución entre iteraciones para escapar de óptimos locales. Fijando su convergencia únicamente en la entropía de la matriz de probabilidades en cada iteración.

El PBIL ha sido ampliamente usado para resolver problemas discretos en la literatura asociados a localización de recursos energéticos distribuidos en redes eléctricas (Grisales-Noreña et al., 2018), planificación de rutas de robots (Kang et al., 2014), problema de optimización de la recarga nuclear (da Silva et al., 2018), estabilización de sistemas de potencia (Folly, 2011), generación automática de horarios (Rao et al., 2019), entre otros (da Silva et al., 2018). Siendo destacado por su efectividad en términos de calidad de solución y tiempos de procesamiento reducidos.

5.2 Algoritmo de aprendizaje incremental basado en poblaciones

Con el objetivo describir el proceso de convergencia, algoritmo PBIL, para tomar ventaja de su rendimiento y velocidad al resolver problemas discretos. Esta sección describe cada uno de los pasos fundamentales de este algoritmo y presenta el algoritmo que describe el proceso iterativo.

5.2.1 Inicialización de parámetros

En el primer paso del algoritmo PBIL, se leen los datos del problema que permiten la evaluación de la función adaptación, y se cargan los parámetros propios del algoritmo. Dentro de los cuales se destacan número de individuos de la población (n_s), número de variables (n_v), número de opciones de cada la variable (n), tasa de aprendizaje máxima y mínima ($LRmax$ y $LRmin$), y la entropía de convergencia del algoritmo ($Etol$).

5.2.2 Población inicial

En el algoritmo PBIL la población de individuos depende en cada iteración de la matriz de probabilidades (PM). Esta matriz indica la probabilidad de aparición de las diferentes opciones de cada variable dentro de la población, es decir, limita el número de apariciones de las opciones posibles de cada variable en los diferentes individuos que componen la población.

Por lo cual para la generación de la población, inicialmente se debe generar la matriz de probabilidades PM^t , por medio de la ecuación (5.1). Esta matriz contiene dentro de sus filas las n opciones que puede tomar cada variable y dentro de sus columnas las j variables que representan el problema.

$$PM^t = \begin{bmatrix} pm_{1,2} & pm_{1,2} & \cdots & pm_{1,j} & \cdots & pm_{1,n_v} \\ pm_{2,1} & pm_{2,2} & \cdots & pm_{2,j} & \cdots & pm_{2,n_v} \\ \vdots & \vdots & \ddots & \vdots & \ddots & \vdots \\ pm_{k,1} & pm_{k,2} & \cdots & pm_{k,j} & \cdots & pm_{k,n_v} \\ \vdots & \vdots & \ddots & \vdots & \ddots & \vdots \\ pm_{n,1} & pm_{n,2} & \cdots & pm_{n,j} & \cdots & pm_{n,n_v} \end{bmatrix}, \quad (5.1)$$

En la primera iteración, dentro de la matriz de probabilidades se asigna el mismo porcentaje de probabilidad a todas las opciones del por medio de la Ecuación (5.2). Donde n corresponde al número de

opciones o diferentes valores discretos que puede tomar cada variable dentro del problema de optimización.

$$pm_{k,j} = 1/n, \{k = 1, 2, \dots, n, j = 1, 2, \dots, n_v\} \quad (5.2)$$

Es importante resaltar que siempre la suma de las probabilidades asignadas a cada variable debe dar igual a uno. Lo cual se garantiza dentro del PBIL empleando la siguiente Ecuación (5.3).

$$\sum_{k=1}^n pm_{k,j} = 1, \{j = 1, 2, \dots, n_v\} \quad (5.3)$$

Posteriormente, empleando PM^t , se genera la población de individuos empleando la Ecuación (5.4). Siendo $x_{k,j}$ el valor asignado a la j -ésima variable del k -ésimo individuo de la población. Donde para generar este valor, se garantiza el porcentaje de apariciones de las n opciones contenidas en cada variable, es decir, la población de individuos debe contener dentro de cada variable los mismos porcentajes de participación de cada una de las opciones contenidas dentro de PM^t .

$$X^t = \begin{bmatrix} x_{1,1} & x_{1,2} & \cdots & x_{1,j} & \cdots & x_{1,n_v} \\ x_{2,1} & x_{2,2} & \cdots & x_{2,j} & \cdots & x_{2,n_v} \\ \vdots & \vdots & \ddots & \vdots & \ddots & \vdots \\ x_{k,1} & x_{k,2} & \cdots & x_{k,j} & \cdots & x_{k,n_v} \\ \vdots & \vdots & \ddots & \vdots & \ddots & \vdots \\ x_{n_s,1} & x_{n_s,2} & \cdots & x_{n_s,j} & \cdots & x_{n_s,n_v} \end{bmatrix}, \quad (5.4)$$

Observación 5.1. El algoritmo PBIL no utiliza límites mínimos y máximos para las variables como los algoritmos presentados con anterioridad. El PBIL asigna un valor a la variable asociado a cada opción dentro de la matriz de probabilidad, es decir, el programador debe conocer a que valor de la variable corresponde a cada opción. En el caso de la

mochila analizado en este documento, la opción 1 indica asignar un cero a la variable. Mientras que la opción 2 representa asignar un uno. Existiendo para este problema la posibilidad de contener dos opciones para cada variable dentro de la matriz de probabilidad, las cuales dentro del proceso iterativo del PBIL se reparte el 100% de participación dentro de la población.

5.2.3 Reglas de avance

Una vez se ha creada la población inicial X^t en la primera iteración del algoritmo. Se procede a la evaluación de la función adaptación de cada individuo, seleccionando el individuo con mejor solución como incumbente del problema ($Xbest$).

El paso siguiente consta de determinar la entropía (E^t). Esta variable está encargada de evaluar la dispersión de las probabilidades en PM^t , empleando la Ecuación (5.5). La cual está en función de la probabilidad de PM^t en la iteración actual y una función logaritmo en base dos.

$$E^t = \frac{- \sum_{k=1}^n \sum_{j=1}^{nv} pm_{k,j} \log_2 (pm_{k,j})}{nv}, \quad (5.5)$$

En el caso en que E^t alcance el criterio de convergencia $Etol$, el algoritmo finaliza. Imprimiendo $Xbest$ y su función adaptación como solución al problema. En otro caso, empleando la entropía en la iteración actual y los límites de la tasa de aprendizaje ($LRmax$ y $LRmin$), es calculada la tasa de aprendizaje en la iteración actual de algoritmo (LR^t), empleando la Ecuación (5.6). En la versión clásica del PBIL $LRmax = 1$ y $LRmin = 0$. Sin embargo, estos algoritmos pueden ser sintonizados para cada problema dentro de estos rangos de valor real.

$$LR^t = LRmax - \frac{LRmax - LRmin}{1 + e^{-10 \cdot (E^t - 0.5)}}, \quad (5.6)$$

Con los valores de E^t y LR^t es actualizada la matriz de probabilidad, por medio de las Ecuaciones (5.7) y (5.8).

$$pm_{k,j}^{Act} = pm_{k,j}^{Ant} + (1 - pm_{k,j}^{Ant}) \cdot LR^t \quad (5.7)$$

$$pm_{k,j}^{Nuevo} = \begin{cases} pm_{k,j}^{Act} & \text{if } k = m \\ \left(1 - pm_{k,j}^{Act}\right) \cdot \frac{pm_{k,j}^{Ant}}{1 - pm_{k,j}^{Ant}} & \text{if } k \neq m \end{cases} \quad (5.8)$$

Inicialmente, la Ecuación (5.7), actualiza el valor de cada componente de la matriz de probabilidades ($pm_{k,j}^{Act}$), empleando los valores anteriores de probabilidad ($pm_{k,j}^{Ant}$) contenidos en PM^t y la LR^t . Posteriormente, con $pm_{k,j}^{Act}$ es obtenida la pm^{t+1} , considerando si la k -ésima opción de cada variable contenida dentro del problema se encuentra o no dentro de la $Xbest$. En la Ecuación (5.8), m corresponde a la opción contenida dentro de la mejor solución hallada por la población en la iteración actual. Para las opciones contenidas dentro de $Xbest$, la probabilidad nueva ($pm_{k,j}^{Nuevo}$) asignada a la matriz pm^{t+1} es igual a la $pm_{k,j}^{Act}$, incrementando la probabilidad de dicha opción dentro de la matriz en el caso en que la k -ésima opción analizada sea igual a la m -ésima opción contenida dentro de $Xbest$. En otro caso, si la k -ésima opción analizada para la j -ésima variable no está contenida dentro de las opciones que conforman a $Xbest$, el PBIL reduce la probabilidad de esta opción dentro de la matriz pm^{t+1} mediante la segunda formulación contenida en la Ecuación (5.8) para el cálculo de $pm_{k,j}^{Nuevo}$.

Posteriormente, con la actualización de la matriz de probabilidades, se procede al análisis del criterio de convergencia del PBIL para finalizar o continuar con el proceso iterativo.

5.2.4 Criterio de parada

El algoritmo PBIL utiliza como criterio de parada únicamente la E^t , finalizando el proceso en el caso en que este valor alcance la tolerancia asignada ($Etol$). Si el proceso finaliza, se extrae la mejor solución

hallada por el PBIL por medio de la decodificación de la matriz de probabilidades. Tomando como opción a fijar en la solución las opciones de cada variable con mayor porcentaje de participación. Posteriormente, se evalúa la función adaptación de esta solución, siendo este valor la solución al problema otorgada por parte del PBIL. En caso en que no se alcance el criterio de convergencia fijado para el PBIL, el algoritmo genera una nueva población con la matriz actualizada y realiza una nueva iteración.

5.2.5 Implementación computacional

En el Algoritmo 4 describe los principales pasos y proceso iterativo necesario para la implementación del algoritmo PBIL.

Data: Lectura de parámetros del problema bajo análisis

Definir los parámetros iniciales del PBIL: $n_s, n_v, n, LR_{max}, LR_{min}, E_{tol}$ y $t = 1$;

Inicializar PM^t empleando la Ec. (5.2) y (5.3);

Generar la población de individuos X^t basados en la información de PM^t ;

Evaluar la función adaptación de cada individuo que compone X^t e identificar el individuo con mejor solución X_{best} ;

Evaluar E^t empleando la Ec. (5.5);

while $E^t \geq E_{tol}$ **do**

 Calcular la tasa de aprendizaje LR^t empleando la Ecuación (5.6);

 Incrementar el contador de iteraciones $t = t + 1$;

 Actualizar la matriz de PM^t empleando E^t, LR^t y X_{best} en las Ecuaciones (5.7) y (5.8);

 Evaluar E^t empleando la Ec. (5.5) ;

Result: Decodificar la solución del problema de PM^t y evaluar la función adaptación

Algoritmo 4: Algoritmo de optimización PBIL

Observación 5.2. En la versión descrita del algoritmo PBIL se incluye un contador de iteraciones para brindar claridad al proceso de avance. Sin embargo, este contador no es indispensable en el proceso del algoritmo.

5.3 Implementación en el software Julia

A continuación se ilustra el proceso de implementación del algoritmo de aprendizaje incremental basado en poblaciones, para brindar solución al problema de la mochila modelado en las Ecuaciones (2.1) a (2.4).

Inicialmente, se realiza el llamado las bibliotecas necesarias para llevar a cabo la implementación del algoritmo PBIL en Julia.

```
using JuMP, HiGHS, DataFrames, LinearAlgebra  
using Random, Statistics, Printf, BenchmarkTools
```

Posteriormente, se debe realizar la carga de los datos asociados al problema bajo análisis. Esta información está completamente detallada en el Capítulo 2 dentro de la Sección 2.1 de este documento.

```
ks = DataFrames.DataFrame([  
    (236, 40, 67), (484, 45, 34), (500, 50, 83), (136, 25, 49),  
    (380, 28, 33), (299, 25, 75), (481, 50, 17), (419, 35, 34),  
    (419, 21, 92), (301, 30, 15), (226, 31, 62), (467, 41, 19),  
    (224, 29, 64), (202, 39, 45), (474, 21, 91), (241, 30, 34),  
    (287, 39, 85), (209, 21, 68), (319, 43, 23), (231, 50, 45),  
    (111, 47, 86), (182, 37, 60), (233, 45, 84), (339, 39, 60),  
    (412, 31, 92), (251, 45, 43), (460, 29, 19), (235, 21, 51),  
    (341, 43, 78), (439, 47, 13),])  
DataFrames.rename!(ks, [:C, :V, :W])  
C = ks.C'; V = ks.V'; W = ks.W';  
Vmax = 861; Wmax = 1215; n = size(C, 2); P = 1:n;
```

Para evaluar la función adaptación de cada individuo generado dentro del proceso evolutivo del PBIL, se construye la función adaptación denominada dentro de Julia `function fitnessf(C,V,W,Vmax,Wmax,x,P)`, como se muestra a continuación.

```
function fitnessf(C,V,W,Vmax,Wmax,x)
    Penalty = 100
    global Vol = sum(V.*x);
    global Wei = sum(W.*x)
    Objval = sum(C.*x) - Penalty*(max(Vol .- Vmax,0)
    + max(Wei .- Wmax,0));
    return Objval
end
```

Antes de definir la función que evalúa el algoritmo PBIL para resolver el problema abordado, se realiza la carga de los parámetros iniciales del algoritmo. Dentro de estos parámetros se encuentra: número de individuos (N_s), número de opciones de en cada variable (n), número de variables contenidas en el problema (N_v), tasa de aprendizaje máxima y mínima (LR_{max} y LR_{min}), y finalmente la tolerancia de la entropía ($Etol$). Todos los parámetros anteriormente mencionados, excepto el número de variables, son parámetros de sintonización del algoritmo PBIL para cada problema. La carga de los parámetros en el software Julia se describe en el siguiente recuadro.

```
Ns=80; n=2; Nv=30; LRmax=1; LRmin=0; Etol=0.001;
```

A continuación, se ilustra el proceso iterativo del algoritmo de aprendizaje incremental basado en poblaciones desarrollado en Julia, el cual se ha desarrollado dentro de la función `function PBIL(Ns,n,Nv,LRmax,LRmin,Etol)`. La cual recibe los parámetros de optimización del PBIL como datos de entrada.

```
function PBIL(Ns,n,Nv,LRmax,LRmin,Etol)
    Pob=zeros(Ns,Nv);
    global PM=ones(n,Nv)*(1/n);
    global NI=round.(PM*Ns);
```

```
for i=1:Nv
    local aux=Ns-NI[1,i];
    global NI[2,i]=aux;
end
global x=zeros(Ns,Nv);
for i=1:Ns
    for j=1:Nv
        local aux=rand((0,1));
        local aux2=0;
        if ((aux==0) && (NI[1,j] !=0) && (aux2==0))
            x[i,j]=0;
            NI[1,j]=NI[1,j]-1;
            aux2=1;
        end
        if ((aux==1) && (NI[2,j] !=0) && (aux2==0))
            x[i,j]=1;
            NI[2,j]=NI[2,j]-1;
            aux2=1;
        end
        if ((aux==0) && (NI[1,j]==0) && (aux2==0))
            x[i,j]=1;
            NI[2,j]=NI[2,j]-1;
            aux2=1;
        end
        if ((aux==1) && (NI[2,j]==0) && (aux2==0))
            x[i,j]=0;
            NI[1,j]=NI[1,j]-1;
            aux2=1;
        end
    end
end
global fo=zeros(Ns,1);
for i=1:Ns
    fo[i,1]=fitnessf(C',V',W',Vmax,Wmax,x[i,:]);
end
global mejorpos, mejorpos = findmax(fo);
global mejorpos=x[mejorpos[1],:];
global E=0;
for i=1:n
```

```

    for j=1:Nv
        global E=E+PM[i,j]*log2(PM[i,j]);
    end
end
E=-E/Nv;
global t=1;
while (E>=Etol)
    global t=t+1;
    global LR=LRmax-((LRmax-LRmin)/(1+exp(-10*(E-0.5))));
    for i=1:n
        for j=1:Nv
            aux=mejorpos[j,1];
            if(aux==0)
                if (i==1)
                    local PMact=PM[i,j]+(1-PM[i,j])*LR;
                    PM[i,j]=PMact;
                end
                if(i==2)
                    local PMact=PM[i,j]+(1-PM[i,j])*LR;
                    PM[i,j]=(1-PMact)*((PM[i,j])/(1-PM[i,j]));
                end
            end
            if(aux==1)
                if(i==1)
                    local PMact=PM[i,j]+(1-PM[i,j])*LR;
                    PM[i,j]=(1-PMact)*((PM[i,j])/(1-PM[i,j]));
                end
                if (i==2)
                    local PMact=PM[i,j]+(1-PM[i,j])*LR;
                    PM[i,j]=PMact;
                end
            end
        end
    end
    global NI=round.(PM*Ns);
    for i=1:Nv
        local aux=Ns-NI[1,i];
        global NI[2,i]=aux;
    end
end

```

```
for i=1:Ns
    for j=1:Nv
        local aux=rand((0,1));
        local aux2=0;
        if ((aux==0) && (NI[1,j] !=0) && (aux2==0))
            x[i,j]=0;
            NI[1,j]=NI[1,j]-1;
            aux2=1;
        end
        if ((aux==1) && (NI[2,j] !=0) && (aux2==0))
            x[i,j]=1;
            NI[2,j]=NI[2,j]-1;
            aux2=1;
        end
        if ((aux==0) && (NI[1,j]==0) && (aux2==0))
            x[i,j]=1;
            NI[2,j]=NI[2,j]-1;
            aux2=1;
        end
        if ((aux==1) && (NI[2,j]==0) && (aux2==0))
            x[i,j]=0;
            NI[1,j]=NI[1,j]-1;
            aux2=1;
        end
    end
end
global fo=zeros(Ns,1);
for i=1:Ns
    fo[i,1]=fitnessf(C',V',W',Vmax,Wmax,x[i,:]);
end
global mejorpos, mejorpos = findmax(fo);
global mejorpos=x[mejorpos[1],:];
global E=0;
for i=1:n
    for j=1:Nv
        global E=E+PM[i,j]*log2(PM[i,j]);
    end
end
E=-E/Nv;
```

```
end
global Xbest=zeros(1,Nv+1);
Xbest[1,Nv+1]=mejorsol;
for i=1:Nv
    if (PM[1,i]>=PM[2,i])
        Xbest[1,i]=0;
    else
        Xbest[1,i]=1;
    end
end
return Xbest;
end
```

Para la implementación de la función que contiene el proceso de avance del algoritmo PBIL, se deben de tener en cuenta las siguientes consideraciones:

- i. Para la construcción la población se requiere la información de la matriz de probabilidad, dentro del software para inicializar esta matriz, se emplea la siguiente la función $PM = \text{ones}(n, nv) * (1/n)$, que genera una matriz de probabilidades iguales para todas las opciones. Posteriormente, esta matriz se multiplica por ns y se almacena dentro de una matriz auxiliar NI aplicando la función `round` para redondear al entero más cercano. Esta matriz auxiliar contiene el número de veces que se puede considerar cada opción dentro de la población. Para satisfacer que la suma de los elementos de cada opción sea igual a ns , se aplica un verificador por medio de una función `for` que permite garantizar esta restricción dentro del proceso de programación. El mismo proceso de asignación de opciones para los ns individuos dentro de NI se aplica a lo largo del proceso iterativo del PBIL.
- ii. La población de individuos para el PBIL, se genera a través de una función que emplea las posiciones de NI como contadores,

restando una unidad cada vez que la opción es empleada y evitando utilizar una opción en el caso en que haya alcanzado o se le haya fijado un valor de cero dentro del proceso iterativo.

- iv. Después de evaluar la función adaptación de cada individuo de la población por medio de la implementación de la función `fitnessf(C', V', W', Vmax, Wmax, x[i, :])`, el mejor individuo y su información se identifica mediante la implementación de la función `mejorsol, mejorpos = findmax(fo)`.
- iv. Después de actualizar la entropía y tasa de aprendizaje en cada iteración, se plantea una función basada en dos `for` anidados para actualizar la matriz de probabilidades en cada iteración.

Con el fin de desarrollar una evaluación estadística básica de la implementación de la metodología de PBIL para resolver el problema de la mochila, se proponen 100 repeticiones del algoritmo, i.e., `rep = 100`, para calcular el valor máximo, mínimo y promedio de función adaptación. Así como la desviación estándar de los datos obtenidos, la desviación entre el valor real de la mejor solución hallada (máximo valor) y la solución promedio, y el tiempo promedio de procesamiento requerido por el algoritmo. La función que permite realizar este análisis se describe a continuación.

```
t1 = time();
rep = 100;
global Guardar = zeros(rep, Nv+1);
    for r = 1:rep
        PBIL(Ns, n, Nv, LRmax, LRmin, Etol)
        global Guardar[r, :] = Xbest;
    end
global Guardar = Guardar[sortperm(Guardar[:, Nv+1]), :];
elapsed_time = time() - t1;
@printf("Valor maximo: %2.1f          \n", Guardar[rep, Nv+1])
@printf("Valor minimo: %2.1f          \n", Guardar[1, Nv+1])
@printf("Valor promedio: %2.1f\n", mean(Guardar[:, Nv+1]))
```

```
@printf("Desviacion estandar: %2.1f\n", std(Guardar[:, Nv+1]))
@printf("Maximo - Media: %2.1f\n", Guardar[rep, Nv+1]-mean(Guardar[:, Nv+1]))
println("Tiempo de ejecucion (promedio): ",
round(elapsed_time/rep, digits = 4), " s");
println("xbest: ", Int.(Guardar[rep, 1:Nv]));
```

Es esta implementación es importante mencionar que:

- i. Se emplea la librería `Statistics` para habilitar las funciones `mean()` y `std()`, que permiten el cálculo del promedio y la desviación estándar; además de la librería `BenchmarkTools` para evaluar el tiempo de ejecución del algoritmo a través del comando `@time()`.
- ii. La mejor solución hallada en cada iteración y la función adaptación asociada a esta se almacena en la matriz `Guardar`. La cual es de tamaño $rep \times nv + 1$

Al realizar las 100 ejecuciones del algoritmo PBIL en el entorno de Julias, se obtienen los resultados que se muestran a continuación.

```
Valor maximo: 8416.0
Valor minimo: 8138.0
Valor promedio: 8345.9
Desviacion estandar: 59.5
Maximo - Media: 70.1
Tiempo de ejecucion (promedio): 0.3516 s
xbest: [0 1 1 0 1 1 1 1 1 1 1 1 0 1 1 1
1 1 1 1 0 0 0 1 1 1 1 1 1 1 1]
```

Analizando estos resultados es posible apreciar que: (i) la mejor solución encontrada en el proceso iterativo corresponde a la solución óptima del problema, i.e., 8416.0. Siendo este valor reportado en el Capítulo 2 por la metodología de solución exacta con el solver `HiGHS` para problemas de programación lineal entera. (ii) los valores mínimo y promedio obtenidos en esta evaluación fueron, 8138.0 y 8345.9, con

una desviación estándar en valor real de 59.5; lo cual representa una desviación del 0.71 %. (iii) la desviación estándar entre la mejor solución (máximo valor) y solución promedio fue de 70.1 unidades de valor real, equivalente a un porcentaje del 0.83%. Este valor en conjunto con la desviación estándar permite identificar que el PBIL obtiene soluciones de excelente calidad con una desviación del 0.86% con respecto a la solución óptima del problema. (iv) en el tiempo promedio de ejecución del algoritmo fue de 0.3516 s. Esto permite ubicar al PBIL como un algoritmo eficiente desde el punto de vista de la calidad de la solución y tiempo de procesamiento promedio.

Observación 5.3. Los resultados obtenidos en esta simulación son únicamente con fines ilustrativos, ya que estos dependen del computador en el que sea ejecutado el algoritmo.

6

Algoritmo genético

En este capítulo se describen e implementan dos versiones del método de optimización denominada Algoritmo Genético, el cual es un algoritmo de optimización clásico basado en los procesos de evolución biológica. El origen de este método se remonta a los años 70's. Desde entonces ha sido utilizado, de manera generosa, en la literatura para abordar problemas de diferentes áreas del conocimiento y en una variedad de problemas, que, desde el punto de vista matemático, pueden ser clasificados como continuos, enteros, binarios o mixtos. En este capítulo se describe la versión poblacional del Algoritmo Genético y la variante denominada Algoritmo Genético de Chu & Beasley. Los dos algoritmos son implementados en el entorno de programación de Julia para resolver el Problema de la Mochila con restricciones de volumen y peso.

6.1 Filosofía e inspiración

De manera general, los algoritmos genéticos están basados en el proceso evolutivo de los cromosomas y adaptación de las especies, es decir, emulan el proceso de selección natural. Entre los procesos biológicos que se desacatan en los algoritmos genéticos, se encuentran diferentes procesos de selección de individuos, donde aquellos con mejores condiciones pueden pasar a una etapa posterior para generar descendencias y en cada una de ellas realizar mejoras dentro de sus procesos (Lo cual hace referencia a un proceso evolutivo). Para esto, las dos versiones tratadas en este libro presentan en común, los operadores de selección, recombinación y mutación, los cuales son los encargados de guiar el proceso de avance del algoritmo hacia individuos mejor adaptados.

Originalmente, este algoritmo fue propuesto por (Holland, 1975). Desde entonces, diversas aplicaciones y variaciones del algoritmo han sido reportadas en la literatura. Siendo la versión presentada por Chu & Beasley (Chu and Beasley, 1997), una de las de mayor apropiación entre la comunidad académica a la hora de resolver problemas de las diferentes ramas de la ciencia.

Los algoritmos genéticos presentan diversas ventajas a la hora de solucionar problemas de difícil solución, entre ellas se destacan:

- i. Al tratarse de un algoritmo de tipo poblacional, en cada iteración explora de manera paralela diferentes soluciones.
- ii. Los operadores genéticos de recombinación y mutación permiten que el algoritmo explore el espacio de solución tanto desde el punto de vista de la intensificación como de la diversificación.
- iii. Por ser un algoritmo elitista, garantiza que en cada iteración se avance hacia un espacio de soluciones de mejor calidad.
- iv. Permite una etapa de mejora después de aplicar los operadores genéticos, lo que guía la búsqueda hacia otras regiones, permitiendo escapar de óptimos locales.

Por tratarse de uno de los métodos clásicos de la optimización combinatoria, los algoritmos genéticos han sido aplicados en un número relevante de investigaciones en todos los campos del conocimiento. Trabajos reportados en radiología (Shiraishi et al., 2011), oncología (Duraipandian et al., 2011), cardiología (Khalil et al., 2006) y ginecología (Marinakis et al., 2009) son algunas de las especialidades con aplicaciones del algoritmo genético en la medicina (Ghaheri et al., 2015). Así mismo, se presentan diferentes investigaciones en agricultura (Maraveas et al., 2023; Roy and De, 2022), aeronáutica (Yinfeng et al., 2021), transporte terrestre (Jiang et al., 2024; Lv et al., 2023), energía (Mythreyee and Nalini, 2023; Ildarabadi et al., 2023), entre muchas otras aplicaciones que pueden ser consultadas en (Alam et al., 2020).

6.2 Algoritmo Genético

En esta sección se describen dos versiones del algoritmo genético, la primera hace referencia al algoritmo genético poblacional y la segunda a la versión conocida en la literatura como algoritmo genético de Chu-Beasley. Los dos algoritmos comparten en esencia los mismos operadores genéticos de selección, recombinación y mutación. Para el caso de la selección se hará uso de la estrategia de selección por torneo acorde al valor de la función de adaptación de cada individuo (cromosoma) participante. La recombinación se hará de un único-punto, mientras que la mutación corresponderá a la deformación de un individuo en un único componente (un gen).

6.2.1 Algoritmo Genético poblacional

La primera parte del algoritmo genético consiste en definir cuáles son los parámetros del algoritmo. Para este caso se tendrán los siguientes parámetros: número de individuos (N_s), número máximo de iteraciones (t_{max}), número de variables que componen el problema (N_v). También

es necesario determinar el número de padres (n_p), que corresponden al número de individuos que participaran en el proceso de selección (torneo), la probabilidad de recombinación (ρ_r), la probabilidad de mutación (ρ_m), y el grado de diversidad del algoritmo (d).

6.2.1.1 Población inicial

La población inicial corresponde a un matriz de dimensiones $N_s \times N_v$, la cual puede ser generada de manera aleatoria o mediante estrategias que permitan inicializar el ciclo evolutivo del algoritmo en puntos del espacio de búsqueda de buena calidad. Este paso se realiza cuando el contador de iteraciones es $t = 0$. La representación de la población inicial X^t , está dada por la Ecuación (6.1), donde cada fila representa cada uno de los cromosomas que conforman la población y las columnas hacen referencia a cada una de las variables que conforman el individuo.

$$X^t = \begin{bmatrix} x_{1,1} & x_{1,2} & \cdots & x_{1,j} & \cdots & x_{1,N_v} \\ x_{2,1} & x_{2,2} & \cdots & x_{2,j} & \cdots & x_{2,N_v} \\ \vdots & \vdots & \ddots & \vdots & \ddots & \vdots \\ x_{k,1} & x_{k,2} & \cdots & x_{k,j} & \cdots & x_{k,N_v} \\ \vdots & \vdots & \ddots & \vdots & \ddots & \vdots \\ x_{N_s,1} & x_{N_s,2} & \cdots & x_{N_s,j} & \cdots & x_{N_s,N_v} \end{bmatrix}, \quad (6.1)$$

En esta matriz, cada gen $x_{k,j}$ contiene al valor asignado a la j -ésima variable del k -ésimo cromosoma. Este valor debe garantizar la factibilidad de la j -ésima variable del problema. Es decir, el valor debe ser asignado entre el límite mínimo y máximo asociados a dicha variable independiente de su naturaleza (continua, binaria o entera). Para generar la matriz inicial de manera aleatoria se emplea la Ecuación (6.2).

$$x_{k,j} = x_j^{\min} + \alpha_j \left(x_j^{\max} - x_j^{\min} \right), \quad \{j = 1, 2, \dots, N_v, k = 1, 2, \dots, N_s\} \quad (6.2)$$

Donde α_j un valor aleatorio en el rango de cero a uno con distribución normal, y $x_j^{\min}$ y $x_j^{\max}$ representan los límites inferiores y superiores aplicables a la j -ésima variable.

6.2.1.2 Función de adaptación

Cada uno de los individuos que hacen parte de la población deben ser evaluados con el fin de determinar la calidad de los mismos. Cuando el individuo es factible, su valor es dado por la función objetivo. Sin embargo, es posible que los individuos sean infactibles, es decir que se viole alguna de las restricciones técnicas del problema que se esté analizando, esto se permite con el fin de explorar otras regiones del espacio de solución, de tal manera que se pueda escapar de óptimos locales durante el proceso de optimización. En este caso, es necesario diseñar una función de adaptación, la cual está compuesta por la suma del valor de función objetivo más una valor asociado a la penalización generado por la infactibilidad, tal como se presenta en la Ecuación (6.3).

$$f_a = f_o + \beta \cdot pen_v \quad (6.3)$$

donde β es un factor calculado de manera heurística, el cual permite darle un valor significativo a cada grado de penalización por concepto de violación de las restricciones técnicas.

6.2.1.3 Reglas de avance

Como regla de avance, el algoritmo genético cuenta con tres operadores básicos que permiten la evolución del algoritmo hacia soluciones de buena calidad. Cada una de ellas se describe continuación:

- **Selección:** Este operador permite seleccionar un conjunto de individuos, de manera aleatoria o por medio de alguna regla de selección, los cuales serán denominados padres y serán los encargados de llevar a cabo el proceso de reproducción, es decir, a

partir de dichos individuos se obtendrán descendientes. Note que este operador es el símil del proceso de reproducción sexual de la naturaleza. Para efectos de este libro, los padres son seleccionados de manera aleatoria desde la población. Como ejemplo, suponga que se cuenta con una población X^t que tiene N_s individuos, donde cada uno de ellos tiene un número de variables $N_v = 7$. En este punto, un número de individuos $n_p = 4$ son seleccionados de manera aleatoria. Tal como se muestra a continuación:

$$\begin{aligned} X_i &= x_{i1} \ x_{i2} \ x_{i3} \ x_{i4} \ x_{i5} \ x_{i6} \ x_{i7} \\ X_j &= x_{j1} \ x_{j2} \ x_{j3} \ x_{j4} \ x_{j5} \ x_{j6} \ x_{j7} \\ X_k &= x_{k1} \ x_{k2} \ x_{k3} \ x_{k4} \ x_{k5} \ x_{k6} \ x_{k7} \\ X_l &= x_{l1} \ x_{l2} \ x_{l3} \ x_{l4} \ x_{l5} \ x_{l6} \ x_{l7} \end{aligned}$$

El paso siguiente, es determinar la pareja de padres p que será seleccionada para realizar el proceso de reproducción. En la literatura especializada existen diversos métodos de selección, sin embargo, en este capítulo se usa el método de selección por torneo. El cual permite que, a partir del conjunto de padres seleccionados de manera aleatoria, se realice un torneo donde los dos individuos con mejor función de adaptación resultaran ganadores (concepto de elitismo). Diferentes diseños de torneos han sido implementados en la literatura, en este libro el torneo se realiza por parejas y el ganador de cada enfrentamiento pasará a la siguiente etapa. Otros métodos de selección pueden ser consultados en (Beasley et al., 1993).

- **Recombinación:** El proceso de reproducción inicia con los individuos (padres) ganadores del torneo en el etapa anterior. Suponga que los ganadores fueron los individuos X_i y X_k .

$$\begin{aligned} X_i &= x_{i1} \ x_{i2} \ x_{i3} \ x_{i4} \ x_{i5} \ x_{i6} \ x_{i7} \\ X_k &= x_{k1} \ x_{k2} \ x_{k3} \ x_{k4} \ x_{k5} \ x_{k6} \ x_{k7} \end{aligned}$$

En la literatura existen diversos métodos de recombinación, los cuales han sido adaptados dependiendo del problema que se quiere solucionar (Spears, 2000). Para este caso, el método utilizado corresponde a la recombinación de un solo punto. Para esto, es necesario elegir una posición aleatoria en el rango $\{2, N_v - 1\}$. Suponga que la posición aleatoria corresponde a $pos = 5$. A partir de dicha posición, parte de los cromosomas de cada padre, darán origen a un nuevo descendiente (hijo) de la siguiente manera: Para las primeras posiciones del hijo, se toman los genes del primer padre, que se encuentran a la izquierda de pos inclusive, más los genes que están a la derecha de pos en el segundo padre, sin incluir pos , tal como se ilustra a continuación:

$$\begin{array}{rcccccccc} X_i = & x_{i1} & x_{i2} & x_{i3} & x_{i4} & x_{i5} & x_{i6} & x_{i7} \\ X_k = & x_{k1} & x_{k2} & x_{k3} & x_{k4} & x_{k5} & x_{k6} & x_{k7} \\ Y_m = & x_{i1} & x_{i2} & x_{i3} & x_{i4} & x_{i5} & x_{k6} & x_{k7} \end{array}$$

Observación 6.1. Note que realizando el proceso inverso descrito anteriormente, se obtiene un segundo descendientes. Sin embargo, en este libro solo se usará la generación de un descendiente.

- **Mutación:** El proceso de mutación, corresponde a una deformación parcial del cromosoma descendiente, la cual puede verse como el intercambio entre dos posiciones, tal como se presenta a continuación:

$$Y_m = x_{i1} \ x_{i2} \ x_{k7} \ x_{i4} \ x_{i5} \ x_{k6} \ x_{i3}$$

Otros métodos de mutación pueden incluir intercambio de un número mayor de genes, aumento o disminución del valor de una variable entre muchas otras formas de mutación, las cuales están ligadas al problema que se esté solucionando (Beasley et al.,

1993). Por ultimo, cabe resaltar que para el caso de variables binarias, la mutación está dada por el cambio de valor de una variable de cero a uno o viceversa.

$$Y_m = \begin{matrix} x_{i1} & x_{i2} & x_{i3} & x_{i4} & x_{i5} & x_{i6} & \overline{x_{i7}} \end{matrix}$$

Observación 6.2. El proceso de recombinación hace referencia a la intensificación del algoritmo, dado que en este proceso se conservan partes de soluciones ya existentes, es decir, se realiza una exploración de soluciones vecinas. Mientras que, la mutación, busca diversificar la búsqueda en el espacio de soluciones, dado que las deformaciones realizadas a los cromosomas permiten que el algoritmo explore espacios de solución en otros vecindarios.

6.2.1.4 Sustitución en la población

En la versión poblacional del algoritmo genético, los operadores genéticos son aplicados N_s veces en cada iteración, es decir, a través de dichos operadores se genera una población Y_k^t del mismo tamaño de la población X_k^t .

$$Y^t = \begin{bmatrix} y_{1,1} & y_{1,2} & \cdots & y_{1,j} & \cdots & y_{1,N_v} \\ y_{2,1} & y_{2,2} & \cdots & y_{2,j} & \cdots & y_{2,N_v} \\ \vdots & \vdots & \ddots & \vdots & \ddots & \vdots \\ y_{k,1} & y_{k,2} & \cdots & y_{k,j} & \cdots & y_{k,N_v} \\ \vdots & \vdots & \ddots & \vdots & \ddots & \vdots \\ y_{N_s,1} & y_{N_s,2} & \cdots & y_{N_s,j} & \cdots & y_{N_s,N_v} \end{bmatrix}, \quad (6.4)$$

Una vez se cuente con las dos poblaciones evaluadas por medio de la función de adaptación de cada individuo, las dos poblaciones se unen en

una sola, para generar una nueva población $X^{(t+1)'}$ tal como se presenta en la Ecuación (6.5).

$$X^{(t+1)'} = X^t \cup Y^t \quad (6.5)$$

Note que en este paso, el tamaño de la población $X^{(t+1)'}$ corresponde a $(2 \times N_s)$ individuos. tal como se observa en la Ecuación (6.6).

$$X^{(t+1)'} = \begin{bmatrix} x_{1,1} & x_{1,2} & \cdots & x_{1,j} & \cdots & x_{1,N_v} \\ x_{2,1} & x_{2,2} & \cdots & x_{2,j} & \cdots & x_{2,N_v} \\ \vdots & \vdots & \ddots & \vdots & \ddots & \vdots \\ x_{N_s,1} & x_{N_s,2} & \cdots & x_{N_s,j} & \cdots & x_{N_s,N_v} \\ y_{N_s+1,1} & y_{N_s+1,2} & \cdots & y_{N_s+1,j} & \cdots & y_{N_s+1,N_v} \\ y_{N_s+2,1} & y_{N_s+2,2} & \cdots & y_{N_s+2,j} & \cdots & y_{N_s+2,N_v} \\ \vdots & \vdots & \ddots & \vdots & \ddots & \vdots \\ y_{2 \times N_s,1} & y_{2 \times N_s,2} & \cdots & y_{2 \times N_s,j} & \cdots & y_{2 \times N_s,N_v} \end{bmatrix}, \quad (6.6)$$

La matriz de la Ecuación 6.6 se ordena de mejor a peor individuo con respecto del valor de función de adaptación, de tal manera que los primeros N_s individuos conformarán la población $X^{(t+1)}$, tal como se define en la Ecuación .

$$X^{t+1} = \left\{ x_k \in X^{(t+1)'} \wedge y_k \in X^{(t+1)'}, \forall k = 1, 2, 3 \dots N_s \right\} \quad (6.7)$$

Observación 6.3. Cabe resaltar que en cada ciclo iterativo del algoritmo genético, mediante los operadores genéticos, se generan N_s individuos nuevos y cada uno de ellos debe ser evaluado por medio de la función de adaptación, esto con el fin de determinar el nivel calidad de cada uno de ello y de esta manera, al ser ordenados e ingresados a la población actual, puedan hacer parte del siguiente ciclo generacional.

6.2.1.5 Criterio de parada

El criterios de parada empleado en este capítulo, mediante el cual se determina que proceso de exploración y explotación del espacio de soluciones se da por terminado, se realiza mediante un número máximo de iteraciones preestablecido por el programador $t_{\max}$.

6.2.1.6 Implementación computacional

El algoritmo 6 presenta de manera general, los pasos descritos en la Sección 6.2.1 para la implementación computacional del algoritmo genético poblacional.

Data: Lectura de parámetros del problema bajo análisis

Definir los valores de N_s , N_v , $t_{\max}$, n_p , ρ_r , ρ_m y d ;
 Crear la población inicial (X) de forma aleatoria acorde a los límites de las variables ($x^{\min}$ y $x^{\max}$);
 Evaluar la función objetivo o de adaptación para cada individuo de la población $f(x_k^0) \forall k = 1, 2, \dots, N_s$;
for $t = 1 : t_{\max}$ **do**
 for $k = 1 : N_s$ **do**
 $p \leftarrow$ aplicar operador de selección por torneo en la matriz X ;
 if $\text{rand}() \leq \rho_r$ **then**
 $Y_k \leftarrow$ aplicar operador de recombinación en p ;
 else
 $Y_k \leftarrow$ un individuo aleatorio del espacio de búsqueda;
 if $\text{rand}() \leq \rho_m$ **then**
 $Y_k \leftarrow$ aplicar operador de mutación al individuo Y_k ;
 $Y_k^t \leftarrow$ almacenar el individuo Y_k ;
 $X^{(t)'} \leftarrow$ Unir las poblaciones $X^t \cup Y^t$;
 Ordenar la matriz $X^{(t)'}$ de mejor a peor individuo;
 $X^{(t+1)} \leftarrow$ los primeros N_s individuos de $X^{(t)'}$;
end for

Result: Reportar la mejor solución encontrada, i.e., $X_1^{t_{\max}}$

Algoritmo 5: Algoritmo genético poblacional

Observación 6.4. Es preciso resaltar que, en este capítulo, los valores de las probabilidades de recombinación ρ_r y mutación ρ_m se toman del 100%, es decir, en cada ciclo generacional se aplica tanto el operador de recombinación como el de mutación.

6.2.1.7 Implementación en el software Julia

En esta sección se describe la aplicación del algoritmo genético poblacional al Problema de la Mochila definido en el modelo matemático (2.1)–(2.4). Para ilustrar la implementación del algoritmo se presentan los siguientes pasos secuenciales.

El primer paso consiste en el llamado a las librerías que son necesarias para la ejecución del algoritmo.

```
using Printf; LinearAlgebra; Random;
using DataFrames; Statistics;
```

Posteriormente, se definen los parámetros del algoritmo.

```
Nv = size(C,2); P = 1:Nv; Ns=100; rep = 100; tmax=50
```

Seguidamente, se procede a cargar los datos que representan el problema bajo análisis. Los datos usados son los mismos que se presentaron en la Sección 2.1.1 de este libro.

```
ks = DataFrames.DataFrame([
(236, 40, 67), (484, 45, 34), (500, 50, 83), (136, 25, 49),
(380, 28, 33), (299, 25, 75), (481, 50, 17), (419, 35, 34),
(419, 21, 92), (301, 30, 15), (226, 31, 62), (467, 41, 19),
(224, 29, 64), (202, 39, 45), (474, 21, 91), (241, 30, 34),
(287, 39, 85), (209, 21, 68), (319, 43, 23), (231, 50, 45),
(111, 47, 86), (182, 37, 60), (233, 45, 84), (339, 39, 60),
(412, 31, 92), (251, 45, 43), (460, 29, 19), (235, 21, 51),
(341, 43, 78), (439, 47, 13),])
DataFrames.rename!(ks, [:C, :V, :W])
```

```
C = ks.C'; V = ks.V'; W = ks.W';  
Vmax = 861; Wmax = 1215;
```

Una vez se cuenta con las librerías y los datos del problema, se realiza la codificación de cada una de las funciones necesarias para el desarrollo del algoritmo genético poblacional. Siendo la primera de ellas, la función de adaptación.

```
function fitnessf(C,V,W,Vmax,Wmax,x,P)  
    Penalty = 100;  
    Vol = sum(V[p]*x[p] for p in P);  
    Wei = sum(W[p]*x[p] for p in P);  
    Objval = sum(C[p]*x[p] for p in P) -  
    Penalty*(max(Vol - Vmax,0) + max(Wei - Wmax,0));  
    return Objval  
end
```

La función anterior, se diseñó acorde con la Ecuación (6.3), donde el factor de penalización usado corresponde a 100 unidades, el cual multiplica las unidades de peso y volumen que están por encima de los límites máximos cuando un individuo viola dichas restricciones.

Las tres siguientes funciones, corresponden a los operadores genéticos, siendo el primero de ellos, el proceso de selección. El cual permite determinar las posiciones de los individuos de la población que fueron elegidos, usando el método de torneo, para pasar a la siguiente etapa.

```
function seleccion(Nv,Ns,x,np)  
    ganadores=[]  
    padres1= rand(1:Ns,np)  
    for i in 1:2:length(padres1)  
        if x[padres1[i],Nv+1] >= x[padres1[i+1],Nv+1]  
            push!(ganadores, padres1[i])  
        else  
            push!(ganadores, padres1[i+1])  
        end  
    end  
    return ganadores  
end
```

Con los individuos ganadores (padres), se continúa con la ejecución del proceso de recombinación de un único punto, el cual se explicó en la Sección 6.2.1.3. Esta función devuelve un descendiente nuevo.

```
function recombinacion(Nv,p,padres,C,W,Wmax,V,Vmax)
    padre1=p[padres[1],:]
    padre2=p[padres[2],:]
    corte=rand(2:Nv-1)
    hijo1=zeros(Float64,1,Nv+1)
    hijo1[1:corte]=padre1[1:corte]
    hijo1[corte+1:Nv]=padre2[corte+1:Nv]
    hijo1[Nv+1]=fitnessf(C,V,W,Vmax,Wmax,hijo1[1:Nv],P)
    return hijo1
end
```

Para finalizar los operadores genéticos, se presenta la función de mutación. La cual es la encargada de deformar el nuevo individuo en un único gen.

```
function mutacion(hijo,Nv,C,W,Wmax,V,Vmax)
    genmutado=rand(1:Nv)
    if hijo[genmutado]==0
        hijo[genmutado]=1
    else
        hijo[genmutado]=0
    end
    hijo[Nv+1]=fitnessf(C,V,W,Vmax,Wmax,hijo[1:Nv],P)
    return hijo
end
```

Observación 6.5. Note que las probabilidades de recombinación ρ_r y mutación ρ_m , no se definieron, lo cual indica que en esta implementación se considera que siempre se deben realizar estos dos operadores, es decir cada una tiene una probabilidad del 100%.

La siguiente parte, corresponde a la implementación del ciclo iterativo del algoritmo genético poblacional.

```
global Guardar = zeros(rep,Nv+1);
for r=1:rep
    global x=round.(rand(Float64, (Ns, Nv+1)))
    for i=1:Ns
        x[i,Nv+1]=fitnessf(C,V,W,Vmax,Wmax,x[i,1:Nv],P)
    end
    x=x[sortperm(x[:, Nv+1]), :]
    global np=4
    for i=1:tmax
        y=zeros(Ns,Nv+1)
        for j=1:Ns
            global padres=seleccion(Nv,Ns,x,np)
            Hijor=recombinacion(Nv,x,padres,C,W,Wmax,V,Vmax)
            global Hijom=mutacion(Hijor,Nv,C,W,Wmax,V,Vmax)
            y[j,:]=Hijom
        end
        x2=[x;y]
        x2=x2[sortperm(x2[:, Nv+1]), :]
        global x=x2[Ns+1:end,1:end]
    end
    global Guardar[r,:]=x[Ns,1:Nv+1];
end
```

La población inicial es creada de manera aleatoria, para lo cual se usan las funciones (rand()), y (round()), con lo cual se garantiza que cada gen tome el valor de cero o uno. Cada uno de los individuos es evaluado con el llamado a la función (fitnessf()). Posteriormente, cada una de las funciones de los operadores genéticos son llamadas en su orden (selección, recombinación y mutación). La ultima fase del algoritmo, corresponde a la actualización de la población. Este proceso iterativo se realiza (tmax) veces.

Note que, cada vez que el algoritmo se ejecuta (tmax) veces, la mejor solución encontrada se almacena en la variable (Guardar). Este proceso se realiza un número rep veces, el cual está definido en 100. Este proceso de repeticiones se realiza con el fin de realizar un análisis

estadístico de la solución encontrada por el algoritmo. Con lo cual es posible obtener el valor máximo, mínimo y promedio de la función adaptación. Así como la desviación estándar y la diferencia entre la mejor solución y la promedió. Finalmente, se calcula el tiempo promedio de ejecución del algoritmo empleando el reloj del computador con la función `time()`. El código usado se muestra a continuación:

```
global Guardar = Guardar[sortperm(Guardar[:, Nv+1]), :];
elapsed_time = time() - t1;
@printf("Valor maximo: %2.1f\n", Guardar[rep, Nv+1]);
@printf("Valor minimo: %2.1f\n", Guardar[1, Nv+1]);
@printf("Valor promedio: %2.1f\n", mean(Guardar[:, Nv+1]));
@printf("Desviacion estandar: %2.1f\n", std(Guardar[:, Nv+1]));
@printf("Maximo - Media: %2.1f\n", Guardar[rep, Nv+1] - mean(Guardar[:, Nv+1]));
println("Tiempo de ejecucion (promedio): ",
        round(elapsed_time/rep, digits = 4), " s");
println("xbest: ", Int.(Guardar[rep, 1:Nv])');
```

Finalmente, una vez se realiza la ejecución del algoritmo genético poblacional con sus correspondientes repeticiones, se obtienen los resultados que se muestran a continuación.

```
Valor maximo: 8416.0;
Valor minimo: 8365.0;
Valor promedio: 8411.4;
Desviacion estandar: 11.6;
Maximo - Media: 4.6;
Tiempo de ejecucion (promedio): 0.0224 s;
xbest: [0 1 1 0 1 1 1 1 1 1 1 0 1 1
        1 1 1 1 1 0 0 0 1 1 1 1 1 1];
```

Los resultados obtenidos muestran que: (i) la mejor solución encontrada en el proceso iterativo, i.e., 8416.0 corresponde al valor reportado por la metodología de solución exacta con el solver `HiGHS` para proble-

mas de programación lineal entera estudiado en el Capítulo 2; (ii) los valores mínimo y promedio obtenidos en esta evaluación fueron 8365.0 y 8411.4, con una desviación estándar en valor real de 11.6, y (iii) en el tiempo promedio de ejecución del algoritmo fue de 0.0224 s.

Observación 6.6. Los resultados obtenidos en esta simulación son únicamente con fines ilustrativos, ya que estos dependen del computador en el que sea ejecutado el algoritmo.

6.2.2 Algoritmo Genético de Chu-Beasley

Para esta versión del algoritmo genético, se usan los mismos parámetros descritos en la Sección 6.2.1 y se siguen los pasos descritos la Sección 6.2.1.1, la Sección 6.2.1.2 y la Sección 6.2.1.3, es decir, se siguen las mismas etapas del algoritmo genético poblacional para la creación de la población inicial y la implementación de los operadores genéticos. El algoritmo cambia su esencia en la sustitución en la población. La cual se describe a continuación.

6.2.2.1 Sustitución en la población

Después de generar el nuevo individuo Y_k^t por medio de los operadores genéticos, este individuo es evaluado por medio de la función de adaptación, y para que este nuevo individuo pase a ser parte de la población X^t , se deben cumplir las siguientes dos reglas:

- El nuevo individuo Y_k^t es mejor que el peor individuo $X_k^t \in X^t$.
- El nuevo individuo cumple con un criterio de diversidad fijado por el programador. Es decir, es diferente al menos a un número mínimo de individuos de la población de padres.

Si el individuo Y_k^t , cumple con las dos condiciones descritas anteriormente, entonces el peor individuo de la población X^t , es eliminado y en su reemplazo ingresa Y_k^t . En este punto, la nueva matriz X^t , es ordenada nuevamente de manera ascendente con respecto al valor de función de adaptación.

Observación 6.7. El grado de diversidad div utilizado en este algoritmo corresponde a 3, es decir se puede tener en la población un máximo de 3 individuos idénticos (trillizos), esto debido a que un número mayor de individuos idénticos en la población, no permite que se explore de manera adecuada el espacio de solución y el algoritmo se queda atrapado en soluciones locales de manera temprana.

6.2.2.2 Criterio de parada

El criterios de parada empleado para el algoritmo genético de Chu-Beasley, es el mismo que se utiliza en la versión poblacional, el cual se realiza mediante un número máximo de iteraciones preestablecido por el programador $t_{\max}$.

6.2.2.3 Implementación computacional

El algoritmo 6 presenta de manera general, los pasos que permiten la implementación computacional del algoritmo genético de Chu-Beasley.

Data: Lectura de parámetros del problema bajo análisis

Definir los valores de N_s , N_v , $t_{\max}$, n_p y ρ_r, ρ_m , d y div ;

Crear la población inicial (X) de forma aleatoria acorde a los límites de las variables ($x^{\min}$ y $x^{\max}$);

Evaluar la función objetivo o de adaptación para cada individuo de la población $f(x_k^0) \forall k = 1, 2, \dots, N_s$;

for $t = 1 : t_{\max}$ **do**

$p \leftarrow$ aplicar operador de selección por torneo en la matriz X^t ;

if $\text{rand}() \leq \rho_r$ **then**

$Y_k \leftarrow$ aplicar operador de recombinación en p ;

else

$Y_k \leftarrow$ un individuo aleatorio del espacio de búsqueda;

if $\text{rand}() \leq \rho_m$ **then**

$Y_k \leftarrow$ aplicar operador de mutación al individuo Y_k ;

if Y_k es mejor que el peor individuo de X^t && $div \leq 3$ **then**

$X^t[N_s, :] \leftarrow Y_k$;

Result: Reportar la mejor solución encontrada, i.e., $X_1^{t_{\max}}$

Algoritmo 6: Algoritmo genético de Chu-Beasley

6.2.2.4 Implementación en el software Julia

En esta sección se describe la aplicación del algoritmo genético de Chu-Beasley al Problema de la Mochila. El modelo matemático del problema se define mediante las ecuaciones(2.1)–(2.4). Para ilustrar la implementación del algoritmo se presentan los siguientes pasos secuenciales.

Igual que en la sección anterior, el primer paso corresponde a el llamado de las librerías que son necesarias para la ejecución del algoritmo.

```
using Printf; LinearAlgebra; Random;
using DataFrames; Statistics;
```

Seguidamente, se procede a cargar los datos que representan el problema de mochila con restricciones de peso y volumen. Los datos

usados son los mismos que se presentaron en la Sección 2.1.1 de este documento.

```
ks = DataFrames.DataFrame([
(236, 40, 67), (484, 45, 34), (500, 50, 83), (136, 25, 49),
(380, 28, 33), (299, 25, 75), (481, 50, 17), (419, 35, 34),
(419, 21, 92), (301, 30, 15), (226, 31, 62), (467, 41, 19),
(224, 29, 64), (202, 39, 45), (474, 21, 91), (241, 30, 34),
(287, 39, 85), (209, 21, 68), (319, 43, 23), (231, 50, 45),
(111, 47, 86), (182, 37, 60), (233, 45, 84), (339, 39, 60),
(412, 31, 92), (251, 45, 43), (460, 29, 19), (235, 21, 51),
(341, 43, 78), (439, 47, 13),])
DataFrames.rename!(ks, [:C, :V, :W])
C = ks.C'; V = ks.V'; W = ks.W';
Vmax = 861; Wmax = 1215;
```

La siguiente codificación corresponde a las funciones necesarias para el desarrollo del algoritmo genético de Chu-Beasley. Siendo la primera de ellas, la función de adaptación.

```
function fitnessf(C,V,W,Vmax,Wmax,x,P)
    Penalty = 100;
    Vol = sum(V[p]*x[p] for p in P);
    Wei = sum(W[p]*x[p] for p in P);
    Objval = sum(C[p]*x[p] for p in P) -
    Penalty*(max(Vol - Vmax,0) + max(Wei - Wmax,0));
    return Objval
end
```

La función anterior, se diseñó acorde con la Ecuación (6.3). Esta función de adaptación, permite contabilizar y penalizar el grado de violación de las restricciones del problema, ya sea por peso o por volumen.

Seguidamente, se procede a programar los operadores genéticos, los cuales inician con el proceso de selección. En este algoritmo se usa la selección por torneo, el cual consiste en seleccionar, de manera aleatoria, dos parejas de individuos y con cada una de ellas se realiza un torneo acorde al valor de función de adaptación. Los ganadores son los padres del descendiente y pasan a la siguiente etapa.

```
function seleccion(Nv,Ns,x,np)
    ganadores=[]
    padres1= rand(1:Ns,np)
    for i in 1:2:length(padres1)
        if x[padres1[i],Nv+1] >= x[padres1[i+1],Nv+1]
            push!(ganadores, padres1[i])
        else
            push!(ganadores, padres1[i+1])
        end
    end
    return ganadores
end
```

Una vez se cuenta con los individuos ganadores (padres), se pasa al proceso de recombinación de un único punto, el cual se explicó en la Sección 6.2.1.3. Esta función devuelve un descendiente nuevo que tiene información genética de los dos ganadores del torneo .

```
function recombinacion(Nv,p,padres,C,W,Wmax,V,Vmax)
    padre1=p[padres[1],:]
    padre2=p[padres[2],:]
    corte=rand(2:Nv-1)
    hijo1=zeros(Float64,1,Nv+1)
    hijo1[1:corte]=padre1[1:corte]
    hijo1[corte+1:Nv]=padre2[corte+1:Nv]
    hijo1[Nv+1]=fitnessf(C,V,W,Vmax,Wmax,hijo1[1:Nv],P)
    return hijo1
end
```

La siguiente, es la función de mutación, la cual es la encargada de deformar el individuo obtenido en el proceso de recombinación. Cabe recordar que la mutación utilizada en este libro, solo cambia un gen del individuo descendiente.

```
function mutacion(hijo,Nv,C,W,Wmax,V,Vmax)
    genmutado=rand(1:Nv)
    if hijo[genmutado]==0
        hijo[genmutado]=1
    else
```

```
        hijo[genmutado]=0
    end
    hijo[Nv+1]=fitnessf(C,V,W,Vmax,Wmax,hijo[1:n],P)
    return hijo
end
```

Antes de iniciar el ciclo iterativo del algoritmo genético de Chu-Beasley, se procede a definir los diferentes parámetros del algoritmo,

```
Nv = size(C,2); P = 1:Nv; Ns=50;
rep = 100; tmax=1000; div=3; np=4;
```

Observación 6.8. Nuevamente, las probabilidades de recombinación ρ_r y mutación ρ_m , no se definieron, lo cual indica que para el algoritmo genético de Chu-Beasley implementado, se considera que siempre se deben realizar estos dos operadores, es decir cada uno tiene una probabilidad del 100%.

El código diseñado para el algoritmo genético de Chu-Beasley, permite en primera medida inicializar la población inicial de manera aleatoria, mediante una codificación binaria. La población inicial es evaluada por medio de la función de adaptación denominada `fitnessf()`. Seguidamente el proceso continua con el llamado de las funciones de los operadores genéticos. Es importante resaltar que, en cada ciclo iterativo, a diferencia del algoritmo genético poblacional, la variante de Chu-Beasley, únicamente genera un individuo, el cual tiene la posibilidad de ingresar a la población. para lo cual, desde el algoritmo se revisa si el descendiente es de buena calidad y su grado de diversidad es menor a 3.

```
global Guardar = zeros(rep,Nv+1);
for r=1:rep
```

MÉTODOS DE OPTIMIZACIÓN METAHEURÍSTICA PARA INGENIERÍA: Con aplicaciones en el software Julia

```
global x=round.(rand(Float64, (Ns, Nv+1)));
for i=1:Ns
    x[i,Nv+1]=fitnessf(C,V,W,Vmax,Wmax,x[i,1:Nv],P);
end
x=x[sortperm(x[:, Nv+1]), :];
for i=1:tmax
    y=zeros(1,Nv+1);
    global padres=seleccion(Nv,Ns,x,np);
    Hijor=recombinacion(Nv,x,padres,C,W,Wmax,V,Vmax);
    global y=mutacion(Hijor,Nv,C,W,Wmax,V,Vmax);
    if x[1,Nv+1]<y[Nv+1]
        global cont=0;
        for j=1:Ns
            if x[j,1:Nv]==y[1:Nv];
                cont=cont+1;
            end
        end
        if cont<div
            global x[1,1:end]=y[1:end];
            global x=x[sortperm(x[:, Nv+1]), :];
        end
    end
end
global Guardar[r,:] = x[Ns,1:Nv+1];
end
```

Note que el algoritmo con sus ciclos generacionales, se repiten un cantidad de $rep=100$ y en cada repetición, el mejor individuo de ciclo iterativo, es almacenado en la matriz Guardar; esto con el fin de realizar un análisis estadístico del desempeño del algoritmo genético de Chu-Beasley aplicado al problema de la mochila.

Para poder realizar el análisis estadístico, el siguiente código fue implementado.

```
global Guardar = Guardar[sortperm(Guardar[:, Nv+1]), :];
elapsed_time = time() - t1;
@printf("Valor maximo: %2.1f\n", Guardar[rep,Nv+1]);
@printf("Valor minimo: %2.1f\n", Guardar[rep,Nv+1]);
```

```
\n", Guardar[1, Nv+1]);  
@printf("Valor promedio: %2.1f  
\n", mean(Guardar[:, Nv+1]));  
@printf("Desviacion estandar: %2.1f  
\n", std(Guardar[:, Nv+1]));  
@printf("Maximo - Media: %2.1f  
\n", Guardar[rep, Nv+1]-mean(Guardar[:, Nv+1]));  
println("Tiempo de ejecucion (promedio): ",  
        round(elapsed_time/rep, digits = 4), " s");  
println("xbest: ", Int.(Guardar[rep, 1:Nv]));
```

Después de realizar el total de repeticiones sobre el ciclo iterativo del algoritmo genético de Chu-Beasley, los datos que se obtuvieron, se presentan a continuación:

```
Valor maximo: 8416.0  
Valor minimo: 8197.0  
Valor promedio: 8347.5  
Desviacion estandar: 51.6  
Maximo - Media: 68.5  
Tiempo de ejecucion (promedio): 0.0104 s  
xbest: [0 1 1 0 1 1 1 1 1 1 1 0 1 1  
        1 1 1 1 1 0 0 0 1 1 1 1 1 1];
```

Los resultados obtenidos muestran que: (i) la mejor solución encontrada en el proceso iterativo, i.e., 8416.0 corresponde al valor reportado por la metodología de solución exacta con el solver `HIGHS` para problemas de programación lineal entera estudiado en el Capítulo 2; (ii) los valores mínimo y promedio obtenidos en esta evaluación fueron 8197.0 y 8347.5, con una desviación estándar en valor real de 51.6, y (iii) en el tiempo promedio de ejecución del algoritmo fue de 0.0104 s.

Observación 6.9. Los resultados obtenidos en esta simulación son únicamente con fines ilustrativos, ya que estos dependen del computador en el que sea ejecutado el algoritmo.

Parte II

Técnicas metaheurísticas en dominio continuo

7

Algoritmo de optimización de búsqueda de cuervos

En este capítulo se describe, analiza e implementa el algoritmo de optimización de búsqueda de cuervos, el cual es un algoritmo basado en el comportamiento inteligente de los cuervos. Este método de optimización ha sido diseñado para abordar problemas de diferente naturaleza (continuo, entero y binario). Además, presenta un adecuado balance entre diversificación e intensificación, así como la capacidad de realizar búsquedas locales o globales acorde a la naturaleza propia del problema abordado. Presenta como principal ventaja el contar con solo dos parámetros de optimización que requieren ser sintonizados. A continuación, se presenta la lógica del algoritmo de búsqueda de cuervos para resolver el problema despacho económico de centrales de generación térmica considerando pérdidas bajo el entorno de programación de Julia.

7.1 Filosofía e inspiración

El algoritmo de búsqueda de cuervos, pertenece a la familia de las técnicas metaheurísticas basadas en población, cuya filosofía hace referencia a el hecho de que los cuervos almacenas sus excesos de comida en un escondite determinado y una vez es requerido, nuevamente vuelven a dichos escondites por la comida que estaba almacenada para alimentarse. Bajo este concepto, entonces se asume que los cuervo tienen la capacidad de almacenar en su memoria el lugar de escondite. Así mismo, se considera que un cuervo puede seguir a otro semejante con el objetivo de descubrir su escondite y robar su alimento. Lo anterior, se asemeja a uno de los comportamientos inteligentes presentado por los cuervos y que han sido reportados en la literatura (Prior et al., 2008). Inicialmente, el algoritmo fue propuesto por (Askarzadeh, 2016b) para resolver seis problemas de ingeniería con funciones objetivo, restricciones y variables de decisión de diferente naturaleza. Acorde con la inspiración natural de este algoritmo, se resaltan las siguientes ventajas que presentan su implementación.

- i. La evolución en la búsqueda de una solución de buena calidad inicia a partir de una población de individuos y su respectiva memoria del escondite de la comida sobrante. Ambas son creadas de manera aleatoria.
- ii. Solo se requiere el ajuste o sintonización de dos parámetros de optimización.
- iii. Presenta un mecanismo de diversificación e intensificación que le permite explorar el espacio de solución de manera local o global.
- iv. En cada iteración, la población es reemplazada acorde a criterios de avance, sin embargo, los individuos de mejor calidad, en cada ciclo generacional son almacenados en la memoria.

Un número creciente de aplicaciones del algoritmo de búsqueda de cuervos para solucionar diferentes problemas de ingeniería, han sido reportados en la literatura especializada, entre los que se destacan los problemas de clasificación (Abdallh and Algamal, 2020), selección de características (De Souza et al., 2018), problema de embalaje de contenedores bidimensionales (Laabadi et al., 2020), problemas de coloración de gráficos (Meraihi et al., 2020), sistemas híbridos de energía (Makhdoomi and Askarzadeh, 2020), localización óptima de compensación reactiva (Cuevas et al., 2020), localización óptima de generadores distribuidos (Sannigrahi and Acharjee, 2018), preservación de datos médicos (Mandala and Rao, 2019), entre otras aplicaciones que pueden ser consultadas en el trabajo de revisión realizado por (Meraihi et al., 2021).

7.2 Algoritmo de búsqueda de cuervos

En esta sección se presenta cada una de las etapas que se deben desarrollar para llevar a cabo la implementación del algoritmo de búsqueda de cuervos para problemas de optimización en ingeniería y ciencia. En este libro se seguirán cada uno de los pasos propuestos en la versión original del algoritmo presentada por (Askarzadeh, 2016b).

7.2.1 Inicialización de parámetros

Partiendo de la definición problema de optimización, la naturaleza de las variables y el conjunto de restricciones, el primer paso del algoritmo consiste en definir los parámetros ajustables. Para esto teniendo en cuenta que el algoritmo está basado en la forma como los cuervos protegen su comida, es necesario asumir que estos viven en bandadas (población). Es por esto que, el primer parámetro a definir es el tamaño de la población, el cual se denota como N_s . Ahora bien, dada la naturaleza evolutiva de los algoritmos poblacionales, se procede a definir el número máximo de ciclos generacionales ($t_{\max}$). El tercer parámetro corresponde al número

de variables (N_v), sin embargo, es importante notar que este parámetro no es ajustable, siendo este dado por las características propias del problema abordado.

Nótese que los tres parámetros descritos anteriormente, son comunes en las técnicas de optimización basadas en poblaciones. Para el caso del algoritmo de búsqueda de cuervos, es necesario definir y ajustar dos parámetros adicionales que deben ser sintonizados para lograr un buen desarrollo del algoritmo; el primero de ellos corresponde al tamaño del vuelo de cada cuervo denominado fl , el cual puede tomar cualquier valor positivo $fl \geq 0$, sin embargo, se debe tener en cuenta que para realizar búsquedas locales este parámetro deberá ser sintonizado con valores en el rango $0 \leq fl \leq 1$. Para problemas donde se requiera de una exploración del espacio de soluciones de manera global, el parámetro deberá ser ajustado a un valor $fl > 1$. El segundo parámetro que debe ser ajustado, corresponde a la probabilidad de conciencia (AP), el cual está en un rango entre $0 \leq AP \leq 1$. Este parámetro garantiza el balance entre diversificación e intensificación, de esta manera, para valores pequeños (ceranos a cero) se aumenta la intensificación, mientras que para valores grandes (ceranos a uno) se incrementa la diversificación.

7.2.2 Población y memoria inicial

La población inicial corresponde a un arreglo matricial de dimensiones $N_s \times N_v$, la cual se genera de manera aleatoria antes de inicializar el ciclo evolutivo del algoritmo de búsqueda de cuervos, es decir, cuando el contador de iteraciones es $t = 0$. La representación de la población inicial X^t , está dada por la Ecuación (7.1), donde cada fila representa la posición actual de cada uno de los individuos (cuervos) que conforman la población y las columnas hacen referencia a cada una de las variables

que conforman el individuo.

$$X^t = \begin{bmatrix} x_{1,1} & x_{1,2} & \cdots & x_{1,j} & \cdots & x_{1,N_v} \\ x_{2,1} & x_{2,2} & \cdots & x_{2,j} & \cdots & x_{2,N_v} \\ \vdots & \vdots & \ddots & \vdots & \ddots & \vdots \\ x_{k,1} & x_{k,2} & \cdots & x_{k,j} & \cdots & x_{k,N_v} \\ \vdots & \vdots & \ddots & \vdots & \ddots & \vdots \\ x_{N_s,1} & x_{N_s,2} & \cdots & x_{N_s,j} & \cdots & x_{N_s,N_v} \end{bmatrix}, \quad (7.1)$$

En esta matriz $x_{k,j}$ corresponde al valor asignado a la j -ésima variable del k -ésimo individuo. Este valor debe garantizar la factibilidad de la j -ésima variable del problema. Es decir, el valor debe ser asignado entre el límite mínimo y máximo asociados a dicha variable. La población inicial se genera empleando la Ecuación (7.2).

$$x_{k,j} = x_j^{\min} + \alpha_j \left(x_j^{\max} - x_j^{\min} \right), \{j = 1, 2, \dots, N_v, k = 1, 2, \dots, N_s\} \quad (7.2)$$

siendo α_j un valor aleatorio en el rango de cero a uno con distribución normal, y $x_j^{\min}$ y $x_j^{\max}$ representan los límites inferiores y superiores aplicables a la j -ésima variable.

De manera similar, la memoria de cada individuo debe ser inicializada. En este punto, los cuervos no tienen experiencia en esconder su alimento ni en encontrar el alimento de otro cuervo, razón por la cual, la memoria m^t , debe ser inicializada de manera aleatoria bajo las mismas condiciones descritas para la población inicial. La Ecuación (7.3), representa la posición de escondite de cada uno de los cuervos. Desde el punto de vista del algoritmo, en cada ciclo iterativo, esta matriz

de dimensiones $N_s \times N_v$, alojará los mejores individuos encontrados.

$$m^t = \begin{bmatrix} m_{1,1} & x_{1,2} & \cdots & m_{1,j} & \cdots & m_{1,N_v} \\ m_{2,1} & x_{2,2} & \cdots & m_{2,j} & \cdots & m_{2,N_v} \\ \vdots & \vdots & \ddots & \vdots & \ddots & \vdots \\ m_{k,1} & x_{k,2} & \cdots & m_{k,j} & \cdots & m_{k,N_v} \\ \vdots & \vdots & \ddots & \vdots & \ddots & \vdots \\ m_{N_s,1} & x_{N_s,2} & \cdots & m_{N_s,j} & \cdots & m_{N_s,N_v} \end{bmatrix}, \quad (7.3)$$

Observación 7.1. En el caso de problemas de optimización de naturaleza discreta, la generación de las variables de decisión $x_{k,j}$ y $m_{k,j}$ son redondeadas al valor entero más cercano. Esto garantizará que cada una de las variables se encuentre dentro del límite mínimo y máximo.

7.2.3 Reglas de avance

Una vez se cuente con la población inicial y la memoria de cada individuo con su respectiva valuación, se procede a iniciar el ciclo iterativo. Para esto, suponga que un cuervo que esta en la fila k , i.e., x_k^t , elige, de manera aleatoria, seguir al cuervo que está ubicado en la posición j , i.e., x_j^t , con el objetivo de descubrir el lugar donde este último escondió su alimento. En este punto, es posible que suceda una de las dos cosas que se describen a continuación.

1. El cuervo x_j no sabe que el cuervo x_k lo está persiguiendo. Bajo este supuesto, el cuervo x_k se apoderará del alimento del cuervo x_j y por lo tanto su posición cambiará. Tal como se describe en la Ecuación (7.4).

$$x_k^{t+1} = x_k^t + r_k \cdot fl_k^t \cdot (m_j^t - x_k^t) \quad (7.4)$$

donde r_k es un número aleatorio con distribución uniforme entre 0 y 1, y fl_k^t corresponde al tamaño del vuelo del cuervo k en a iteración t , tal como se describió en la Sección 7.2.

Observación 7.2. Note que por medio de este supuesto, el algoritmo intensifica la búsqueda en el espacio de solución.

2. El cuervo x_j^t sabe que el cuervo x_k^t lo está persiguiendo. El cuervo x_j^t volará a una posición aleatoria. Es decir, guiará al cuervo x_k^t a una nueva posición lejos del escondite de su alimento.

Observación 7.3. Note que bajo este criterio, es como el algoritmo diversifica la búsqueda en el espacio de soluciones.

De manera general, la forma como el algoritmo avanza en el espacio de solución, puede describirse por medio de la Ecuación (7.5).

$$x_k^{t+1} = \begin{cases} x_k^t + r_i \cdot fl_k^t \cdot (m_j^t - x_k^t) & r_j \geq AP_i^t \\ \text{Una posición aleatoria} & r_j < AP_i^t \end{cases}, \{ \forall k = 1, 2, \dots, N_s \}. \quad (7.5)$$

Observación 7.4. En este punto, es importante que el lector tenga presente que en cada iteración del algoritmo, todos los individuos de la población x_k^t , serán reemplazados bajo uno de los criterios de la Ecuación (7.5), esto sin importar el valor de su función objetivo.

El proceso de avance del algoritmo hacia regiones prometedoras (mejores soluciones), se complementa con el almacenamiento de los mejores individuos en la memoria m . En este punto, una solución x_k^{t+1} es almacenada en la memoria m_j^{t+1} únicamente en el caso que $f(x_k^{t+1})$ sea mejor que $f(m_j^t)$ tal como se presenta en la Ecuación (7.6)

$$m_j^{t+1} = \begin{cases} x_k^{t+1} & f(x_k^{t+1}) \text{ es mejor que } f(m_j^t) \\ m_j^t & \text{En otro caso} \end{cases} \quad (7.6)$$

7.2.4 Criterio de parada

En este libro se conserva el criterio de parada que fue presentado originalmente para el algoritmo de CSA en el trabajo publicado por (Askarzadeh, 2016b), el cual indica que, el proceso de exploración y explotación del espacio de soluciones se da por terminado cuando se alcanza el número máximo de iteraciones preestablecido por el programador $t_{\max}$.

Observación 7.5. Después de que el algoritmo finalice la totalidad de ciclos iterativos, la matriz m deberá ser ordenada ascendentemente con respecto al valor de función objetivo o función de adaptación de cada individuo. Si se está tratando con un problema de maximización, la mejor solución quedará almacenada en la última fila de la matriz. Pero si por el contrario, se está tratando con un problema de minimización, el mejor individuo quedará localizado en la primera fila.

7.2.5 Implementación computacional

El algoritmo 7 presenta de manera general, los pasos descritos desde la Sección 7.2.1 a la Sección 7.2.4 para la implementación computacional del algoritmo de optimización búsqueda de cuervos.

Data: Lectura de parámetros del problema bajo análisis

Definir los límites de las variables que componen el problema, i.e., $x^{\min}$ y $x^{\max}$;

Definir los valores de N_s , N_v , $t_{\max}$, fl y AP ;

Crear la población inicial (x^0) de forma aleatoria acorde a los límites de las variables ($x^{\min}$ y $x^{\max}$);

Crear la memoria inicial (m^0) de forma aleatoria acorde a los límites de las variables ($x^{\min}$ y $x^{\max}$);

Evaluar la función objetivo o de adaptación para cada individuo de la población $f(x_k^0) \forall k = 1, 2, \dots, N_s$;

Evaluar la función objetivo o de adaptación para cada individuo en la memoria $f(m_k^0) \forall k = 1, 2, \dots, N_s$;

for $t = 1 : t_{\max}$ **do**

for $k = 1 : N_s$ **do**

 Elegir de manera aleatoria un cuervo a seguir, i.e., x_j ;

if $r_j \geq AP$ **then**

$x_k^{t+1} = x_k^t + r_k * fl_k^t * (m_j^t - x_k^t)$;

else

x_k^{t+1} = posición aleatoria del espacio de búsqueda;

if $f(x_k^{t+1})$ es mejor que $f(m_j^t)$ **then**

$m_j^{t+1} = x_k^{t+1}$

else

$m_j^{t+1} = m_j^t$;

Ordenar la matriz m en orden ascendente;

Result: Reportar la mejor solución encontrada, i.e., $m_1^{t_{\max}}$

Algoritmo 7: Algoritmo de optimización búsqueda de cuervos

7.3 Implementación en el software Julia

En esta sección se presenta la implementación del algoritmo de optimización de búsqueda cuervos aplicado al problema de despacho económico de centrales térmicas multiperíodo, el cual fue presentado en la Sección 2.2 y definido mediante las ecuaciones (2.9)–(2.11).

Como primera medida, es necesario hacer el llamado de las librerías que serán necesarias para la ejecución del algoritmo, cuyos paquetes deben ser instalados previamente, tal como se describió en la Sección 2.4. Las librerías utilizadas en este capítulo, son las que se presentan a continuación.

```
using DataFrames, LinearAlgebra, Random;  
using Printf, Statistics, BenchmarkTools;
```

El siguiente paso en la secuencia del algoritmo, consiste en cargar los datos del problema. En este caso corresponde a la matriz de coeficientes de pérdidas, coeficientes de costo, límites mínimo y máximo de cada generador y las demandas de potencia activa en cada periodo.

```
B = [0.00013630    0.00006750    0.00007839  
      0.00006750    0.00015450    0.00009828  
      0.00007839    0.00009828    0.00016140];  
Generadores = DataFrames.DataFrame([  
  (1, 0.006085, 10.04025, 136.9125, 5, 150),  
  (2, 0.005915, 9.760576, 59.15500, 15, 100),  
  (3, 0.005250, 8.662500, 328.1250, 50, 250),]);  
DataFrames.rename!(Generadores,  
  [:g,:a,:b,:c,:pmin,:pmax]);  
a = Generadores.a; b = Generadores.b; c = Generadores.c;  
pmin = Generadores.pmin; pmax = Generadores.pmax;  
Periodos = DataFrames.DataFrame([  
  (1, 210), (2, 230), (3, 200), (4, 180),  
  (5, 240), (6, 180), (7, 150), (8, 100),  
  (9, 80 ), (10,130), (11,190), (12,280),  
  (13,310), (14,320), (15,350), (16,380),  
  (17,400), (18,420), (19,450), (20,460),  
  (21,470), (22,420), (23,350), (24,220),]);
```

```
DataFrames.rename!(Periodos, [:h, :PD]);  
PD = Periodos.PD;
```

Seguidamente, se procede a determinar el valor de los parámetros y variables, tanto del problema como aquellos que son propios del algoritmo, tal como número de períodos h , cantidad de generadores g , número de variables N_v , número de individuos en la población N_s , número máximo de iteraciones t_{max} entre otros.

```
h=size(Periodos.h,1);  
g = size(Generadores,1);  
H = 1:h;  
G = 1:g;  
alpha=1;  
Nv=g*h;  
Ns=1000;  
rep=100;  
global tmax=5000;  
global fl=3.5;  
global AP= 0.05;
```

Del cuadro anterior es preciso resaltar que, los valores de fl y AP , fueron hallados de manera heurística, y los mismos deben ser sintonizados para cada problema que se quiera solucionar, incluso para el mismo problema, estos pueden variar en relación con la cantidad de variables que se consideren.

Una vez se cuenta con los datos y los parámetros calculados en el entorno de Julia, se procede a implementar el proceso iterativo del algoritmo de búsqueda de cuervos. Para este caso, se desarrollaron tres funciones. La primera consiste en la función de adaptación, mediante la cual se realizará la evaluación de cada uno de los individuos de la población x y la memoria m .

```
function fitnessf(a,b,c,G,H,pg,PD,alpha,B)  
    Penalty = 35;  
    Penalization = 0;  
    for h in H  
        Pgen = sum(pg[k,h] for k in G)
```

```

Pdl = PD[h] +
      alpha*sum(sum(pg[k,h]*B[k,m]*pg[m,h]
      for m in G) for k in G)
if Pgen < Pdl
    Penalization = Penalization +
                  abs(Penalty*(Pgen - Pdl))+Penalty;
end
end
Objval = sum(sum(a[k]*pg[k,h]^2+b[k]*pg[k,h]+c[k]
      for k in G) for h in H) + Penalization;
return Objval
end

```

Del cuadro anterior, es preciso mencionar que, la función de adaptación permite que tanto en la población x como en la memoria m se cuente con individuos infactibles, pero únicamente aquellas infactibilidades que se presentan cuando la demanda es mayor que la generación, dichos individuos son penalizados, es decir tendrán un costo mayor, de tal manera que, durante el proceso iterativo, estos serán desechados. En esta implementación no es necesario realizar penalizaciones relacionadas con los límites mínimo y máximo de cada generador, toda vez que, estos límites se obligan a cumplir durante la generación de cada individuo dentro de la población. En los Capítulos 8 y 9, la función de adaptación sí considera dichas penalizaciones. Note que el cálculo de potencia generada P_{gen} y P_{dl} son necesarios para cumplir con la restricción (2.10), mientras que el cálculo de $Objval$ hace referencia a la Ecuación (2.9) del modelo de despacho económico multiperíodo descrito en el capítulo 2 de este libro.

La segunda función desarrollada, hace referencia a la inicialización tanto de la población x como de la memoria m .

```

function Pob_Mem_inicial(g,h,Nv,Ns,G,pmin,pmax)
    xo=zeros(Ns,Nv+1)
    mo=zeros(Ns,Nv+1)
    for i=1:Ns
        global Pg_x=zeros(g,h)
        global Pg_m=zeros(g,h)
    end
end

```

```
    for k in G
        Pg_x[k,1:h]=rand(pmin[k]:pmax[k],1,h)
        Pg_m[k,1:h]=rand(pmin[k]:pmax[k],1,h)
        if k==1
            xo[i,1:k*h]=Pg_x[k,1:h]
            mo[i,1:k*h]=Pg_m[k,1:h]
        elseif k==2
            xo[i,h+1:k*h]=Pg_x[k,1:h]
            mo[i,h+1:k*h]=Pg_m[k,1:h]
        else
            xo[i,2*h+1:k*h]=Pg_x[k,1:h]
            mo[i,2*h+1:k*h]=Pg_m[k,1:h]
        end
    end
    xo[i,Nv+1]=fitnessf(a,b,c,G,H,Pg_x,PD,alpha,B)
    mo[i,Nv+1]=fitnessf(a,b,c,G,H,Pg_m,PD,alpha,B)
end
return mo, xo
end
```

En el cuadro anterior, se observa que para la generación de cada individuo de la población y de la memoria (Pg_x y Pg_m), se obliga a que estos sean factibles desde del punto de vista de los límites mínimo y máximo de cada generador. Así mismo, se puede observar que cada individuo es evaluado mediante el llamado a la función de adaptación ($fitnessf()$) descrita anteriormente.

La tercera función implementada corresponde a la función *CSA*, la cual contiene el proceso evolutivo la búsqueda de cuervos descrita en la Sección 7.2.

```

function CSA(mt,xt,Nv,Ns,g,h,AP,fl,pmin,pmax,G)
    for i=1:Ns
        global pos=rand(1:Ns,1)
        global xi=zeros(Nv)
        global Pg_x=zeros(g,h)
        if rand() > AP
            xi = xt[i,1:Nv] +
                (fl*rand()).*(mt[pos[1],1:Nv] - xt[i,1:Nv])
            for k=1:g
                if k==1
                    Pg_x[k,1:h]= xi[1:k*h]
                elseif k==2
                    Pg_x[k,1:h]= xi[h+1:k*h]
                else
                    Pg_x[k,1:h]= xi[2*h+1:k*h]
                end
            end
            cont=0
            for k=1:h
                for m=1:g
                    if Pg_x[m,k] < pmin[m] ||
                        Pg_x[m,k] > pmax[m]
                        cont=cont+1
                    end
                end
            end
            if cont==0
                xt[i,1:Nv]=xi
                xt[i,Nv+1]=
                    fitnessf(a,b,c,G,H,Pg_x,PD,alpha,B)
                if xt[i,Nv+1]<mt[i,Nv+1]
                    mt[i,1:Nv]=xt[i,1:Nv]
                    mt[i,Nv+1]=xt[i,Nv+1]
                end
            end
        else
            for k in G
                Pg_x[k,1:h]=rand(pmin[k]:pmax[k],1,h)
                if k==1

```

```

        xt[i,1:k*h]=Pg_x[k,1:h]
    elseif k==2
        xt[i,h+1:k*h]=Pg_x[k,1:h]
    else
        xt[i,2*h+1:k*h]=Pg_x[k,1:h]
    end
end
xt[i,Nv+1]=
fitnessf(a,b,c,G,H,Pg_x,PD,alpha,B)
if xt[i,Nv+1]<mt[i,Nv+1]
    mt[i,1:Nv]=xt[i,1:Nv]
    mt[i,Nv+1]=xt[i,Nv+1]
end
end
end
return mt, xt
end

```

En esta función, se observa que la variable *pos*, hace referencia a la selección aleatoria del cuervo que se desea seguir. Así mismo, se identifican las dos opciones que pueden suceder cuando un cuervo *i* sigue a un cuervo *j*, la primera, es que el cuervo *j* no se da cuenta que lo están siguiendo, esto sucede cuando $rand() > AP$; y la nueva posición del cuervo *i* se calcula acorde en a la Ecuación (7.4). El segundo caso, es cuando el cuervo *j* sabe que lo están siguiendo, este caso se da cuando $rand() \leq AP$, lo que da como resultado una nueva posición del cuervo *i*, generada de manera aleatoria. Esta función es ejecutada *tmax* iteraciones y en cada una de las iteraciones retorna las matrices de la población *x* y la memoria *m*.

Una vez se cuenta con las implementaciones descritas anteriormente, se procede a ejecutar el algoritmo 100 veces, esto con el fin de realizar un análisis estadístico de las solución encontrada.

```

t1 = time();
global Guardar = zeros(rep,Nv+1);
for r=1:rep
    global m, x =Pob_Mem_inicial(g,h,Nv,Ns,G,pmin,pmax);

```

```

for j=1:tmax
    m, x = CSA(m,x,Nv,Ns,g,h,AP,fl,pmin,pmax,G);
end
m=m[sortperm(m[:, Nv+1]), :];
global Guardar[r,:] = m[1,:];
end
global Guardar = Guardar[sortperm(Guardar[:, Nv+1]), :];
et = time() - t1;

```

En el cuadro anterior, se identifica que la ejecución del algoritmo se repite una cantidad de *rep* veces (100 en este caso). en cada repetición es necesario llamar la función *Pob_Mem_inicial* la cual inicializa las matrices *x* y *m*. La función *CSA* se ejecuta una cantidad de *tmax*rep* veces (100X5000 en este caso). Así mismo la función *fitnessf* es llamada dentro de estas dos funciones en cada repetición del proceso iterativo. En cada repetición, la mejor solución encontrada es almacenada en la matriz *Guardar*. Después de finalizar la totalidad de repeticiones, la matriz *Guardar* es ordenada de menor a mayor con respecto al valor de función objetivo, es decir con los valores que se encuentran almacenados en la columna $Nv + 1$. Finalmente, la matriz *Guardar* es utilizada para realizar el análisis estadístico de los resultados obtenidos. Tal como se presenta a continuación.

```

@printf("Valor maximo: %2.1f\n", Guardar[rep,Nv+1])
@printf("Valor minimo: %2.1f\n", Guardar[1,Nv+1])
@printf("Valor promedio: %2.1f\n", mean(Guardar[:,Nv+1]))
@printf("Desviacion estandar: %2.1f\n", std(Guardar[:,Nv+1]))
@printf("Media - Minimo: %2.1f\n", mean(Guardar[:,Nv+1])-Guardar[1,Nv+1])
println("Tiempo promedio: ", round(et/rep, digits = 4), " s");
println("xbest:");
Pg_m=zeros(g,h)
for k=1:g

```

```
    if k==1
        local Pg_m[k,1:h]= Guardar[1,1:k*h]
    elseif k==2
        Pg_m[k,1:h]= Guardar[1,h+1:k*h]
    else
        Pg_m[k,1:h]= Guardar[1,2*h+1:k*h]
    end
end
for k=1:g
    @printf("Gen.  %d      \t",k)
end
@printf("\n")
for i in H
    for k in G
        @printf("%0.5f \t",Pg_m[k,i])
    end
    @printf("\n")
end
```

Note que al tratarse de un problema de minimización, la mejor solución hallada, es la que se encuentra almacenada en la primera fila de la matriz *Guardar*. Si se trabaja un problema de maximización, la mejor solución quedará almacenada en última fila de la matriz *Guardar*.

El análisis del conjunto de soluciones halladas por el algoritmo, permite determinar que el algoritmo siempre converge a una solución de buena calidad, esto comparado con la solución óptima del problema presentada en la Sección 2.2.3. Si bien, se observa que la mejor solución encontrada, por el algoritmo metaheurístico de optimización de búsqueda de cuervos presenta un valor de 82562.6, no es la solución óptima del problema, sí es posible asegurar que la solución hallada es repetible, lo que quiere decir que, a pesar de que el algoritmo basa su desarrollo en números aleatorios, cada vez que se ejecute, este siempre va a encontrar una solución de buena calidad cercana a la mejor encontrada en este análisis. Lo anterior se sustenta en que, las 100 soluciones almacenadas presentan una desviación estándar tan solo de 380.4 unidades. En el siguiente recuadro, también se presenta la mejor solución hallada para el

MÉTODOS DE OPTIMIZACIÓN METAHEURÍSTICA PARA INGENIERÍA: Con
aplicaciones en el software Julia

problema de despacho económico *xbest*, donde cada columna representa cada uno de los generadores y cada fila hace referencia a cada hora de análisis (de 1 a 24 horas). Finalmente, cada valor dentro de *xbest*, indica la cantidad de potencia que fue despachada desde cada generador en cada hora del periodo de análisis.

```
Valor maximo: 84317.9
Valor minimo: 82562.6
Valor promedio: 83232.0
Desviacion estandar: 380.4
Media - Minimo: 669.4
Tiempo promedio: 94.6327 s
xbest:
```

Gen. 1	Gen. 2	Gen. 3
40.65524	38.02836	136.91341
57.46003	25.15754	154.18096
64.24416	16.50145	124.15497
23.42491	47.02265	113.68634
59.55867	51.24787	136.14340
9.28205	16.21487	159.44941
47.11142	30.55698	74.91971
27.46029	19.88287	53.91008
9.81066	20.79728	50.21094
54.11852	19.54651	58.26675
31.25957	31.32765	132.12326
84.86251	41.15685	163.51081
78.56663	86.37253	156.52322
73.73824	90.17331	168.50834
85.62945	72.46132	207.24551
117.90884	64.33180	215.56016
146.45765	56.46092	216.58777
145.56810	53.30578	243.15123
140.71974	96.68968	237.45568
147.68660	99.58162	238.62508
149.41051	98.58743	249.24823
128.79830	89.98612	222.76239
100.55567	85.21037	178.87156
50.98567	83.12655	91.53813

Observación 7.6. Los resultados de tiempos obtenidos en esta simulación son únicamente con fines ilustrativos, ya que estos dependen del computador en el que sea ejecutado el algoritmo.

8

Algoritmo JAYA

En este capítulo se describe, analiza e implementa el algoritmo de optimización JAYA, el cual es un algoritmo de optimización diseñado para problemas de variable continua que toma ventaja de las dinámicas poblacionales y de avance basado en mejor solución, para solucionar problemas no lineales de alta complejidad de una manera simple, con un bajo número de parámetros de optimización. En esta sección este algoritmo se programa en el software Julia para resolver el problema despacho económico de centrales de generación térmica considerando pérdidas.

8.1 Filosofía e inspiración

El nombre de este algoritmo es tomado del sánscrito y significa victoria (Zitar et al., 2022). Este algoritmo de optimización se basa en

poblaciones, tomando ventaja de las estrategias de avance de los algoritmos evolutivos bío-inspirados para generar su criterio de avance y convergencia, permitiéndole alcanzar soluciones de excelente calidad con un nivel de complejidad en términos de programación bajo. El JAYA emplea la estrategia de supervivencia del individuo más apto, dirigiendo el resto de individuos de la población hacia regiones de mejor calidad, empleando dos poblaciones, la actual y la de desplazamiento. Siendo los individuos de la población actual remplazados por los de desplazamiento en los casos que la función adaptación sea mejorar. En términos de programación, este algoritmo es simple, presentando como parámetros de optimización dos factores: número de individuos y de iteraciones máximas; alcanzando un alto desempeño en la literatura para resolver problemas complejos de de ingeniería. Las características de mayor relevancia del JAYA se describen a continuación.

- i. Emplea una población para analizar el espacio de la solución, la cual identifica la mejor y peor solución en cada iteración para generar el desplazamiento de cada individuo dentro del proceso iterativo del algoritmo.
- ii. En cada iteración del algoritmo la población se ordena de mejor a peor solución en relación con la función objetivo obtenida por cada individuo. Siendo para un caso de minimización la mejor solución el individuo que obtenga la menor función adaptación dentro de la población (problema analizado en esta sección). Mientras que para un caso de maximización será el de mayor valor en términos de la función adaptación.
- iii. El JAYA emplea una segunda población generada por el desplazamiento de los individuos de la iteración actual, para lo cual emplea una ecuación que considera la posición actual de cada individuo, y la diferencia de este con la mejor y la peor solución. Generando durante que a medida que el algoritmo avanza en el proceso iterativo, el espacio de búsqueda se reduzca, generando

que en la última iteración los individuos converjan casi a una misma solución.

- iv. El JAYA utiliza dos parámetros para explorar y explotar el espacio de soluciones número de iteraciones. Los cuales corresponden al número de individuos y de iteraciones máximas.

El algoritmo JAYA es considerado como un método de optimización metaheurístico nuevo en relación con técnicas más clásicas como el PSO, GA y senos y cosenos. Siendo este propuesto ocho años atrás de esta publicación (Rao, 2016). Sin embargo, a pesar de no ser un algoritmo clásico, ha sido ampliamente utilizado y adaptado para resolver problemas de ingeniería compuestos por variables continuas y discretas. Dentro de las investigaciones más destacadas donde ha sido empleado el JAYA se destaca: flujo óptimo de potencia (Warid, 2020), identificación de parámetros de células solares (Luu and Nguyen, 2020), el problema de la mochila (Luu and Nguyen, 2020) (versión discreta), problema de formación de equipos (El-Ashmawi et al., 2020), estructuras de armadura (Degertekin et al., 2018), reconocimiento de emociones faciales (?), intercambiador de calor (More and Venkata Rao, 2020), estimación de parámetros en baterías de iones de litio (Wang et al., 2018), entre otros. Debido a la naturaleza compleja de algunos problemas de optimización, la forma simple del JAYA lo convierte en una excelente herramienta de baja complejidad para resolver problemas de difícil solución, como es el caso de algunos de los problemas antes mencionados.

8.2 Algoritmo JAYA

En esta sección se describen los pasos fundamentales del JAYA, los cuales son detallados de tal forma que los lectores de este documento puedan interiorizar las principales herramientas de este algoritmo de optimización. Para esta labor se aborda la forma clásica del JAYA,

ya que al ser adaptado para múltiples problemas ha tenido diferentes variantes en los últimos años (Zitar et al., 2022).

8.2.1 Inicialización de parámetros

Al iniciar el algoritmo JAYA, se deben leer los parámetros propios que corresponden al número de individuos que compondrán la población (n_s) y el número de iteraciones máximas que ejecutará el algoritmo durante el proceso iterativo (t_{max}). A diferencia de otros algoritmos de optimización como el AG, PSO, entre otros. El JAYA no tiene otros parámetros de control a los anteriormente mencionados, por lo cual tanto n_s como t_{max} deben ser ajustados para cada problema con el objetivo de obtener el mejor rendimiento del algoritmo. Adicionalmente, dentro de este paso del algoritmo se deben leer los parámetros y variables que describen el problema, y permiten evaluar la función adaptación en la cual el JAYA basará su proceso de avance y convergencia hacia la mejor solución posible.

8.2.2 Población inicial

El JAYA basa su proceso iterativo en una población. La cual en la primera iteración se genera de manera aleatoria, mientras desde la segunda iteración hasta alcanzar el t_{max} se actualiza iteración a iteración, empleando la información contenida dentro de la población actual (iteración t), la mejor solución y peor solución contenida dentro de la población. La Ecuación (8.1) describe la forma de la población:

$$X^t = \begin{bmatrix} x_{1,1} & x_{1,2} & \cdots & x_{1,j} & \cdots & x_{1,n_v} \\ x_{2,1} & x_{2,2} & \cdots & x_{2,j} & \cdots & x_{2,n_v} \\ \vdots & \vdots & \ddots & \vdots & \ddots & \vdots \\ x_{k,1} & x_{k,2} & \cdots & x_{k,j} & \cdots & x_{k,n_v} \\ \vdots & \vdots & \ddots & \vdots & \ddots & \vdots \\ x_{n_s,1} & x_{n_s,2} & \cdots & x_{n_s,j} & \cdots & x_{n_s,n_v} \end{bmatrix}, \quad (8.1)$$

En la anterior ecuación, el número de variables que describe el problema se denota con n_v , siendo este valor definido por la naturaleza del problema. Para la construcción de la población, el número de filas corresponde al número de individuos fijados para la población n_s , mientras el número de columnas está relacionado con n_v . De esta forma la población en cada iteración (X^t) corresponde a una matriz de dimensiones $n_s \times n_v$, siendo t el contador de iteraciones. En esta matriz $x_{k,j}$ corresponde al valor asignado a la j -ésima variable del k -ésimo individuo. Este valor siempre debe respetar los límites mínimos y máximos para la j -ésima variable del problema.

En la Ecuación (4.1) se presenta la estructura general para la construcción de la población inicial.

$$x_{k,j}^1 = x_j^{\min} + \alpha_j (x_j^{\max} - x_j^{\min}), \{ \forall j = 1, 2, \dots, n_v, \forall k = 1, 2, \dots, n_s \} \quad (8.2)$$

La generación de la población inicial ($t = 1$), se realiza de forma aleatoria, para lo cual es empleada la Ecuación (8.2). Empleando los valores máximos y mínimos asignados a cada variable ($x_j^{\min}$ y $x_j^{\max}$), y un valor aleatorio α_j con un rango de cero a uno con distribución normal para generar un valor dentro del rango permitido.

Observación 8.1. Cuando se analizan problemas de variables discretas o mixta, las variables son generadas con la Ecuación (8.2), siendo posteriormente redondeadas al valor entero más cercano para llevarlas al dominio discreto. Esto garantizará la factibilidad de cada una de las soluciones contenidas en $x_{k,j}$. Esta acción evita generar penalizaciones adicionales en la función adaptación. Las cuales podrían hacer caer el algoritmo en óptimos locales, aumentando el tiempo de procesamiento del algoritmo.

8.2.3 Evaluación de la función adaptación y selección del mejor y peor individuo

En este paso del algoritmo, se evalúa la función adaptación de cada individuo que compone la población, almacenando este valor en la columna correspondiente a la posición $n_v + 1$, por lo cual se hace necesario integrar una columna adicional a la población. Tal y como se presenta en la Ecuación (8.3). Donde, $f(X_{n_s})$ corresponde al valor de la función adaptación alcanzado por el n_s individuo de la población.

$$X^t = \begin{bmatrix} x_{1,1} & x_{1,2} & \cdots & x_{1,j} & \cdots & x_{1,n_v} & f(x_1) \\ x_{2,1} & x_{2,2} & \cdots & x_{2,j} & \cdots & x_{2,n_v} & f(x_2) \\ \vdots & \vdots & \ddots & \vdots & \ddots & \vdots & \vdots \\ x_{k,1} & x_{k,2} & \cdots & x_{k,j} & \cdots & x_{k,n_v} & f(x_k) \\ \vdots & \vdots & \ddots & \vdots & \ddots & \vdots & \vdots \\ x_{n_s,1} & x_{n_s,2} & \cdots & x_{n_s,j} & \cdots & x_{n_s,n_v} & f(x_{n_s}) \end{bmatrix}, \quad (8.3)$$

Posterior a la evaluación de la función adaptación, los individuos de la población son ordenados en relación con la función adaptación. En el caso de un problema de minimización, como es el caso del despacho óptimo de potencia considerando pérdidas aquí analizado, la población se organiza en orden ascendente. Por lo que el mejor y peor individuo serían los individuos x_1 y x_{n_s} , respectivamente. En el caso de abordar un problema de maximización existen dos caminos para el ordenamiento. (i) invertir el sentido de la selección del peor individuo con un ordenamiento ascendente, es decir, el mejor individuo será x_{n_s} , mientras el peor individuo corresponderá a x_1 . (ii) ordenar de forma descendente, lo cual respeta el orden de selección básico del algoritmo: mejor solución (x_1) y peor solución (x_{n_s}). El problema abordado en esta sección es de minimización, por lo cual se ordena de forma ascendente y se respeta el orden anterior.

8.2.4 Reglas de avance

Para el proceso de avance del JAYA, se genera una población auxiliar (y^t) que desplaza las posiciones de los individuos de la población durante cada iteración t (x^t), empleando la información de x_1 (mejor solución) y peor solución x_{n_s} . La matriz auxiliar para el avance se presenta en la Ecuación (8.4), la cual presenta el mismo tamaño de la matriz descrita en la Ecuación (8.3), integrando el mismo número de individuos, variables y función adaptación. Para cada individuo de la población auxiliar se calcula el valor de la función adaptación, almacenando este valor en la posición n_v+1 .

$$y^t = \begin{bmatrix} y_{1,1} & y_{1,2} & \cdots & y_{1,j} & \cdots & y_{1,n_v} & f(y_1) \\ 2,1 & y_{2,2} & \cdots & y_{2,j} & \cdots & y_{2,n_v} & f(y_2) \\ \vdots & \vdots & \ddots & \vdots & \ddots & \vdots & \vdots \\ y_{k,1} & y_{k,2} & \cdots & y_{k,j} & \cdots & y_{k,n_v} & f(y_k) \\ \vdots & \vdots & \ddots & \vdots & \ddots & \vdots & \vdots \\ y_{n_s,1} & y_{n_s,2} & \cdots & y_{n_s,j} & \cdots & y_{n_s,n_v} & f(y_{n_s}) \end{bmatrix}, \quad (8.4)$$

Para obtener cada una de las componentes de la matriz auxiliar que desplaza los individuos de población en el espacio de solución, se emplea la Ecuación (8.5). En esta ecuación, $x_{k,j}^t$ corresponde al valor de la j -ésima variable del k -ésimo individuo en la iteración t , $x_{1,j}^t$ y $x_{n_s,j}^t$ denotan el valor de la j -ésima variable de la mejor y peor solución en la iteración t . Mientras que, R_1 y R_2 representan valores aleatorios en el rango de cero a uno con distribución normal, los cuales garantizan la diversidad de las soluciones; permitiendo que el JAYA pueda escapar de óptimos locales.

$$y_{k,j}^t = x_{k,j}^t + R_1(x_{1,j}^t - |x_{k,j}^t|) - R_2(x_{n_s,j}^t - |x_{k,j}^t|), \\ \{\forall j = 1, 2, \dots, n_v, \forall k = 1, 2, \dots, n_s\} \quad (8.5)$$

Observación 8.2. Para reducir el tiempo de procesamiento del algoritmo, la construcción de las componentes de la matriz auxiliar pueden ser obtenidos de manera matricial. Generando el mismo efecto que componente a componente usado en la subsección anterior.

8.2.5 Sustitución en la población

Después de haber generado y evaluado la función adaptación de y_k^t , se realiza la actualización de los individuos de la población en función del valor de la función adaptación. Para el proceso de sustitución se emplea la Ecuación (8.6).

$$x_k^{t+1} = \begin{cases} y_k^t & f(y_k^t) < f(x_k^t) \\ x_k^t & f(y_k^t) \geq f(x_k^t) \end{cases}, \quad \{\forall k = 1, 2, \dots, n_s\} \quad (8.6)$$

El k -ésimo individuo de la población actual es remplazado por el k -ésimo individuo de la población auxiliar solo si la función adaptación obtenida por y_k^t es inferior a la del x_k^t , en caso contrario la población actual mantiene el individuo sin modificación. Lo anterior aplica para un caso de minimización como el abordado en esta sección, en un caso de maximización el individuo x_k^t solo será remplazado si el mismo individuo de la población auxiliar presenta un valor de función adaptación mayor.

Posterior a la actualización de la población en la iteración $t + 1$, se realiza el ordenamiento de la población identificando nuevamente la mejor y peor solución en la iteración actual del algoritmo. Finalizada esta acción, el algoritmo actualiza la posición de la población en cada iteración, identificando la mejor y peor solución, hasta que el criterio de convergencia haya sido alcanzado.

8.2.6 Criterios de parada

En el algoritmo de optimización JAYA, el proceso de exploración y explotación del espacio de soluciones finaliza cuando el número de iteraciones máximas es alcanzado. Cuando esto ocurre, el JAYA retorna como solución la mejor solución encontrada al finalizar el proceso iterativo.

Observación 8.3. Para evitar desperdiciar tiempo de procesamiento en iteraciones que no mejoran la solución. Es posible integrar un contador de iteraciones de no mejoras, el cual finalice el proceso de búsqueda en el caso de que sea alcanzado un número de iteraciones de no mejorar máximas (m_{max}). Siendo este un tercer parámetro de sintonización del JAYA en el caso en que este sea considerado.

8.2.7 Implementación computacional

El algoritmo 8 presenta los pasos fundamentales del proceso iterativo del JAYA y el proceso de convergencia en general.

Data: Lectura de parámetros del problema bajo análisis

Definir los límites de las variables que componen el problema, i.e., $x^{\min}$ y $x^{\max}$;

Definir los valores de n_s , n_v , $t_{\max}$ y $m_{\max}$;

Inicializar el contador de iteraciones de no mejora $m = 0$;

for $t = 1 : t_{\max}$ **do**

if $t = 1$ **then**

Crear la población inicial (x^t) de forma aleatoria empleando la Ecuación (8.2);

Evaluar la función de adaptación para cada individuo de la población $f(x_k^t) \forall k = 1, 2, \dots, n_s$;

Ordenar la población x^t y determinar la mejor solución (x_1^t) y peor solución de la población ($x_{n_s}^t$);

if $t \geq 2$ **then**

Obtener la matriz auxiliar y^t empleando la Ecuación (8.5);

Evaluar la función adaptación de todos los individuos de y^t ;

Actualizar los individuos de x^t empleando la Ecuación (8.6);

Actualizar la mejor solución (x_1^t) y peor solución de la población ($x_{n_s}^t$);

if $f(x_1^t) \geq f(x_1^{t-1})$ **then**

Hacer $m = m + 1$;

else

Hacer $m = 0$;

if $m \geq m_{\max}$ **then**

Reportar la mejor solución actual contenida en x_1^t ;

break;

Algoritmo 8: Algoritmo de optimización JAYA

8.3 Implementación en el software Julia

A continuación se describe la aplicación del algoritmo JAYA para resolver el problema de despacho económico de centrales de generación térmica considerando pérdidas de potencia, presentado y analizado en la Sección 2.2 en el Capítulo 2.

```
using DataFrames, LinearAlgebra, Random;  
using Statistics, Printf, BenchmarkTools;
```

Inicialmente, dentro del programa desarrollo en Julia, se deben utilizar "llamar" las bibliotecas necesarias para el procesamiento y presentación de la información en pantalla. Para la ejecución del JAYA se debe emplear las bibliotecas DataFrames, LinearAlgebra, Random, Statistics, Printf y BenchmarkTools; por medio de la implementación de la función using.

```
B = [0.00013630    0.00006750    0.00007839  
      0.00006750    0.00015450    0.00009828  
      0.00007839    0.00009828    0.00016140];  
Generadores = DataFrames.DataFrame([  
  (1, 0.006085, 10.04025, 136.9125, 5, 150),  
  (2, 0.005915, 9.760576, 59.15500, 15, 100),  
  (3, 0.005250, 8.662500, 328.1250, 50, 250),])  
DataFrames.rename!(Generadores,  
[:g, :a, :b, :c, :xmin, :xmax]);  
a = Generadores.a; b = Generadores.b; c = Generadores.c;  
xmin = Generadores.xmin;  
xmax = Generadores.xmax;  
Periodos = DataFrames.DataFrame([  
  (1, 210), (7, 150), (13, 310), (19, 450),  
  (2, 230), (8, 100), (14, 320), (20, 460),  
  (3, 200), (9, 80), (15, 350), (21, 470),  
  (4, 180), (10, 130), (16, 380), (22, 420),  
  (5, 240), (11, 190), (17, 400), (23, 350),  
  (6, 180), (12, 280), (18, 420), (24, 220),])  
DataFrames.rename!(Periodos, [:h, :PD]);
```

```
PD = Periodos.PD;  
t = size(PD,1); g = size(Generadores,1);  
H = 1:t; G = 1:g; alpha = 1;
```

Posteriormente, se deben cargar la información del problema a resolver. En esta sección, el JAYA abordará el problema de despacho económico de centrales de generación térmica considerando pérdidas de potencia. La descripción de esta información se encuentra en la Sección 2.2 en el Capítulo 2.

```
function fitnessf(a,b,c,xmin,xmax,G,H,pg,PD,alpha,B)  
    global Penalty = 100;  
    global Penalization = 0;  
  
    for h in H  
        Pgen = sum(pg[k,h] for k in G)  
        Pdl = PD[h] +  
            alpha*sum(sum(pg[k,h]*B[k,m]*pg[m,h]  
  
        for m in G) for k in G)  
        if Pgen < Pdl  
            global Penalization = Penalization  
            - Penalty*min(Pgen - Pdl,0);  
        end  
    end  
    for h in H  
        for g in G  
            if pg[g,h] < xmin[g]  
                Penalization = Penalization  
                + Penalty*max(xmin[g]-pg[g,h],0);  
            end  
            if pg[g,h] > xmax[g]  
                Penalization = Penalization  
                + Penalty*max(pg[g,h]-xmax[g],0);  
            end  
        end  
    end  
    Objval = sum(sum(a[k]*pg[k,h]^2+b[k]*pg[k,h]+c[k]  
    for k in G) for h in H) + Penalization;
```

```
        return Objval  
    end
```

Para evaluación la función adaptación relacionada con cada individuo dentro de las poblaciones empleadas para el avance del JAYA, se utiliza la función descrita en el recuadro anterior. Es importante destacar que esta es ampliamente descrita en el Capítulo 3 dentro de la Sección 3.6.

Para inicializar el algoritmo JAYA se deben establecer los parámetros de inicio correspondientes al número de iteraciones ($t_{\max}$), número de individuos (N_s), número de variables del problema (N_v), número de iteraciones máximas de no mejora ($n_{\max}$), contador de iteraciones de no mejora (m), número de periodos de tiempo analizados (N_t), vector de límites inferior y superior de potencia de los generadores (lb y ub) y memoria de almacenamiento de mejores valores de función adaptación obtenidas por el JAYA en cada iteración (fa). La lectura de parámetros en el entorno de JULIA se ilustra a continuación.

```
global tmax=5000; global Ns=120;  
global Nv=72; global mmax=1000;  
global m=0; global Nt = size(Periodos,1);  
global lb = zeros(Nv,1); global ub = zeros(Nv,1)  
for j in G  
    global lb[(j-1)*Nt+1:j*Nt] = xmin[j]*ones(Nt,1);  
    global ub[(j-1)*Nt+1:j*Nt] = xmax[j]*ones(Nt,1);  
end  
global fa=zeros(tmax,1);
```

En la anterior información es importante resaltar que:

- i. Los parámetros n_s , $t_{\max}$ y $m_{\max}$ deben ser ajustados para cada tipo de problema. Los valores reportados anteriormente fueron hallados mediante prueba de error con el objetivo de encontrar la mejor solución posible para el problema por parte del JAYA.
- ii. N_v está asociado a la codificación del problema, lo cual se clarifica en la Sección 3.5 del Capítulo 3.

- iii. El almacenamiento de las funciones adaptación se realiza para aplicar el criterio de iteraciones de no mejora. En el caso en que este criterio de convergencia no sea considerado, este valor se puede eliminar.

Posterior a la carga de los parámetros del algoritmo se realiza la construcción de la población inicial, el JAYA se ejecuta empleando la función `jaya (tmax, Ns, Nv, mmax, m, fa, lb, ub)`, la cual emplea las funciones y parámetros antes descritos para encontrar el despacho de potencia de las plantas térmicas que presente los menores costos. Empleando la información contenida en el Algoritmo 8, se construye el proceso iterativo del JAYA en Julia, descrito a continuación.

```
function jaya (tmax, Ns, Nv, mmax, m, fa, xmin, xmax, lb, ub)
    for t=1:tmax
        if t==1
            global x=zeros (Ns, Nv+1);
            global cont=0;
            for e=1:Ns
                x[e, 1:Nv]=lb+rand(Nv, 1) .* (ub-lb);
            end
            for e = 1:Ns
                for u = 1:Nv
                    if x[e, u] < lb[u, 1]
                        global x[e, u] = lb[u, 1];
                    elseif x[e, u] > ub[u, 1]
                        global x[e, u] = ub[u, 1];
                    end
                end
            end
        for i=1:Ns
            global aux=hcat (x[i, 1:24], x[i, 25:48], x[i, 49:72]);
            x[i, end]=fitnessf(a, b, c, xmin, xmax, G, H, aux', PD, alpha, B);
        end
        x=x[sortperm(x[:, Nv+1]), :];
        global peor_individuo=x[Ns, 1:Nv];
        global mejor_individuo=x[1, 1:Nv];
    end
```

```

if t>=2
    global y=zeros(Ns,Nv+1);
    y[:,1:Nv]=x[:,1:Nv]+rand(Ns,Nv)
    .*(repeat(mejor_individuo', outer = [Ns, 1, 1])
    -abs.(x[:,1:Nv]))-rand(Ns,Nv)
    .*(repeat(peor_individuo', outer = [Ns, 1, 1])
    -abs.(x[:,1:Nv]));
    for e = 1:Ns
        for u = 1:Nv
            if y[e,u] < lb[u,1]
                global y[e,u] = lb[u,1];
            elseif y[e,u] > ub[u,1]
                global y[e,u] = ub[u,1];
            end
        end
    end
    for i=1:Ns
        global aux=
            hcat(y[i,1:24],y[i,25:48],y[i,49:72]);
        y[i,end]=fitnessf(a,b,c,xmin,xmax,G,H,aux'
        ,PD,alpha,B);
    end
    for i=1:Ns
        if y[i,end]<x[i,end]
            x[i,:]=y[i,:];
        end
    end
    x=x[sortperm(x[:, Nv+1]), :];
    global peor_individuo=x[Ns,1:Nv];
    global mejor_individuo=x[1,1:Nv];
end
    global fa[t,1]=x[1,Nv+1];
    if t>=2
        if fa[t-1,1]==fa[t,1]
            m=m+1;
        else
            m=0;
        end
    end
end

```

```
        if m==mmax
            break;
        end
    end
    return x[1, :];
end
```

Para la implementación de la función que contiene el proceso de avance del algoritmo JAYA, se deben de tener en cuenta las siguientes consideraciones:

- i. Para construir la población del JAYA (x), inicialmente se genera una matriz de ceros de tamaño $ns \times nv$, con el objetivo de reservar los espacios de memoria y garantizarlos vacíos. Posteriormente, esta matriz se llena con valores aleatorios entre los límites de potencia de los generadores.
- ii. Dentro del JAYA, se incluye una sección de verificación de los límites inferior y superior de cada generador para los diferentes individuos de las poblaciones empleadas dentro del proceso de avance del algoritmo. Con el objetivo de garantizar los límites de potencia y evitar emplear una penalización adicional dentro de la función adaptación, lo cual podría afectar la calidad de solución y tiempos de procesamiento del algoritmo. En el problema abordado, se consideran 3 generadores con límites mínimos y máximos de potencia almacenados en los vectores lb y ub .
- iv. Para realizar la evaluación de la función adaptación de cada individuo, es necesario decodificar su información de su forma vectorial de tamaño $1 \times nv$ a la forma matricial de 3×24 requerida por la función adaptación propuesta. Para llevar a cabo esta labor se emplea la función de Julia `hcat`.
- v. El desplazamiento de los individuos dentro del espacio de solución se realiza empleando a y^t , esta matriz se construye de forma

matricial. Para generar las matrices de mejor y peor individuo se emplea la función `repeat` de Julia, que permite repetir un vector tantas veces sea necesario, en este caso Ns veces. Al construir esta matriz en cada iteración, se verifican los límites de potencia empleando la función descrita en el ítem anterior.

- vi. Para validar el contador de no mejora del JAYA, iteración a iteración, es almacenada la mejor solución del problema en la variable `fa`. Comparando los valores de los valores de la iteración anterior con la actual, el algoritmo contabiliza las iteraciones en que la función adaptación no mejora, finalizando el proceso de búsqueda al alcanzar $nmax$.

Para realizar el análisis estadístico de la efectividad del JAYA en términos de calidad de solución y tiempos de procesamiento se aplica el siguiente algoritmo.

```
t1 = time();
rep = 100;
global Guardar = zeros(rep,nv+1);
    for r = 1:rep
        global Guardar[r,:] = jaya(tmax,Ns,Nv,mmax,
            m,fa,xmin,xmax,lb,ub);
    end
global Guardar = Guardar[sortperm(Guardar[:,Nv+1]),:];
elapsed_time = time() - t1;
@printf("Valor maximo: %2.1f\n",Guardar[rep,Nv+1])
@printf("Valor minimo: %2.1f\n",Guardar[1,Nv+1])
@printf("Valor promedio: %2.1f\n",mean(Guardar[:,Nv+1]))
@printf("Desviacion estandar: %2.1f\n",std(Guardar[:,Nv+1]))
@printf("Maximo - Media: %2.1f\n",
Guardar[rep,Nv+1]-mean(Guardar[:,Nv+1]));
println("Tiempo de ejecucion (promedio): ",
round(elapsed_time/rep, digits = 4), " s");
```

El anterior algoritmo ejecuta 100 veces el JAYA, empleando la variable `rep`, almacenando la mejor solución y función adaptación obtenida en cada iteración del JAYA en la matriz `Guardar`. Posterior a las ejecuciones del algoritmo, es obtenido el valor máximo, mínimo y promedio de la función adaptación. Así como la desviación estándar y la diferencia en real entre la mejor solución y la promedió. Finalmente, se calcula el tiempo promedio de ejecución del algoritmo empleando el reloj del computador con la función `time()`. Al realizar las 100 ejecuciones del algoritmo propuesto, es posible obtener los resultados descritos a continuación.

Valor maximo: 86381.3		
Valor minimo: 83149.9		
Valor promedio: 83985.3		
Desviacion estandar: 464.3		
Maximo - Media: 2395.9		
Tiempo promedio: 23.5286 s		
Gen. 1	Gen. 2	Gen. 3
29.17829	45.04027	141.52118
8.07154	24.91132	120.22265
65.20667	79.20309	177.49918
130.71395	100.00000	244.37465
36.29713	51.81026	148.66810
5.00000	15.00000	81.42220
68.87067	82.66329	181.13095
136.25712	100.00000	250.00000
25.63468	41.66672	137.95668
5.00000	15.00000	60.87358
79.93195	93.09340	192.06420
147.26231	100.00000	250.00000
18.57869	34.94223	130.84546
5.00000	16.40787	111.08522
92.75667	100.00000	205.00750
114.33776	100.00000	227.42801
39.87253	55.20681	152.25059
22.10151	38.30072	134.39812
103.51150	100.00000	216.19304
79.93195	93.09340	192.06420

MÉTODOS DE OPTIMIZACIÓN METAHEURÍSTICA PARA INGENIERÍA: Con
aplicaciones en el software Julia

18.57869	34.94223	130.84546
54.28266	68.87117	166.64136
114.33776	100.00000	227.42801
32.73241	48.42144	145.09165

Al realizar un análisis estadístico, empleando la librería *Statistics*, a los resultados obtenidos por el JAYA implementado en el software Julia. Es posible apreciar que: (i) la mejor solución hallada corresponde a un costo total de 83149.9, presentando una diferencia con la mejor solución global de 879.9 unidades (82269.9). (ii) El JAYA obtuvo como valores máximo y promedio un total de 86381.3 y 83985.3, Con una desviación estándar en valor real de 464.3, lo que equivale a una desviación del 0.55%. Presentando una diferencia entre la mejor solución y la solución promedio de 835.4, equivalente a una diferencia del 0.99%. Estos valores demuestran que el JAYA obtiene soluciones de buena calidad en relación con la calidad de la solución, con una excelente repetibilidad de la mejor solución cada vez que es ejecutado. (iii) El tiempo de procesamiento promedio requerido por el JAYA para resolver el problema es de 23.5 segundos, lo cual lo convierte en un método eficiente desde el punto de vista del procesamiento; y el más rápido de los propuestos para resolver el problema de despacho económico de plantas térmicas dentro de este documento.

Observación 8.4. Los resultados obtenidos por el JAYA se presentaron en el recuadro anterior, donde cada generador respeta sus límites de potencia mínima y máxima. Es importante resaltar que la mejor respuesta cambiara cada vez que el JAYA es ejecutado. Esto se debe al ser basado en números aleatorios. Sin embargo, la calidad de la solución se mantendrá en un margen similar a los resultados anteriormente descritos.

9

Algoritmo de optimización de búsqueda por vórtices

En este capítulo se presenta la descripción general del algoritmo de búsqueda de vórtices y su aplicación a problemas de optimización de dominio continuo. Este algoritmo pertenece a la familia de las técnicas de optimización de trayectoria inspirados en fenómenos físicos, ya que explora y explota el espacio de soluciones a partir de una solución inicial mediante regla de avance probabilístico aplicadas a un espacio de soluciones de radio variable. El algoritmo de búsqueda de vórtices será implementado para el problema de despacho económico de centrales térmicas.

9.1 Filosofía e inspiración

El algoritmo de optimización de búsqueda por vórtices corresponde a una técnica de optimización de la familia de los algoritmos de búsqueda de trayectoria, el cual es inspirado en el comportamiento de los vórtices de fluidos agitados en tuberías (Gharehchopogh et al., 2021), es un algoritmo metaheurístico inspirado en un fenómeno físico. Esta estrategia de optimización fue originalmente propuesta para resolver modelos de optimización puramente matemáticos, demostrando un desempeño exitoso, según los autores de (Doğan and Ölmez, 2015a). Las características más relevantes de esta técnica de optimización metaheurística se presentan a continuación.

- i. Evoluciona a partir del centro geométrico de una hiper-elipse que define el espacio de soluciones generando mediante una función de probabilidad Gausiana multivariada un conjunto de soluciones uniformemente distribuidas en todo el espacio de solución.
- ii. Explora y explota el espacio de soluciones con un movimiento parecido a los vórtices de un fluido en un punto de sumidero, siendo este la mejor solución actual del problema.
- iii. Reduce el tamaño de soluciones a medida que avanza el proceso iterativo a través de una función no lineal decreciente, con el fin de equilibrar las etapas de exploración y explotación del espacio de soluciones.
- iv. Sólo cuenta con dos parámetros de control para explorar y explotar el espacio de soluciones, siendo estos el tamaño de la población y el número de iteraciones.

En la literatura especializada, el algoritmo metaheurístico de búsqueda por vórtices ha sido aplicado a múltiples problemas de optimización de naturaleza, continua, discreta y mixta, dentro de los que se

destacan: selección de componentes en filtros analógicos activos (Doğan and Ölmez, 2015b), balance de cargas en redes eléctricas desbalanceadas (Cortés-Cañedo et al., 2021), detección de ataques en sistemas cibernéticos (Krishnan et al., 2022), estimación de parámetros en motores de inducción (Montano et al., 2023) y el problema de cinemática inversa de manipuladores de robots en serie (Toz, 2020), entre otros.

9.2 Algoritmo de búsqueda de vórtices

Considerando las exitosas aplicaciones presentadas en la literatura especializada de la técnica de optimización de búsqueda de vórtices, el objetivo de este capítulo es presentar de forma general el algoritmo de búsqueda por vórtices, con énfasis particular en sus reglas de avance, corrección de las soluciones y criterios de parada, entre otros aspectos relevantes, tal como se verá en las siguientes subsecciones.

9.2.1 Centro de la hiper-elipse y vecindario

La técnica de optimización de búsqueda por vórtices explora y explota el espacio de soluciones a partir de un punto inicial localizado en el centro del hiper-espacio de soluciones. Este punto es conocido como el valor medio μ_t (en $t = 0$), y se obtiene a partir de los límites máximos y mínimos asociados a las variables de decisión (Doğan and Ölmez, 2015a). Como punto de partida, este centro de la hiper-elipse se calcula, como se muestra en la Ecuación (9.1).

$$\mu_t = \frac{x^{\max} + x^{\min}}{2}. \quad (9.1)$$

A partir de este punto inicial en el espacio de soluciones, se genera un vecindario de soluciones uniformemente distribuida, que se construye a partir de una función de probabilidad Gausiana $x(y|\mu, \Sigma)$, con la

estructura dada en (9.2).

$$x(y|\mu, \Sigma) = \frac{1}{\sqrt{(2\pi)^{n_v} |\Sigma|}} \exp \left\{ -\frac{1}{2} (y - \mu)^\top \Sigma^{-1} (y - \mu) \right\}, \quad (9.2)$$

donde n_v representa el número de variables o la dimensión del problema, siendo y un vector de variables aleatorias y Σ la matriz de covarianzas de dimensión $n_v \times n_v$.

Observación 9.1. Si los elementos de la diagonal de la matriz Σ son iguales (varianza equivalente) y los elementos fuera de la diagonal son definidos como cero (covarianza nula), entonces la forma de la distribución normal será hiper-esférica. En caso de que los elementos de la diagonal sean diferentes y se mantengan valores cero en las covarianzas, entonces la distribución tomará una forma hiper-elíptica.

Para construir inicialmente la matriz Σ con las características anteriormente descritas, se parte del radio inicial del sistema en $t = 0$, i.e., r_t , el cual se determina según (9.3).

$$r_t = \frac{x^{\max} - x^{\min}}{2}, \quad (9.3)$$

posteriormente se define la matriz Σ como (9.4).

$$\Sigma = \sigma_t I_{n_v \times n_v}, \quad (9.4)$$

donde σ_t se define como r_t e $I_{n_v \times n_v}$ es una matriz identidad de dimensiones apropiadas.

Teniendo en cuenta el vecindario de soluciones que se genera a partir de la distribución de probabilidad $x(y|\mu, \Sigma)$, es necesario verificar que

estas estén dentro del rango admisible de las variables de solución, para lo cual se aplica la siguiente regla de revisión.

$$x_{k,j}^t = \begin{cases} x_{k,j} & x_j^{\min} \leq x_{k,j} \leq x_j^{\max} \\ x_j^{\min} & x_{k,j} < x_j^{\min} \\ x_j^{\max} & x_{k,j} > x_j^{\max} \end{cases}, \quad \{\forall j = 1, 2, \dots, n_v, \forall k = 1, 2, \dots, n_s\}, \quad (9.5)$$

siendo n_s el número de soluciones contenidas en el vecindario.

Observación 9.2. En el caso de problemas de optimización de naturaleza discreta, la generación de las variables de decisión a través de la regla (9.5) son redondeadas al valor entero más cercano. Esto garantizará la factibilidad de cada una de las soluciones contenidas en $x_{k,j}$.

9.2.2 Reglas de avance

El algoritmo de búsqueda por vórtices definirá el nuevo centro de la hiper-elipse a partir de la mejor contenida vecindario, siempre que esta tenga mejor desempeño que el centro anterior. Para determinar la mejor solución el vecindario, i.e., x_{best}^t se evalúa toda la población X^t en la función objetivo del problema o función de adaptación, con lo cual se tiene:

$$x_{\text{best}}^t = \{x_k^t / x_k^t \rightarrow \min(f(x_k^t)), \forall k = 1, 2, \dots, n_s\}, \quad (9.6)$$

donde x_k^t es la k -ésima solución contenida en la población y $f(x_k^t)$ corresponde a la función objetivo (también conocida como función de adaptación o desempeño) evaluada en la solución x_k^t .

La regla de sustitución para el nuevo centro será la dada por (9.7).

$$\mu_t = \begin{cases} \mu_{t-1} & f(\mu_t) \leq f(x_{\text{best}}^t) \\ x_{\text{best}}^t & f(\mu_t) > f(x_{\text{best}}^t) \end{cases}, \quad (9.7)$$

Observación 9.3. La aplicación de la regla de sustitución para el centro de la hiper-elipse dada en (9.7) funciona para problemas de minimización. En caso de problemas de maximización, los símbolos de menor igual que y mayor que, deberán ser intercambiados.

9.2.3 Reducción del radio

Con el objetivo de pasar de la etapa de exploración del espacio de soluciones a la explotación de regiones promisorias, el algoritmo de búsqueda por vórtices propone una reducción paulatina del radio a medida que aumenta el número de iteraciones. Originalmente los autores de (Doğan and Ölmez, 2015a) proponen la utilización de la función gamma inversa incompleta; sin embargo, los autores de (Montoya et al., 2020b) una modificación a esta regla a partir de una función exponencial decreciente como sigue:

$$r_t = \left(1 - \frac{t}{t_{\max}}\right) \exp\left\{-b \frac{t}{t_{\max}}\right\} \left(\frac{x^{\max} - x^{\min}}{2}\right), \quad (9.8)$$

siendo b el parámetro de decrecimiento de la función exponencial, para el cual los autores de (Montoya et al., 2020b) recomiendan que se asigne el valor de 6.

9.2.4 Criterios de parada

En la aplicación de técnicas de optimización metaheurística, debido a que la solución del problema es aproximada mediante programación

secuencial, es típico encontrar dos criterios de parada comúnmente empleados en la literatura especializada, como siguen:

- i. El proceso de exploración y explotación del espacio de soluciones se da por terminado cuando se alcanza el número máximo de iteraciones preestablecido por el programador, o
- ii. cuando durante $m_{\max}$ número de iteraciones la función de desempeño no presenta ninguna mejora en su valor.

Nótese que para implementar el segundo criterio de parada, se requiere de la implementación de un contador de no mejoras de la función objetivo. En caso de implementar este criterio de parada, el valor de $m_{\max}$ es predefinido por el usuario y se recomienda como un valor entre 10 y 30% del total de iteraciones.

9.2.5 Implementación computacional

En el Algoritmo 9 se describe la implementación general de la técnica de optimización de búsqueda por vórtices para la solución de problemas de optimización de dominio continuo en ingeniería y ciencia.

Data: Datos del problema de optimización bajo análisis

Definir los límites inferiores de las variables, i.e., $x^{\min}$ y $x^{\max}$;
 Definir los valores de n_s , $t_{\max}$, $m_{\max}$ y determinar el valor de n_v ;
 Generar el centro inicial de la hiper-elipse en $t = 0$, i.e., μ_t empleando (9.1);
 Generar el radio de la hiper-elipse en $t = 0$, i.e., r_t a través de (9.3);
 Hacer $m = 0$;
for $t = 1 : t_{\max}$ **do**
 Definir la matriz de covarianzas Σ empleando (9.3);
 for $k = 1 : n_s$ **do**
 Obtener la potencial solución vecina x_k^t con (9.2);
 Verificar/corregir la solución vecina x_k^t empleando la regla dada en
 (9.5) Evaluar la función de adaptación para x_k^t , i.e., $f(x_k^t)$;
 Determinar la mejor solución contenida en el vecindario, i.e., x_{best}^t ,
 empleando (9.6);
 Actualizar el centro de la hiper-elipse, i.e., μ_t empleando la regla dada
 en (9.7);
 Actualizar el radio r_t de la hiperelipse empleando la función (9.8);
 if $f(x_{\text{best}}^t) \geq f(x_{\text{best}}^{t-1})$ **then**
 Hacer $m = m + 1$;
 else
 Hacer $m = 0$;
 if $m \geq m_{\max}$ **then**
 Reportar la mejor solución actual contenida en x_{best}^t ;
 break;
end for

Result: Reportar la mejor solución encontrada, i.e., x_{best}^t

Algoritmo 9: Algoritmo de optimización de búsqueda por vórtices

9.3 Implementación en el software Julia

En esta sección se describe la aplicación del algoritmo de optimización de búsqueda por vórtices al problema de despacho económico de centrales térmicas multiperíodo definido en el modelo matemático (2.9)–(2.11). Para ilustrar la implementación del algoritmo de búsqueda por vórtices se presentan los siguientes pasos secuenciales.

En primer lugar, se definen los parámetros iniciales del sistema y se calculan el número de generadores y número de períodos de estudio a partir de los datos iniciales del sistema. El llamado a librerías, así como los datos iniciales del sistema se ilustran a continuación.

```
using DataFrames, Random, Distributions
B = [0.00013630  0.00006750  0.00007839
      0.00006750  0.00015450  0.00009828
      0.00007839  0.00009828  0.00016140];
Generadores = DataFrames.DataFrame([
(1, 0.006085, 10.04025, 136.9125, 5, 150),
(2, 0.005915, 9.760576, 59.15500, 15, 100),
(3, 0.005250, 8.662500, 328.1250, 50, 250),])
DataFrames.rename!(Generadores,
[:g, :a, :b, :c, :pmin, :pmax]);
a = Generadores.a; b = Generadores.b; c = Generadores.c;
pmin = Generadores.pmin; pmax = Generadores.pmax;
Periodos = DataFrames.DataFrame([
(1, 210), (7, 150), (13, 310), (19, 450),
(2, 230), (8, 100), (14, 320), (20, 460),
(3, 200), (9, 80 ), (15, 350), (21, 470),
(4, 180), (10, 130), (16, 380), (22, 420),
(5, 240), (11, 190), (17, 400), (23, 350),
(6, 180), (12, 280), (18, 420), (24, 220),])
DataFrames.rename!(Periodos, [:h, :PD]);
PD = Periodos.PD;
t = size(PD,1); g = size(Generadores,1);
H = 1:t; G = 1:g; alpha = 1;
```

En la implementación anterior, es posible observar que se están empleando la librería *Distributions*, debido a que método de búsqueda por vórtices emplea funciones de probabilidad normal multivariada que sólo están disponibles en esta librería.

En segundo lugar, como se explicó en el Capítulo 3, la esencia en la evolución de una técnica de optimización metaheurística corresponde a la implementación de la función de adaptación del problema, la cual permite explorar y explotar el espacio de soluciones desde la

infactibilidad. A continuación se ilustra la función de adaptación implementada para el despacho de centrales térmicas. Nótese que la función `round(pg[k,h], digits = 4)` se emplean para redondear el valor de la variable de decisión a sus primeros cuatro decimales, esto con el fin de evitar impresiones numéricas durante el proceso de búsqueda de la técnica de optimización.

```
function fitnessf(a,b,c,pmin,pmax,G,H,pg)
    Penalty = 1000; Penalization = 0;
    for h in H
        Pgen = round((sum(pg[k,h] for k in G)),
            digits = 4)
        Pdl = round((PD[h] +
            alpha*sum(sum(pg[k,h]*B[k,m]*pg[m,h]
                for m in G) for k in G)), digits = 4)
        if Pgen < Pdl
            Penalization = Penalization +
            Penalty*abs(Pdl - Pgen) + Penalty;
        end
        for k in G
            if round(pg[k,h], digits = 4) < pmin[k,1]
                Penalization = Penalization +
                Penalty*abs(pmin[k,1]-pg[k,h]) + Penalty;
            end
            if round(pg[k,h], digits = 4) > pmax[k,1]
                Penalization = Penalization +
                Penalty*abs(pg[k,h] - pmax[k,1]) + Penalty;
            end
        end
    end
    Objval = sum(sum(a[k]*pg[k,h]^2+b[k]*pg[k,h]+c[k]
        for k in G) for h in H) + Penalization;
    return Objval
end
```

En tercer lugar, se realiza la implementación del algoritmo de búsqueda por vórtices (ver Algoritmo 9) a través de una función `ABV(Ns, Nv, tmax, lb, ub)`. Esto se ilustra a continuación.

```
function ABV(Ns,Nv,tmax,lb,ub)
global Mu = (1/2)*(lb + ub);
global gmin = Inf; beta = 1e-3;
for t = 1:tmax
    global r = (1+beta-t/tmax)*exp(-6*t/tmax)
    *(ub-lb).*rand(Nv,1)/2;
    global Sigma = zeros(Nv,Nv);
    for k = 1:Nv
        global Sigma[k,k] = r[k];
    end
    global X = rand(MvNormal(Mu[:,1], Sigma),Ns)';
    global Objvalx = zeros(Ns,1);
    for k = 1:Ns
        for j = 1:Nv
            if X[k,j] < lb[j,1]
                global X[k,j] = lb[j,1];
            elseif X[k,j] > ub[j,1]
                global X[k,j] = ub[j,1];
            end
        end
        global xaux = zeros(Ng,Nt);
        for j in G
            global xaux[j,:] = X[k,(j-1)*Nt+1:j*Nt]
        end
        global Objvalx[k,1] =
            fitnessf(a,b,c,pmin,pmax,G,H,xaux);
    end
    global minpos = argmin(Objvalx);
    global fmin = Objvalx[argmin(Objvalx)];
    global Iterbest = X[minpos[1],:];
    if fmin < gmin
        global gmin = fmin; global xbest = Iterbest;
        global Mu = xbest;
    end
    println([t, gmin])
end
return xbest, gmin
end
```

De la implementación anterior, es importante mencionar que:

- i. En la función de reducción del radio, i.e., r_t , se ha agregado una constante denominada `beta` con un valor de 1×10^{-3} , esto con el fin de asegurar que la función de probabilidad genere el vecindario cumpliendo las características de invertibilidad requeridos en las distribuciones normales de probabilidad (i.e., que pueda ser descompuesta a través de la forma de *Cholesky*) (Dostál et al., 2011).
- ii. La función `X = rand(MvNormal(Mu[:,1], Sigma),Ns)'` crea el vecindario de soluciones uniformemente distribuidas, por medio de la definición (9.2), a partir de la matriz de covarianzas y el radio. Nótese que en este caso, se crea toda la población vecina, para lo cual se le agregó a la función la entrada `Ns`, que en este caso, corresponde al número de potenciales soluciones.

Observación 9.4. En la implementación de la función que recrea el algoritmo de búsqueda de vórtices `ABV(Ns, Nv, tmax, lb, ub)`, no se ha implementado el criterio de parada local por no mejoras consecutivas de la función objetivo, por lo cual se invita al lector a proponer las modificaciones necesarias para realizar dicha implementación.

Con el fin de desarrollar una evaluación estadística básica de la implementación de la metodología de optimización de búsqueda de vórtices para resolver el problema de despacho económico multiperíodo en centrales térmica, se propone realizar múltiples veces la ejecución del algoritmo, que como se muestra a continuación, se realizó para 100 repeticiones, i.e., $rep = 100$.

```
using Printf, Statistics, BenchmarkTools
t1 = time(); rep = 100;
Ns = 1000; Ng = size(Generadores,1);
Nt = size(Periodos,1); Nv = Ng*Nt;
global tmax = 5000;
lb = zeros(Nv,1); ub = zeros(Nv,1)
for j in G
    global lb[(j-1)*Nt+1:j*Nt] = pmin[j]*ones(Nt,1);
    global ub[(j-1)*Nt+1:j*Nt] = pmax[j]*ones(Nt,1);
end
global Guardar = zeros(rep,Nv+1);
    for r = 1:rep
        ABV(Ns,Nv,tmax,lb,ub)
        global Guardar[r,1:Nv] = xbest;
        global Guardar[r,Nv+1] = gmin;
    end
global Guardar = Guardar[sortperm(Guardar[:, Nv+1]), :];
et = time() - t1;
@printf("Valor maximo: %2.1f\n",Guardar[rep,Nv+1])
@printf("Valor minimo: %2.1f\n",Guardar[1,Nv+1])
@printf("Valor promedio: %2.1f\n",mean(Guardar[:,Nv+1]))
@printf("Desviacion estandar: %2.1f\n",std(Guardar[:,Nv+1]))
@printf("Maximo - Media: %2.1f\n",Guardar[rep,Nv+1]-mean(Guardar[:,Nv+1]))
println("Tiempo promedio: ",
round(et/rep, digits = 4), " s");
Power = Guardar[1,1:Nv]';
Pg = zeros(Ng,Nt);
for k in G
    @printf("Gen. %d \t",k)
    Pg[k,:] = Power[(k-1)*Nt+1:k*Nt];
end
@printf("\n")
for h in H
    for k in G
```

```

        @printf("%0.5f \t",Pg[k,h])
    end
    @printf("\n")
end

```

La principal característica de la implementación anterior, es que se ha definido un tamaño de vecindario (o población) de 1000 individuos, con un total de 5000 iteraciones. La variable `Guardar` almacena la mejor solución encontrada en cada repetición, es decir, los valores de `xbest` y `gmin`, con el fin de realizar el estudio estadístico de los resultados por medio de la librería `Statistics`. Finalmente, una vez se realiza la ejecución del algoritmo de búsqueda por vórtices con sus correspondientes repeticiones, se obtienen los resultados que se muestran a continuación.

```

Valor maximo: 83191.9
Valor minimo: 82556.2
Valor promedio: 82800.6
Desviacion estandar: 147.0
Maximo - Media: 391.3
Tiempo promedio: 358.7319 s

```

Gen. 1	Gen. 2	Gen. 3
72.96550	58.65187	83.30289
6.03550	63.97388	82.93094
70.81314	63.44551	187.78881
138.54826	93.92321	242.51227
30.18969	68.24467	138.29103
10.29903	32.06223	59.03194
134.68537	38.61262	158.74123
140.35098	99.34730	246.44848
55.40703	34.25462	115.11214
9.09135	17.58303	54.20134
82.00124	74.67773	208.76205
148.26028	99.29946	249.70749
11.40700	42.20275	130.86027
22.40009	43.70544	65.93343
112.15370	97.39010	187.69373
119.33222	80.50008	242.26869

MÉTODOS DE OPTIMIZACIÓN METAHEURÍSTICA PARA INGENIERÍA: Con aplicaciones en el software Julia

55.61713	72.65413	118.52981
45.05394	52.49149	96.67152
132.15029	96.03633	190.83947
92.66867	55.34500	217.48431
61.82603	25.06025	96.88376
73.54572	67.87664	147.92220
112.93183	81.17441	248.18756
44.30203	75.69717	105.78015

Los resultados obtenidos muestran que: (i) la mejor solución encontrada en el proceso iterativo, i.e., 8416.0 corresponde al valor reportado por la metodología de solución exacta con el solver `HiGHS` para problemas de programación lineal entera estudiado en el Capítulo 2; (ii) los valores mínimo y promedio obtenidos en esta evaluación fueron 8248.0 y 8395.5, con una desviación estándar en valor real de 20.5, y (iii) en el tiempo promedio de ejecución del algoritmo fue de 1.1907 s.

Observación 9.5. Los resultados obtenidos en esta simulación son únicamente con fines ilustrativos, ya que estos dependen del computador en el que sea ejecutado el algoritmo.

Parte III

Técnicas metaheurísticas en dominio discreto-continuo

10

Algoritmo de optimización de distribución normal generalizada

En este capítulo se presentan la técnica de optimización metaheurística basada en distribución normal generalizada, el cual pertenece a la familia de las metodologías de solución de naturaleza poblacional inspiradas en matemáticas. Este algoritmo explora y explota el espacio de soluciones a través de mecanismos probabilísticos que equilibran ambas etapas, siendo el primero orientado a la explotación local empleando la información de desviación estándar, la media de la población y la mejor solución actual; mientras que la exploración radica en la utilización de información de individuos aleatorios en la población con el fin de orientar las soluciones hacia nuevas regiones del espacio de solución. Esta técnica será empleada para resolver un modelo de programación no lineal entero mixto a través de una codificación híbrida.

10.1 Filosofía e inspiración

El algoritmo de optimización de distribución normal generalizada corresponde a una técnica de optimización metaheurística de naturaleza poblacional de la familia de las técnicas inspiradas en matemáticas. Este algoritmo fue inicialmente propuesto por (Zhang et al., 2020) para resolver el problema de extracción de parámetros de módulos fotovoltaicos. Las principales características de esta técnica de optimización se resumen a continuación (Zhang, 2022).

- i. Evoluciona a partir de una población inicial creada de manera aleatoria dentro del rango de valores permitidos para las variables de decisión.
- ii. Se basa en funciones de probabilidad para generar nuevas soluciones potenciales a partir de la solución actual y la mejor solución contenida en la población. Lo cual se logra a través de la media general de la población, así como la media y desviación estándar generalizada para cada individuo de la población.
- iii. Asigna el 50% de probabilidad a las etapas de exploración y explotación del espacio de soluciones para generar la población descendiente.
- iv. Sólo contiene dos parámetros de control para explorar y explotar el espacio de soluciones, siendo estos el tamaño de la población y el número de iteraciones.

En la literatura especializada, el algoritmo de optimización de distribución normal generalizada ha sido ampliamente utilizado para resolver problemas de ciencia e ingeniería que involucran problemas de naturaleza continua, así como problemas de dominio discreto. Algunas de estas aplicaciones incluyen: selección óptima de conductores en sistemas de distribución trifásicos desbalanceados (Vega-Forero et al., 2023),

solución de problemas de diseño mecánico (Mehta et al., 2023), estimación de parámetros en transformadores de distribución monofásicos (Camelo-Daza et al., 2024), despacho económico de centrales térmicas considerando penetración de generación renovable (Hoque et al., 2023), clasificación de las características más relevantes en paquetes de información (Tubishat et al., 2022) y ubicación y dimensionamiento óptimo de fuentes fotovoltaicas en redes de distribución (Montoya et al., 2022a), entre otros.

10.2 Algoritmo de optimización de distribución normal generalizada

Considerando las exitosas aplicaciones presentadas en la literatura especializada de la técnica de optimización de distribución normal generalizada, el objetivo de este capítulo es presentar de forma general la estructura matemática de este algoritmo, con énfasis particular en sus reglas de avance, corrección de las soluciones y criterios de parada, entre otros aspectos relevantes, tal como se verá en las siguientes subsecciones.

10.2.1 Población inicial

El algoritmo de optimización de distribución normal generalizada de optimización inspirada en las características asignables a una distribución de probabilidad de tipo Gaussiana, es decir, la media y la desviación estándar, las cuales son empleadas para explorar y explotar el espacio de soluciones a partir de una población inicial (Zhang et al., 2020). La población inicial se define como X^t , con t como el contador de iteraciones, que inicia con $t = 0$. Esta población es un arreglo bidimensional de dimensiones $n_s \times n_v$, siendo n_s el número de soluciones (parámetro de control) y n_v el número de variables (definido por el problema de optimización estudiado). En la Ecuación (10.1) se presenta la estructura

general de la población inicial.

$$X^t = \begin{bmatrix} x_{1,1} & x_{1,2} & \cdots & x_{1,j} & \cdots & x_{1,n_v} \\ x_{2,1} & x_{2,2} & \cdots & x_{2,j} & \cdots & x_{2,n_v} \\ \vdots & \vdots & \ddots & \vdots & \ddots & \vdots \\ x_{k,1} & x_{k,2} & \cdots & x_{k,j} & \cdots & x_{k,n_v} \\ \vdots & \vdots & \ddots & \vdots & \ddots & \vdots \\ x_{n_s,1} & x_{n_s,2} & \cdots & x_{n_s,j} & \cdots & x_{n_s,n_v} \end{bmatrix}, \quad (10.1)$$

donde $x_{k,j}$ el valor asignado a la j -ésima variable de decisión en la k -ésima fila de la población. Nótese que para generar estos valores, se aplica la regla definida en (10.2).

$$x_{k,j} = x_j^{\min} + \alpha_j \left(x_j^{\max} - x_j^{\min} \right), \{j = 1, 2, \dots, n_v, k = 1, 2, \dots, n_s\} \quad (10.2)$$

siendo α_j un valor aleatorio generado con una distribución uniforme en el rango de cero a uno, y $x_j^{\min}$ y $x_j^{\max}$ representan los límites inferiores y superiores aplicables a la j -ésima variable.

Observación 10.1. Cuando se tiene problemas de naturaleza discreta o problemas con parte discreta y parte continua en la codificación, la población inicial será generada con distribución uniforme entre los límites inferior y superior asignable a las variables (ver Ecuación (10.2)). Posteriormente, la parte discreta sera redondeada al valor entero más cercano, con el objetivo de preservar la factibilidad de las variables $x_{k,j}$ que pertenezcan al dominio discreto.

10.2.2 Reglas de avance

El algoritmo de optimización de distribución normal generalizada explota y explora el espacio de soluciones a partir de dos estrategias

denominadas búsqueda local y búsqueda global, las cuales son realizadas para cada individuo contenido en la población (Montoya et al., 2022a). Las principales características de cada una de estas etapas se define a continuación.

10.2.2.1 Búsqueda local

La búsqueda local, es una estrategia de explotación del espacio de soluciones en las vecindades de la k -ésima solución x_k^t contenida en la población, con el fin de refinar dicha solución. En el caso del algoritmo de optimización de distribución generalizada, esta búsqueda se hace teniendo en cuenta la mejor solución contenida en la población actual, i.e., $x_{\text{best}}^t \in X^t$, la media generalizada μ_k y desviación estándar generalizada σ_k para el individuo k , así como la media general de la población, i.e., y^t (Zhang et al., 2020). Para obtener la mejor solución de la población actual, se aplica la regla dada en (10.3).

$$x_{\text{best}}^t = \{x_k^t / x_k^t \rightarrow \min(f(x_k^t)), \forall k = 1, 2, \dots, n_s\}, \quad (10.3)$$

siendo $f(x_k^t)$ la función objetivo (también conocida como función de adaptación o desempeño) evaluada en la solución x_k^t . Nótese que la aplicación de la Ecuación (10.3) está pensada para problemas de optimización con sentido de minimización; sin embargo, en el caso de maximización, se deberá cambiar la función mínimo por la función máximo.

Una vez obtenida la mejor solución de la población, se requiere del cálculo de la media general de la población, la cual se denomina y^t . Esta puede ser obtenida como se define en (10.4).

$$y^t = \frac{1}{n_s} \sum_{k=1}^{n_s} x_k^t, \quad (10.4)$$

Teniendo en cuenta la mejor solución contenida en la población y la media de la población, se obtiene la media y la desviación estándar

generalizadas, como se define en (10.5) y (10.6).

$$\mu_k^t = \frac{1}{3} (x_k^t + x_{\text{best}}^t + y^t), \quad (10.5)$$

$$\sigma_k^t = \sqrt{\frac{1}{3} \left((x_k^t - \mu_k^t)^2 + (x_{\text{best}}^t - \mu_k^t)^2 + (y^t - \mu_k^t)^2 \right)}, \quad (10.6)$$

En el caso de la búsqueda local, el potencial individuo que hará parte de la solución del problema toma la forma dada en la Ecuación (10.7).

$$v_k^t = \mu_k^t + \sigma_k^t \cdot \eta \quad (10.7)$$

donde $\sigma_k^t \cdot \eta$ representa el producto elemento a elemento entre los vectores μ_k^t y η , siendo η definido como sigue:

$$\eta = \begin{cases} \sqrt{-\log(\lambda_1)} \cos(2\pi\lambda_2) & r_1 \leq r_2 \\ \sqrt{-\log(\lambda_1)} \cos(2\pi\lambda_2 + \pi) & r_1 > r_2 \end{cases} \quad (10.8)$$

donde λ_1 y λ_2 son vectores de dimensión $n_v \times 1$ generados con distribución uniforme en el rango de cero a uno, y los parámetros r_1 y r_2 son escalares en el rango de cero a uno generados aleatoriamente.

Observación 10.2. Las funciones de raíz cuadrada, logaritmo y coseno definidas en (10.6) y (10.8) son aplicadas a operaciones vectoriales, con lo cual estas son aplicadas en particular a cada elemento del vector resultante, es decir, son funciones de $g(\mathbf{x}) : \mathbb{R}^{n_v} \rightarrow \mathbb{R}^{n_v}$.

10.2.2.2 Búsqueda global

La búsqueda global en el algoritmo de distribución normal generalizada hace referencia a la exploración del espacio de soluciones en regiones

que aún no han sido revisadas con profundidad (Zhang, 2022), en esencia, este es el mecanismo que le permite a esta técnica metaheurística escapar de soluciones óptimas locales en búsqueda de la solución global. Para realizar esta búsqueda, teniendo en cuenta la solución x_k^t , se general tres puntos sobre la población, i.e., (p_1, p_2, p_3) , tal que $p_1 \in \{1, \dots, n_s\}$, $p_2 \in \{1, \dots, n_s\}$ y $p_3 \in \{1, \dots, n_s\}$, siendo $p_1 \neq p_2 \neq p_3 \neq k$, es decir, tres soluciones contenidas en la población X^t distintas entre si y diferentes del k -ésimo individuo analizado.

Teniendo en cuenta las características asignables a los puntos p_1 , p_2 y p_3 , el algoritmo de distribución normal generalizada define dos vectores de seguimiento v_1 y v_2 , con la estructura que se muestra a continuación.

$$v_1 = \begin{cases} x_k^t - x_{p_1}^t & f(x_k^t) \leq f(x_{p_1}^t) \\ x_{p_1}^t - x_k^t & f(x_k^t) > f(x_{p_1}^t) \end{cases} \quad (10.9)$$

$$v_2 = \begin{cases} x_{p_2}^t - x_{p_3}^t & f(x_{p_2}^t) \leq f(x_{p_3}^t) \\ x_{p_3}^t - x_{p_2}^t & f(x_{p_2}^t) > f(x_{p_3}^t) \end{cases} \quad (10.10)$$

Observación 10.3. El cálculo de los vectores de seguimiento v_1 y v_2 dados por (10.9) y (10.10) está definido para problemas de minimización, en caso contrario, los símbolos de menor igual que y mayor que deberán ser intercambiados.

Una vez se han definido los vectores de seguimiento v_1 y v_2 , se procede a obtener la potencial solución v_k^t , como se muestra en (10.11).

$$v_k^t = x_k^t + \beta \times (|\lambda_3| \times v_1) + (1 - \beta) \times (|\lambda_4| \times v_2), \quad (10.11)$$

siendo λ_3 y λ_4 y β valores escalares aleatorios que define la importancia de cada vector de seguimiento y son generados con distribución uniforme en el rango de cero a uno.

Observación 10.4. Los parámetros λ_3 y λ_4 en la propuesta original de los autores de (Zhang et al., 2020) corresponden a escalares, sin embargo, en caso de problemas de optimización más complejos, estos podrían corresponder a vectores con distribución uniforme con la misma estructura de los vectores λ_1 y λ_2 .

10.2.3 Sustitución en la población

Una vez se han generado las potenciales nuevas soluciones, i.e., v_k^t , se requiere conocer si estas harán parte de la población en la siguiente iteración, para lo anterior, es necesario conocer el valor de la función de desempeño $f(y_k^t)$. Sin embargo, será necesario antes de evaluar en la función de adaptación, verificar el cumplimiento de los límites mínimos y máximos de las variables de decisión, como sigue:

$$v_{k,j}^t = \begin{cases} v_{k,j} & x_j^{\min} \leq v_{k,j} \leq x_j^{\max} \\ x_j^{\min} & v_{k,j} < x_j^{\min} \\ x_j^{\max} & v_{k,j} > x_j^{\max} \end{cases}, \quad \{\forall j = 1, 2, \dots, n_v\}. \quad (10.12)$$

Una vez revisada la potencial solución v_k^t , se procede a realizar la sustitución en la población, aplicando el criterio definido en (10.13).

$$x_k^t = \begin{cases} v_k^t, & f(v_k^t) < f(x_k^t) \\ x_k^t, & f(v_k^t) \geq f(x_k^t) \end{cases} \quad (10.13)$$

Es importante aclarar que la regla de substitución de la población definida en (10.13) es aplicable a problemas de minimización; sin embargo, en el caso de maximización se cambian los signos de menor que y mayor igual que, por símbolos de mayor que y menor igual que, respectivamente.

Observación 10.5. La selección del potencial individuo solución v_k^t según la búsqueda local dada por (10.7) o la búsqueda global dada en (10.11) estará determinada con una probabilidad de 50%, es decir, v_k^t se obtendrá de la Ecuación (10.7) si $\delta \leq 1/2$, en otro caso, este será obtenido a partir de la Ecuación (10.11). Nótese que el valor del parámetro δ corresponde a un valor aleatorio en el rango de cero a uno generado para cada individuo de la población.

10.2.4 Criterios de parada

La finalización del proceso de exploración y explotación del espacio de soluciones mediante el algoritmo de distribución normal generalizada, se dará cuando:

- i. se alcance el número máximo de iteraciones preestablecido por el programador, o
- ii. si durante $m_{\max}$ número de iteraciones la función de desempeño no presenta ninguna mejora en su valor.

10.2.5 Implementación computacional

En el Algoritmo 10 se describe la implementación general de la técnica de optimización de distribución normal generalizada.

Data: Datos del problema de optimización bajo análisis

Definir $x^{\min}$, $x^{\max}$, n_s , $t_{\max}$, $m_{\max}$ y n_v ;

Crear la población inicial con (10.1) empleando la regla (10.2) con $t = 0$;

Evaluar la función de adaptación $f(x_k^t) \forall k = 1, 2, \dots, n_s$;

Determinar la solución x_{best}^t usando la Ecuación (10.3) y hacer $m = 0$;

for $t = 1 : t_{\max}$ **do**

 Obtener el valor de la media generalizada y^t empleando la fórmula (10.4);

for $k = 1 : n_s$ **do**

 Generar δ , λ_1 , λ_2 , λ_3 , λ_4 y β ;

if $\delta \leq 1/2$ **then**

 Calcular μ_k^t con (10.5) y σ_k^t con (10.6);

 Generar r_1 y r_2 y definir el vector η usando la Ecuación (10.8);

 Construir el vector v_k^t empleando la regla de avance dada en (10.7);

else

 Determinar los valores de los puntos p_1 , p_2 y p_3 ;

 Obtener los vectores de seguimiento v_1 y v_2 aplicando las formulas (10.9) y (10.10);

 Construir el vector v_k^t empleando la regla de avance dada en (10.11);

 Revisar/corregir el vector v_k^t empleando la Ecuación (10.12);

 Aplicar la regla de sustitución en la población dada en (10.13);

 Actualizar la mejor solución x_{best}^t ;

if $f(x_{\text{best}}^t) == f(x_{\text{best}}^{t-1})$ **then**

 Hacer $m = m + 1$;

else

 Hacer $m = 0$;

if $m \geq m_{\max}$ **then**

 Reportar la mejor solución actual contenida en x_{best}^t ;

break;

Result: Reportar la mejor solución encontrada, i.e., x_{best}^t

Algoritmo 10: Algoritmo de optimización de distribución normal generalizada

10.3 Implementación en el software Julia

En esta sección se describe la aplicación del algoritmo de optimización de distribución normal generalizada al problema de programación no lineal entero mixto definido por el modelo (2.12)–(2.15). Para ilustrar la implementación de este algoritmo, se consideran los siguientes pasos secuenciales.

El primer paso de la implementación, consiste en definir los datos iniciales del sistema, como sigue:

```
using DataFrames
Parametros = DataFrames.DataFrame([
(1,5,10,0.11, -6, 5), (2, 1, 9, 0.50,-6, 8),
(3,9,10,0.61,-13,10), (4, 1, 4, 0.17,-7,12),
(5,7, 1,0.44,-15, 5), (6, 2, 7, 0.56,-9, 9),
(7,2, 1,0.45,-11,11), (8, 8, 4, 0.60,-5, 5),
(9,3, 5,0.79,-14,14), (10,1, 9, 0.51,-8,14),]);
DataFrames.rename!(Parametros, [:k, :a, :b, :c, :xmin, :xmax]);
a = Parametros.a; b = Parametros.b; c = Parametros.c;
xmin = Parametros.xmin; xmax = Parametros.xmax;
n = size(Parametros,1); N = 1:n;
```

En segundo lugar, se construye la función adaptación para el problema de optimización la cual se denomina `fitnessf(a,b,c,n,y,x,N)`, como se muestra a continuación.

```
function fitnessf(a,b,c,n,y,x,N)
    Penalty = 100;
    Objval=(sum(y[k]*(a[k]*sin(b[k]*x[k]))+c[k]*x[k]^2)
for k in N) +
    Penalty*(max(sum(y[k] for k in N)-n/2,0));
return Objval
end
```

En tercer, se ilustra el proceso iterativo del algoritmo de distribución normal generalizada, el cual se ha construido a partir de una función denominada `function GNDO(X,Nv,Ns,tmax,xmax,xmin)`.

Que recibe la parametrización del algoritmo, es decir, límites superiores e inferiores de las variables, población inicial y número de iteraciones, entre otros.

```
function GNDO (X,Nv,Ns,tmax,xmax,xmin)
    for t = 1:tmax
        global xbest = X[1,:]
        for k = 1:Ns
            global delta = rand(Float64)
            global xkt = X[k,:];
            r1 = rand(Float64); r2 = rand(Float64);
            global lambda1 = rand(Float64,1,Nv+1)';
            global lambda2 = rand(Float64,1,Nv+1)';
            if r1 <= r2
                global eta = sqrt.(-log.(lambda1)).*
                    cos.(2*pi.*lambda2);
            else
                global eta = sqrt.(-log.(lambda1)).*
                    cos.(2*pi*lambda2 +
                        pi.*ones(Nv+1,1));
            end
            beta = rand(Float64);
            global lambda3 = rand(Float64,1,Nv+1)';
            global lambda4 = rand(Float64,1,Nv+1)';
            if delta <= 1/2
                global yt = (1/Ns)*sum(X; dims=1)';
                global ukt = (1/3)*(xbest + xkt + yt);
                global sigmakt = sqrt.((xkt - ukt).^2 +
                    (xbest - ukt).^2 + (yt - ukt).^2);
                global vkt = ukt + eta.*sigmakt;
            else
                p1 = rand(1:Ns); p2 = rand(1:Ns);
                p3 = rand(1:Ns);
                while (p1 == k || p2 == k || p3 == k ||
                    p1 == p2 || p1 == p3 || p2 == p3)
                    p1 = rand(1:Ns); p2 = rand(1:Ns);
                    p3 = rand(1:Ns);
                end
                xp1t = X[p1,:]; xp2t = X[p2,:];
                xp3t = X[p3,:];
```

```
        if xkt[Nv+1] <= xplt[Nv+1]
            global v1 = xkt - xplt;
        else
            global v1 = xplt - xkt;
        end
        if xp2t[Nv+1] <= xp3t[Nv+1]
            global v2 = xp2t - xp3t;
        else
            global v2 = xp3t - xp2t;
        end
        global vkt = beta*abs.(lambda3).*v1 +
            (1-beta)*abs.(lambda4).*v2;
    end
    global vkt[1:n] = abs.(round.(vkt[1:n]));
    for m = 1:Nv
        if m <= n
            if vkt[m] > 1
                global vkt[m] = 1;
            elseif vkt[m] < 0
                global vkt[m] = 0;
            end
        else
            if vkt[m] < xmin[m-n]
                global vkt[m] = xmin[m-n];
            elseif vkt[m] > xmax[m-n]
                global vkt[m] = xmax[m-n];
            end
        end
    end
    global y1 = vkt[1:n];
    global x1 = vkt[n+1:Nv];
    global Objval = fitnessf(a,b,c,n,y1,x1,N);
    global vkt[Nv+1] = Objval;
    if Objval < X[k,Nv+1];
        X[k,:] = vkt
    end
end
X = X[sortperm(X[:, Nv+1]), :]
end
```

```
    return xbest  
end
```

Un detalle importante de la implantación anterior corresponde al uso de la función `sortperm()`, la cual permite ordenar un vector o una matriz en forma ascendente. En el caso de matrices, estas se ordenan respecto de una columna específica, así pues, para el caso de la población inicial, esta es ordenada respecto de la última columna, que es donde se guardado el valor de la función de adaptación, para ello se emplea el comando `X = X[sortperm(X[:, Nv+1]), :]`. Por otro lado, se invita al lector a revisar la forma en la que se han generado los valores de los parámetros λ_3 y λ_4 , ya que se ha empleado una estructura vectorial, sin embargo, sería interesante revisar el impacto que tienen estos, cuando son generados de manera escalar.

Observación 10.6. En la implementación de la función `function GNDO(X, Nv, Ns, tmax, xmax, xmin)`, que determina la secuencia general del algoritmo de distribución normal generalizada, no se ha implementado el criterio de parada local por no mejoras consecutivas de la función objetivo, por lo cual se invita al lector a proponer las modificaciones necesarias para realizar dicha implementación.

Con el fin de desarrollar una evaluación estadística básica de la implementación de la metodología de optimización de distribución normal generalizada para resolver el problema de la mochila, se propone realizar múltiples veces la ejecución del algoritmo, que como se muestra a continuación, se realizó para 100 repeticiones, i.e., `rep = 100`.

```
using Printf, Statistics, BenchmarkTools, Random  
t1 = time(); rep = 100;  
Ns = 10000; Nv = 2*n; tmax = 100;  
global Guardar = zeros(rep, Nv+1);  
    for r = 1:rep
```

```

global X = rand(Float64,Ns,Nv+1);
for j = 1:Ns
    for k = 1:Nv
        if k <= n
            X[j,k] = rand(0:1);
        else
            X[j,k] = xmin[k-n] + rand(Float64)
            * (xmax[k-n]-xmin[k-n]);
        end
    end
    global y = X[j,1:n]
    global x = X[j,n+1:Nv];
    global X[j,Int(Nv+1)] =
    fitnessf(a,b,c,n,y,x,N)
end
X = X[sortperm(X[:, Nv+1]), :]
GNDO(X,Nv,Ns,tmax,xmax,xmin)
global Guardar[r,:] = xbest;
println([r, Guardar[r,Nv+1]])
end
global Guardar = Guardar[sortperm(Guardar[:, Nv+1]), :];
et = time() - t1;
@printf("Valor maximo: %2.4f\n",Guardar[rep,Nv+1])
@printf("Valor minimo: %2.4f\n",Guardar[1,Nv+1])
@printf("Valor promedio: %2.4f\n",mean(Guardar[:,Nv+1]))
@printf("Desv. estandar: %2.4f\n",std(Guardar[:,Nv+1]))
@printf("|Minimo - Media|: %2.4f\n",abs(Guardar[1,Nv+1]- mean(Guardar[:,Nv+1])))
println("Tiempo promedio: ",
round(et/rep, digits = 4), " s");
y = Guardar[1,1:n]; x = Guardar[1,n+1:Nv];
@printf("Binaria\tContinua\n")
for k in N
    @printf("%0.0f\t%0.5f \n",y[k],x[k])
end

```

Es esta implementación es importante mencionar que:

- i. El algoritmo de distribución normal generalizada se ha ajustado para un tamaño de población de 10000 individuos. Esto se hace debido a que el problema tiene una combinación de variables binarias y continuas, con lo cual se garantiza que haya gran diversidad en cuanto a la componente continua del espacio de soluciones. Además, cada búsqueda se realiza con un total de 100 iteraciones.
- ii. La población inicial se construye por etapas, donde las primeras n variables corresponden a la parte binaria del problema y la siguiente parte, es decir, desde $n + 1$ hasta $2n$ corresponde a la parte continua del espacio de soluciones.
- iii. El vector $y = X[j, 1:n]$ guarda los valores de las variables binarias, mientras que el vector $x = X[j, n+1:Nv]$ hace lo correspondiente para las variables continuas.
- iv. La matriz `Guardar[rep, 1:Nv]` contiene todas las soluciones del proceso de repetición, y como este es un problema de minimización, y la matriz ha sido ordenada en forma ascendente, siempre la mejor solución estará en la primera fila, y la peor solución en la última fila.

Nótese que una vez se realiza la ejecución del algoritmo de distribución normal generalizada con sus correspondientes repeticiones, se obtienen los resultados que se muestran a continuación.

```
Valor maximo: -28.9002
Valor minimo: -30.8501
Valor promedio: -30.3673
Desv. estandar: 0.6823
|Minimo - Media|: 0.4827
Tiempo promedio: 72.9788 s
Binaria Continua
1      -0.15703
0      -0.25376
```

1	-0.15686
0	-1.19386
1	-1.39483
0	0.86804
0	-0.29886
1	-0.38904
1	-0.30761
0	0.74630

Los resultados obtenidos muestran que: (i) la mejor solución encontrada en el proceso iterativo, i.e., -30.8501 corresponde al valor reportado por la metodología de solución exacta con el solver `Ipopt` para problemas de programación lineal entera estudiado en el Capítulo 2; (ii) los valores máximo y promedio obtenidos en esta evaluación fueron -28.9002 y -30.3673, con una desviación estándar en valor real de 0.6823, y (iii) en el tiempo promedio de ejecución del algoritmo fue de 72.9788s. Nótese que el error absoluto entre el óptimo y el valor medio es de 0.4827, lo que implica que en promedio el algoritmo tiene soluciones muy cercanas a su valor medio, más precisamente, con una variación de 1.5647% respecto al valor promedio.

Observación 10.7. Los resultados obtenidos en esta simulación son únicamente con fines ilustrativos, ya que estos dependen del computador en el que sea ejecutado el algoritmo, así como de los parámetros asignados por el programador.

11

Algoritmo de optimización por cúmulo de partículas

En este capítulo se presenta el proceso iterativo del algoritmo de optimización por cúmulo de partículas (PSO). Este algoritmo es uno de los pilares de los algoritmos de optimización basados en poblaciones. Siendo sus principales atributos la implementación del conocimiento local de la población de individuos o partículas (componente cognitivo), asociado a cada individuo. Así como la utilización del conocimiento global, relacionado con la mejor solución de la población (componente social). En cada iteración del PSO, este le brinda la posibilidad a la población de actualizar la mejor solución asociada a cada individuo, así como la global, basándose en la función adaptación de la población de partículas en cada iteración para tomar esta decisión, y siendo la mejor solución global la solución del problema. El PSO emplea valores aleatorios dentro de la generación de nuevos individuos para escapar

de óptimos locales, mejorando la exploración del espacio de solución, siendo un algoritmo clásico en la solución de problemas con variables de dominio continuo. Sin embargo, el aporte del PSO en este documento es la implementación de este para brindar solución a problemas de variables de naturaleza discreta y continua (mixtos).

11.1 Filosofía e inspiración

El algoritmo de optimización PSO, plantea su estrategia de avance en la forma en que cazan las bandadas de aves y bancos de peces (Kennedy and Eberhart, 1995). Este tipo de animales basan su estrategia de casa en dos componentes (Grisales-Noreña et al., 2018): (i.) el primer componente conocido como componente cognitivo, está asociado a la inteligencia de cada individuo dentro de la nube de partículas (población). El cual se mueve dentro del espacio de solución, recordando las posiciones que le permitieron hallar la mejor solución, y observando al líder de la nube para realizar cada desplazamiento dentro del espacio de solución. (ii.) la segunda componente, conocida como la social, corresponde al conocimiento de la nube de partículas. En cada iteración se localiza la mejor solución dentro del espacio de solución (nube). Esta solución es la líder de la población de partículas y es encargada de guiar la búsqueda en cada iteración. El líder puede cambiar iteración a iteración, siendo seleccionado dentro de los individuos de la población por medio de la comparación de las funciones adaptación alcanzadas por estos. Empleando la componente cognitiva y social, valores aleatorios y una variable de control de la exploración conocido como inercia; el PSO en cada iteración del proceso iterativo desplaza los individuos de la población "partículas" dentro del espacio de solución, con el objetivo de encontrar la mejor solución posible. Dentro de las principales características presentadas por el PSO se encuentran:

- i. Emplea una población para analizar el espacio de la solución, donde cada individuo lo identifica como partícula. La cual analiza

y almacena la mejor solución local y global en cada iteración por medio de la evaluación de la función adaptación.

- ii. Iteración a iteración, para cada individuo se genera un vector de velocidades que permite el desplazamiento de la población dentro del espacio de solución. Empleando el conocimiento propio y global de la solución, la inercia, y valores aleatorios para salir de óptimos locales. Permitiendo a la posición de una partícula ser actualizada en términos de su función adaptación.
- iii. Dentro del proceso iterativo, en cada iteración se actualiza la mejor solución del problema y su posición dentro del espacio de solución. Esto permite a las partículas intercambiar el lugar del líder dentro de la exploración, y guiar el resto de partículas hacia la búsqueda de una mejor solución. Cuando el proceso de búsqueda del PSO finaliza, todas las partículas convergen a una misma solución, lo cual en la naturaleza del algoritmo indica que los individuos llegan al punto de mejor alimento dentro del espacio de solución a consumir el alimento.
- iv. El PSO emplea como criterio de convergencia un número máximo de iteraciones e iteraciones de no mejora. Teniendo en cuenta 7 parámetros de optimización que le permiten ajustarse en mejor medida al problema de optimización asignado. Siendo esta una de las estrategias de optimización con mayor número de parámetros de optimización.

El algoritmo PSO es considerado un referente dentro de las técnicas de optimización metaheurísticas, permitiendo dar solución a problemas con variables de dominio continuo, discreto y mixto (Jain et al., 2022), como es el caso analizado en este documento. Siendo empleado para brindar solución a múltiples problemas dentro de diferentes campos del saber existentes en el mundo: flujo óptimo de potencia (Shayeghi et al., 2009), dimensionamiento y localización de recursos energéticos

distribuidos (Cortés-Caicedo et al., 2023), estrategias de gestión de la energía en microrredes (Grisales-Noreña et al., 2023), modelado y control de robots (Aner et al., 2024), diagnóstico de fallas (Hong and Pula, 2024), predicción de la temperatura del carbón (Guo et al., 2024), diseño mecánico (Cabal et al., 2019), entre otros.

A continuación se describe la metodología del algoritmo PSO de forma general y se presenta la aplicación en JULIA de este para brindar solución al problema de programación no lineal entera mixta propuesto en la Sección 2.3 de este documento.

11.2 Algoritmo PSO

Dentro de esta sección se describen los pasos fundamentales del proceso iterativo del PSO (Grisales-Noreña et al., 2018).

11.2.1 Inicialización de parámetros

Al inicio del proceso iterativo del PSO se deben cargar los parámetros que le permiten la exploración y explotación del espacio de solución. Los primeros parámetros de control corresponde al número máximo de iteraciones ($tmax$) e iteraciones de no mejora ($nmmax$); encargados de regular la exploración del algoritmo. Posteriormente, se debe fijar el número de partículas o individuos que componen la población (n_s), número de variables que componen el problema (n_v), inercia máxima y mínima ($Inermax$ e $Inermin$), factor de componente cognitivo y social ($Phi1$ y $Phi2$), y velocidad mínima y máxima ($Velmax$ y $Velmin$). A diferencia de otros algoritmos descritos en secciones anteriores, la nomenclatura del número de iteraciones de no mejora se cambia un poco, por el uso de variables dentro del desarrollo de la Julia. Siendo, todos los parámetros anteriores, variables de control de la convergencia del PSO. Excepto N_v que está relacionada con el problema bajo análisis.

11.2.2 Población inicial

El PSO explora el espacio de solución identificando soluciones de buena calidad por medio de la implementación de una población, nube de partículas, que actualiza la posición de sus individuos dentro del espacio de solución en cada iteración. La Ecuación (11.1) presenta en forma matricial la población, siendo el número de filas asociado al número de partículas y las columnas al número de variables que componen el problema.

$$X^t = \begin{bmatrix} x_{1,1} & x_{1,2} & \cdots & x_{1,j} & \cdots & x_{1,n_v} \\ x_{2,1} & x_{2,2} & \cdots & x_{2,j} & \cdots & x_{2,n_v} \\ \vdots & \vdots & \ddots & \vdots & \ddots & \vdots \\ x_{k,1} & x_{k,2} & \cdots & x_{k,j} & \cdots & x_{k,n_v} \\ \vdots & \vdots & \ddots & \vdots & \ddots & \vdots \\ x_{n_s,1} & x_{n_s,2} & \cdots & x_{n_s,j} & \cdots & x_{n_s,n_v} \end{bmatrix}, \quad (11.1)$$

La población o nube de partículas en cada iteración (X^t), corresponde a una matriz de dimensiones $n_s \times n_v$, siendo t el contador de iteraciones. En esta matriz $x_{k,j}$ corresponde al valor asignado a la j -ésima variable del k -ésimo individuo. Este valor siempre debe respetar los límites mínimos y máximos para la j -ésima variable del problema.

$$x_{k,j}^1 = x_j^{\min} + \alpha_j (x_j^{\max} - x_j^{\min}), \quad \{\forall j = 1, 2, \dots, n_v, \forall k = 1, 2, \dots, n_s\} \quad (11.2)$$

La nube de partículas inicial se genera de forma aleatoria empleado la ecuación (11.2). En esta ecuación $x_j^{\min}$ y $x_j^{\max}$, corresponden a los valores máximos y mínimos asignados a la j -ésima variable. Siendo α_j un valor aleatorio en el rango de cero a uno con distribución normal. Al ser la población inicial generada en la primera iteración, t toma un valor de uno.

Adicionalmente, dentro del proceso iterativo, el algoritmo PSO emplea una matriz de velocidades que permite el desplazamiento de

las partículas dentro del espacio de solución iteración a iteración. Esta matriz se debe generar desde la primera iteración del algoritmo, siendo este proceso llevado de forma aleatoria. La matriz de velocidades Vel se presenta en la Ecuación ,

$$Vel^t = \begin{bmatrix} vel_{1,1} & vel_{1,2} & \cdots & vel_{1,j} & \cdots & vel_{1,n_v} \\ vel_{2,1} & vel_{2,2} & \cdots & vel_{2,j} & \cdots & vel_{2,n_v} \\ \vdots & \vdots & \ddots & \vdots & \ddots & \vdots \\ vel_{k,1} & vel_{k,2} & \cdots & vel_{k,j} & \cdots & vel_{k,n_v} \\ \vdots & \vdots & \ddots & \vdots & \ddots & \vdots \\ vel_{n_s,1} & vel_{n_s,2} & \cdots & vel_{n_s,j} & \cdots & vel_{n_s,n_v} \end{bmatrix}, \quad (11.3)$$

Dentro de la matriz de velocidades, $vel_{k,j}$ corresponde al valor asignado de velocidad a la j -ésima variable del k -ésimo individuo. Este valor siempre debe respetar los límites mínimos y máximos de $Velmin$ y $Velmax$, los cuales se generalizaron para todas las variables del problema abordado en este capítulo. Siendo posible controlar la velocidad de cada variable y aumentar el número de parámetros de optimización del problema en el caso en que esto sea necesario. La Ecuación (11.4) permite la generación aleatoria de la matriz de probabilidades en la primera iteración.

$$vel_{k,j}^1 = Velmin + \alpha_j (Velmax - Velmin), \\ \{j = 1, 2, \dots, n_v, k = 1, 2, \dots, n_s\} \quad (11.4)$$

Observación 11.1. En la generación de la primera población y matriz de velocidades no se hace necesario verificar los límites dado que las ecuaciones que las generan garantizan los valores dentro de los límites asignados. Presentando las variables discretas, la necesidad de ser ajustadas por medio de la función de redondeo dentro de los rangos de los valores discretos de cada problema, dado que se generan inicialmente en el dominio continuo por la naturaleza del PSO.

Finalmente, para generar la primera población de individuos se aplica la Ecuación (11.5). La cual permite desplazar la población dentro del espacio de solución considerando el efecto de la matriz de velocidades. Después de ejecutar esta ecuación, se hace necesario redondear y ajustar las variables discretas del problema. Así como analizar si los límites de las variables continuas están en sus rangos, fijándolas en los límites asignados a cada variable, en el caso en que estos sean sobrepasados.

$$X^t = X^t + Vel^t, \quad (11.5)$$

Dentro del proceso de ajuste de las componentes de la nube de partículas, se redondean cada una de las variables discretas contenidas dentro de la matriz ($\text{round}(x_{k,j})$). Siendo la función round de Julia encargada de redondear cada variable (esta función puede cambiar en otros tipos de lenguajes de programación). Posteriormente, se garantiza que las variables discretas no violen los límites asignados para el problema. Para la verificación de los límites de las variables discretas y continuas se aplica la Ecuación (11.6).

$$x_{k,j} = \begin{cases} x_j^{\min} & x_{k,j} < x_j^{\min} \\ x_j^{\max} & x_{k,j} > x_j^{\max} \end{cases}, \quad \{\forall j = 1, 2, \dots, n_v, \forall k = 1, 2, \dots, n_s\} \quad (11.6)$$

Observación 11.2. La verificación de los límites de las variables contenidas dentro de la nube de partículas garantizan la factibilidad de cada una de las soluciones. Esta acción evita generar penalizaciones adicionales dentro de la función adaptación, agilizando la convergencia de los algoritmos. A criterio del programador, la factibilización de las soluciones puede ser incluida como una penalización de la función adaptación.

11.2.3 Evaluación de la función adaptación y selección de mejor solución local y global

Posterior a la generación de la nube de partículas, se realiza la evaluación de la función adaptación de cada una de las partículas. Posterior a esto, al ser la primera iteración del PSO, se almacena la posición actual de cada partícula dentro de la nube como la mejor solución local (*mejorposi*) y su función adaptación como la mejor solución hallada por la partícula hasta el momento (*aptitudi*). Posteriormente, dentro de la nube se identifica la partícula con mejor solución adaptación y se almacena su posición y función adaptación como la mejor posición (*mejorposg*) y solución global (*aptitudg*). Siendo esta solución, la líder de la nube de partículas.

11.2.4 Reglas de avance

A partir de la segunda iteración y hasta alcanzar el número máximo de iteraciones, el PSO aplica el siguiente criterio de avance:

$$Inercia^t = Inermax - (((Inermax - Inermin) * t) / tmax), \quad (11.7)$$

En cada iteración, el PSO actualiza el valor de la inercia ($Inercia^t$), el cual está en función de los valores máximo y mínimo asignados a esta variable ($Inermax$ e $Inermin$), tomando valores dentro de un rango de 1 a 0. Así como el $tmax$ y valor de la iteración actual t .

$$\begin{aligned} Vel_{k,j}^t = & Vel_{k,j}^{t-1} * Inercia^t + \beta_j * Phi1 * (mejorposi_{k,j} - x_{k,j}^{t-1}) \\ & + \gamma_j * Phi2 * (mejorposg_{k,j} - x_{k,j}^{t-1}), \end{aligned} \quad (11.8)$$

Posteriormente, se calcula el valor de Vel^t empleando la Ecuación (11.8). Para lo cual se utilizan los valores de dicha matriz y nube de partículas en la iteración anterior (Vel^{t-1} y X^{t-1}), la $inercia^t$, las variables contenidas en la mejor solución local (*mejorposi*) y global (*mejorposg*), los factores cognitivos ($Phi1$) y social ($Phi1$), y dos variables β_j y γ_j que toman un valor aleatorio en el rango de cero a uno

con distribución normal en cada iteración del PSO. Siendo estos valores aleatorios empleados con el objetivo de escapar de óptimos locales y realizar una adecuada exploración del espacio de solución. Dentro de esta ecuación $mejorpos_{i,k,j}$ y $mejorpos_{g,k,j}$, corresponden a la j -ésima variable de la k -ésima partícula de la mejor solución local y global.

$$vel_{k,j} = \begin{cases} Velmin & vel_{k,j} < Velmin \\ Velmax & vel_{k,j} > Velmax \end{cases},, \{ \forall j = 1, 2, \dots, n_v, \forall k = 1, 2, \dots, n_s \} \quad (11.9)$$

Después de actualizar la matriz de velocidades, se verifica que todas las componentes de esta se encuentren dentro de los límites mínimos y máximos fijados por el PSO usando la Ecuación (11.9). Estos límites de velocidad están asociados al paso máximo de variación que se le desea permitir a cada variable del problema dentro del espacio de búsqueda. Valores pequeños generarán una mayor explotación del espacio de solución, requiriendo de un tiempo mayor de convergencia para el algoritmo. Un paso grande en el límite de la velocidad, disminuirá el tiempo de convergencia del PSO; sin embargo, lo puede llevar a caer en óptimos locales. Para cada variable puede existir un límite de velocidad diferente. Sin embargo, para este problema se asignó el mismo límite de velocidad para todas las variables, siendo esta una decisión que se debe tomar mediante prueba y error.

Finalmente, el desplazamiento de la población dentro del espacio de solución se genera empleando las Ecuaciones (11.5) y (11.6). Dando inicio a una nueva iteración del algoritmo.

Observación 11.3. Para reducir el tiempo de procesamiento del algoritmo, la construcción de las componentes de la nube de partículas y matriz de velocidades pueden ser construidas de manera matricial. Generando el mismo efecto que al ser generadas componente a componente, como se describió anteriormente.

11.2.5 Sustitución de la mejor solución local y global

Después el desplazamiento de la nube de partículas dentro del espacio de solución, se realiza la evaluación de la función adaptación de cada partícula empleando la función de adaptación. Posteriormente, para cada partícula se realiza la comparación entre la función adaptación de la iteración actual con la mejor función adaptación alcanzada por la partícula dentro del proceso iterativo *aptitudi*. En el caso que la función adaptación de la partícula actual mejore el valor almacenado en *aptitudi*, la *mejorposi* y *aptitudi* son actualizadas. Este proceso se realiza empleando la Ecuación (11.10), la cual aplica para todo i perteneciente al conjunto de partículas que componen la nube. En esta ecuación x_i corresponde al i -ésimo individuo de la población, y $f(x_i)$ a la función adaptación del este individuo.

$$[mejorposi, aptitudi] = \begin{cases} [x_i, f(x_i)] & f(x_i) < aptitudi \\ [mejorposi, aptitudi] & f(x_i) \geq aptitudi \end{cases} \quad (11.10)$$

Posteriormente, aplicando un criterio similar, es actualizada la mejor solución de la nube de partículas empleando la Ecuación (11.11).

$$[mejorposg, aptitudg] = \begin{cases} [x_i, f(x_i)] & f(x_i) < aptitudg \\ [mejorposg, aptitudg] & f(x_i) \geq aptitudg \end{cases} \quad (11.11)$$

11.2.6 Criterios de parada

El PSO converge a una solución cuando el número de iteraciones máximas o número de iteraciones de no mejora es alcanzado. Cuando esto ocurre, el PSO retorna como solución la mejor solución dentro la nube de partículas al momento de alcanzar el criterio de convergencia.

Observación 11.4. Para evitar desperdiciar tiempo de procesamiento en iteraciones que no mejoran la solución. se integró un contador de

iteraciones de no mejoras, el cual finaliza el proceso de búsqueda en el caso de que sea alcanzado un número de iteraciones de no mejorar máximas (*nmmmax*). Siendo otro parámetro de sintonización del PSO.

11.2.7 Implementación computacional

El algoritmo 11 presenta el proceso iterativo del PSO, realizando una descripción general del proceso de convergencia.

Data: Lectura de parámetros del problema bajo análisis

Definir los límites de las variables que componen el problema, i.e., n_v , $x^{\min}$ y $x^{\max}$;

Definir los valores de n_s , *Inermax*, *Inermin*, *Phi1*, *Phi2*, *Velmax*, *Velmin*, *tmax*, y *nmmax*;

Inicializar el contador de iteraciones de no mejora *contnm* = 0;

for $t = 1 : tmax$ **do**

if $t = 1$ **then**

 Crear la nube de partículas (x^t) de forma aleatoria empleando la Ecuación (11.2);
 Crear la matriz de velocidades (Vel^t) de forma aleatoria empleando la Ecuación (11.4);
 Actualizar las posiciones de las partículas empleando la Ecuaciones (11.5) y (11.6);
 Evaluar la función de adaptación para cada individuo de la población $f(x_k^t) \forall k = 1, 2, \dots, n_s$;
 Identificar y almacenar *mejorposi* y *aptitudi*;
 Identificar y almacenar *mejorposg* y *aptitudg*;

if $t \geq 2$ **then**

 Calcular la *inercia* ^{t} empleando la Ecuación (11.7);
 Actualizar Vel^t empleando las ecuaciones (11.8) y (11.9);
 Actualizar X^t empleando la Ecuaciones (11.5) y (11.6);
 Evaluar la función de adaptación para cada individuo de la población $f(x_k^t) \forall k = 1, 2, \dots, n_s$;
 Actualizar *mejorposi* y *aptitudi* empleando la Ecuación (11.10);
 Actualizar *mejorposg* y *aptitudg* empleando la Ecuación (11.11);

if $f(x_1^t) == f(x_1^{t-1})$ **then**

 Hacer *contnm* = *contnm* + 1;

else

 Hacer *contnm* = 0;

if $m \geq nmmax$ **then**

 Retornar *mejorposg* y *aptitudg* como solución del problema;
 break;

Result: Reportar la mejor solución encontrada, i.e., *mejorposg*

Algoritmo 11: Algoritmo de optimización PSO

11.3 Implementación en el software Julia

En esta sección se describe la aplicación del algoritmo PSO al problema de programación no lineal entero mixto definido por el modelo (2.12)–(2.15). Para ilustrar la implementación de este algoritmo, se considera lo siguiente.

Inicialmente, se definen los datos iniciales del problema, lo cual se describe a continuación:

```
using DataFrames
Parametros = DataFrames.DataFrame([
(1,5,10,0.11, -6, 5), (2, 1,9,0.50,-6, 8),
(3,9,10,0.61,-13,10), (4, 1,4,0.17,-7,12),
(5,7, 1,0.44,-15, 5), (6, 2,7,0.56,-9, 9),
(7,2, 1,0.45,-11,11), (8, 8,4,0.60,-5, 5),
(9,3, 5,0.79,-14,14), (10,1,9,0.51,-8,14),]);
DataFrames.rename!(Parametros,
[:k, :a, :b, :c, :xmin, :xmax]);
a = Parametros.a; b = Parametros.b; c = Parametros.c;
xmin = Parametros.xmin; xmax = Parametros.xmax;
n = size(Parametros,1); N = 1:n;
```

Posteriormente, se desarrolla la función adaptación para el problema de optimización mixto propuesto, denominada dentro del PSO `fitnessf(a,b,c,n,y,x,N)`, tal y como se presenta a continuación.

```
function fitnessf(a,b,c,n,y,x,N)
    Penalty = 100;
    Objval=(sum(y[k]*(a[k]*sin(b[k]*x[k])+c[k]*x[k]^2)
for k in N) + Penalty*(max(sum(y[k]
for k in N)-n/2,0)));
    return Objval
end
```

Después de tener los datos del problema y la función adaptación programada, se realiza la lectura de los parámetros del PSO y el número de variables del problema (N_v). Como parámetros del PSO encuentran:

el número máximo de iteraciones (t_{max}), número de individuos (N_s), inercia máxima y mínima (In_{max} e In_{min}), factor cognitivo y social (Φ_1 y Φ_2), límite máximo y mínimo de velocidad (Vel_{max} y Vel_{min}), contador de iteraciones de no mejora ($cont_{nm}$) y máximo número de iteraciones de no mejora (nm_{max}). Es importante destacar que se utilizaron los parámetros clásicos usados en la literatura para este algoritmo, reportados en (Grisales-Noreña et al., 2018).

```
global tmax=1000; global Ns=1000; global Nv=2*n;  
global X=zeros(Ns,Nv+1); global Inmax=1;  
global Inmin=0; global Phi1=1.494;  
global Phi2=1.494; global Velmax=0.1;  
global Velmin=-Velmax; contnm=0; nmmax=100;
```

El algoritmo PSO fue desarrollado dentro de la función `pso(Ns,Nv,tmax,Inmax,Inmin,Phi1,...)`. Este recibe como datos de entrada los parámetros del algoritmo y N_v , entregando como dato de salida la solución al problema propuesta por este y la función adaptación asociada.

```
function pso(Ns,Nv,tmax,Inmax,Inmin,Phi1,Phi2,Velmax,  
Velmin)  
    global X=zeros(Ns,Nv+1);  
    for t=1:tmax  
        if t==1  
            global X = rand(Float64,Ns,Nv+1);  
            for j = 1:Ns  
                for k = 1:Nv  
                    if k <= n  
                        X[j,k] = rand(0:1);  
                    else  
                        X[j,k] = xmin[k-n] + rand(Float64)  
                            * (xmax[k-n]-xmin[k-n]);  
                    end  
                end  
            end  
            global Vel= Velmin.*ones(Ns,Nv)  
                + (Velmax-Velmin).*rand(Ns,Nv);  
            X[:,1:Nv]=X[:,1:Nv]+Vel;
```

```

X[:,1:n] = abs.(round.(X[:,1:n]));
for j=1:Ns
    for m = 1:Nv
        if m <= n
            if X[j,m] > 1
                global X[j,m] = 1;
            elseif X[j,m] < 0
                global X[j,m] = 0;
            end
        else
            if X[j,m] < xmin[m-n]
                global X[j,m] = xmin[m-n];
            elseif X[j,m] > xmax[m-n]
                global X[j,m] = xmax[m-n];
            end
        end
    end
end
for j=1:Ns
    global y1 = X[j,1:n]
    global x1 = X[j,n+1:Nv];
    global X[j,Int(Nv+1)] =
        fitnessf(a,b,c,n,y1,x1,N)
end
global mejorposi=X[:,1:Nv];
global aptitudi=X[:,Nv+1];
global aptitudg,
mejorposg = findmin(aptitudi);
global mejorposg=X[mejorposg[1],1:Nv];
end
if t>=2
    global Inercia=Inermax-
        (((Inermax-Inermin)*t)/tmax)
    Vel=Inercia*Vel
    +(Phi1.*rand(Ns,Nv)).*(mejorposi-X[:,1:Nv])
    +(Phi2.*rand(Ns,Nv)).*(repeat(mejorposg', outer
    = [Ns, 1, 1])-X[:,1:Nv]);
    X[:,1:Nv]=X[:,1:Nv]+Vel;
    X[:,1:n] = abs.(round.(X[:,1:n]));

```

```
for j=1:Ns
    for m = 1:Nv
        if m <= n
            if X[j,m] > 1
                global X[j,m] = 1;
            elseif X[j,m] < 0
                global X[j,m] = 0;
            end
        else
            if X[j,m] < xmin[m-n]
                global X[j,m] = xmin[m-n];
            elseif X[j,m] > xmax[m-n]
                global X[j,m] = xmax[m-n];
            end
        end
    end
end
global aux=1;
for j=1:Ns
    global y1 = X[j,1:n]
    global x1 = X[j,n+1:Nv];
    global X[j,Int(Nv+1)] =
    fitnessf(a,b,c,n,y1,x1,N)
    if X[j,Int(Nv+1)] < aptitudi[j,1]
        aptitudi[j,1]=X[j,Int(Nv+1)];
        mejorposi[j,:]=X[j,1:Nv];
    end
    if X[j,Int(Nv+1)] < aptitudg
        aptitudg=X[j,Int(Nv+1)];
        mejorposg=X[j,1:Nv];
        global aux=0;
    end
    if aux==1
        global contnm=contnm+aux;
    else
        global contnm=0;
    end
end
end
```

```
        if contnm==nmmax
            break;
        end
    end
    global xbest=[mejorposg;aptitudg];
    return xbest;
end
```

Dentro de los detalles a destacar dentro de la function `pso(Ns,Nv,tmax,Inermax,Inermin,Phi1,...)` desarrollada en el recuadro anterior, se destaca lo siguiente.

- i. Cada vez que se genera la posición para una nube de partículas, se discretizan las variables que lo requieren y se aplica una función de factibilización que garantiza que la solución codificada dentro de cada individuo garantice los límites impuestos a cada variable.
- ii. La actualización de la matriz de velocidades emplea una formulación matricial que permite agilizar la convergencia del algoritmo, respetando el criterio de avance descrito en secciones anteriores. Dentro de esta formulación, la función `repeat` de Julia permite generar una matriz de Ns individuos compuestos por la *mejorposg* en cada fila. Mientras que la función `rand(Ns,Nv)` genera una matriz aleatoria de valores reales de rango [0-1].

Con el fin de desarrollar una evaluación estadística básica de la implementación del PSO para resolver el problema de programación no lineal entera mixta, se propone realizar múltiples veces la ejecución del algoritmo, que como se muestra a continuación, se realizó para 100 repeticiones, i.e., `rep = 100`.

```
t1 = time(); rep = 100;
global Guardar = zeros(rep,Nv+1);
for r = 1:rep
    pso(Ns,Nv,tmax,Inermax,Inermin,Phi1,Phi2,Velmax,Velmin)
    global Guardar[r,:] = xbest;
    println([r, Guardar[r,Nv+1]])
end
```

```
end
global Guardar = Guardar[sortperm(Guardar[:, Nv+1]), :];
et = time() - t1;
@printf("Valor maximo: %2.4f\n", Guardar[rep, Nv+1])
@printf("Valor minimo: %2.4f\n", Guardar[1, Nv+1])
@printf("Valor promedio: %2.4f\n", mean(Guardar[:, Nv+1]))
@printf("Desv. estandar: %2.4f\n", std(Guardar[:, Nv+1]))
@printf("|Minimo - Media|: %2.4f\n", abs(Guardar[1, Nv+1] - mean(Guardar[:, Nv+1])))
println("Tiempo promedio: ",
round(et/rep, digits = 4), " s");
y = Guardar[1, 1:n]; x = Guardar[1, n+1:Nv];
@printf("Binaria\tContinua\n")
for k in N
    @printf("%0.0f\t%0.5f \n", y[k], x[k])
end
```

Es esta implementación, es importante mencionar que:

- i. El algoritmo PSO se ha ajustado para un tamaño de población de 1000 individuos con un número de iteraciones máxima de 1000. Esto se hace debido a que el problema tiene una combinación de variables binarias y continuas, con lo cual se garantiza que haya gran diversidad en cuanto a la componente continua del espacio de soluciones. Además, cada búsqueda se realiza con un total de 100 iteraciones.
- ii. La población inicial se construye por etapas, donde las primeras n variables corresponden a la parte binaria del problema y la siguiente parte, es decir, desde $n + 1$ hasta $2 * n$ corresponde a la parte continua del espacio de soluciones.
- iii. El vector $y = X[j, 1:n]$ guarda los valores de las variables

binarias, mientras que el vector $x = X[j, n+1:Nv]$ hace lo correspondiente para las variables continuas.

- iv. La matriz `Guardar[rep, 1:Nv]` contiene todas las soluciones del proceso de repetición, almacenando en la posición $Nv + 1$ la función adaptación asociada a cada ejecución del PSO. Como este es un problema de minimización, y la matriz ha sido ordenada en forma ascendente, siempre la mejor solución estará en la primera fila, y la peor solución en la última fila.

Cada vez que se realiza la ejecución del PSO, considerando las repeticiones planteadas, se obtienen los resultados que se muestran a continuación.

```
Valor maximo: -25.8528
Valor minimo: -30.8501
Valor promedio: -29.6266
Desv. estandar: 1.3065
|Minimo - Media|: 1.2235
Tiempo promedio: 12.014 s
Binaria Continua
1      -1.17740
0      -4.74112
1       2.90272
0      10.75384
1      -7.02185
0       8.91271
0      -3.71467
1      -2.35051
1      -0.99436
0      -7.45762
```

Al realizar un análisis estadístico, empleando la librería `Statistics`, a los resultados obtenidos por el PSO implementado en el software Julia. Es posible apreciar que: (i) la mejor solución hallada corresponde a un valor de función adaptación de -30.8501, alcanzado la mejor solución reportada por el método exacto en el Capítulo 2. (ii) El

PSO obtuvo como valores máximo y promedio un total de -25.8528 y -29.6266, con una desviación estándar en valor real de 1.3065 unidades en la función adaptación, lo que equivale a una desviación del 4.40 %. Presentando una diferencia entre la mejor solución y la solución promedio de 1.2235 unidades de valor real, equivalente a una diferencia del 3.96 %. Estos valores demuestran que el PSO obtiene soluciones de buena calidad en relación con la calidad de la solución, con una excelente repetibilidad de la mejor solución cada vez que es ejecutado. Alcanzando la mejor solución reportada por el método exacto para este problema. (iii) El tiempo de procesamiento promedio requerido por el PSO para resolver el problema es de 12.01 segundos, lo cual lo convierte en un método eficiente desde el punto de vista de los tiempos de procesamiento.

Observación 11.5. Los resultados obtenidos en esta simulación son únicamente con fines ilustrativos, ya que estos dependen del computador en el que sea ejecutado el algoritmo, así como de los parámetros asignados por el programador.

12

Algoritmo de optimización por enjambre de Salpas

En este capítulo se presenta el algoritmo de optimización por enjambre de salpas. El cual es un algoritmo reciente y pertenece a la familia de los algoritmos poblacionales. El algoritmo basa su búsqueda en dos tipos de movimientos, el primero se basa en la mejor solución encontrada en cada iteración, conservando los límites de cada variable de decisión. Mientras que el segundo se basa en la ley de movimiento de Newton, para lo cual son necesarios dos individuos vecinos. Cada movimiento tiene un componente aleatorio, lo cual permite no solo realizar búsquedas locales, sino que permite que el algoritmo avance hacia otras regiones del espacio de solución, escapando de soluciones óptimas locales. Como ventaja principal, se destaca que el algoritmo no cuenta con parámetros de sintonización adicionales a aquellos que son comunes en los algoritmos poblacionales, tal como son el tamaño de la

población y número máximo de iteraciones. Esto obedece que las reglas de avance están fundamentadas en tres parámetros denominados c_1 , c_2 y c_3 , donde el primero de ellos está relacionado con la iteración actual y el número máximo de iteraciones, mientras que los dos restantes son totalmente aleatorios, es decir, ninguno de estos tres parámetros requiere de un proceso de sintonización.

12.1 Filosofía e inspiración

El algoritmo por enjambre de salpas, está basado en la locomoción de las salpas, las cuales son un tipo de organismo marino que presentan similitud con las medusas. La forma como se desplazan en el océano es ordenada y con cambios rápidos; sin embargo, no ha sido descifrada de manera exacta la causa de dichos movimientos. Diferentes investigadores atribuyen este compartimiento a la búsqueda de comida. Así mismo, se cree que durante el proceso de búsqueda de comida, existe una salpa líder y el resto de las salpas ordenadas son seguidoras. El algoritmo fue propuesto por Mirjalili et al. (2017b) para resolver cinco problemas de diseño de ingeniería clásicos y diseño de perfiles aerodinámicos para vehículos aeronáuticos bajo contextos mono-objetivo y multi-objetivo. En cada iteración, el algoritmo actualiza la población en su totalidad, basado en dos reglas de avance que serán discutidas en las siguientes secciones. Cada una de las reglas genera la misma cantidad de individuos, que sumados conforman el enjambre total. En cada ciclo generacional, el mejor individuo es almacenado como la posición donde se encuentra el alimento y hace referencia a la salpa líder. El resto de individuos son considerados seguidores. El primer criterio de avance tiene un componente aleatorio que consta de 2 parámetros que normaliza el rango de avance entre el valor mínimo y máximo de cada variable de decisión. Mientras que, el segundo criterio de avance depende de los valores de los individuos en la iteración actual. Dentro de las principales características presentadas por el algoritmo por enjambre de salpas se

encuentran:

- i. Para explorar el espacio de soluciones, el algoritmo utiliza una población. La cual es actualizada en cada iteración, almacenando siempre la solución incumbente.
- ii. En cada iteración se desplaza la mitad de individuos con mejor solución hacia la mejor solución encontrada hasta el momento. En este proceso, el tamaño del avance está determinado por dos componentes aleatorios. La otra mitad de individuos son desplazados considerando la posición actual de la salpa y de la que está mejor posicionada en la cadena, lo que permite intensificar la búsqueda de la solución.
- iii. Dentro del proceso iterativo, la salpa líder guía el resto de salpas, es decir, tiene la capacidad de llevar las salpas seguidoras hacia el mismo punto de convergencia acorde con las reglas de avance.
- iv. El algoritmo por enjambre de salpas emplea como criterio de convergencia un número máximo de iteraciones e iteraciones de no mejora. Teniendo en cuenta movimientos basados en la calidad de los individuos que conformar la población y números aleatorios.

El algoritmo por enjambre de salpas, ha sido explorado y aplicado en la solución de diferentes problemas de ingeniería reales lo cuales han sido reportados en la literatura especializada. Así mismo, diferentes autores han propuesto variantes del algoritmo, entre las que se destaca la versión de variables binarias, hibridación y versiones multi-objetivo (Abualigah et al., 2020).

Entre las aplicaciones del algoritmo a problemas reales se encuentra la selección de características (Ibrahim et al., 2017), control de sistemas de potencia (Sahu et al., 2018) Procesamiento de imágenes(Ibrahim et al., 2018), entrenamiento de redes neuronales (Abusnaina et al., 2018),

sistemas de energía renovables (Rezk and Fathy, 2017; Ahmed et al., 2024), programación de tareas (Sun et al., 2018) entre muchas otras aplicaciones.

12.2 Algoritmo por enjambre de salpas

En esta sección se describe de manera detallada cada una de las etapas que se requieren para la implementación del algoritmo por enjambre de salpas.

12.2.1 Inicialización de parámetros

Al inicio del proceso iterativo del algoritmo por enjambre de salpas, es necesario definir los parámetros que le permiten la exploración y explotación del espacio de solución. Los parámetros requeridos en este algoritmo, corresponde a aquellos que son comunes en la implementación de algoritmos poblacionales, tal como el número máximo de iteraciones ($tmax$) e iteraciones de no mejora ($nmax$); encargados de regular la exploración del algoritmo y el número de partículas o individuos que componen la población (n_s), los cuales son variables y determinan la convergencia del algoritmo. Finalmente, el número de variables que componen el problema (n_v) está relacionado con el problema bajo análisis.

12.2.2 Población inicial

De manera similar con otras técnicas de optimización poblacionales, en este algoritmo, la posición de cada una de las salpas se define en un espacio de búsqueda n -dimensional. Es decir, todo el conjunto de salpas que conforman la población, es almacenado en una matriz denominada X^t , donde t corresponde a la iteración actual del algoritmo. La Ecuación (12.1) presenta en forma matricial la población, siendo el número de filas asociado al número de partículas n_s y las columnas al número de

variables n_v que componen el problema.

$$X^t = \begin{bmatrix} x_{1,1} & x_{1,2} & \cdots & x_{1,j} & \cdots & x_{1,n_v} \\ x_{2,1} & x_{2,2} & \cdots & x_{2,j} & \cdots & x_{2,n_v} \\ \vdots & \vdots & \ddots & \vdots & \ddots & \vdots \\ x_{k,1} & x_{k,2} & \cdots & x_{k,j} & \cdots & x_{k,n_v} \\ \vdots & \vdots & \ddots & \vdots & \ddots & \vdots \\ x_{n_s,1} & x_{n_s,2} & \cdots & x_{n_s,j} & \cdots & x_{n_s,n_v} \end{bmatrix}, \quad (12.1)$$

En esta matriz $x_{k,j}$ corresponde al valor asignado a la j -ésima variable del k -ésimo individuo. Este valor siempre debe respetar los límites mínimos y máximos para la j -ésima variable del problema. En la primera iteración del algoritmo, la población es creada de forma aleatoria dentro de los rangos límites de las variables, por medio de la ecuación (12.2).

$$x_{k,j}^1 = x_j^{\min} + \alpha_j (x_j^{\max} - x_j^{\min}), \quad \{\forall j = 1, 2, \dots, n_v, \forall k = 1, 2, \dots, n_s\} \quad (12.2)$$

donde α_j es un valor aleatorio en el rango de cero a uno con distribución normal. Mientras $x_j^{\min}$ y $x_j^{\max}$ representan los límites inferiores y superiores asignados a la j -ésima variable. Al ser la población inicial generada en la primera iteración, t toma un valor de uno.

12.2.3 Función de adaptación

Cada una de las salpas que hacen parte de la población deben ser evaluadas en términos de la función adaptación para determinar su calidad. Cuando el individuo es factible, basta con calcular el valor de función objetivo. Sin embargo, es posible que algunos de los individuos que conforman la población sea infactibles, es decir, que se viole alguna de las restricciones técnicas del problema que se esté analizando. En este caso, es necesario diseñar una función de adaptación, la cual está compuesta por la suma del valor de función objetivo más un valor asociado

a la penalización generado por la infactibilidad, tal como se presenta en la ecuación (12.3).

$$f_a = f_o + \beta \cdot pen_v \quad (12.3)$$

donde β es un factor calculado de manera heurística, el cual permite darle un valor significativo a cada grado de penalización por concepto de violación de las restricciones técnicas. El cual debe ser ajustado para cada problema bajo análisis.

12.2.4 Reglas de avance

La evolución del algoritmo se realiza mediante dos estrategias de avance, donde cada una de ellas es aplicada al 50% de los individuos de la población en cada ciclo generacional. La primera estrategia de avance está dada por la ecuación (12.4) y se aplica a la primera mitad de la población.

$$x_{k,j}^t = \begin{cases} F_j + c_1 ((ub_j - lb_j) c_2 + lb_j) & c_3 \geq 0.5 \\ F_j - c_1 ((ub_j - lb_j) c_2 + lb_j) & c_3 < 0.5 \end{cases} \begin{cases} \forall j = 1, 1, 2 \dots N_v \\ \forall k = 1, 2, 3 \dots N_s/2 \end{cases} \quad (12.4)$$

donde F_j hace referencia a la posición donde se encuentra el alimento, es decir, F_j corresponde al mejor individuo que se tiene en la iteración t . ub_j y lb_j corresponden a los límites máximos y mínimos de cada una de las variables del problema. c_2 y c_3 son números aleatorios uniformemente distribuidos en el rango $[0, 1]$. Mientras que c_1 se calcula mediante la ecuación (12.5).

$$c_1 = 2e^{-\left(\frac{4t}{t_{max}}\right)^2} \quad (12.5)$$

Donde t corresponde a la iteración actual, y t_{max} es el número máximo de iteraciones, definido previamente como uno de los parámetros del algoritmo.

Observación 12.1. Note que c_1 es un parámetro dinámico que varía con el número de iteraciones, es decir, se constituye como el balance entre la exploración y explotación del espacio de búsqueda.

Observación 12.2. Para efectos de la implementación del algoritmo por enjambre de salpas propuesto en este capítulo, dado que el problema abordado tiene una naturaleza mixta (variables continuas y binarias), los límites de las variables ub_j y lb_j son reemplazadas por x_{min} y x_{max} para el caso de las variables continuas, mientras que para las variables binarias se utilizan las variables y_{min} y y_{max} .

La segunda regla de avance, se aplica a la otra mitad de los individuos de la población. Este avance se basa en la Ley del Movimiento de Newton (Mirjalili et al., 2017b). Para lo cual se usa la ecuación presentada en (12.6).

$$x_j^t = \frac{1}{2} (x_j^t + x_{j-1}^t) \quad \forall j = \frac{N_s}{2} + 1 \dots N_s \quad (12.6)$$

12.2.5 Sustitución de la mejor solución

Para cada iteración que se realice en el espacio de solución, se procede a la evaluación de cada salpa empleando la función de adaptación propuesta para cada problema particular acorde a la ecuación (12.3). Posteriormente, todas las salpas que se encuentran en la población son ordenadas de mejor a peor calidad acorde con el valor de la función de adaptación. De esta manera, la mejor solución es almacenada en la primera fila de la población X_1^t . Este individuo se compara con el individuo almacenado en F_j . En el caso que la función adaptación

de la salpa almacenada en X_1^t sea mejor que el valor de la función de adaptación de F_j , entonces se procede a actualizar $F_j = X_1^t$.

12.2.6 Criterios de parada

El algoritmo por enjambre de salpas converge a una solución cuando el número de iteraciones máximas o número de iteraciones de no mejora es alcanzado. Cuando esto ocurre, el algoritmo retorna como solución la mejor solución que se encuentre almacenada en F_j al momento de alcanzar el criterio de convergencia. En este libro el criterio de parada se da cuando se alcanza el número máximo de iteraciones ($tmax$).

12.2.7 Implementación computacional

El algoritmo 12 presenta el proceso iterativo del algoritmo por enjambre de salpas, realizando una descripción general del proceso de convergencia.

12.3 Implementación en el software Julia

En esta sección se describe la aplicación del algoritmo por enjambre de salpas al problema de programación no lineal entero mixto definido por el modelo (2.12)–(2.15). A continuación se muestra la implementación del algoritmo por bloques en el software Julia.

Data: Lectura de parámetros del problema bajo análisis

Definir los límites de las variables que componen el problema, i.e., n_s , $y^{\min}$, $y^{\max}$, $x^{\min}$ y $x^{\max}$;

Definir los valores de n_s , $tmax$, y $nmax$;

Crear la población (x^t) de forma aleatoria empleando la Ecuación (12.1);

Evaluar la función de adaptación para cada individuo de la población

$f(x_k^t) \forall k = 1, 2, \dots, n_s$;

Identificar y almacenar F_j ;

for $t = 1 : tmax$ **do**

 Definir c_1 usando la ecuación (12.5);

for $i = 1 : N_s$ **do**

if $i < N_s/2$ **then**

$X[i, :] \leftarrow x_i^t$ usando la ecuación (12.4);

else

$X[i, :] \leftarrow x_i^t$ usando la ecuación (12.6);

 Evaluar la función de adaptación para cada individuo de la población

$f(x_i^t) \forall i = 1, 2, \dots, n_s$;

 Actualizar F ;

Result: Reportar la mejor solución encontrada, i.e., F

Algoritmo 12: Algoritmo de optimización por enjambre de salpas

Inicialmente, se definen los datos iniciales del problema, los cuales fueron descritos en el Capítulo 2.

```
using DataFrames
Parametros = DataFrames.DataFrame([
(1, 5, 10, 0.11, -6, 5), (2, 1, 9, 0.50, -6, 8),
(3, 9, 10, 0.61, -13, 10), (4, 1, 4, 0.17, -7, 12),
(5, 7, 1, 0.44, -15, 5), (6, 2, 7, 0.56, -9, 9),
(7, 2, 1, 0.45, -11, 11), (8, 8, 4, 0.60, -5, 5),
(9, 3, 5, 0.79, -14, 14), (10, 1, 9, 0.51, -8, 14),])
DataFrames.rename!(Parametros,
[:k, :a, :b, :c, :xmin, :xmax]);
a = Parametros.a; b = Parametros.b; c = Parametros.c;
xmin = Parametros.xmin; xmax = Parametros.xmax;
n = size(Parametros, 1); N = 1:n;
```

Posteriormente, se desarrolla la función adaptación para el problema de optimización mixto, denominada `fitnessf(a,b,c,n,y,x,N)`, tal y como se presenta a continuación.

```
function fitnessf(a,b,c,n,y,x,N)
    Penalty = 100;
    Objval=(sum(y[k]*(a[k]*sin(b[k]*x[k]))+c[k]*x[k]^2)
    for k in N) + Penalty*(max(sum(y[k]
    for k in N)-n/2,0)));
    return Objval
end
```

Después de tener los datos del problema y la función adaptación programada, se procede a definir los parámetros y variables del algoritmo de optimización por enjambre de salpas, tales como el número de variables del problema (N_v), el número máximo de iteraciones (t_{max}), el número de individuos (N_s) y con el fin de generalizar se definen las variables $y_{min}=0$ y $y_{max}=1$ para los límites de las variables binarias.

```
Ns = 1500;
Nv = 2*n;
tmax = 10000;
global ymin=0;
global ymax=1;
```

Seguidamente, se procede a programar la función principal denominada `SSA(tmax,Ns,Nv,n,ymax,ymin,F,xmax,xmin,X)`, la cual corresponde al algoritmo por enjambre de salpas, tal como se presenta a continuación:

```
function SSA(tmax,Ns,Nv,n,ymax,ymin,F,xmax,xmin,X)
    for t=2:tmax
        global c1 = 2*exp(-(4*t/tmax)^2);
        for i=1:Ns
            if i<Ns/2
                for j=1:Nv
                    global c2=rand();
                    global c3=rand();
                    if j<=n && c3<=0.5
```

```

        X[i,j]=F[1,j]+
        c1*((ymax-ymin)*c2+ymin)
    elseif j<=n && c3>0.5
        X[i,j]=F[1,j]-
        c1*((ymax-ymin)*c2+ymin)
    elseif j>n && c3<=0.5
        X[i,j]=F[1,j]+
        c1*((xmax[j-n]-
        xmin[j-n])*c2+xmin[j-n])
        if X[i,j]>xmax[j-n]
            X[i,j]=xmax[j-n]
        elseif X[i,j]<xmin[j-n]
            X[i,j]=xmin[j-n]
        end
    elseif j>n && c3>=0.5
        X[i,j]=F[1,j]-
        c1*((xmax[j-n]-xmin[j-n])
        *c2+xmin[j-n])
        if X[i,j]>xmax[j-n]
            X[i,j]=xmax[j-n]
        elseif X[i,j]<xmin[j-n]
            X[i,j]=xmin[j-n]
        end
    end
    if j<=n && X[i,j]<0.5
        X[i,j]=0
    elseif j<=n && X[i,j]>=0.5
        X[i,j]=1
    end
end
else
    X[i,1:Nv]= (X[i,1:Nv].+ X[i-1,1:Nv])./2
    for j=1:n
        if X[i,j]<0.5
            X[i,j]=0
        else
            X[i,j]=1
        end
    end
end
end

```

```
end
global y = X[i,1:n];
global x = X[i,n+1:Nv];
global X[i,Int(Nv+1)] =
    fitnessf(a,b,c,n,y,x,N);
end
X = X[sortperm(X[:, Nv+1]), :];
if X[1,Nv+1] < F[1,Nv+1]
    F[1,:] = X[1,:];
end
end
return F
end
```

Es preciso resaltar que dado que el problema abordado, presenta tanto variables continuas como binarias, donde cada una de las variables tiene un límite mínimo y máximo ($x_{\max}$ y $x_{\min}$ para las variables continuas, y $y_{\min}$ y $y_{\max}$ para las variables binarias) se hace necesario usar la regla de avance definida en la ecuación (12.4) variable por variable. Así mismo, los parámetros aleatorios c_2 y c_3 se generan para cada una de las variables, mientras que el parámetros c_1 solo se calcula una vez por iteración. Debido a que al aplicar la regla de avance descrita por la ecuación (12.4), a las variables continuas, es posible que el resultado sea un valor que viole el límite máximo o mínimo, se debe realizar una reparación de las mismas. Para lo cual, si se viola el límite inferior, entonces la variable se fija en el valor mínimo permitido, pero si se viola el límite superior, entonces la variable se fija en su valor máximo. Para el caso de las variables binarias, todas las variables que tomen valores superiores o iguala a 0.5 se fijan en 1 y el en caso contrario se fijan en cero.

Observación 12.3. Note que en cada ciclo iterativo se tiene la posibilidad de actualizar el valor del individuo F , el cual al final del ciclo iterativo es el mejor individuo encontrado, pero además, es preciso

resaltar que, en cada ciclo iterativo, toda población se genera con base en dicha solución, razón por la cual, a este individuo se le conoce como la salpa líder del enjambre y al resto de salpas se denomina seguidores.

Con el fin de desarrollar una evaluación estadística básica de la implementación del algoritmo por enjambre de salpas para resolver el problema de programación no lineal entera mixta, se propone realizar la ejecución del algoritmo 100 veces (rep=100), tal como se muestra a continuación.

```
t1 = time();
rep = 100;
global Guardar = zeros(rep,Nv+1);
for r = 1:rep
    global X = rand(Float64,Ns,Nv+1);
    for j = 1:Ns
        for k = 1:Nv
            if k <= n
                X[j,k] = rand(0:1);
            else
                X[j,k] = xmin[k-n] +
                    rand(Float64) * (xmax[k-n]-xmin[k-n]);
            end
        end
        global y = X[j,1:n];
        global x = X[j,n+1:Nv];
        global X[j,Int(Nv+1)] = fitnessf(a,b,c,n,y,x,N);
    end
    X = X[sortperm(X[:, Nv+1]), :];
    global F=zeros(1,Nv+1);
    F[1,:]=X[1,:];
    global Guardar[r,:]=
        SSA(tmax,Ns,Nv,n,ymax,ymin,F,xmax,xmin,X);
    println([r, Guardar[r,Nv+1]])
end
```

Del recuadro anterior, se identifica que el algoritmo se corrió un total de 100 repeticiones, y que por cada repetición, el algoritmo iteró un total de 10000 veces con una población de 1500 individuos. Para cada una de las repeticiones, el mejor individuo encontrado se almacenó en la matriz denominada Guardar, es decir, para realizar el análisis estadístico se contó con 100 soluciones encontradas por el algoritmo.

```
global Guardar = Guardar[sortperm(Guardar[:, Nv+1]), :];  
et = time() - t1;  
@printf("Valor maximo: %2.4f  
\n", Guardar[rep, Nv+1])  
@printf("Valor minimo: %2.4f  
\n", Guardar[1, Nv+1])  
@printf("Valor promedio: %2.4f  
\n", mean(Guardar[:, Nv+1]))  
@printf("Desviacion estandar: %2.4f  
\n", std(Guardar[:, Nv+1]))  
@printf("|Minimo - Media|: %2.4f  
\n", abs(Guardar[1, Nv+1] - mean(Guardar[:, Nv+1])))  
println("Tiempo promedio: ",  
round(et/rep, digits = 4), " s");  
y = Guardar[1, 1:n]; x = Guardar[1, n+1:Nv];  
@printf("Binaria\tContinua\n")  
for k in N  
    @printf("%0.0f\t%0.5f \n", y[k], x[k])  
end
```

Una vez finalizada la ejecución de las cien repeticiones, los resultados obtenidos por el algoritmo implementado son los que se reportan a continuación:

```
Valor maximo: -21.8906  
Valor minimo: -30.8501  
Valor promedio: -28.0320  
Desviacion estandar: 1.3283  
|Minimo - Media|: 2.8181  
Tiempo promedio: 13.9437 s  
Binaria Continua  
1          -0.15701
```

0	3.89671
1	-0.15687
0	8.24974
1	-1.39457
0	4.94992
0	-4.52220
1	-0.38905
1	-0.30768
0	-2.45077

Los resultados obtenidos por el algoritmo por enjambre de salpas implementado en el software Julia. Es posible apreciar que: (i) la mejor solución hallada corresponde a un valor de función adaptación de -30.8501, alcanzado la mejor solución reportada por el método exacto en el Capítulo 2. (ii) El algoritmo obtuvo como valores máximo y promedio un total de -21.8906 y -28.0320, con una desviación estándar en valor real de 1.3283 unidades en la función adaptación. Presentando una diferencia entre la mejor solución y la solución promedio de 2.8181 unidades de valor real. Estos valores demuestran que el algoritmo por enjambre de salpas obtiene soluciones de buena calidad en relación con la calidad de la solución, con una excelente repetibilidad de la mejor solución cada vez que es ejecutado. Alcanzando la mejor solución reportada por el método exacto para este problema, (iii) El tiempo de procesamiento promedio requerido por el algoritmo para cada repetición es de 13.9437 segundos, lo cual lo convierte en un método eficiente desde el punto de vista de los tiempos de procesamiento.

Observación 12.4. Los resultados obtenidos en esta simulación son únicamente con fines ilustrativos, ya que estos dependen del computador en el que sea ejecutado el algoritmo, así como de los parámetros asignados por el programador.

13

Conclusiones, recomendaciones y trabajos futuros

13.1 Conclusiones

Este libro inicia con un análisis de los principales conceptos y metodologías empleados al resolver problema de optimización no lineales, con el objetivo de entregar al lector las herramientas necesarias para entender, formular y resolver problemas de optimización de alta complejidad con múltiple tipo de variables (discretas, continuas y mixtas). Todo lo anterior fue descrito desde un enfoque general y minimalista que le permita al lector obtener soluciones simples de buena calidad. Permitiéndole familiarizarse con los conceptos y metodologías planteadas en el resto del documento.

Dentro del Capítulo 1 se realizó la descripción de la formulación matemática que compone un problema de optimización de forma general.

Describiendo, posteriormente, las diferentes metodologías de solución que existen de acuerdo a la naturaleza del problema. Posteriormente, fueron descritas las metodologías de optimización clásicas y modernas aplicadas en la actualidad. Cerrando el Capítulo con las aplicaciones existentes dentro del campo de la ingeniería de la optimización. Este Capítulo fija las bases para cualquier persona que desee iniciar en el camino de aprendizaje de metodologías la optimización aplicada a problemas de alta complejidad.

En el Capítulo 2 de este documento se formularon y parametrizaron tres problemas de optimización que consideran variables en el dominio discreto, entero y mixto. Los cuales corresponden a problemas clásicos reportados en la literatura. El primer problema de naturaleza discreta fue el problema de la mochila, el segundo de naturaleza continua estuvo asociado al problema de despacho económico de centrales térmicas considerando pérdidas de potencia. Mientras que, el tercer problema fue desarrollado dentro de este documento para ofrecer un problema de optimización no lineal entero mixto de alta complejidad, que presente dentro de sus soluciones tanto variables de dominio discreto como continuo. Este último fue identificado dentro del documento como problema de programación no lineal entera mixta. Siendo este problema de optimización un importante aporte de este documento a la literatura.

Posterior a la formulación y descripción de los problemas de optimización. Se realizó la solución exacta de cada uno de ellos empleando el entorno de Julia. Este entorno ofrece la posibilidad de aplicar herramientas de optimización no lineales de alta efectividad para resolver problemas de naturaleza continua, entera y mixta; como los problemas abordados en este documento. Dentro del Capítulo 2 en forma de tutorial, se explicó como realizar la formulación de cada uno de los problemas planteados en Julia, y como construir el conjunto de ecuaciones que lo representa.

Para brindar solución a los problemas de optimización no lineales, fue empleado el solucionador de problemas de programación lineal o lineal entera denominado `HIGHs`, dentro del entorno de optimización

JuMP. La selección de esta herramienta por parte de los autores, se debe a la alta efectividad reportada en la literatura para esta herramienta de optimización. Dejando la puerta abierta a los lectores de este documento de plantear otros entornos de optimización exacta del software Julia, para resolver los problemas de optimización planteados. Para cada problema se analizó la solución obtenida por la herramienta de optimización, logrando garantizar la obtención de la solución óptima en el problema de la mochila y despacho económico de centrales térmicas. Esto no fue posible en el problema de optimización mixto, dado que las no linealidades de este problema generar un espacio de solución altamente complejo, que no permite a las metodologías exactas garantizar una solución óptima. Sin embargo, a pesar de que el solucionador HiGHS no declaro llegar al óptimo para este problema, algunas de las metodologías de optimización metaheurística convergieron en la misma solución reportada por el optimizador, lo cual hace pensar a los autores que la respuesta reportada por estos corresponde al óptimo del problema. La alta complejidad de este problema fue ilustrada y analizada en la Sección 2.3.1 de Capítulo 2. Mientras las soluciones obtenidas por la herramienta exacta de software Julia fueron usadas como base para el análisis de los resultados obtenidos por las técnicas de solución metaheurísticas propuestas.

A pesar del enfoque general brindado a los Capítulos 1 y 2, este documento se centra en métodos de optimización metaheurísticos. Por lo cual, el Capítulo 3 fue planteado para analizar la clasificación de las técnicas de optimización metaheurísticas, describir las metodologías basadas en población y trayectorias, realizar una descripción general de la algoritmia y programación necesaria para llevar a cabo la implementación de las técnicas metaheurísticas. Realizando en este análisis una descripción general de la metodología de avance de estas estrategias en forma de pseudocódigo. Las secciones iniciales de este capítulo le permitirán al lector identificar las principales características dentro de las estrategias de avance de las principales técnicas de optimización metaheurísticas, así como plantear un esquema de seguimiento de los

procesos iterativos de las mismas. Finalmente, dentro del Capítulo 3 se propusieron las codificaciones y se desarrollaron en el entorno de Julia las funciones adaptación de cada problema propuesto. Fijando de esta manera una base para la implementación de las técnicas de optimización. Estas herramientas guiarán a los lectores en la formulación y solución de problemas de optimización diferentes a los abordados en este documento.

Dentro de este documento se presentaron, analizaron, formularon, programaron y validaron nueve técnicas de optimización de alta implementación dentro de la literatura especializada. Es importante resaltar que todas pueden ser adaptadas para resolver los tres problemas de optimización propuestos, siguiendo las recomendaciones de este libro. Sin embargo, con el objetivo de realizar un documento simple y dinámico. Los autores de este documento tomaron la determinación de proponer tres metodologías de optimización para resolver cada problema, fragmentando de esta manera el documento en tres macro capítulos.

En la parte I del documento, que agrupo los Capítulos 4, 5 y 6, se propusieron tres técnicas metaheurísticas para resolver el problema de la mochila con variables de dominio discreto: algoritmo de senos y cosenos, algoritmo de aprendizaje incremental basado en poblaciones y el algoritmo genético. En esta sección se analizó la filosofía de inspiración, reglas de avance, criterios de convergencia, desarrollo en el entorno de programación Julia, y validación de la efectividad bajo un análisis estadístico simple del algoritmo de senos y cosenos, algoritmo de aprendizaje incremental basado en poblaciones y el algoritmo genético. El análisis estadístico ejecutó 100 veces cada metodología, analizando el valor máximo, mínimo y promedio de la función adaptación. Realizando además el análisis de la desviación estándar de las soluciones, y de la diferencia entre la mejor solución y solución promedio. Permitiendo al lector identificar la calidad de las soluciones obtenidas y la repetibilidad de las mismas; cada vez que la técnica metaheurística es ejecutada. Adicionalmente, se analizó el tiempo de procesamiento promedio requerido por cada metodología de optimización. El proceso

de análisis anteriormente descrito se implementó también en las partes II y III, que agrupan el resto de capítulos de este documento. Proponiendo de esta manera una metodología paso a paso, que permitirá al lector de este documento interpretar, formular y reproducir las metodologías de optimización propuestas; y realizar un análisis de calidad de solución y tiempos de procesamiento en función del problema analizado.

Los resultados obtenidos en la parte I, demostraron que al resolver el problema de la mochila, todas las metodologías alcanzaron la solución óptima del problema reportado por la metodología exacta en el Capítulo 2, con desviaciones estándar y desviaciones entre la mejor solución y solución promedio reducidas, y con tiempos de procesamiento bajos. Estando el tiempo de procesamiento directamente ligado a la máquina empleada para ejecutar el programa que describe cada técnica de optimización. Lo anterior demuestra que los métodos propuestos para resolver el problema en dominio discreto son adecuados en términos de la solución, con una excelente relación entre la calidad de la solución y los tiempos de procesamiento.

En la parte II de este documento, se emplearon los métodos de optimización de búsqueda de cuervos, algoritmo JAYA y el algoritmo de optimización de búsqueda por vórtices; para brindar solución al problema de optimización en dominio continuo: despacho óptimo de centrales térmicas considerando pérdidas de potencia. Posterior a describir la metodología de avance y convergencia de cada algoritmo, y realizar el desarrollo de cada uno en el entorno de programación Julia. Fue posible identificar mediante la aplicación del análisis estadístico descrito en párrafos anteriores que, ninguno de los algoritmos, llega a la solución óptima reportada para el problema en la sección 2.2.3. Sin embargo, llegan a soluciones próximas de buena calidad, con valores de desviación estándar reducidas. Con una excelente relación entre mejor solución y solución promedio. Lo anterior demuestra que este problema es altamente complejo, y que las técnicas de optimización metaheurísticas propuestas obtienen soluciones de excelente calidad, sin garantizar el óptimo global; tal cual lo enuncian sus autores en los

documentos base. Adicionalmente, las metodologías requieren tiempos de procesamiento bajos para hallar la solución del problema. Siendo el JAYA el método de optimización más rápido, seguido del algoritmo de optimización de búsqueda de cuervos y el algoritmo de optimización de búsqueda por vórtices.

Finalmente, en la parte III de este documento se propuso el algoritmo de optimización de distribución normal generalizada, el algoritmo de optimización por cúmulo de partículas y el algoritmo de optimización basado en Salpas; para brindar solución al problema de optimización mixto propuesto dentro de la Sección 2.3. Al evaluar la efectividad de las metodologías de optimización empleando el análisis estadístico simple, fue posible apreciar que de los tres problemas este fue el de mayor complejidad, al combinar variables de naturaleza discreta y continua. Requiriendo ampliar el número de iteraciones e individuos con el objetivo de alcanzar soluciones de buena calidad. Analizando los resultados alcanzados por las metodologías, fue posible apreciar que estas alcanzan la solución reportada por la metodología exacta en la sección 2.3.2, con desviaciones estándar reducidas en relación con la dimensión del problema abordado. Presentando tiempos de procesamiento cortos cada vez que los algoritmos de optimización fueron ejecutados.

13.2 Recomendaciones

Para los usuarios de este libro de investigación, enfocado en el análisis y aplicación de técnicas de optimización metaheurísticas a problemas de naturaleza discreta, continua y mixta. Los autores de este documento realizan una serie de recomendaciones simples, las cuales se describen a continuación.

- i. Al momento de analizar y replicar los modelos propuestos con la metodología exacta de solución. Se recomienda al lector inicialmente interpretar las ecuaciones que componen el modelo

matemático. Para posteriormente, realizar el seguimiento y montaje de las líneas de código en el entorno de Julia. Empleando los ejemplos numéricos incluidos en el Capítulo 2.

- ii. Antes de iniciar con el análisis de las técnicas de optimización se recomienda al lector analizar y plantear ejemplos de las codificaciones y funciones adaptación propuestas dentro del Capítulo 3. Esto con el objetivo de interpretar la forma en que son generadas las soluciones para cada problema y como se evalúa su efecto en las funciones objetivo y conjunto de restricciones propuestas. Es importante resaltar que se pueden replantear tanto las codificaciones como las funciones adaptación, en función del conocimiento del usuario de este documento. Considerando las soluciones exactas descritas en el Capítulo 2 como herramienta de validación.
- iii. Para la interpretación de las técnicas de optimización metaheurísticas propuestas, se recomienda analizar para cada una de ellas de forma detalla la sección de filosofía, reglas de avance y algoritmo que describe su proceso iterativo.
- iv. La implementación reportada en Julia de los algoritmos sigue fielmente los algoritmos propuestos por el algoritmo que describe la estrategia de avance de cada técnica. Por lo cual se invita al lector que identifique las diferentes etapas dentro de los códigos propuestas, e implemente la descripción de la misma dentro del proceso del algoritmo detallado en cada capítulo. Esto con el objetivo que sea más fácil la implementación de los algoritmos en el entorno en Julia para su validación.
- v. Solo será posible emplear como punto de comparación las soluciones reportadas por la metodología exacta en el Capítulo 2. Dado que las soluciones reportadas por las técnicas metaheurísticas son difíciles de reproducir, dado a la naturaleza aleatoria de los

procesos de avance de los algoritmos propuestos. Sin embargo, las desviaciones estándar de las soluciones deben estar en rangos similares siempre que se mantengan los parámetros de cada algoritmo.

- vi. Los parámetros de las técnicas de optimización deben ser ajustados para cada problema bajo análisis, se recomienda realizar un análisis de prueba de error cada vez que los parámetros sean modificados o los algoritmos sean adaptados a otros problemas. Es posible emplear una técnica de optimización para sintonizar los parámetros.
- vii. Los tiempos de procesamiento requeridos por las metodologías de optimización no son estándar. Estos dependerán de la máquina empleada para ejecutar el código. Se recomienda la implementación de máquinas robustas para reducir los tiempos de procesamiento, en el caso en que este sea un factor de importancia.

13.3 Trabajos futuros

Como trabajos futuros de este documento se plantean los siguientes:

- i. Emplear otras metodologías exactas del software Julia para solucionar los problemas propuestos, con el objetivo de mejorar el desempeño de las repuestas y explorar nuevas herramientas de optimización.
- ii. Aplicar las metodologías de solución propuestas a todos los problemas empleados como escenarios base, e identificar cuál es la más adecuada para cada problema en términos de calidad de la solución y tiempos de procesamiento.
- iii. Proponer una metodología de optimización que permita sintonizar los parámetros de los algoritmos y mejorar su desempeño en

términos de la relación entre calidad de la solución y tiempos de procesamiento.

- iv. Plantear técnicas de optimización modernas para resolver los problemas descritos en este documento.
- v. Analizar y modelar estrategias de procesamiento en paralelo que permitan mejorar el desempeño de las metodologías propuestas.
- vi. Analizar y plantear el procedimiento para convertir las técnicas metaheurísticas analizadas en este documento en métodos de optimización multi-objetivo de alta eficiencia.

Bibliografía

Ghada Yousif Abdallh and Zakariya Yahya Algamal. A qsar classification model of skin sensitization potential based on improving binary crow search algorithm. *Electronic Journal of Applied Statistical Analysis*, 13(1):86–95, 2020.

Mohamed Abdel-Basset, Laila Abdel-Fatah, and Arun Kumar Sangai. Metaheuristic algorithms: A comprehensive review. In *Computational intelligence for multimedia big data on the cloud with engineering applications*, pages 185–231. Elsevier, Amsterdam, Netherlands, 2018. URL <https://doi.org/10.1016/b978-0-12-813314-9.00010-4>.

A.Y. Abdelaziz, E.S. Ali, and S.M. Abd Elazim. Flower pollination algorithm to solve combined economic and emission dispatch problems. *Engineering Science and Technology, an International Journal*, 19(2):980–990, June 2016. ISSN 2215-0986. doi: 10.1016/j.jestch.2015.11.005.

Ruba Abu Khurma, Ibrahim Aljarah, Ahmad Sharieh, Mohamed Abd Elaziz, Robertas Damaševičius, and Tomas Krilavičius. A review of the modification strategies of the nature inspired algorithms for feature selection problem. *Mathematics*, 10(3):464, January 2022. ISSN 2227-7390. doi: 10.3390/math10030464.

Laith Abualigah, Mohammad Shehab, Mohammad Alshinwan, and

Hamzeh Alabool. Salp swarm algorithm: a comprehensive survey. *Neural Computing and Applications*, 32:11195–11215, 2020.

Laith Abualigah, Mohamed Abd Elaziz, Ahmad M. Khasawneh, Mohammad Alshinwan, Rehab Ali Ibrahim, Mohammed A. A. Al-qaness, Seyedali Mirjalili, Putra Sumari, and Amir H. Gandomi. Meta-heuristic optimization algorithms for solving real-world mechanical engineering design problems: a comprehensive survey, applications, comparative analysis, and results. *Neur. Comp. App.*, 34(6):4081–4110, Jan. 2022. URL <https://doi.org/10.1007/s00521-021-06747-4>.

Ahmed A Abusnaina, Sobhi Ahmad, Radi Jarrar, and Majdi Mafarja. Training neural networks using salp swarm algorithm for pattern classification. In *Proceedings of the 2nd international conference on future networks and distributed systems*, pages 1–6, 2018.

Jeffrey O. Agushaka and Absalom E. Ezugwu. Initialisation Approaches for Population-Based Metaheuristic Algorithms: A Comprehensive Review. *App. Sci.*, 12(2):896, jan 2022. URL <https://doi.org/10.3390/app12020896>.

Iman Ahmadianfar, Omid Bozorg-Haddad, and Xuefeng Chu. Gradient-based optimizer: A new metaheuristic optimization algorithm. *Information Sciences*, 540:131–159, November 2020. ISSN 0020-0255. doi: 10.1016/j.ins.2020.06.037.

Ijaz Ahmed, Muhammad Rehan, Abdul Basit, Saddam Hussain Malik, Waqas Ahmed, and Keum-Shik Hong. Adaptive salp swarm algorithm for sustainable economic and environmental dispatch under renewable energy sources. *Renewable Energy*, 223:119944, 2024.

Tanweer Alam, Shamimul Qamar, Amit Dixit, and Mohamed Benaïda. Genetic algorithm: Reviews, implementations, and applications. *arXiv preprint arXiv:2007.12673*, 2020.

- Elsayed Atif Aner, Mohammed Ibrahim Awad, and Omar M Shehata. Performance evaluation of pso-pid and pso-flc for continuum robot's developed modeling and control. *Scientific Reports*, 14(1):733, 2024.
- Tero Ansio, Hannu Kivijarvi, and Margareta Soismaa. The past, present, and future of operations research at the helsinki school of economics and business administration. *Interfaces*, 10(3):83–89, June 1980. ISSN 1526-551X. doi: 10.1287/inte.10.3.83.
- Alireza Askarzadeh. A novel metaheuristic method for solving constrained engineering optimization problems: Crow search algorithm. *Computers & Structures*, 169:1–12, June 2016a. ISSN 0045-7949. doi: 10.1016/j.compstruc.2016.03.001.
- Alireza Askarzadeh. A novel metaheuristic method for solving constrained engineering optimization problems: crow search algorithm. *Computers & structures*, 169:1–12, 2016b.
- Noor H. Awad, Mostafa Z. Ali, and Rehab M. Duwairi. Cultural algorithm with improved local search for optimization problems. In *2013 IEEE Congress on Evolutionary Computation*. IEEE, June 2013. doi: 10.1109/cec.2013.6557582.
- Shumeet Baluja. *Population-based incremental learning: A method for integrating genetic search based function optimization and competitive learning*. School of Computer Science, Carnegie Mellon University Pittsburgh, PA, 1994.
- Jagdish Chand Bansal, Prathu Bajpai, Anjali Rawat, and Atulya K. Nagar. *Sine Cosine Algorithm*, pages 15–33. Springer Nature Singapore, 2023. ISBN 9789811997228. doi: 10.1007/978-981-19-9722-8_2.
- Muhammed BasheerJasser, Mohamad Sarmini, and Rauf Yaseen. Ant colony optimization (aco) and a variation of bee colony optimization (bco) in solving tsp problem: A comparative study. *International*

Journal of Computer Applications, 96(9):1–8, June 2014. ISSN 0975-8887. doi: 10.5120/16819-6587.

David Beasley, David R Bull, and Ralph Robert Martin. An overview of genetic algorithms: Part 1, fundamentals. *University computing*, 15(2):56–69, 1993.

Jeff Bezanson, Alan Edelman, Stefan Karpinski, and Viral B Shah. Julia: A fresh approach to numerical computing. *SIAM review*, 59(1):65–98, 2017. URL <https://doi.org/10.1137/141000671>.

Anupam Biswas, K. K. Mishra, Shailesh Tiwari, and A. K. Misra. Physics-inspired optimization algorithms: A survey. *Optim. Lett.*, 2013:1–16, 2013. URL <https://doi.org/10.1155/2013/438152>.

Mirosław Blocho. *Heuristics, metaheuristics, and hyperheuristics for rich vehicle routing problems*, pages 101–156. Elsevier, 2020. doi: 10.1016/b978-0-12-815715-2.00009-9.

Sara Yulieth Bocanegra, Oscar Danilo Montoya, and Alexander Molina. Sine-cosine optimization approach applied to the parametric estimation in single-phase transformers by considering voltage and current measures. *DYNA*, 88(219):19–27, November 2021. ISSN 0012-7353. doi: 10.15446/dyna.v88n219.93670.

Freddy Bolanos, Jose Edison Aedo, and Fredy Rivera. Comparison of learning rules for adaptive population-based incremental learning algorithms. In *Proceedings on the International Conference on Artificial Intelligence (ICAI)*, page 1. The Steering Committee of The World Congress in Computer Science, Computer . . . , 2012.

Brian Borchers. *MINLP: Branch and Bound Methods*, pages 2138–2142. Springer US, 2008. ISBN 9780387747590. doi: 10.1007/978-0-387-74759-0_378.

- Miguel Angel Rodriguez Cabal, Luis Fernando Grisales Noreña, Juan Gonzalo Ardila Marín, and Oscar Danilo Montoya Giraldo. Optimal design of transmission shafts: a continuous genetic algorithm approach. *Statistics, Optimization & Information Computing*, 7(4): 802–815, 2019.
- Valentina Cacchiani, Manuel Iori, Alberto Locatelli, and Silvano Martello. Knapsack problems — an overview of recent advances. part ii: Multiple, multidimensional, and quadratic knapsack problems. *Computers & Operations Research*, 143:105693, July 2022. ISSN 0305-0548. doi: 10.1016/j.cor.2021.105693.
- Alberto Cambini and Laura Martein. Springer Berlin Heidelberg, 2009. ISBN 9783540708766. doi: 10.1007/978-3-540-70876-6.
- Juan David Camelo-Daza, Diego Noel Betancourt-Alonso, Oscar Danilo Montoya, and Ernesto Gómez-Vargas. Parameter estimation in single-phase transformers via the generalized normal distribution optimizer while considering voltage and current measurements. *Results in Engineering*, 21:101760, March 2024. ISSN 2590-1230. doi: 10.1016/j.rineng.2024.101760.
- Michael W. Carter, Camille C. Price, and Ghaith Rabadi. *Operations Research: A Practical Introduction*. Chapman and Hall/CRC, August 2018. ISBN 9781315153223. doi: 10.1201/9781315153223.
- Josu Ceberio, Ekhine Irurozki, Alexander Mendiburu, and Jose A Lozano. A review on estimation of distribution algorithms in permutation-based combinatorial optimization problems. *Progress in Artificial Intelligence*, 1:103–117, 2012.
- P.C. Chu and J.E. Beasley. A genetic algorithm for the generalised assignment problem. *Computers & Operations Research*, 24(1):17–23, 1997. ISSN 0305-0548. doi: [https://doi.org/10.1016/S0305-0548\(96\)00032-9](https://doi.org/10.1016/S0305-0548(96)00032-9). URL

[https://www.sciencedirect.com/science/
article/pii/S0305054896000329](https://www.sciencedirect.com/science/article/pii/S0305054896000329).

Antonio J. Conejo, Enrique Castillo, Roberto Mínguez, and Raquel García-Bertrand. *Decomposition Techniques in Mathematical Programming*. Springer-Verlag, 2006. ISBN 3540276858. doi: 10.1007/3-540-27686-6.

Brandon Cortés-Caicedo, Luis Fernando Grisales-Noreña, Oscar Danilo Montoya, and Rubén Iván Bolaños. Optimization of bess placement, technology selection, and operation in microgrids for minimizing energy losses and co2 emissions: A hybrid approach. *Journal of Energy Storage*, 73:108975, 2023.

Brandon Cortés-Caicedo, Luis Fernando Grisales-Noreña, Oscar Danilo Montoya, and Rubén Iván Bolaños. Minimizing the annual costs in ac distribution microgrids through optimal bess location, selection, and operation using a hybrid approach. *Journal of Energy Storage*, 84:110894, 2024.

Brandon Cortés-Caicedo, Laura Sofía Avellaneda-Gómez, Oscar Danilo Montoya, Lazaro Alvarado-Barrios, and Harold R. Chamorro. Application of the vortex search algorithm to the phase-balancing problem in distribution systems. *Energies*, 14(5):1282, February 2021. ISSN 1996-1073. doi: 10.3390/en14051282.

Erik Cuevas, Jorge Gálvez, Omar Avalos, Erik Cuevas, Jorge Gálvez, and Omar Avalos. An enhanced crow search algorithm applied to energy approaches. *Recent Metaheuristics Algorithms for Parameter Identification*, pages 27–49, 2020.

Mario César-Vélez and José Alejandro Montoya. Metaheurísticos: Una alternativa para la solución de problemas combinatorios en administración de operaciones. *Revista EIA*, (8):99–115, December 2007.

- Márcio Henrique da Silva, Ana Paula Legey, and Antônio Carlos de A Mól. The evolution of pbil algorithm when used to solve the nuclear reload optimization problem. *Annals of Nuclear Energy*, 113:393–398, 2018.
- Keshav Dahal, Stephen Remde, Peter Cowling, and Nic Colledge. Improving metaheuristic performance by evolving a variable fitness function. In *Evolutionary Computation in Combinatorial Optimization*, pages 170–181. Springer Berlin Heidelberg, 2008. URL https://doi.org/10.1007/978-3-540-78604-7_15.
- Stéphane Dauzère-Pérès, Junwen Ding, Liji Shen, and Karim Tamsaouet. The flexible job shop scheduling problem: A review. *European Journal of Operational Research*, 314(2):409–432, April 2024. ISSN 0377-2217. doi: 10.1016/j.ejor.2023.05.017.
- Rodrigo Clemente Thom De Souza, Leandro dos Santos Coelho, Camila Andrade De Macedo, and Juliano Pierezan. A v-shaped binary crow search algorithm for feature selection. In *2018 IEEE congress on evolutionary computation (CEC)*, pages 1–8. IEEE, 2018.
- Hasan Deeb, Archana Sarangi, Debahuti Mishra, and Shubhendu Kumar Sarangi. Improved black hole optimization algorithm for data clustering. *Journal of King Saud University - Computer and Information Sciences*, 34(8):5020–5029, September 2022. ISSN 1319-1578. doi: 10.1016/j.jksuci.2020.12.013.
- Sadik Ozgur Degertekin, Luciano Lamberti, and Ibrahim Behram Ugur. Sizing, layout and topology design optimization of truss structures using the jaya algorithm. *Applied soft computing*, 70:903–928, 2018.
- B. H. Dickman and M. J. Gilman. Monte carlo optimization. *Journal of Optimization Theory and Applications*, 60(1):149–157, January 1989. ISSN 1573-2878. doi: 10.1007/bf00938806.

- Zdeněk Dostál, Tomáš Kozubek, Alexandros Markopoulos, and Martin Menšík. Cholesky decomposition of a positive semidefinite matrix with known kernel. *Applied Mathematics and Computation*, 217(13): 6067–6077, March 2011. ISSN 0096-3003. doi: 10.1016/j.amc.2010.12.069.
- Berat Doğan and Tamer Ölmez. A new metaheuristic for numerical function optimization: Vortex search algorithm. *Information Sciences*, 293:125–145, February 2015a. ISSN 0020-0255. doi: 10.1016/j.ins.2014.08.053.
- Berat Doğan and Tamer Ölmez. Vortex search algorithm for the analog active filter component selection problem. *AEU - International Journal of Electronics and Communications*, 69(9):1243–1253, September 2015b. ISSN 1434-8411. doi: 10.1016/j.aeue.2015.05.005.
- Shiyamala Duraipandian, Wei Zheng, Joseph Ng, Jeffrey JH Low, A Ilancheran, and Zhiwei Huang. In vivo diagnosis of cervical precancer using raman spectroscopy and genetic algorithm techniques. *Analyst*, 136(20):4328–4336, 2011.
- Ernesto Díaz-López, Adrian Martínez-Prieto, and Daniel Gálvez-Lio. Una implementación de la meta-heurística “optimización en mallas variables” en la arquitectura cuda. *Revista Cubana de Ciencias Informáticas*, 10(3):42–57, July 2016.
- Walaa H El-Ashmawi, Ahmed F Ali, and Adam Slowik. An improved jaya algorithm with a modified swap operator for solving team formation problem. *Soft Computing*, 24:16627–16641, 2020.
- M.E. El-Hawary. An overview of economic dispatch for general hydro-thermal electric power systems. *IFAC Proceedings Volumes*, 12(5):225–233, September 1979. ISSN 1474-6670. doi: 10.1016/s1474-6670(17)65252-2.

- Komla A Folly. Performance evaluation of power system stabilizers based on population-based incremental learning (pbil) algorithm. *International Journal of Electrical Power & Energy Systems*, 33(7): 1279–1287, 2011.
- Asma Benmessaoud Gabis, Yassine Meraihi, Seyedali Mirjalili, and Amar Ramdane-Cherif. A comprehensive survey of sine cosine algorithm: variants and applications. *Artificial Intelligence Review*, 54(7):5469–5540, June 2021. ISSN 1573-7462. doi: 10.1007/s10462-021-10026-y.
- Alejandro Garces. *Mathematical Programming for Power Systems Operation with Applications in Python: From Theory to Applications in Python*. Wiley, November 2021. ISBN 9781119747291. doi: 10.1002/9781119747291.
- Alejandro Garcés Ruiz. *Optimización convexa, aplicaciones en operación y dinámica de sistemas de potencia*. Universidad Tecnológica de Pereira, 2020. ISBN 9789587224658. doi: 10.22517/9789587224658.
- Ali Ghaheri, Saeed Shoar, Mohammad Naderan, and Sayed Shahabuddin Hoseini. The applications of genetic algorithms in medicine. *Oman medical journal*, 30(6):406, 2015.
- Farhad Soleimanian Gharehchopogh, Isa Maleki, and Zahra Asheghi Dizaji. Chaotic vortex search algorithm: metaheuristic algorithm for feature selection. *Evolutionary Intelligence*, 15(3):1777–1808, March 2021. ISSN 1864-5917. doi: 10.1007/s12065-021-00590-1.
- Saeed Gholizadeh, Masood Danesh, and Changiz Gheyratmand. A new newton metaheuristic algorithm for discrete performance-based design optimization of steel moment frames. *Computers & Structures*, 234:106250, July 2020. ISSN 0045-7949. doi: 10.1016/j.compstruc.2020.106250.

- LF Grisales-Noreña, Brandon Cortés-Caicedo, Oscar Danilo Montoya, JC Hernández, and G Alcalá. A battery energy management system to improve the financial, technical, and environmental indicators of colombian urban and rural networks. *Journal of Energy Storage*, 65: 107199, 2023.
- Luis Fernando Grisales-Noreña, Daniel Gonzalez Montoya, and Carlos Andres Ramos-Paja. Optimal sizing and location of distributed generators based on pbil and pso techniques. *Energies*, 11(4):1018, 2018.
- Luis Fernando Grisales-Noreña, Oscar Danilo Montoya, Jesús C Hernández, Carlos Andres Ramos-Paja, and Alberto-Jesus Perea-Moreno. A discrete-continuous pso for the optimal integration of d-statcoms into electrical distribution systems by considering annual power loss and investment costs. *Mathematics*, 10(14):2453, 2022.
- Jun Guo, Changming Chen, Hu Wen, Guobin Cai, and Yin Liu. Prediction model of goaf coal temperature based on pso-gru deep neural network. *Case Studies in Thermal Engineering*, 53:103813, 2024.
- Ramakrishna Guttula, Venkateswara Rao Nandanavanam, and Vella Satyanarayana. Design and optimization of microstrip patch antenna via improved metaheuristic algorithm. *Wireless Personal Communications*, 120(2):1721–1739, June 2021. ISSN 1572-834X. doi: 10.1007/s11277-021-08531-y.
- John H Holland. *Adaptation in natural and artificial systems*. MIT press, 1975.
- Ying-Yi Hong and Rolando A Pula. Diagnosis of photovoltaic faults using digital twin and pso-optimized shifted window transformer. *Applied Soft Computing*, 150:111092, 2024.
- Sadmanul Hoque, Md. Rashidul Islam, Md Shafiullah, Saymun Adnan, and Md Samiul Azam. Generalized normal distribution optimization

- algorithm for economic dispatch with renewable resources integration. *Journal of Energy and Power Technology*, 05(03):1–19, September 2023. ISSN 2690-1692. doi: 10.21926/jept.2303030.
- Xiaodong Huang and Weibai Li. A new multi-material topology optimization algorithm and selection of candidate materials. *Computer Methods in Applied Mechanics and Engineering*, 386:114114, December 2021. ISSN 0045-7825. doi: 10.1016/j.cma.2021.114114.
- Amjad A. Hudaib and Hussam N. Fakhouri. Supernova optimizer: A novel natural inspired meta-heuristic. *Modern Applied Science*, 12(1): 32, December 2017. ISSN 1913-1844. doi: 10.5539/mas.v12n1p32.
- Abdelhameed Ibrahim, Ali Ahmed, Sherif Hussein, and Aboul Ella Hassanien. Fish image segmentation using salp swarm algorithm. In *The international conference on advanced machine learning technologies and applications (AMLT2018)*, pages 42–51. Springer, 2018.
- Hadeel Tariq Ibrahim, Wamidh Jalil Mazher, Osman Nuri Uçan, and Oğuz Bayat. Feature selection using salp swarm algorithm for real biomedical datasets. 2017.
- Rahim Ildarabadi, Hossein Lotfi, and Mohammad Ebrahim Hajiabadi. Resilience enhancement of distribution grids based on the construction of tie-lines using a novel genetic algorithm. *Energy Systems*, pages 1–31, 2023.
- Rabih A. Jabr. Optimization of ac transmission system planning. *IEEE Transactions on Power Systems*, 28(3):2779–2787, August 2013. ISSN 1558-0679. doi: 10.1109/tpwrs.2012.2228507.
- Meetu Jain, Vibha Saihpal, Narinder Singh, and Satya Bir Singh. An overview of variants and advancements of pso algorithm. *Applied Sciences*, 12(17):8392, 2022.

- Zoran Jakšić, Swagata Devi, Olga Jakšić, and Koushik Guha. A comprehensive review of bio-inspired optimization algorithms including applications in microelectronics and nanophotonics. *Bio-mimetics*, 8(3):278, June 2023. ISSN 2313-7673. doi: 10.3390/biomimetics8030278.
- Xiao-Yun Jiang, Wen-Chao Chen, and Yu-Tong Liu. A study on the vehicle routing problem considering infeasible routing based on the improved genetic algorithm. *International Journal of Engineering & Technology Innovation*, 14(1), 2024.
- Bo-Yeong Kang, Miao Xu, Jaesung Lee, and Dae-Won Kim. Robil: Robot path planning based on pbil algorithm. *International Journal of Advanced Robotic Systems*, 11(9):147, 2014.
- Sourabh Katoch, Sumit Singh Chauhan, and Vijay Kumar. A review on genetic algorithm: past, present, and future. *Multimed. Tools. Appl.*, 80(5):8091–8126, oct 2020. URL <https://doi.org/10.1007/s11042-020-10139-6>.
- James Kennedy and Russell Eberhart. Particle swarm optimization (pso). In *Proc. IEEE international conference on neural networks, Perth, Australia*, volume 4, pages 1942–1948, 1995.
- Ahmad S Khalil, Brett E Bouma, and Mohammad R Kaazempur Mofrad. A combined fem/genetic algorithm for vascular soft tissue elasticity estimation. *Cardiovascular Engineering*, 6:93–102, 2006.
- V. Gokula Krishnan, P. Vasantha Lakshmi, AG Noorul Julaiha, S. Lincy Jemina, A. Sunitha, and V. Divya. Vortex search algorithm based machine learning classification for ids. In *2022 6th International Conference on Trends in Electronics and Informatics (ICOEI)*. IEEE, April 2022. doi: 10.1109/icoei53556.2022.9776878.

- Meeta Kumar, Anand J. Kulkarni, and Suresh Chandra Satapathy. Socio evolution & learning optimization algorithm: A socio-inspired optimization methodology. *Future Generation Computer Systems*, 81: 252–272, April 2018. ISSN 0167-739X. doi: 10.1016/j.future.2017.10.052.
- Soukaina Laabadi, Mohamed Naimi, Hassan El Amri, and Boujemâa Achchab. A binary crow search algorithm for solving two-dimensional bin packing problem with fixed orientation. *Procedia Computer Science*, 167:809–818, 2020.
- Changlun Li, Ke Liang, Yuan Chen, and Mingzhang Pan. An exploitation-boosted sine cosine algorithm for global optimization. *Engineering Applications of Artificial Intelligence*, 117:105620, January 2023. ISSN 0952-1976. doi: 10.1016/j.engappai.2022.105620.
- Na Lin, Luwei Fu, Liang Zhao, Ammar Hawbani, Zhiyuan Tan, Ahmed Al-Dubai, and Geyong Min. A novel nomad migration-inspired algorithm for global optimization. *Computers and Electrical Engineering*, 100:107862, May 2022. ISSN 0045-7906. doi: 10.1016/j.compeleceng.2022.107862.
- The Vinh Luu and Ngoc Son Nguyen. Parameters extraction of solar cells using modified jaya algorithm. *Optik*, 203:164034, 2020.
- Bowen Lv, Bin Yang, and Ek Peng Chew. Optimization of multistage timeliness transit consolidation problem using adaptive-weighted genetic algorithm. *Annals of Operations Research*, pages 1–32, 2023.
- Yiwei Ma, Yanling Tian, and Xianping Liu. Structural optimization design of machine tools based on parallel artificial neural networks and genetic algorithms. *Neural Computing and Applications*, 35 (36):25201–25221, March 2023. ISSN 1433-3058. doi: 10.1007/s00521-023-08371-w.

- Sina Makhdoomi and Alireza Askarzadeh. Optimizing operation of a photovoltaic/diesel generator hybrid energy system with pumped hydro storage by a modified crow search algorithm. *Journal of Energy Storage*, 27:101040, 2020.
- Jyothi Mandala and MVP Chandra Sekhara Rao. Privacy preservation of data using crow search with adaptive awareness probability. *Journal of information security and applications*, 44:157–169, 2019.
- Chrysanthos Maraveas, Panagiotis G Asteris, Konstantinos G Arvanitis, Thomas Bartzanas, and Dimitrios Loukatos. Application of bio and nature-inspired algorithms in agricultural engineering. *Archives of Computational Methods in Engineering*, 30(3):1979–2012, 2023.
- Yannis Marinakis, Georgios Dounias, and Jan Jantzen. Pap smear diagnosis using a hybrid intelligent scheme focusing on genetic algorithm based feature selection and nearest neighbor classification. *Computers in Biology and Medicine*, 39(1):69–78, 2009.
- G. R. M. Mastinu and M Gobbi. On the optimal design of railway passenger vehicles. *Proceedings of the Institution of Mechanical Engineers, Part F: Journal of Rail and Rapid Transit*, 215(2):111–124, March 2001. ISSN 2041-3017. doi: 10.1243/0954409011531440.
- Pranav Mehta, Betül Sultan Yıldız, Nantiwat Pholdee, Sumit Kumar, Ali Riza Yildiz, Sadiq M. Sait, and Sujin Bureerat. A novel generalized normal distribution optimizer with elite oppositional based learning for optimization of mechanical engineering problems. *Materials Testing*, 65(2):210–223, February 2023. ISSN 2195-8572. doi: 10.1515/mt-2022-0259.
- Yassine Meraihi, Mohammed Mahseur, and Dalila Acheli. A modified binary crow search algorithm for solving the graph coloring problem. *International Journal of Applied Evolutionary Computation (IJAEC)*, 11(2):28–46, 2020.

Yassine Meraihi, Asma Benmessaoud Gabis, Amar Ramdane-Cherif, and Dalila Acheli. A comprehensive survey of crow search algorithm and its applications. *Artificial Intelligence Review*, 54(4):2669–2716, 2021.

Microsoft. Visual Studio Code. Software available online (<https://www.julia-vscode.org/>), April 2015. URL <https://www.julia-vscode.org/>.

Seyedali Mirjalili. Sca: A sine cosine algorithm for solving optimization problems. *Knowledge-Based Systems*, 96:120–133, March 2016. ISSN 0950-7051. doi: 10.1016/j.knosys.2015.12.022.

Seyedali Mirjalili and Andrew Lewis. The whale optimization algorithm. *Advances in Engineering Software*, 95:51–67, May 2016. ISSN 0965-9978. doi: 10.1016/j.advengsoft.2016.01.008.

Seyedali Mirjalili, Seyed Mohammad Mirjalili, and Abdolreza Hatamlou. Multi-verse optimizer: a nature-inspired algorithm for global optimization. *Neural Computing and Applications*, 27(2):495–513, March 2015. ISSN 1433-3058. doi: 10.1007/s00521-015-1870-7.

Seyedali Mirjalili, Amir H. Gandomi, Seyedeh Zahra Mirjalili, Shahrzad Saremi, Hossam Faris, and Seyed Mohammad Mirjalili. Salp swarm algorithm: A bio-inspired optimizer for engineering design problems. *Advances in Engineering Software*, 114:163–191, December 2017a. ISSN 0965-9978. doi: 10.1016/j.advengsoft.2017.07.002.

Seyedali Mirjalili, Amir H Gandomi, Seyedeh Zahra Mirjalili, Shahrzad Saremi, Hossam Faris, and Seyed Mohammad Mirjalili. Salp swarm algorithm: A bio-inspired optimizer for engineering design problems. *Advances in engineering software*, 114:163–191, 2017b.

Himanshu Mittal, Ashish Tripathi, Avinash Chandra Pandey, and Raju Pal. Gravitational search algorithm: a comprehensive analysis of

recent variants. *Multimedia Tools and Applications*, 80(5):7581–7608, October 2020. ISSN 1573-7721. doi: 10.1007/s11042-020-09831-4.

Jhon Montano, Oscar Daniel Garzón, Diego Alejandro Herrera-Jaramillo, Oscar Danilo Montoya, Fabio Andrade, and Andres Tobon. Estimating the parameters of a three-phase induction motor using the vortex search algorithm. *Iranian Journal of Science and Technology, Transactions of Electrical Engineering*, November 2023. ISSN 2364-1827. doi: 10.1007/s40998-023-00673-y.

O D Montoya, W Gil-González, and L F Grisales-Noreña. Sine-cosine algorithm for parameters' estimation in solar cells using datasheet information. *Journal of Physics: Conference Series*, 1671(1):012008, October 2020a. ISSN 1742-6596. doi: 10.1088/1742-6596/1671/1/012008.

Oscar Danilo Montoya. Solving a Classical Optimization Problem Using GAMS Optimizer Package: Economic Dispatch Problem Implementation. *Ingeniería y Ciencia*, 13(26):39–63, November 2017. ISSN 2256-4314. doi: 10.17230/ingciencia.13.26.2.

Oscar Danilo Montoya, Walter Gil-Gonzalez, and L. F. Grisales-Noreña. Vortex search algorithm for optimal power flow analysis in dc resistive networks with cpls. *IEEE Transactions on Circuits and Systems II: Express Briefs*, 67(8):1439–1443, August 2020b. ISSN 1558-3791. doi: 10.1109/tcsii.2019.2938530.

Oscar Danilo Montoya, Jorge Alexander Alarcon-Villamil, and Jesus C. Hernández. Operating cost reduction in distribution networks based on the optimal phase-swapping including the costs of the working groups and energy losses. *Energies*, 14(15):4535, July 2021a. ISSN 1996-1073. doi: 10.3390/en14154535.

Oscar Danilo Montoya, Federico Martin Serra, Cristian Hernan De Angelo, Harold R. Chamorro, and Lazaro Alvarado-Barrios. Heuris-

tic methodology for planning ac rural medium-voltage distribution grids. *Energies*, 14(16):5141, August 2021b. ISSN 1996-1073. doi: 10.3390/en14165141.

Oscar Danilo Montoya, Luis Fernando Grisales-Noreña, and Carlos Andres Ramos-Paja. Optimal allocation and sizing of pv generation units in distribution networks via the generalized normal distribution optimization approach. *Computers*, 11(4):53, March 2022a. ISSN 2073-431X. doi: 10.3390/computers11040053.

Oscar Danilo Montoya, Alexander Molina-Cabrera, and Walter Gil-González. A possible classification for metaheuristic optimization algorithms in engineering and science. *Ingeniería*, 27(3):e19815, August 2022b. ISSN 0121-750X. doi: 10.14483/23448393.19815.

Oscar Danilo Montoya, Walter Gil-González, Rubén Iván Bolaños, Diego Fernando Muñoz-Torres, Jesús C. Hernández, and Luis Fernando Grisales-Noreña. Effective power coordination of besus in distribution grids via the sine-cosine algorithm. In *2024 IEEE Green Technologies Conference (GreenTech)*. IEEE, April 2024a. doi: 10.1109/greentech58819.2024.10520484.

Oscar Danilo Montoya, Carlos Alberto Ramírez-Vanegas, and José Rodrigo González-Granada. Dynamic active and reactive power compensation in distribution networks using pv-statcoms: A tutorial using the julia software. *Results in Engineering*, 21:101876, March 2024b. ISSN 2590-1230. doi: 10.1016/j.rineng.2024.101876.

Kiran Chunilal More and R Venkata Rao. Design optimization of plate-fin heat exchanger by using modified jaya algorithm. In *Advanced Engineering Optimization Through Intelligent Techniques: Select Proceedings of AEOTIT 2018*, pages 165–172. Springer, 2020.

Seyed Jalaaladdin Mousavirad and Hossein Ebrahimpour-Komleh. Human mental search: a new population-based metaheuristic optimi-

zation algorithm. *Applied Intelligence*, 47(3):850–887, April 2017. ISSN 1573-7497. doi: 10.1007/s10489-017-0903-6.

Venkataraman Muthiah-Nakarajan and Mathew Mithra Noel. Galactic swarm optimization: A new global optimization metaheuristic inspired by galactic motion. *Applied Soft Computing*, 38:771–787, January 2016. ISSN 1568-4946. doi: 10.1016/j.asoc.2015.10.034.

M Mythreyee and A Nalini. Genetic algorithm based smart grid system for distributed renewable energy sources. *Computer Systems Science & Engineering*, 45(1), 2023.

Lumír Pečený, Pavol Meško, Rudolf Kampf, and Jozef Gašparík. Optimisation in transport and logistic processes. *Transportation Research Procedia*, 44:15–22, 2020. ISSN 2352-1465. doi: 10.1016/j.trpro.2020.02.003.

Harun Pirim, Engin Bayraktar, and Burak Eksioğlu. *Tabu Search: A Comparative Study*. I-Tech Education and Publishing, September 2008. doi: 10.5772/5637.

Cristina Bianca Pop, Tudor Cioara, Ionut Anghel, Marcel Antal, Viorica Rozina Chifu, Claudia Antal, and Ioan Salomie. Review of bio-inspired optimization applications in renewable-powered smart grids: Emerging population-based metaheuristics. *Energy Reports*, 8:11769–11798, November 2022. ISSN 2352-4847. doi: 10.1016/j.egyr.2022.09.025.

Helmut Prior, Ariane Schwarz, and Onur Güntürkün. Mirror-induced behavior in the magpie (*pica pica*): evidence of self-recognition. *PLoS biology*, 6(8):e202, 2008.

Thanigaivelan R., Suchithra M., Kaliappan S., and Mothilal T. *Metaheuristics Algorithm and Optimization of Engineering and Complex Systems*. IGI Global, June 2024. ISBN 9798369333150. doi: 10.4018/979-8-3693-3314-3.

- Marada Srinivasa Rao, Kaki Leela Prasad, and Pilaka Anusha. Automatic timetable generation using pbil algorithm. *I-Manager's Journal on Information Technology*, 8(2), 2019.
- R Rao. Jaya: A simple and new optimization algorithm for solving constrained and unconstrained optimization problems. *International Journal of Industrial Engineering Computations*, 7(1):19–34, 2016.
- R.V. Rao, V.J. Savsani, and D.P. Vakharia. Teaching–learning-based optimization: A novel method for constrained mechanical design optimization problems. *Computer-Aided Design*, 43(3):303–315, March 2011. ISSN 0010-4485. doi: 10.1016/j.cad.2010.12.015.
- Singiresu S. Rao. *Engineering Optimization: Theory and Practice*. Wiley, June 2009. ISBN 9780470549124. doi: 10.1002/9780470549124.
- Singiresu S Rao. *Engineering Optimization Theory and Practice*. Wiley, October 2019. ISBN 9781119454816. doi: 10.1002/9781119454816.
- Hegazy Rezk and Ahmed Fathy. A novel optimal parameters identification of triple-junction solar cell based on a recently meta-heuristic water cycle algorithm. *Solar Energy*, 157:778–791, 2017.
- Miguel Ángel Rodríguez-Cabal, Luis Fernando Grisales Noreña, Carlos Alberto Ramírez Vanegas, and Andrés Arias Londoño. Application of the sine-cosine algorithm to the optimal design of a closed coil helical spring. *Transactions on Energy Systems and Engineering Applications*, 2(2):33–38, December 2021. ISSN 2745-0120. doi: 10.32397/tesea.vol2.n2.5.
- Omar Rodríguez-Abreo, Juvenal Rodríguez-Reséndiz, José Manuel Álvarez Alvarado, and Alfonso García-Cerezo. Metaheuristic parameter identification of motors using dynamic response relations. *Sensors*, 22(11):4050, May 2022. ISSN 1424-8220. doi: 10.3390/s22114050.

- Saeyeon Roh, Vinh V. Thai, Hyunmi Jang, and Gi-Tae Yeo. The best practices of port sustainable development: a case study in korea. *Maritime Policy & Management*, 50(2):254–280, October 2021. ISSN 1464-5254. doi: 10.1080/03088839.2021.1979266.
- Juan Antonio Rojas-Quintero, Juan Antonio Rojas-Estrada, Jorge Villalobos-Chin, Victor Santibañez, and Eusebio Bugarin. Optimal controller applied to robotic systems using covariant control equations. *International Journal of Control*, 95(6):1576–1589, January 2021. ISSN 1366-5820. doi: 10.1080/00207179.2020.1865570.
- Sayan Kumar Roy and Debashis De. Genetic algorithm based internet of precision agricultural things (iopat) for agriculture 4.0. *Internet of Things*, 18:100201, 2022.
- Prakash Chandra Sahu, Ramesh Chandra Prusty, and Sidhartha Panda. Salp swarm optimized multistage pdf plus (1+ pi) controller in agc of multi source based nonlinear power system. In *Soft Computing Systems: Second International Conference, ICSCS 2018, Kollam, India, April 19–20, 2018, Revised Selected Papers 2*, pages 789–800. Springer, 2018.
- Harvey M Salkin and Cornelis A De Kluyver. The knapsack problem: a survey. *Naval Research Logistics Quarterly*, 22(1):127–144, 1975.
- Surajit Sannigrahi and Parimal Acharjee. Implementation of crow search algorithm for optimal allocation of dg and dstatcom in practical distribution system. In *2018 International Conference on Power, Instrumentation, Control and Computing (PICC)*, pages 1–6. IEEE, 2018.
- Tareq M. Shami, Ayman A. El-Saleh, Mohammed Alswaitti, Qasem Al-Tashi, Mhd Amen Summakieh, and Seyedali Mirjalili. Particle swarm optimization: A comprehensive survey. *IEEE Access*, 10:10031–10061, 2022. ISSN 2169-3536. doi: 10.1109/access.2022.3142859.

- H Shayeghi, HA Shayanfar, S Jalilzadeh, and A Safari. A pso based unified power flow controller for damping of power system oscillations. *Energy conversion and management*, 50(10):2583–2592, 2009.
- Junji Shiraishi, Qiang Li, Daniel Appelbaum, and Kunio Doi. Computer-aided diagnosis and artificial intelligence in clinical imaging. In *Seminars in nuclear medicine*, volume 41, pages 449–462. Elsevier, 2011.
- Ramteen Sioshansi and Antonio J. Conejo. *Optimization in engineering*. Springer, Ney York, NY, USA, 2017. URL <https://doi.org/10.1007/978-3-319-56769-3>.
- Dongran Song, Junbo Liu, Jian Yang, Mei Su, Yun Wang, Xuebing Yang, Lingxiang Huang, and Young Hoon Joo. Optimal design of wind turbines on high-altitude sites based on improved yin-yang pair optimization. *Energy*, 193:116794, February 2020. ISSN 0360-5442. doi: 10.1016/j.energy.2019.116794.
- Alireza Soroudi. *Power System Optimization Modeling in GAMS*. Springer International Publishing, 2017. ISBN 9783319623504. doi: 10.1007/978-3-319-62350-4.
- William M Spears. *Evolutionary Algorithms: the role of mutation and recombination*. Springer Science & Business Media, 2000.
- Zai-Xing Sun, Rong Hu, Bin Qian, Bo Liu, and Guo-Lin Che. Salp swarm algorithm based on blocks on critical path for reentrant job shop scheduling problems. In *Intelligent Computing Theories and Application: 14th International Conference, ICIC 2018, Wuhan, China, August 15-18, 2018, Proceedings, Part I 14*, pages 638–648. Springer, 2018.
- Tohru Suwa and Hamid Hadim. Optimal placement of heat generating components at various levels of electronics packaging. *Microelec-*

- MÉTODOS DE OPTIMIZACIÓN METAHEURÍSTICA PARA INGENIERÍA: Con aplicaciones en el software Julia
tronics Journal, 41(2–3):129–134, February 2010. ISSN 0026-2692. doi: 10.1016/j.mejo.2010.01.004.
- Kenneth Sörensen and Fred W. Glover. *Metaheuristics*, pages 960–970. Springer US, 2013. ISBN 9781441911537. doi: 10.1007/978-1-4419-1153-7_1167.
- Craig A. Tovey. Simulated simulated annealing. *American Journal of Mathematical and Management Sciences*, 8(3–4):389–407, February 1988. ISSN 2325-8454. doi: 10.1080/01966324.1988.10737246.
- Metin Toz. Chaos-based vortex search algorithm for solving inverse kinematics problem of serial robot manipulators with offset wrist. *Applied Soft Computing*, 89:106074, April 2020. ISSN 1568-4946. doi: 10.1016/j.asoc.2020.106074.
- Mohammad Tubishat, Zainab Rawshdeh, Hazim Jarrah, Zenab Mohamed Elgamal, Ashraf Elnagar, and Maen T. Alrashdan. Dynamic generalized normal distribution optimization for feature selection. *Neural Computing and Applications*, 34(20):17355–17370, May 2022. ISSN 1433-3058. doi: 10.1007/s00521-022-07398-9.
- Burak Urazel and Kemal Keskin. A new solution approach for non-convex combined heat and power economic dispatch problem considering power loss. *Energy*, 278:128031, 2023.
- Christian Anker Vandet and Jeppe Rich. Optimal placement and sizing of charging infrastructure for evs under information-sharing. *Technological Forecasting and Social Change*, 187:122205, February 2023. ISSN 0040-1625. doi: 10.1016/j.techfore.2022.122205.
- Julián Alejandro Vega-Forero, Jairo Stiven Ramos-Castellanos, and Oscar Danilo Montoya. Application of the generalized normal distribution optimization algorithm to the optimal selection of conductors in three-phase asymmetric distribution networks. *Energies*, 16(3):1311, January 2023. ISSN 1996-1073. doi: 10.3390/en16031311.

- Ravipudi Venkata Rao. *Jaya: An Advanced Optimization Algorithm and its Engineering Applications*. Springer International Publishing, 2019. ISBN 9783319789224. doi: 10.1007/978-3-319-78922-4.
- Chimmiri Venkateswarlu. A metaheuristic tabu search optimization algorithm: Applications to chemical and environmental processes. In *Optimization Problems in Engineering [Working Title]*. IntechOpen, jun 2021. URL <https://doi.org/10.5772/intechopen.98240>.
- Long Wang, Zijun Zhang, Chao Huang, and Kwok Leung Tsui. A gpu-accelerated parallel jaya algorithm for efficiently estimating lithium battery model parameters. *Applied Soft Computing*, 65:12–20, 2018. ISSN 1568-4946. doi: <https://doi.org/10.1016/j.asoc.2017.12.041>. URL <https://www.sciencedirect.com/science/article/pii/S1568494617307780>.
- Warid Warid. Optimal power flow using the amtpg-jaya algorithm. *Applied Soft Computing*, 91:106252, 2020.
- Xuemeng Weng, Ping Xuan, Ali Asghar Heidari, Zhennao Cai, Huiling Chen, Romany F. Mansour, and Mahmoud Ragab. A vertical and horizontal crossover sine cosine algorithm with pattern search for optimal power flow in power systems. *Energy*, 271:127000, May 2023. ISSN 0360-5442. doi: 10.1016/j.energy.2023.127000.
- Xin-She Yang. *Engineering Optimization: An Introduction with Metaheuristic Applications*. Wiley, June 2010. ISBN 9780470640425. doi: 10.1002/9780470640425.
- Yunhong Yang, Xingzhong Xiong, and Yuehao Yan. Uav formation trajectory planning algorithms: A review. *Drones*, 7(1):62, January 2023. ISSN 2504-446X. doi: 10.3390/drones7010062.
- Xiaoja Ye, Zhennao Cai, Chenglang Lu, Huiling Chen, and Zhifang Pan. Boosted sine cosine algorithm with application to medical diagnosis.

Computational and Mathematical Methods in Medicine, 2022:1–21,
June 2022. ISSN 1748-670X. doi: 10.1155/2022/6215574.

LIU Yinfeng, Soheil Jafari, and Theoklis Nikolaidis. Advanced optimization of gas turbine aero-engine transient performance using linkage-learning genetic algorithm: Part ii, optimization in flight mission and controller gains correlation development. *Chinese Journal of Aeronautics*, 34(4):568–588, 2021.

Tao Zhai, Jiabin Liu, and Daqing Wang. Optimization path of agricultural products marketing channel based on innovative industrial chain. *Economic Change and Restructuring*, 56(6):3949–3977, March 2023. ISSN 1574-0277. doi: 10.1007/s10644-023-09495-8.

Yiying Zhang. An improved generalized normal distribution optimization and its applications in numerical problems and engineering design problems. *Artificial Intelligence Review*, 56(1):685–747, April 2022. ISSN 1573-7462. doi: 10.1007/s10462-022-10182-9.

Yiying Zhang, Zhigang Jin, and Seyedali Mirjalili. Generalized normal distribution optimization and its applications in parameter extraction of photovoltaic models. *Energy Conversion and Management*, 224: 113301, November 2020. ISSN 0196-8904. doi: 10.1016/j.enconman.2020.113301.

Donglin Zhu, Siwei Wang, Changjun Zhou, Shaoqiang Yan, and Jiankai Xue. Human memory optimization algorithm: A memory-inspired optimizer for global optimization problems. *Expert Systems with Applications*, 237:121597, March 2024. ISSN 0957-4174. doi: 10.1016/j.eswa.2023.121597.

Raed Abu Zitar, Mohammed Azmi Al-Betar, Mohammed A Awadallah, Iyad Abu Doush, and Khaled Assaleh. An intensive and comprehensive overview of jaya algorithm, its versions and applications.

MÉTODOS DE OPTIMIZACIÓN METAHEURÍSTICA PARA INGENIERÍA: Con
aplicaciones en el software Julia

Archives of Computational Methods in Engineering, 29(2):763–792,
2022.

Este libro fue compuesto en caracteres
Garamond a 11 puntos, impreso sobre papel
Bond de 75 gramos y encuadernado con el
método *hot melt* en Bogotá, Colombia.

MÉTODOS DE OPTIMIZACIÓN METAHEURÍSTICA PARA INGENIERÍA

La optimización en ingeniería y ciencia es un campo de las matemáticas en constante desarrollo, que ha cobrado relevancia en el mundo moderno gracias al avance de la computación y a la necesidad de encontrar mejores soluciones para problemas con recursos limitados. En este contexto, el presente libro presenta diversas herramientas de optimización, pertenecientes a la familia de los algoritmos metaheurísticos, para abordar problemas de optimización en tres dominios diferentes.

El texto se enfoca en tres áreas de optimización. La primera aborda problemas discretos con variables binarias, como el problema de la mochila. La segunda trata problemas continuos con variables reales, ejemplificado por el despacho económico de centrales térmicas. La tercera combina ambos tipos de variables, presentando un problema de programación no lineal entero mixto, común en ingeniería y ciencia.

Está dirigido a estudiantes de ingeniería y matemáticas interesados en métodos de optimización y programación en Julia. Proporciona un enfoque práctico sobre la formulación de problemas de optimización y la implementación de técnicas exactas y metaheurísticas, facilitando la comprensión y programación de algoritmos complejos en este *software*.

Área: Computación y tecnología de la información
Subárea: Programación y desarrollo de *software*

ECOE
EDICIONES

transformate



INTEGRA®
operador de transporte masivo

Incluye

- ▶ Problemas de optimización en tres dominios específicos: discreto, continuo y mixto, con aplicaciones en ingeniería y ciencia.
- ▶ El desarrollo y programación de técnicas metaheurísticas seleccionadas en Julia, brindando un enfoque práctico y aplicable al lector.
- ▶ Algoritmos metaheurísticos con diferente filosofía e inspiración, facilitando la elección de técnicas según el escenario y necesidades del usuario.
- ▶ Tutoriales detallados que explican la lógica y programación de cada algoritmo, mejorando la comprensión y habilidades en optimización.

Óscar Danilo Montoya Giraldo

Es doctor en Ingeniería, magíster en Ingeniería Eléctrica e ingeniero electricista, todos títulos de la Universidad Tecnológica de Pereira. Actualmente, es profesor asociado en la Universidad Distrital Francisco José de Caldas. Ha publicado más de 300 artículos en revistas y conferencias indexadas. Es investigador y miembro del IEEE y del GCEM. odmontoyag@udistrital.edu.co

Luis Fernando Grisales Noreña

Es doctor en Energías Renovables por la Universidad de Jaén y en Ingeniería por la Universidad Nacional de Colombia. Magíster en Ingeniería Eléctrica e Ingeniero Electricista por la Universidad Tecnológica de Pereira. Actualmente, es profesor en la Universidad de Talca, investigador senior en Minciencias y autor de más de 170 artículos. luis.grisales@utalca.cl

Rubén Iván Bolaños

Es doctor en Ingeniería, magíster en Ingeniería Eléctrica e ingeniero electricista, todos títulos de la Universidad Tecnológica de Pereira. Actualmente, es docente en el Instituto Tecnológico Metropolitano de Medellín. Ha publicado artículos en revistas indexadas y es investigador junior en Minciencias. Miembro de IEEE y de los grupos de investigación TransFórmate y MATYER. rubenbolanos@it-m.edu.co

ISBN 978-958-508-378-3



e-ISBN 978-958-508-379-0 (PDF)